

Python 3 — Biblioteca Estándar

Tomo 2 de 6

Contenido de este tomo:

1. `functools`— Higher-order functions and operations on callable objects
2. `operator`— Standard operators as functions
3. File and Directory Access
4. `pathlib`— Object-oriented filesystem paths
5. `os.path`— Common pathname manipulations
6. `stat`— Interpreting `stat()` results
7. `filecmp`— File and Directory Comparisons
8. `tempfile`— Generate temporary files and directories
9. `glob`— Unix style pathname pattern expansion
10. `fnmatch`— Unix filename pattern matching
11. `linecache`— Random access to text lines
12. `shutil`— High-level file operations
13. Data Persistence
14. `pickle`— Python object serialization
15. `copyreg`— Register pickle support functions
16. `shelve`— Python object persistence
17. `marshal`— Internal Python object serialization
18. `dbm`— Interfaces to Unix “databases”
19. `sqlite3`— DB-API 2.0 interface for SQLite databases
20. Data Compression and Archiving
21. The compression package
22. `compression.zstd`— Compression compatible with the Zstandard format
23. `zlib`— Compression compatible with gzip
24. `gzip`— Support for gzip files
25. `bz2`— Support for bzip2 compression

- 26. lzma— Compression using the LZMA algorithm
- 27. zipfile— Work with ZIP archives
- 28. tarfile— Read and write tar archive files
- 29. File Formats
- 30. csv— CSV File Reading and Writing
- 31. configparser— Configuration file parser
- 32. tomlib— Parse TOML files
- 33. netrc— netrc file processing
- 34. plistlib— Generate and parse Apple.plistfiles
- 35. Cryptographic Services
- 36. hashlib— Secure hashes and message digests
- 37. hmac— Keyed-Hashing for Message Authentication
- 38. secrets— Generate secure random numbers for managing secrets
- 39. Generic Operating System Services
- 40. os— Miscellaneous operating system interfaces
- 41. io— Core tools for working with streams
- 42. time— Time access and conversions
- 43. logging— Logging facility for Python
- 44. logging.config— Logging configuration
- 45. logging.handlers— Logging handlers

functools— Higher-order functions and operations on callable objects

Source code: `Lib/functools.py`

The `functools` module is for higher-order functions: functions that act on or return other functions. In general, any callable object can be treated as a function for the purposes of this module.

The `functools` module defines the following functions:

@functools.cache(user_function)

Simple lightweight unbounded function cache. Sometimes called “memoize”. Returns the same as `lru_cache(maxsize=None)`, creating a thin wrapper around a dictionary lookup for the function arguments. Because it never needs to evict old values, this is smaller and faster than `lru_cache()` with a size limit. For example: `@cachedef factorial(n): return n * factorial(n-1) if n else 1` `>>> factorial(10)` # no previously cached result, makes 11 recursive calls `3628800` `>>> factorial(5)` # no new calls, just returns the cached res

@functools.cached_property(func)

Transform a method of a class into a property whose value is computed once and then cached as a normal attribute for the life of the instance. Similar to `property()`, with the addition of caching. Useful for expensive computed properties of instances that are otherwise effectively immutable. Example: `class DataSet: def __init__(self, sequence_of_numbers): self._data = tuple(sequence_of_numbers) @cached_property def stdev(self): return statistics.stdev(self._data)` The mechanics of `cached_property()` are somewhat dif

functools.cmp_to_key(func)

Transform an old-style comparison function to a key function. Used with tools that accept key functions (such as `sorted()`, `min()`, `max()`, `heapq.nlargest()`, `heapq.nsmallest()`, `itertools.groupby()`). This function is primarily used as a transition tool for programs being converted from Python 2 which supported the use of comparison functions. A comparison function is any callable that accepts two arguments, compares them, and returns a negative number for less-than, zero for equality, or a positive number

@functools.lru_cache(user_function)

@functools.lru_cache(maxsize=128, typed=False)

Decorator to wrap a function with a memoizing callable that saves up to the `maxsize` most recent calls. It can save time when an expensive or I/O bound function is periodically called with the same arguments. The cache is thread-safe so that the wrapped function can be used in multiple threads. This means that the underlying data structure will remain coherent during concurrent updates. It is possible for the wrapped function to be called more than once if another thread makes an additional call bef

@functools.total_ordering

Given a class defining one or more rich comparison ordering methods, this class decorator supplies the rest. This simplifies the effort involved in specifying all of the possible rich comparison operations: The class must define one of `__lt__()`, `__le__()`, `__gt__()`, or `__ge__()`. In addition, the class should supply an `__eq__()` method. For example: `@total_ordering class Student: def __is_valid_operand(self, other): return (hasattr(other, "lastname") and hasattr(other, "firstname")) def __eq__(self, other): if not self._is_`

functools.Placeholder

A singleton object used as a sentinel to reserve a place for positional arguments when calling `partial()` and `partialmethod()`. Added in version 3.14.

functools.partial(func, /, *args, **keywords)

Return a new `partial` object which when called will behave like `func` called with the positional arguments `args` and keyword arguments `keywords`. If more arguments are supplied to the call, they are appended to `args`. If additional keyword arguments are supplied, they extend and override `keywords`. Roughly equivalent to: `def partial(func, /, *args, **keywords): def newfunc(*more_args, **more_keywords): return func(*args, *more_args, **(keywords | more_keywords)) newfunc.func = func newfunc.args = args newfunc.keywords = keywords return newfunc`

`functools.partialmethod(func, /, *args, **keywords)`

Return a new `partialmethod` descriptor which behaves like `partial` except that it is designed to be used as a method definition rather than being directly callable. `func` must be a descriptor or a callable (objects which are both, like normal functions, are handled as descriptors). When `func` is a descriptor (such as a normal Python function, `classmethod()`, `staticmethod()`, `abstractmethod()` or another instance of `partialmethod`), calls to `__get__` are delegated to the underlying descriptor, and an appropriate `partial` object

`functools.reduce(function, iterable, /[, initial])`

Apply `function` of two arguments cumulatively to the items of `iterable`, from left to right, so as to reduce the `iterable` to a single value. For example, `reduce(lambda x, y: x+y, [1, 2, 3, 4, 5])` calculates `((((1+2)+3)+4)+5)`. The left argument, `x`, is the accumulated value and the right argument, `y`, is the update value from the `iterable`. If the optional `initial` is present, it is placed before the items of the `iterable` in the calculation, and serves as a default when the `iterable` is empty. If `initial` is not given and

`@functools.singledispatch`

Transform a function into a single-dispatch generic function. To define a generic function, decorate it with the `@singledispatch` decorator. When defining a function using `@singledispatch`, note that the dispatch happens on the type of the first argument: `>>> from functools import singledispatch >>> @singledispatch... def func(arg, verbose=False): ... if verbose: ... print("Let me just say, ", end=" ") ... print(arg)` To add overloaded implementations to the function, use the `register()` attribute of the generic function, which

`functools.singledispatchmethod(func)`

Transform a method into a single-dispatch generic function. To define a generic method, decorate it with the `@singledispatchmethod` decorator. When defining a method using `@singledispatchmethod`, note that the dispatch happens on the type of the first non-self argument: `class Negator: @singledispatchmethod def neg(self, arg): raise NotImplementedError("Cannot negate a") @neg.register def _(self, arg: int): return -arg @neg.register def _(self, arg: bool): return not arg` `singledispatchmethod` supports nesting with other

`functools.update_wrapper(wrapper, wrapped, assigned=WRAPPER_ASSIGNMENTS, updated=WRAPPER_UPDATES)`

Update a wrapper function to look like the wrapped function. The optional arguments are tuples to specify which attributes of the original function are assigned directly to the matching attributes on the wrapper function and which attributes of the wrapper function are updated with the corresponding attributes from the original function. The default values for these arguments are the module level constants `WRAPPER_ASSIGNMENTS` (which assigns to the wrapper function's `__module__`, `__name__`, `__qualname__`, `__a`

`@functools.wraps(wrapped, assigned=WRAPPER_ASSIGNMENTS, updated=WRAPPER_UPDATES)`

This is a convenience function for invoking `update_wrapper()` as a function decorator when defining a wrapper function. It is equivalent to `partial(update_wrapper, wrapped=wrapped, assigned=assigned, updated=updated)`. For example: `>>> from functools import wraps >>> def my_decorator(f): ... @wraps(f) ... def wrapper(*args, **kwargs): ... print('Calling decorated function') ... return f(*args, **kwargs) ... return wrapper ... >>> @my_decorator ... def example(): ... """ Docstring """ ... print('Called example function') ... >>> example()` Call in

partialObjects

partialobjects are callable objects created by `partial()`. They have three read-only attributes:

partial.func

A callable object or function. Calls to the `partialobject` will be forwarded to `func` with new arguments and keywords.

partial.args

The leftmost positional arguments that will be prepended to the positional arguments provided to a `partialobject` call.

partial.keywords

The keyword arguments that will be supplied when the `partialobject` is called.

`partialobjects` are like `function` objects in that they are callable, weak referenceable, and can have attributes. There are some important differences. For instance, the `__name__` and `__doc__` attributes are not created automatically.

operator— Standard operators as functions

Source code: `Lib/operator.py`

The `operator` module exports a set of efficient functions corresponding to the intrinsic operators of Python. For example, `operator.add(x,y)` is equivalent to the expression `x+y`. Many function names are those used for special methods, without the double underscores. For backward compatibility, many of these have a variant with the double underscores kept. The variants without the double underscores are preferred for clarity.

The functions fall into categories that perform object comparisons, logical operations, mathematical operations and sequence operations.

The object comparison functions are useful for all objects, and are named after the rich comparison operators they support:

`operator.lt(a,b)`

`operator.le(a,b)`

`operator.eq(a,b)`

`operator.ne(a,b)`

`operator.ge(a,b)`

`operator.gt(a,b)`

`operator.__lt__(a,b)`

`operator.__le__(a,b)`

`operator.__eq__(a,b)`

`operator.__ne__(a,b)`

`operator.__ge__(a,b)`

`operator.__gt__(a,b)`

Perform “rich comparisons” between `a` and `b`. Specifically, `lt(a,b)` is equivalent to `a < b`, `le(a,b)` is equivalent to `a <= b`, `eq(a,b)` is equivalent to `a == b`, `ne(a,b)` is equivalent to `a != b`, `gt(a,b)` is equivalent to `a > b` and `ge(a,b)` is equivalent to `a >= b`. Note that these functions can return any value, which may or may not be interpretable as a Boolean value. See [Comparisons](#) for more information about rich comparisons.

The logical operations are also generally applicable to all objects, and support truth tests, identity tests, and boolean operations:

`operator.not_(obj)`

`operator.__not__(obj)`

Return the outcome of `not obj`. (Note that there is no `__not__()` method for object instances; only the interpreter core defines this operation. The result is affected by the `__bool__()` and `__len__()` methods.)

`operator.truth(obj)`

Return `True` if `obj` is true, and `False` otherwise. This is equivalent to using the `bool` constructor.

`operator.is_(a,b)`

Return `a is b`. Tests object identity.

`operator.is_not(a,b)`

Return `a is not b`. Tests object identity.

operator.is_none(a)

Returns None. Tests object identity. Added in version 3.14.

operator.is_not_none(a)

Returns not None. Tests object identity. Added in version 3.14.

The mathematical and bitwise operations are the most numerous:

operator.abs(obj)**operator.__abs__(obj)**

Return the absolute value of obj.

operator.add(a,b)**operator.__add__(a,b)**

Return a+b, for a and b numbers.

operator.and_(a,b)**operator.__and__(a,b)**

Return the bitwise and of a and b.

operator.floordiv(a,b)**operator.__floordiv__(a,b)**

Return a//b.

operator.index(a)**operator.__index__(a)**

Return a converted to an integer. Equivalent to a.__index__(). Changed in version 3.10: The result always has exact type int. Previously, the result could have been an instance of a subclass of int.

operator.inv(obj)**operator.invert(obj)****operator.__inv__(obj)****operator.__invert__(obj)**

Return the bitwise inverse of the number obj. This is equivalent to ~obj.

operator.lshift(a,b)**operator.__lshift__(a,b)**

Return a shifted left by b.

operator.mod(a,b)**operator.__mod__(a,b)**

Return a%b.

operator.mul(a,b)**operator.__mul__(a,b)**

Return a*b, for a and b numbers.

operator.matmul(a,b)

operator.__matmul__(a,b)

Return a@b. Added in version 3.5.

operator.neg(obj)

operator.__neg__(obj)

Return obj negated (-obj).

operator.or_(a,b)

operator.__or__(a,b)

Return the bitwise or of a and b.

operator.pos(obj)

operator.__pos__(obj)

Return obj positive (+obj).

operator.pow(a,b)

operator.__pow__(a,b)

Return a**b, for a and b numbers.

operator.rshift(a,b)

operator.__rshift__(a,b)

Return a shifted right by b.

operator.sub(a,b)

operator.__sub__(a,b)

Return a-b.

operator.truediv(a,b)

operator.__truediv__(a,b)

Return a/b where 2/3 is .66 rather than 0. This is also known as “true” division.

operator.xor(a,b)

operator.__xor__(a,b)

Return the bitwise exclusive or of a and b.

Operations which work with sequences (some of them with mappings too) include:

operator.concat(a,b)

operator.__concat__(a,b)

Return a+b for a and b sequences.

operator.contains(a,b)

operator.__contains__(a,b)

Return the outcome of the test b in a. Note the reversed operands.

operator.countOf(a,b)

Return the number of occurrences of b in a.

operator.delitem(a,b)

operator.__delitem__(a,b)

Remove the value of a at index b.

operator.getitem(a,b)**operator.__getitem__(a,b)**

Return the value of a at index b.

operator.indexOf(a,b)

Return the index of the first occurrence of b in a.

operator.setitem(a,b,c)**operator.__setitem__(a,b,c)**

Set the value of a at index b to c.

operator.length_hint(obj,default=0)

Return an estimated length for the object obj. First try to return its actual length, then an estimate using `object.__length_hint__()`, and finally return the default value. Added in version 3.4.

The following operation works with callables:

operator.call(obj,/*args,kwargs)****operator.__call__(obj,/*args,**kwargs)**

Return `obj(*args,**kwargs)`. Added in version 3.11.

The `operator` module also defines tools for generalized attribute and item lookups. These are useful for making fast field extractors as arguments for `map()`, `sorted()`, `itertools.groupby()`, or other functions that expect a function argument.

operator.attrgetter(attr)**operator.attrgetter(*attrs)**

Return a callable object that fetches `attr` from its operand. If more than one attribute is requested, returns a tuple of attributes. The attribute names can also contain dots. For example: `Afterf=attrgetter('name')`, the `callf(b)` returns `b.name`. `Afterf=attrgetter('name','date')`, the `callf(b)` returns `(b.name,b.date)`. `Afterf=attrgetter('name.first','name.last')`, the `callf(b)` returns `(b.name.first,b.name.last)`. Equivalent to: `def attrgetter(*items): if any(not isinstance(item,str) for item in items): raise TypeError('attr`

operator.itemgetter(item)**operator.itemgetter(*items)**

Return a callable object that fetches `item` from its operand using the operand's `__getitem__()` method. If multiple items are specified, returns a tuple of lookup values. For example: `Afterf=itemgetter(2)`, the `callf(r)` returns `sr[2]`. `Afterg=itemgetter(2,5,3)`, the `callg(r)` returns `(r[2],r[5],r[3])`. Equivalent to: `def itemgetter(*items): if len(items)==1: item=items[0] def g(obj): return obj[item] else: def g(obj): return tuple(obj[item] for item in items) return g` The items can be any type accepted by the operand's `__getitem__()` method.

operator.methodcaller(name,/*args,kwargs)**

Return a callable object that calls the method `name` on its operand. If additional arguments and/or keyword arguments are given, they will be given to the method as well. For example: `Afterf=methodcaller('name')`, the `callf(b)` returns `sb.name()`. `Afterf=methodcaller('name','foo',bar=1)`, the `callf(b)` returns `sb.name('foo',bar=1)`. Equivalent to: `def methodcaller(name,/*args,**kwargs): def caller(obj): return getattr(obj,name)(*args,**kwargs) return caller`

Mapping Operators to Functions

This table shows how abstract operations correspond to operator symbols in the Python syntax and the functions in the `operator` module.

Operation	Syntax	Function	Addition	<code>a + b</code>	<code>add(a, b)</code>	Concatenation	<code>seq1 + seq2</code>	<code>concat(seq1, seq2)</code>	Containment Test	<code>obj in seq</code>	<code>contains(seq, obj)</code>																														
Division	<code>a / b</code>	<code>truediv(a, b)</code>	Division	<code>a // b</code>	<code>floordiv(a, b)</code>	Bitwise And	<code>a & b</code>	<code>and_(a, b)</code>	Bitwise Exclusive Or	<code>a ^ b</code>	<code>xor(a, b)</code>	Bitwise Inversion	<code>~a</code>	<code>invert(a)</code>	Bitwise Or	<code>a b</code>	<code>or_(a, b)</code>	Exponentiation	<code>a ** b</code>	<code>pow(a, b)</code>	Identity	<code>a is b</code>	<code>is_(a, b)</code>	Identity	<code>a is not b</code>	<code>is_not(a, b)</code>	Identity	<code>a is None</code>	<code>is_none(a)</code>	Identity	<code>a is not None</code>	<code>is_not_none(a)</code>	Indexed Assignment	<code>obj[k] = v</code>	<code>setitem(obj, k, v)</code>	Indexed Deletion	<code>del obj[k]</code>	<code>delitem(obj, k)</code>	Indexing	<code>obj[k]</code>	<code>getitem</code>

In-place Operators

Many operations have an “in-place” version. Listed below are functions providing a more primitive access to in-place operators than the usual syntax does; for example, the statement `x+=y` is equivalent to `x=operator.iadd(x,y)`. Another way to put it is to say that `z=operator.iadd(x,y)` is equivalent to the compound statement `z=x;z+=y`.

In those examples, note that when an in-place method is called, the computation and assignment are performed in two separate steps. The in-place functions listed below only do the first step, calling the in-place method. The second step, assignment, is not handled.

For immutable targets such as strings, numbers, and tuples, the updated value is computed, but not assigned back to the input variable:

```
>>> a = 'hello'
>>> iadd(a, ' world')
'hello world'
>>> a
'hello'
```

For mutable targets such as lists and dictionaries, the in-place method will perform the update, so no subsequent assignment is necessary:

```
>>> s = ['h', 'e', 'l', 'l', 'o']
>>> iadd(s, [' ', 'w', 'o', 'r', 'l', 'd'])
['h', 'e', 'l', 'l', 'o', ' ', 'w', 'o', 'r', 'l', 'd']
>>> s
['h', 'e', 'l', 'l', 'o', ' ', 'w', 'o', 'r', 'l', 'd']
```

`operator.iadd(a,b)`

`operator.__iadd__(a,b)`

`a=iadd(a,b)` is equivalent to `a+=b`.

`operator.iand(a,b)`

`operator.__iand__(a,b)`

`a=iand(a,b)` is equivalent to `a&=b`.

`operator.iconcat(a,b)`

`operator.__iconcat__(a,b)`

`a=iconcat(a,b)` is equivalent to `a+=b` for `a` and `b` sequences.

operator.ifloordiv(a,b)

operator.__ifloordiv__(a,b)

a=ifloordiv(a,b)is equivalent to a//b.

operator.ilshift(a,b)

operator.__ilshift__(a,b)

a=ilshift(a,b)is equivalent to a<<=b.

operator.imod(a,b)

operator.__imod__(a,b)

a=imod(a,b)is equivalent to a%=b.

operator.imul(a,b)

operator.__imul__(a,b)

a=imul(a,b)is equivalent to a*=b.

operator.imatmul(a,b)

operator.__imatmul__(a,b)

a=imatmul(a,b)is equivalent to a@=b.Added in version 3.5.

operator.ior(a,b)

operator.__ior__(a,b)

a=ior(a,b)is equivalent to a|=b.

operator.ipow(a,b)

operator.__ipow__(a,b)

a=ipow(a,b)is equivalent to a**=b.

operator.irshift(a,b)

operator.__irshift__(a,b)

a=irshift(a,b)is equivalent to a>>=b.

operator.isub(a,b)

operator.__isub__(a,b)

a=isub(a,b)is equivalent to a-=b.

operator.itruediv(a,b)

operator.__itruediv__(a,b)

a=itruediv(a,b)is equivalent to a/=b.

operator.ixor(a,b)

operator.__ixor__(a,b)

a=ixor(a,b)is equivalent to a^=b.

File and Directory Access

The modules described in this chapter deal with disk files and directories. For example, there are modules for reading the properties of files, manipulating paths in a portable way, and creating temporary files. The full list of modules in this chapter is:

- `pathlib`— Object-oriented filesystem pathsBasic useExceptionsPure pathsGeneral propertiesOperatorsAccessing individual partsMethods and propertiesConcrete pathsParsing and generating URIsExpanding and resolving pathsQuerying file type and statusReading and writing filesReading directoriesCreating files and directoriesCopying, moving and deletingPermissions and ownershipPattern languageComparison to
- `os.path`— Common pathname manipulations
- `stat`— Interpreting`stat()`results
- `filecmp`— File and Directory ComparisonsThe`dircmp`class
- `tempfile`— Generate temporary files and directoriesExamplesDeprecated functions and variables
- `glob`— Unix style pathname pattern expansionExamples
- `fnmatch`— Unix filename pattern matching
- `linecache`— Random access to text lines
- `shutil`— High-level file operationsDirectory and files operationsPlatform-dependent efficient copy operations`copytree` example`lrmmtree` exampleArchiving operationsArchiving exampleArchiving example with`base_dir`Querying the size of the output terminal

See also

Module `os`

Module `io`

Built-in function `open()`

Operating system interfaces, including functions to work with files at a lower level than Pythonfile objects.

Python's built-in I/O library, including both abstract classes and some concrete classes such as file I/O.

The standard way to open files for reading and writing with Python.

pathlib— Object-oriented filesystem paths

Added in version 3.4.

Source code: [Lib/pathlib/](#)

This module offers classes representing filesystem paths with semantics appropriate for different operating systems. Path classes are divided between pure paths, which provide purely computational operations without I/O, and concrete paths, which inherit from pure paths but also provide I/O operations.

If you've never used this module before or just aren't sure which class is right for your task, `Path` is most likely what you need. It instantiates a concrete path for the platform the code is running on.

Pure paths are useful in some special cases; for example:

- If you want to manipulate Windows paths on a Unix machine (or vice versa). You cannot instantiate `WindowsPath` when running on Unix, but you can instantiate `PureWindowsPath`.
- You want to make sure that your code only manipulates paths without actually accessing the OS. In this case, instantiating one of the pure classes may be useful since those simply don't have any OS-accessing operations.

See also

PEP 428: The `pathlib` module – object-oriented filesystem paths.

See also

For low-level path manipulation on strings, you can also use the `os.path` module.

Basic use

Importing the main class:

```
>>> from pathlib import Path
```

Listing subdirectories:

```
>>> p = Path('.')
>>> [x for x in p.iterdir() if x.is_dir()]
[PosixPath('.hg'), PosixPath('docs'), PosixPath('dist'),
 PosixPath('__pycache__'), PosixPath('build')]
```

Listing Python source files in this directory tree:

```
>>> list(p.glob('**/*.py'))
[PosixPath('test_pathlib.py'), PosixPath('setup.py'),
 PosixPath('pathlib.py'), PosixPath('docs/conf.py'),
 PosixPath('build/lib/pathlib.py')]
```

Navigating inside a directory tree:

```
>>> p = Path('/etc')
>>> q = p / 'init.d' / 'reboot'
>>> q
PosixPath('/etc/init.d/reboot')
>>> q.resolve()
PosixPath('/etc/rc.d/init.d/halt')
```

Querying path properties:

```
>>> q.exists()
True
>>> q.is_dir()
```

False

Opening a file:

```
>>> with q.open() as f: f.readline()
...
'#!/bin/bash\n'
```

Exceptions

exceptionpathlib.UnsupportedOperation

An exception inheriting `NotImplementedError` that is raised when an unsupported operation is called on a path object. Added in version 3.13.

Pure paths

Pure path objects provide path-handling operations which don't actually access a filesystem. There are three ways to access these classes, which we also call flavours:

classpathlib.PurePath(*pathsegments)

A generic class that represents the system's path flavour (instantiating it creates either a `PurePosixPath` or a `PureWindowsPath`):>>> `PurePath('setup.py')` # Running on a Unix machine `PurePosixPath('setup.py')` Each element of `pathsegments` can be either a string representing a path segment, or an object implementing the `os.PathLike` interface where the `__fspath__()` method returns a string, such as another path object:>>> `PurePath('foo', 'some/path', 'bar')` `PurePosixPath('foo/some/path/bar')` >>> `PurePath(Path('foo'), Pa`

classpathlib.PurePosixPath(*pathsegments)

A subclass of `PurePath`, this path flavour represents non-Windows filesystem paths:>>> `PurePosixPath('/etc/hosts')` `PurePosixPath('/etc/hosts')` `pathsegments` is specified similarly to `PurePath`.

classpathlib.PureWindowsPath(*pathsegments)

A subclass of `PurePath`, this path flavour represents Windows filesystem paths, including UNC paths:>>> `PureWindowsPath('c:/Users/Ximénez')` `PureWindowsPath('c:/Users/Ximénez')` >>> `PureWindowsPath('//server/share/file')` `PureWindowsPath('//server/share/file')` `pathsegments` is specified similarly to `PurePath`.

Regardless of the system you're running on, you can instantiate all of these classes, since they don't provide any operation that does system calls.

General properties

Paths are immutable and hashable. Paths of a same flavour are comparable and orderable. These properties respect the flavour's case-folding semantics:

```
>>> PurePosixPath('foo') == PurePosixPath('F00')
False
>>> PureWindowsPath('foo') == PureWindowsPath('F00')
True
>>> PureWindowsPath('F00') in { PureWindowsPath('foo') }
True
>>> PureWindowsPath('C:') < PureWindowsPath('d:')
True
```

Paths of a different flavour compare unequal and cannot be ordered:

```
>>> PureWindowsPath('foo') == PurePosixPath('foo')
False
>>> PureWindowsPath('foo') < PurePosixPath('foo')
```

```
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
TypeError: '<' not supported between instances of 'PureWindowsPath' and 'PurePosixPath'
```

Operators

The slash operator helps create child paths, like `os.path.join()`. If the argument is an absolute path, the previous path is ignored. On Windows, the drive is not reset when the argument is a rooted relative path (e.g., `r'foo'`):

```
>>> p = PurePath('/etc')
>>> p
PurePosixPath('/etc')
>>> p / 'init.d' / 'apache2'
PurePosixPath('/etc/init.d/apache2')
>>> q = PurePath('bin')
>>> '/usr' / q
PurePosixPath('/usr/bin')
>>> p / '/an_absolute_path'
PurePosixPath('/an_absolute_path')
>>> PureWindowsPath('c:/Windows', '/Program Files')
PureWindowsPath('c:/Program Files')
```

A path object can be used anywhere an object implementing `os.PathLike` is accepted:

```
>>> import os
>>> p = PurePath('/etc')
>>> os.fspath(p)
'/etc'
```

The string representation of a path is the raw filesystem path itself (in native form, e.g. with backslashes under Windows), which you can pass to any function taking a file path as a string:

```
>>> p = PurePath('/etc')
>>> str(p)
'/etc'
>>> p = PureWindowsPath('c:/Program Files')
>>> str(p)
'c:\\Program Files'
```

Similarly, calling `bytes` on a path gives the raw filesystem path as a bytes object, as encoded by `os.fsencode()`:

```
>>> bytes(p)
b'/etc'
```

■ **Nota:** Note that calling `bytes` is only recommended under Unix. Under Windows, the unicode form is the canonical representation of filesystem paths.

Accessing individual parts

To access the individual “parts” (components) of a path, use the following property:

PurePath.parts

A tuple giving access to the path’s various components: `>>>p=PurePath('/usr/bin/python3')>>>p.parts('/', 'usr', 'bin', 'python3')>>>p=PureWindowsPath('c:/Program Files/PSF')>>>p.parts('c:\\', 'Program Files', 'PSF')`(note how the drive and local root are regrouped in a single part)

Methods and properties

Pure paths provide the following methods and properties:

PurePath.parser

The implementation of the `os.path` module used for low-level path parsing and joining: either `posixpath` or `ntpath`. Added in version 3.13.

PurePath.drive

A string representing the drive letter or name, if any:>>>PureWindowsPath('c:/Program Files/').drive'c:'>>>PureWindowsPath('/Program Files/').drive">>>PurePosixPath('/etc').drive"UNC shares are also considered drives:>>>PureWindowsPath('//host/share/foo.txt').drive'\\\\host\\share'

PurePath.root

A string representing the (local or global) root, if any:>>>PureWindowsPath('c:/Program Files/').root'\\>>>PureWindowsPath('c:Program Files/').root">>>PurePosixPath('/etc').root/'UNC shares always have a root:>>>PureWindowsPath('//host/share').root'\\If the path starts with more than two successive slashes,PurePosixPathcollapses them:>>>PurePosixPath('//etc').root'/'>>>PurePosixPath('///etc').root'/'>>>PurePosixPath('////etc').root'/'NoteThis behavior conforms toThe Open Group Base Specific

PurePath.anchor

The concatenation of the drive and root:>>>PureWindowsPath('c:/Program Files/').anchor'c:\\>>>PureWindowsPath('c:Program Files/').anchor'c:'>>>PurePosixPath('/etc').anchor'/'>>>PureWindowsPath('//host/share').anchor'\\\\host\\share\\'

PurePath.parents

An immutable sequence providing access to the logical ancestors of the path:>>>p=PureWindowsPath('c:/foo/bar/setup.py')>>>p.parents[0]PureWindowsPath('c:/foo/bar')>>>p.parents[1]PureWindowsPath('c:/foo')>>>p.parents[2]PureWindowsPath('c:/')Changed in version 3.10:The parents sequence now supportslicesand negative index values.

PurePath.parent

The logical parent of the path:>>>p=PurePosixPath('/a/b/c/d')>>>p.parentPurePosixPath('/a/b/c')You cannot go past an anchor, or empty path:>>>p=PurePosixPath('/')>>>p.parentPurePosixPath('/')>>>p=PurePosixPath('.')>>>p.parentPurePosixPath('.')NoteThis is a purely lexical operation, hence the following behaviour:>>>p=PurePosixPath('foo/..')>>>p.parentPurePosixPath('foo')If you want to walk an arbitrary filesystem path upwards, it is recommended to first callPath.resolve()so as to resolve symlinks

PurePath.name

A string representing the final path component, excluding the drive and root, if any:>>>PurePosixPath('my/library/setup.py').name'setup.py'UNC drive names are not considered:>>>PureWindowsPath('//some/share/setup.py').name'setup.py'>>>PureWindowsPath('//some/share').name''

PurePath.suffix

The last dot-separated portion of the final component, if any:>>>PurePosixPath('my/library/setup.py').suffix'.py'>>>PurePosixPath('my/library.tar.gz').suffix'.gz'>>>PurePosixPath('my/library').suffix""This is commonly called the file extension.Changed in version 3.14:A single dot (".") is considered a valid suffix.

PurePath.suffixes

A list of the path's suffixes, often called file extensions:>>>PurePosixPath('my/library.tar.gar').suffixes['.tar', '.gar']>>>PurePosixPath('my/library.tar.gz').suffixes['.tar', '.gz']>>>PurePosixPath('my/library').suffixes[]Changed in version 3.14:A single dot (".") is considered a valid suffix.

PurePath.stem

The final path component, without its suffix:>>>PurePosixPath('my/library.tar.gz').stem'library.tar'>>>PurePosixPath('my/library.tar').stem'library'>>>PurePosixPath('my/library').stem'library'Changed in version 3.14:A single dot (".") is considered a valid suffix.

PurePath.as_posix()

Return a string representation of the path with forward slashes
(/):>>>p=PureWindowsPath('c:\\windows')>>>str(p)'c:\\windows'>>>p.as_posix()'/windows'

PurePath.is_absolute()

Return whether the path is absolute or not. A path is considered absolute if it has both a root and (if the flavour allows) a drive:>>>PurePosixPath('/a/b').is_absolute()True>>>PurePosixPath('a/b').is_absolute()False>>>PureWindowsPath('c:/a/b').is_absolute()True>>>PureWindowsPath('/a/b').is_absolute()False>>>PureWindowsPath('c:').is_absolute()False>>>PureWindowsPath('//some/share').is_absolute()True

PurePath.is_relative_to(other)

Return whether or not this path is relative to
theotherpath.>>>p=PurePath('/etc/passwd')>>>p.is_relative_to('/etc')True>>>p.is_relative_to('/usr')FalseThis method is string-based; it neither accesses the filesystem nor treats “.” segments specially. The following code is equivalent:>>>u=PurePath('/usr')>>>u==poruinp.parentsFalseAdded in version 3.9.Deprecated since version 3.12, removed in version 3.14:Passing additional arguments is deprecated; if supplied, they are joined withother.

PurePath.is_reserved()

WithPureWindowsPath, returnTrueif the path is considered reserved under Windows,Falseotherwise.
WithPurePosixPath,Falseis always returned.Changed in version 3.13:Windows path names that contain a colon, or end with a dot or a space, are considered reserved. UNC paths may be reserved.Deprecated since version 3.13, will be removed in version 3.15:This method is deprecated; useos.path.isreserved()to detect reserved paths on Windows.

PurePath.joinpath(*pathsegments)

Calling this method is equivalent to combining the path with each of the givenpathsegmentsin turn:>>>PurePosixPath('/etc').joinpath('passwd')PurePosixPath('/etc/passwd')>>>PurePosixPath('/etc').joinpath(PurePosixPath('passwd'))PurePosixPath('/etc/passwd')>>>PurePosixPath('/etc').joinpath('init.d','apache2')PurePosixPath('/etc/init.d/apache2')>>>PureWindowsPath('c:').joinpath('/Program Files')PureWindowsPath('c:/Program Files')

PurePath.full_match(pattern,*,case_sensitive=None)

Match this path against the provided glob-style pattern. ReturnTrueif matching is successful,Falseotherwise.
For example:>>>PurePath('a/b.py').full_match('a/*.py')True>>>PurePath('a/b.py').full_match('*.py')False>>>PurePath('/a/b/c.py').full_match('/a/**')True>>>PurePath('/a/b/c.py').full_match('**/*.py')TrueSee alsoPattern language documentation.As with other methods, case-sensitivity follows platform defaults:>>>PurePosixPath('b.py').full_match('*.PY')False>>>PureWindowsPath('b.py').full_matc

PurePath.match(pattern,*,case_sensitive=None)

Match this path against the provided non-recursive glob-style pattern. ReturnTrueif matching is successful,Falseotherwise.This method is similar tofull_match(), but empty patterns aren't allowed (ValueErroris raised), the recursive wildcard “***” isn't supported (it acts like non-recursive “**”), and if a relative pattern is provided, then matching is done from the right:>>>PurePath('a/b.py').match('*.py')True>>>PurePath('/a/b/c.py').match('b/*.py')True>>>PurePath('/a/b/c.py').match('a/*.py')False

PurePath.relative_to(other,walk_up=False)

Compute a version of this path relative to the path represented byother. If it's impossible,ValueErroris raised:>>>p=PurePosixPath('/etc/passwd')>>>p.relative_to('/')PurePosixPath('etc/passwd')>>>p.relative_to('/etc')PurePosixPath('passwd')>>>p.relative_to('/usr')Traceback (most recent call last):File"<stdin>", line1, in<module>File"pathlib.py", line941, inrelative_toraiseValueError(error_message.format(str(self),str(formatted)))ValueError:'/etc/passwd' is not in the subpath of '/usr' OR one pa

PurePath.with_name(name)

Return a new path with the name changed. If the original path doesn't have a name, ValueError is raised: >>> p = PureWindowsPath('c:/Downloads/pathlib.tar.gz') >>> p.with_name('setup.py') PureWindowsPath('c:/Downloads/setup.py') >>> p = PureWindowsPath('c:/') >>> p.with_name('setup.py') Traceback (most recent call last): File "<stdin>", line 1, in <module> File "/home/antoine/cpython/default/Lib/pathlib.py", line 751, in with_name raise ValueError("%r has an empty name" % (self,)) ValueError: PureWindowsPath('c:/') has an empty name

PurePath.with_stem(stem)

Return a new path with the stem changed. If the original path doesn't have a name, ValueError is raised: >>> p = PureWindowsPath('c:/Downloads/draft.txt') >>> p.with_stem('final') PureWindowsPath('c:/Downloads/final.txt') >>> p = PureWindowsPath('c:/Downloads/pathlib.tar.gz') >>> p.with_stem('lib') PureWindowsPath('c:/Downloads/lib.gz') >>> p = PureWindowsPath('c:/') >>> p.with_stem("") Traceback (most recent call last): File "<stdin>", line 1, in <module> File "/home/antoine/cpython/default/Lib/pathlib.py", line 861, in with_stem

PurePath.with_suffix(suffix)

Return a new path with the suffix changed. If the original path doesn't have a suffix, the new suffix is appended instead. If the suffix is an empty string, the original suffix is removed: >>> p = PureWindowsPath('c:/Downloads/pathlib.tar.gz') >>> p.with_suffix('.bz2') PureWindowsPath('c:/Downloads/pathlib.tar.bz2') >>> p = PureWindowsPath('README') >>> p.with_suffix('.txt') PureWindowsPath('README.txt') >>> p = PureWindowsPath('README.txt') >>> p.with_suffix("") PureWindowsPath('README') Changed in version 3.14: A single

PurePath.with_segments(*pathsegments)

Create a new path object of the same type by combining the given path segments. This method is called whenever a derivative path is created, such as from parent and relative_to(). Subclasses may override this method to pass information to derivative paths, for example: from pathlib import PurePosixPath class MyPath(PurePosixPath): def __init__(self, *pathsegments, session_id): super().__init__(*pathsegments) self.session_id = session_id def with_segments(self, *pathsegments): return type(self)(*pathsegments, session_id=

Concrete paths

Concrete paths are subclasses of the pure path classes. In addition to operations provided by the latter, they also provide methods to do system calls on path objects. There are three ways to instantiate concrete paths:

class pathlib.Path(*pathsegments)

A subclass of PurePath, this class represents concrete paths of the system's path flavour (instantiating it creates either a PosixPath or a WindowsPath): >>> Path('setup.py') PosixPath('setup.py') pathsegments is specified similarly to PurePath.

class pathlib.PosixPath(*pathsegments)

A subclass of Path and PurePosixPath, this class represents concrete non-Windows filesystem paths: >>> PosixPath('/etc/hosts') PosixPath('/etc/hosts') pathsegments is specified similarly to PurePath. Changed in version 3.13: Raises UnsupportedOperation on Windows. In previous versions, NotImplementedError was raised instead.

class pathlib.WindowsPath(*pathsegments)

A subclass of Path and PureWindowsPath, this class represents concrete Windows filesystem paths: >>> WindowsPath('c:/Users/Ximénez') WindowsPath('c:/Users/Ximénez') pathsegments is specified similarly to PurePath. Changed in version 3.13: Raises UnsupportedOperation on non-Windows platforms. In previous versions, NotImplementedError was raised instead.

You can only instantiate the class flavour that corresponds to your system (allowing system calls on non-compatible path flavours could lead to bugs or failures in your application):

```
>>> import os
>>> os.name
'posix'
>>> Path('setup.py')
PosixPath('setup.py')
>>> PosixPath('setup.py')
PosixPath('setup.py')
>>> WindowsPath('setup.py')
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
  File "pathlib.py", line 798, in __new__
    % (cls.__name__,))
UnsupportedOperation: cannot instantiate 'WindowsPath' on your system
```

Some concrete path methods can raise an `OSError` if a system call fails (for example because the path doesn't exist).

Parsing and generating URIs

Concrete path objects can be created from, and represented as, 'file' URIs conforming to RFC 8089.

■ Nota: `NoteFile` URIs are not portable across machines with different filesystem encodings.

`classmethod Path.from_uri(uri)`

Return a new path object from parsing a 'file' URI. For example: `>>>p=Path.from_uri('file:///etc/hosts')PosixPath('/etc/hosts')` On Windows, DOS device and UNC paths may be parsed from URIs: `>>>p=Path.from_uri('file:///c:/windows')WindowsPath('c:/windows')` `>>>p=Path.from_uri('file://server/share')WindowsPath('//server/share')` Several variant forms are supported: `>>>p=Path.from_uri('file:///server/share')WindowsPath('//server/share')` `>>>p=Path.from_uri('file://server/share')WindowsPath('//server/shar`

`Path.as_uri()`

Represent the path as a 'file' URI. `ValueError` is raised if the path isn't absolute. `>>>p=PosixPath('/etc/passwd')` `>>>p.as_uri()` `'file:///etc/passwd'` `>>>p=WindowsPath('c:/Windows')` `>>>p.as_uri()` `'file:///c:/Windows'` `Deprecated since version 3.14, will be removed in version 3.19: Calling this method from PurePath rather than Path is possible but deprecated. The method's use of os.fspath() makes it strictly impure.`

Expanding and resolving paths

`classmethod Path.home()`

Return a new path object representing the user's home directory (as returned by `os.path.expanduser()` with `~`-construct). If the home directory can't be resolved, `RuntimeError` is raised. `>>>Path.home()PosixPath('/home/antoine')` Added in version 3.5.

`Path.expanduser()`

Return a new path with expanded `~`- and `~user`-constructs, as returned by `os.path.expanduser()`. If a home directory can't be resolved, `RuntimeError` is raised. `>>>p=PosixPath('~films/Monty Python')` `>>>p.expanduser()PosixPath('/home/eric/films/Monty Python')` Added in version 3.5.

`classmethod Path.cwd()`

Return a new path object representing the current directory (as returned by `os.getcwd()`): `>>>Path.cwd()PosixPath('/home/antoine/pathlib')`

`Path.absolute()`

Make the path absolute, without normalization or resolving symlinks. Returns a new path object: `>>>p=Path('tests')` `>>>pPosixPath('tests')` `>>>p.absolute()PosixPath('/home/antoine/pathlib/tests')`

Path.resolve(strict=False)

Make the path absolute, resolving any symlinks. A new path object is returned:
`>>>p=Path()>>>pPosixPath('.')>>>p.resolve()PosixPath('/home/antoine/pathlib')"` components are also eliminated (this is the only method to do so):
`>>>p=Path('docs/./setup.py')>>>p.resolve()PosixPath('/home/antoine/pathlib/setup.py')` If a path doesn't exist or a symlink loop is encountered, and `strict=True`, `OSError` is raised. If `strict=False`, the path is resolved as far as possible and any remainder is appended without

Path.readlink()

Return the path to which the symbolic link points (as returned by `os.readlink()`):
`>>>p=Path('mylink')>>>p.symlink_to('setup.py')>>>p.readlink()PosixPath('setup.py')` Added in version 3.9. Changed in version 3.13: Raises `UnsupportedOperation` if `os.readlink()` is not available. In previous versions, `NotImplementedError` was raised.

Querying file type and status

Changed in version

3.8: `exists()`, `is_dir()`, `is_file()`, `is_mount()`, `is_symlink()`, `is_block_device()`, `is_char_device()`, `is_fifo()`, `is_socket()` now return `False` instead of raising an exception for paths that contain characters unrepresentable at the OS level.

Changed in version 3.14: The methods given above now return `False` instead of raising any `OSError` exception from the operating system. In previous versions, some kinds of `OSError` exception are raised, and others suppressed. The new behaviour is consistent with `os.path.exists()`, `os.path.isdir()`, etc. Use `stat()` to retrieve the file status without suppressing exceptions.

Path.stat(*,follow_symlinks=True)

Return an `os.stat_result` object containing information about this path, like `os.stat()`. The result is looked up at each call to this method. This method normally follows symlinks; to stat a symlink add the argument `follow_symlinks=False`, or use `lstat()`.
`>>>p=Path('setup.py')>>>p.stat().st_size956>>>p.stat().st_mtime1327883547.852554` Changed in version 3.10: The `follow_symlinks` parameter was added.

Path.lstat()

Like `Path.stat()` but, if the path points to a symbolic link, return the symbolic link's information rather than its target's.

Path.exists(*,follow_symlinks=True)

Return `True` if the path points to an existing file or directory. `False` will be returned if the path is invalid, inaccessible or missing. Use `Path.stat()` to distinguish between these cases. This method normally follows symlinks; to check if a symlink exists, add the argument `follow_symlinks=False`.
`>>>Path('.').exists()True>>>Path('setup.py').exists()True>>>Path('/etc').exists()True>>>Path('nonexistentfile').exists()False` Changed in version 3.12: The `follow_symlinks` parameter was added.

Path.is_file(*,follow_symlinks=True)

Return `True` if the path points to a regular file. `False` will be returned if the path is invalid, inaccessible or missing, or if it points to something other than a regular file. Use `Path.stat()` to distinguish between these cases. This method normally follows symlinks; to exclude symlinks, add the argument `follow_symlinks=False`. Changed in version 3.13: The `follow_symlinks` parameter was added.

Path.is_dir(*,follow_symlinks=True)

Return `True` if the path points to a directory. `False` will be returned if the path is invalid, inaccessible or missing, or if it points to something other than a directory. Use `Path.stat()` to distinguish between these cases. This method normally follows symlinks; to exclude symlinks to directories, add the argument `follow_symlinks=False`. Changed

in version 3.13: The `follow_symlinks` parameter was added.

Path.is_symlink()

Return `True` if the path points to a symbolic link, even if that symlink is broken. `False` will be returned if the path is invalid, inaccessible or missing, or if it points to something other than a symbolic link. Use `Path.stat()` to distinguish between these cases.

Path.is_junction()

Return `True` if the path points to a junction, and `False` for any other type of file. Currently only Windows supports junctions. Added in version 3.12.

Path.is_mount()

Return `True` if the path is a mount point: a point in a file system where a different file system has been mounted. On POSIX, the function checks whether `path's` parent, `path/..`, is on a different device than `path`, or whether `path/.` and `path` point to the same i-node on the same device — this should detect mount points for all Unix and POSIX variants. On Windows, a mount point is considered to be a drive letter root (e.g. `c:\`), a UNC share (e.g. `\\server\share`), or a mounted filesystem directory. Added in version 3.12.

Path.is_socket()

Return `True` if the path points to a Unix socket. `False` will be returned if the path is invalid, inaccessible or missing, or if it points to something other than a Unix socket. Use `Path.stat()` to distinguish between these cases.

Path.is_fifo()

Return `True` if the path points to a FIFO. `False` will be returned if the path is invalid, inaccessible or missing, or if it points to something other than a FIFO. Use `Path.stat()` to distinguish between these cases.

Path.is_block_device()

Return `True` if the path points to a block device. `False` will be returned if the path is invalid, inaccessible or missing, or if it points to something other than a block device. Use `Path.stat()` to distinguish between these cases.

Path.is_char_device()

Return `True` if the path points to a character device. `False` will be returned if the path is invalid, inaccessible or missing, or if it points to something other than a character device. Use `Path.stat()` to distinguish between these cases.

Path.samefile(other_path)

Return whether this path points to the same file as `other_path`, which can be either a `Path` object, or a string. The semantics are similar to `os.path.samefile()` and `os.path.samestat()`. An `OSError` can be raised if either file cannot be accessed for some reason. >>> `p=Path('spam')` >>> `q=Path('eggs')` >>> `p.samefile(q)` `False` >>> `p.samefile('spam')` `True` Added in version 3.5.

Path.info

A `PathInfo` object that supports querying file type information. The object exposes methods that cache their results, which can help reduce the number of system calls needed when switching on file type. For example: >>> `p=Path('src')` >>> `if p.info.is_symlink():...print('symlink')...elif p.info.is_dir():...print('directory')...elif p.info.exists():...print('something else')...else:...print('not found')...directory` If the path was generated from `Path.iterdir()` then this attribute is initialized with some info

Reading and writing files

Path.open(mode='r', buffering=-1, encoding=None, errors=None, newline=None)

Open the file pointed to by the path, like the built-in `open()` function
does:>>>p=Path('setup.py')>>>with p.open() as f: ... f.readline() ... '#!/usr/bin/env python3\n'

Path.read_text(encoding=None, errors=None, newline=None)

Return the decoded contents of the pointed-to file as a string:>>>p=Path('my_text_file')>>>p.write_text('Text file contents')18>>>p.read_text() 'Text file contents' The file is opened and then closed. The optional parameters have the same meaning as `open()`. Added in version 3.5. Changed in version 3.13: The `newline` parameter was added.

Path.read_bytes()

Return the binary contents of the pointed-to file as a bytes object:>>>p=Path('my_binary_file')>>>p.write_bytes(b'Binary file contents')20>>>p.read_bytes() b'Binary file contents' Added in version 3.5.

Path.write_text(data, encoding=None, errors=None, newline=None)

Open the file pointed to in text mode, write data to it, and close the file:>>>p=Path('my_text_file')>>>p.write_text('Text file contents')18>>>p.read_text() 'Text file contents' An existing file of the same name is overwritten. The optional parameters have the same meaning as `open()`. Added in version 3.5. Changed in version 3.10: The `newline` parameter was added.

Path.write_bytes(data)

Open the file pointed to in bytes mode, write data to it, and close the file:>>>p=Path('my_binary_file')>>>p.write_bytes(b'Binary file contents')20>>>p.read_bytes() b'Binary file contents' An existing file of the same name is overwritten. Added in version 3.5.

Reading directories

Path.iterdir()

When the path points to a directory, yield path objects of the directory contents:>>>p=Path('docs')>>>for child in p.iterdir(): child ... PosixPath('docs/conf.py') PosixPath('docs/_templates') PosixPath('docs/make.bat') PosixPath('docs/index.rst') PosixPath('docs/_build') PosixPath('docs/_static') PosixPath('docs/Makefile') The children are yielded in arbitrary order, and the special entries '.' and '..' are not included. If a file is removed from or added to the directory after creating the iterator, it is uns

Path.glob(pattern, *, case_sensitive=None, recurse_symlinks=False)

Glob the given relative pattern in the directory represented by this path, yielding all matching files (of any kind):>>>sorted(Path('.').glob('*.*py')) [PosixPath('pathlib.py'), PosixPath('setup.py'), PosixPath('test_pathlib.py')]>>>sorted(Path('.').glob('*/*.*py')) [PosixPath('docs/conf.py')]>>>sorted(Path('.').glob('*/*/*.*py')) [PosixPath('build/lib/pathlib.py'), PosixPath('docs/conf.py'), PosixPath('pathlib.py'), PosixPath('setup.py'), PosixPath('test_pathlib.py')] Note: The paths are returned in no particular order.

Path.rglob(pattern, *, case_sensitive=None, recurse_symlinks=False)

Glob the given relative pattern recursively. This is like calling `Path.glob()` with `"/**/"` added in front of the pattern. Note: The paths are returned in no particular order. If you need a specific order, sort the results. See also `Pattern` language and `Path.glob()` documentation. Raises an `auditlog event` `pathlib.Path.rglob` with arguments `self, pattern`. Changed in version 3.12: The `case_sensitive` parameter was added. Changed in version 3.13: The `recurse_symlinks` parameter was added. Changed in version 3.13: The `pattern` parameter

Path.walk(top_down=True, on_error=None, follow_symlinks=False)

Generate the file names in a directory tree by walking the tree either top-down or bottom-up. For each directory in the directory tree rooted at `self` (including `self` but excluding '.' and '..'), the method yields a 3-tuple of (`dirpath`, `dirname`, `filenames`). `dirpath` is a `Path` to the directory currently being walked, `dirname` is a list of strings

for the names of subdirectories indirpath(excluding '.' and '..'), and filenames is a list of strings for the names of the non-directory files indirpath. To get a full path

Creating files and directories

Path.touch(mode=0o666,exist_ok=True)

Create a file at this given path. If mode is given, it is combined with the process's umask value to determine the file mode and access flags. If the file already exists, the function succeeds when exist_ok is true (and its modification time is updated to the current time), otherwise FileExistsError is raised. See also The open(), write_text() and write_bytes() methods are often used to create files.

Path.mkdir(mode=0o777,parents=False,exist_ok=False)

Create a new directory at this given path. If mode is given, it is combined with the process's umask value to determine the file mode and access flags. If the path already exists, FileExistsError is raised. If parents is true, any missing parents of this path are created as needed; they are created with the default permissions without taking mode into account (mimicking the POSIX mkdir -p command). If parents is false (the default), a missing parent raises FileNotFoundError. If exist_ok is false (the default), File

Path.symlink_to(target,target_is_directory=False)

Make this path a symbolic link pointing to target. On Windows, a symlink represents either a file or a directory, and does not morph to the target dynamically. If the target is present, the type of the symlink will be created to match. Otherwise, the symlink will be created as a directory if target_is_directory is true or a file symlink (the default) otherwise. On non-Windows platforms, target_is_directory is ignored. >>> p = Path('mylink') >>> p.symlink_to('setup.py') >>> p.resolve() PosixPath('/home/antoine

Path.hardlink_to(target)

Make this path a hard link to the same file as target. Note The order of arguments (link, target) is the reverse of os.link()'s. Added in version 3.10. Changed in version 3.13: Raises UnsupportedOperation if os.link() is not available. In previous versions, NotImplementedError was raised.

Copying, moving and deleting

Path.copy(target,*,follow_symlinks=True,preserve_metadata=False)

Copy this file or directory tree to the given target, and return a new Path instance pointing to target. If the source is a file, the target will be replaced if it is an existing file. If the source is a symlink and follow_symlinks is true (the default), the symlink's target is copied. Otherwise, the symlink is recreated at the destination. If preserve_metadata is false (the default), only directory structures and file data are guaranteed to be copied. Set preserve_metadata to true to ensure that file and d

Path.copy_into(target_dir,*,follow_symlinks=True,preserve_metadata=False)

Copy this file or directory tree into the given target_dir, which should be an existing directory. Other arguments are handled identically to Path.copy(). Returns a new Path instance pointing to the copy. Added in version 3.14.

Path.rename(target)

Rename this file or directory to the given target, and return a new Path instance pointing to target. On Unix, if target exists and is a file, it will be replaced silently if the user has permission. On Windows, if target exists, FileExistsError will be raised. target can be either a string or another path object: >>> p = Path('foo') >>> p.open('w').write('some text') >>> target = Path('bar') >>> p.rename(target) PosixPath('bar') >>> target.open().read() 'some text' The target path may be absolute or relative. Relative pa

Path.replace(target)

Rename this file or directory to the given `target`, and return a new `Path` instance pointing to `target`. If `target` points to an existing file or empty directory, it will be unconditionally replaced. The target path may be absolute or relative. Relative paths are interpreted relative to the current working directory, not the directory of the `Path` object. Changed in version 3.8: Added return value, return the new `Path` instance.

`Path.move(target)`

Move this file or directory tree to the given `target`, and return a new `Path` instance pointing to `target`. If the `target` doesn't exist it will be created. If both this path and the `target` are existing files, then the target is overwritten. If both paths point to the same file or directory, or the `target` is a non-empty directory, then `OSError` is raised. If both paths are on the same filesystem, the move is performed with `os.replace()`. Otherwise, this path is copied (preserving metadata and symlinks) and then deleted.

`Path.move_into(target_dir)`

Move this file or directory tree into the given `target_dir`, which should be an existing directory. Returns a new `Path` instance pointing to the moved path. Added in version 3.14.

`Path.unlink(missing_ok=False)`

Remove this file or symbolic link. If the path points to a directory, use `Path.rmdir()` instead. If `missing_ok` is false (the default), `FileNotFoundError` is raised if the path does not exist. If `missing_ok` is true, `FileNotFoundError` exceptions will be ignored (same behavior as the `POSIX rm -f` command). Changed in version 3.8: The `missing_ok` parameter was added.

`Path.rmdir()`

Remove this directory. The directory must be empty.

Permissions and ownership

`Path.owner(*, follow_symlinks=True)`

Return the name of the user owning the file. `KeyError` is raised if the file's user identifier (UID) isn't found in the system database. This method normally follows symlinks; to get the owner of the symlink, add the argument `follow_symlinks=False`. Changed in version 3.13: Raises `UnsupportedOperation` if the `pwd` module is not available. In earlier versions, `NotImplementedError` was raised. Changed in version 3.13: The `follow_symlinks` parameter was added.

`Path.group(*, follow_symlinks=True)`

Return the name of the group owning the file. `KeyError` is raised if the file's group identifier (GID) isn't found in the system database. This method normally follows symlinks; to get the group of the symlink, add the argument `follow_symlinks=False`. Changed in version 3.13: Raises `UnsupportedOperation` if the `grp` module is not available. In earlier versions, `NotImplementedError` was raised. Changed in version 3.13: The `follow_symlinks` parameter was added.

`Path.chmod(mode, *, follow_symlinks=True)`

Change the file mode and permissions, like `os.chmod()`. This method normally follows symlinks. Some Unix flavours support changing permissions on the symlink itself; on these platforms you may add the argument `follow_symlinks=False`, or use `lchmod()`.
>>> p = Path('setup.py')
>>> p.stat().st_mode
33277
>>> p.chmod(0o444)
>>> p.stat().st_mode
33060
Changed in version 3.10: The `follow_symlinks` parameter was added.

`Path.lchmod(mode)`

Like `Path.chmod()` but, if the path points to a symbolic link, the symbolic link's mode is changed rather than its target's.

Pattern language

The following wildcards are supported in patterns for `full_match()`, `glob()` and `rglob()`:

******(entire segment)

*****(entire segment)

*****(part of a segment)

?

[seq]

[!seq]

Matches any number of file or directory segments, including zero.

Matches one file or directory segment.

Matches any number of non-separator characters, including zero.

Matches one non-separator character.

Matches one character in `seq`, where `seq` is a sequence of characters. Range expressions are supported; for example, `[a-z]` matches any lowercase ASCII letter. Multiple ranges can be combined: `[a-zA-Z0-9_]` matches any ASCII letter, digit, or underscore.

Matches one character not in `seq`, where `seq` follows the same rules as above.

For a literal match, wrap the meta-characters in brackets. For example, `"[?]"` matches the character `"?"`.

The `"**"` wildcard enables recursive globbing. A few examples:

Pattern	Meaning
<code>"**/*"</code>	Any path with at least one segment.
<code>"**/*.py"</code>	Any path with a final segment ending <code>".py"</code> .
<code>"assets/**"</code>	Any path starting with <code>"assets/"</code> .
<code>"assets/**/*"</code>	Any path starting with <code>"assets/"</code> , excluding <code>"assets/"</code> itself.

■ Nota: NoteGlobbing with the `"**"` wildcard visits every directory in the tree. Large directory trees may take a long time to search.

Changed in version 3.13: Globbing with a pattern that ends with `"**"` returns both files and directories. In previous versions, only directories were returned.

In `Path.glob()` and `rglob()`, a trailing slash may be added to the pattern to match only directories.

Changed in version 3.11: Globbing with a pattern that ends with a pathname components separator (separated by `sep`) returns only directories.

Comparison to the `glob` module

The patterns accepted and results generated by `Path.glob()` and `Path.rglob()` differ slightly from those by the `glob` module:

- Files beginning with a dot are not special in `pathlib`. This is like passing `include_hidden=True` to `glob.glob()`.
- `"**"` pattern components are always recursive in `pathlib`. This is like passing `recursive=True` to `glob.glob()`.
- `"**"` pattern components do not follow symlinks by default in `pathlib`. This behaviour has no equivalent in `glob.glob()`, but you can pass `recurse_symlinks=True` to `Path.glob()` for compatible behaviour.
- Like all `PurePath` and `Path` objects, the values returned from `Path.glob()` and `Path.rglob()` don't include trailing slashes.

- The values returned from `pathlib'spath.glob()` and `path.rglob()` include the path as a prefix, unlike the results of `glob.glob(root_dir=path)`.
- The values returned from `pathlib'spath.glob()` and `path.rglob()` may include the path itself, for example when globbing `"**"`, whereas the results of `glob.glob(root_dir=path)` never include an empty string that would correspond to `path`.

Comparison to the `os.path` modules

`pathlib` implements path operations using `PurePath` and `Path` objects, and so it's said to be object-oriented. On the other hand, the `os.path` modules supply functions that work with low-level `str` and `bytes` objects, which is a more procedural approach. Some users consider the object-oriented style to be more readable.

Many functions in `os.path` support `bytes` paths and paths relative to directory descriptors. These features aren't available in `pathlib`.

Python's `str` and `bytes` types, and portions of the `os.path` modules, are written in C and are very speedy. `pathlib` is written in pure Python and is often slower, but rarely slow enough to matter.

`pathlib`'s path normalization is slightly more opinionated and consistent than `os.path`. For example, whereas `os.path.abspath()` eliminates `".."` segments from a path, which may change its meaning if symlinks are involved, `Path.absolute()` preserves these segments for greater safety.

`pathlib`'s path normalization may render it unsuitable for some applications:

- `pathlib` normalizes `Path("my_folder/")` to `Path("my_folder")`, which changes a path's meaning when supplied to various operating system APIs and command-line utilities. Specifically, the absence of a trailing separator may allow the path to be resolved as either a file or directory, rather than a directory only.
- `pathlib` normalizes `Path("./my_program")` to `Path("my_program")`, which changes a path's meaning when used as an executable search path, such as in a shell or when spawning a child process. Specifically, the absence of a separator in the path may force it to be looked up in `PATH` rather than the current directory.

As a consequence of these differences, `pathlib` is not a drop-in replacement for `os.path`.

Corresponding tools

Below is a table mapping various `os` functions to their corresponding `PurePath`/`Path` equivalent.

<code>os</code>	<code>and</code>	<code>os.path</code>	<code>pathlib</code>	<code>os.path.dirname()</code>	<code>PurePath.parent</code>	<code>os.path.basename()</code>	<code>PurePath.name</code>
		<code>os.path.splitext()</code>		<code>PurePath.stem</code>	<code>,</code>	<code>PurePath.suffix</code>	<code>os.path.join()</code>
		<code>PurePath.joinpath()</code>		<code>os.path.isabs()</code>		<code>PurePath.is_absolute()</code>	<code>os.path.relpath()</code>
		<code>PurePath.relative_to()</code>		<code>[1]</code>		<code>os.path.expanduser()</code>	<code>Path.expanduser()</code>
		<code>[2]</code>		<code>os.path.realpath()</code>		<code>Path.resolve()</code>	<code>os.path.abspath()</code>
		<code>Path.absolute()</code>		<code>[3]</code>		<code>os.path.exists()</code>	<code>Path.exists()</code>
		<code>os.path.isfile()</code>		<code>Path.is_file()</code>		<code>os.path.isdir()</code>	<code>Path.is_dir()</code>
		<code>os.path.islink()</code>		<code>Path.is_symlink()</code>		<code>os.path.isjunction()</code>	<code>Path.is_junction()</code>
		<code>os.path.ismount()</code>		<code>Path.is_mount()</code>		<code>os.path.samefile()</code>	<code>Path.samefile()</code>
		<code>os.getcwd()</code>		<code>Path.cwd()</code>		<code>os.stat()</code>	<code>Path.stat()</code>
		<code>os.lstat()</code>		<code>Path.lstat()</code>		<code>os.listdir()</code>	<code>Path.listdir()</code>

Footnotes

`os.path.relpath()` calls `os.path.abspath()` to make paths absolute and remove `".."` parts, whereas `PurePath.relative_to()` is a lexical operation that raises `ValueError` when its inputs' anchors differ (e.g. if one path is absolute and the other relative.)

`os.path.expanduser()` returns the path unchanged if the home directory can't be resolved, whereas `Path.expanduser()` raises `RuntimeError`.

`os.path.abspath()` removes `".."` components without resolving symlinks, which may change the meaning of the path, whereas `Path.absolute()` leaves any `".."` components in the path.

`os.walk()` always follows symlinks when categorizing paths into `dirname`s and `filename`s, whereas `Path.walk()` categorizes all symlinks into `filename`s when `follow_symlinks` is `false` (the default.)

Protocols

The `pathlib.types` module provides types for static type checking.

Added in version 3.14.

`class pathlib.types.PathInfo`

`exists(*, follow_symlinks=True)`

`is_dir(*, follow_symlinks=True)`

`is_file(*, follow_symlinks=True)`

`is_symlink()`

A typing.Protocol describing the `Path.info` attribute. Implementations may return cached results from their methods. `exists(*, follow_symlinks=True)` Return `True` if the path is an existing file or directory, or any other kind of file; return `False` if the path doesn't exist. If `follow_symlinks` is `False`, return `True` for symlinks without checking if their targets exist. `is_dir(*, follow_symlinks=True)` Return `True` if the path is a directory, or a symbolic link pointing to a directory; return `False` if the path is (or points to) any other kind of file, or if it doesn't exist. If `follow_symlinks` is `False`, return `True` only if the path is a directory (without following symlinks); return `False` if the path is any other kind of file, or if it doesn't exist.

Return `True` if the path is an existing file or directory, or any other kind of file; return `False` if the path doesn't exist. If `follow_symlinks` is `False`, return `True` for symlinks without checking if their targets exist.

Return `True` if the path is a directory, or a symbolic link pointing to a directory; return `False` if the path is (or points to) any other kind of file, or if it doesn't exist. If `follow_symlinks` is `False`, return `True` only if the path is a directory (without following symlinks); return `False` if the path is any other kind of file, or if it doesn't exist.

Return `True` if the path is a file, or a symbolic link pointing to a file; return `False` if the path is (or points to) a directory or other non-file, or if it doesn't exist. If `follow_symlinks` is `False`, return `True` only if the path is a file (without following symlinks); return `False` if the path is a directory or other non-file, or if it doesn't exist.

Return `True` if the path is a symbolic link (even if broken); return `False` if the path is a directory or any kind of file, or if it doesn't exist.

os.path— Common pathname manipulations

Source code: `Lib/genericpath.py`, `Lib/posixpath.py` (for POSIX) and `Lib/ntpath.py` (for Windows).

This module implements some useful functions on pathnames. To read or write files see `open()`, and for accessing the filesystem see `os` module. The path parameters can be passed as strings, or bytes, or any object implementing the `os.PathLike` protocol.

Unlike a Unix shell, Python does not do any automatic path expansions. Functions such as `os.expanduser()` and `os.expandvars()` can be invoked explicitly when an application desires shell-like path expansion. (See also `glob` module.)

See also

The `pathlib` module offers high-level path objects.

■ Nota: Note All of these functions accept either only bytes or only string objects as their parameters. The result is an object of the same type, if a path or file name is returned.

■ Nota: Note Since different operating systems have different path name conventions, there are several versions of this module in the standard library. `os.path` module is always the path module suitable for the operating system Python is running on, and therefore usable for local paths. However, you can also import and use the individual modules if you want to manipulate a path that is always in one of the different formats. They all have the same interface: `posixpath` for UNIX-style paths, `ntpath` for Windows paths

Changed in version 3.8: `exists()`, `lexists()`, `isdir()`, `isfile()`, `islink()`, and `ismount()` now return `False` instead of raising an exception for paths that contain characters or bytes unrepresentable at the OS level.

os.path.abspath(path)

Return a normalized absolutized version of the pathname `path`. On most platforms, this is equivalent to calling `normpath(join(os.getcwd(), path))`. See also `os.path.join()` and `os.path.normpath()`. Changed in version 3.6: Accepts a path-like object.

os.path.basename(path, /)

Return the base name of pathname `path`. This is the second element of the pair returned by passing `path` to the function `split()`. Note that the result of this function is different from the Unix `basename` program; where `basename` for `"/foo/bar/"` returns `"bar"`, the `basename()` function returns an empty string `""`. Changed in version 3.6: Accepts a path-like object.

os.path.commonpath(paths)

Return the longest common sub-path of each pathname in the iterable `paths`. Raise `ValueError` if `paths` contain both absolute and relative pathnames, if `paths` are on different drives, or if `paths` is empty. Unlike `commonprefix()`, this returns a valid path. Added in version 3.5. Changed in version 3.6: Accepts a sequence of path-like objects. Changed in version 3.13: Any iterable can now be passed, rather than just sequences.

os.path.commonprefix(list, /)

Return the longest string prefix (taken character-by-character) that is a prefix of all strings in `list`. If `list` is empty, return the empty string `""`. Warning This function may return invalid paths because it works a character at a time. If you need a common path prefix, then the algorithm implemented in this function is not secure. Use `commonpath()` for finding a common path prefix.

```
>>> os.path.commonprefix(['/usr/lib', '/usr/local/lib'])
'/usr/'
>>> os.path.commonpath(['/usr/lib', '/usr/local/lib'])
'/usr/'
```

os.path.dirname(path, /)

Return the directory name of pathname `path`. This is the first element of the pair returned by passing `path` to the function `split()`. Changed in version 3.6: Accepts a path-like object.

os.path.exists(path)

Return `True` if `path` refers to an existing path or an open file descriptor. Returns `False` for broken symbolic links. On some platforms, this function may return `False` if permission is not granted to execute `os.stat()` on the requested file, even if the `path` physically exists. Changed in version 3.3: `path` can now be an integer: `True` is returned if it is an open file descriptor, `False` otherwise. Changed in version 3.6: Accepts `apath`-like object.

os.path.lexists(path)

Return `True` if `path` refers to an existing path, including broken symbolic links. Equivalent to `exists()` on platforms lacking `os.lstat()`. Changed in version 3.6: Accepts `apath`-like object.

os.path.expanduser(path)

On Unix and Windows, return the argument with an initial component of `~` or `~user` replaced by that user's home directory. On Unix, an initial `~` is replaced by the environment variable `HOME` if it is set; otherwise the current user's home directory is looked up in the password directory through the built-in module `pwd`. An initial `~user` is looked up directly in the password directory. On Windows, `USERPROFILE` will be used if set, otherwise a combination of `HOMEPATH` and `HOMEDRIVE` will be used. An initial `~user` is handled

os.path.expandvars(path)

Return the argument with environment variables expanded. Substrings of the form `$name` or `${name}` are replaced by the value of environment variable `name`. Malformed variable names and references to non-existing variables are left unchanged. On Windows, `%name%` expansions are supported in addition to `$name` and `${name}`. Changed in version 3.6: Accepts `apath`-like object.

os.path.getatime(path, /)

Return the time of last access of `path`. The return value is a floating-point number giving the number of seconds since the epoch (see the `time` module). Raise `OSError` if the file does not exist or is inaccessible.

os.path.getmtime(path, /)

Return the time of last modification of `path`. The return value is a floating-point number giving the number of seconds since the epoch (see the `time` module). Raise `OSError` if the file does not exist or is inaccessible. Changed in version 3.6: Accepts `apath`-like object.

os.path.getctime(path, /)

Return the system's `ctime` which, on some systems (like Unix) is the time of the last metadata change, and, on others (like Windows), is the creation time for `path`. The return value is a number giving the number of seconds since the epoch (see the `time` module). Raise `OSError` if the file does not exist or is inaccessible. Changed in version 3.6: Accepts `apath`-like object.

os.path.getsize(path, /)

Return the size, in bytes, of `path`. Raise `OSError` if the file does not exist or is inaccessible. Changed in version 3.6: Accepts `apath`-like object.

os.path.isabs(path, /)

Return `True` if `path` is an absolute pathname. On Unix, that means it begins with a slash, on Windows that it begins with two (back)slashes, or a drive letter, colon, and (back)slash together. See also `abspath()`. Changed in version 3.6: Accepts `apath`-like object. Changed in version 3.13: On Windows, returns `False` if the given path starts with exactly one (back)slash.

os.path.isfile(path)

Return `True` if `path` is an existing regular file. This follows symbolic links, so both `islink()` and `isfile()` can be true for the same path. Changed in version 3.6: Accepts `apath`-like object.

os.path.isdir(path, /)

Return True if path is an existing directory. This follows symbolic links, so both `islink()` and `isdir()` can be true for the same path. Changed in version 3.6: Accepts `apath`-like object.

os.path.isjunction(path)

Return True if path refers to an existing directory entry that is a junction. Always return False if junctions are not supported on the current platform. Added in version 3.12.

os.path.islink(path)

Return True if path refers to an existing directory entry that is a symbolic link. Always return False if symbolic links are not supported by the Python runtime. Changed in version 3.6: Accepts `apath`-like object.

os.path.ismount(path)

Return True if path is a mount point: a point in a file system where a different file system has been mounted. On POSIX, the function checks whether path's parent, `path/..`, is on a different device than path, or whether `path/..` and path point to the same i-node on the same device — this should detect mount points for all Unix and POSIX variants. It is not able to reliably detect bind mounts on the same filesystem. On Linux systems, it will always return True for btrfs subvolumes, even if they aren't

os.path.isdevdrive(path)

Return True if path is located on a Windows Dev Drive. A Dev Drive is optimized for developer scenarios, and offers faster performance for reading and writing files. It is recommended for use for source code, temporary build directories, package caches, and other IO-intensive operations. May raise an error for an invalid path, for example, one without a recognizable drive, but returns False on platforms that do not support Dev Drives. See the Windows documentation for information on enabling and

os.path.isreserved(path)

Return True if path is a reserved pathname on the current system. On Windows, reserved filenames include those that end with a space or dot; those that contain colons (i.e. file streams such as "name:stream"), wildcard characters (i.e. `*?*<>`), pipe, or ASCII control characters; as well as DOS device names such as "NUL", "CON", "CONIN\$", "CONOUT\$", "AUX", "PRN", "COM1", and "LPT1". Note This function approximates rules for reserved paths on most Windows systems. These rules change over time in various

os.path.join(path, /, *paths)

Join one or more path segments intelligently. The return value is the concatenation of path and all members of *paths, with exactly one directory separator following each non-empty part, except the last. That is, the result will only end in a separator if the last part is either empty or ends in a separator. If a segment is an absolute path (which on Windows requires both a drive and a root), then all previous segments are ignored and joining continues from the absolute path segment. On Linux, for

os.path.normcase(path, /)

Normalize the case of a pathname. On Windows, convert all characters in the pathname to lowercase, and also convert forward slashes to backward slashes. On other operating systems, return the path unchanged. Changed in version 3.6: Accepts `apath`-like object.

os.path.normpath(path)

Normalize a pathname by collapsing redundant separators and up-level references so that `A/B, A/B/, A/./B` and `A/foo/./B` become `A/B`. This string manipulation may change the meaning of a path that contains symbolic links. On Windows, it converts forward slashes to backward slashes. To normalize case, use `normcase()`. Note On POSIX systems, in accordance with IEEE Std 1003.1 2013 Edition; 4.13 Pathname Resolution, if a pathname begins with exactly two slashes, the first component following the leading c

os.path.realpath(path,/,*,strict=False)

Return the canonical path of the specified filename, eliminating any symbolic links encountered in the path (if they are supported by the operating system). On Windows, this function will also resolve MS-DOS (also called 8.3) style names such as C:\PROGRAM~1 to C:\ProgramFiles. By default, the path is evaluated up to the first component that does not exist, is a symlink loop, or whose evaluation raises `OSError`. All such components are appended unchanged to the existing part of the path. Some errors t

os.path.ALLOW_MISSING

Special value used for the `strict` argument in `realpath()`. Added in version 3.14.

os.path.relpath(path,start=os.curdir)

Return a relative filepath to either from the current directory or from an optional `start` directory. This is a path computation: the filesystem is not accessed to confirm the existence or nature of `path` or `start`. On Windows, `ValueError` is raised when `path` and `start` are on different drives. `start` defaults to `os.curdir`. Changed in version 3.6: Accepts a path-like object.

os.path.samefile(path1,path2,/)

Return `True` if both `pathname` arguments refer to the same file or directory. This is determined by the device number and i-node number and raises an exception if an `os.stat()` call on either `pathname` fails. Changed in version 3.2: Added Windows support. Changed in version 3.4: Windows now uses the same implementation as all other platforms. Changed in version 3.6: Accepts a path-like object.

os.path.sameopenfile(fp1,fp2)

Return `True` if the file descriptors `fp1` and `fp2` refer to the same file. Changed in version 3.2: Added Windows support. Changed in version 3.6: Accepts a path-like object.

os.path.samestat(stat1,stat2,/)

Return `True` if the `stat` tuples `stat1` and `stat2` refer to the same file. These structures may have been returned by `os.fstat()`, `os.lstat()`, or `os.stat()`. This function implements the underlying comparison used by `samefile()` and `sameopenfile()`. Changed in version 3.4: Added Windows support.

os.path.split(path,/)

Split the `pathname` `path` into a pair, `(head,tail)` where `tail` is the last `pathname` component and `head` is everything leading up to that. The `tail` part will never contain a slash; if `path` ends in a slash, `tail` will be empty. If there is no slash in `path`, `head` will be empty. If `path` is empty, both `head` and `tail` are empty. Trailing slashes are stripped from `head` unless it is the root (one or more slashes only). In all cases, `join(head,tail)` returns a path to the same location as `path` (but the strings may differ). Also see t

os.path.splitdrive(path,/)

Split the `pathname` `path` into a pair, `(drive,tail)` where `drive` is either a mount point or the empty string. On systems which do not use drive specifications, `drive` will always be the empty string. In all cases, `drive+tail` will be the same as `path`. On Windows, splits a `pathname` into `drive/UNC sharepoint` and relative path. If the path contains a drive letter, `drive` will contain everything up to and including the colon: `>>>splitdrive("c:/dir")("c:", "/dir")` If the path contains a UNC path, `drive` will contain the

os.path.splitroot(path,/)

Split the `pathname` `path` into a 3-item tuple, `(drive,root,tail)` where `drive` is a device name or mount point, `root` is a string of separators after the drive, and `tail` is everything after the root. Any of these items may be the empty string. In all cases, `drive+root+tail` will be the same as `path`. On POSIX systems, `drive` is always empty. The `root` may be empty (if `path` is relative), a single forward slash (if `path` is absolute), or two forward slashes (implementation-defined per IEEE Std 1003.1-2017; 4.13 Pathname Resolution

os.path.splitext(path, /)

Split the pathname `path` into a pair `(root, ext)` such that `root+ext==path`, and the extension, `ext`, is empty or begins with a period and contains at most one period. If the path contains no extension, `ext` will be `""`:>>> `splitext("bar")` `("bar", "")` If the path contains an extension, then `ext` will be set to this extension, including the leading period. Note that previous periods will be ignored:>>> `splitext('foo.bar.exe')` `('foo.bar', '.exe')`>>> `splitext('/foo/bar.exe')` `('/foo/bar', '.exe')` Leading periods of the last co

os.path.supports_unicode_filenames

True if arbitrary Unicode strings can be used as file names (within limitations imposed by the file system).

stat— Interpretingstat()results

Source code:Lib/stat.py

Thestatmodule defines constants and functions for interpreting the results ofos.stat(),os.fstat()andos.lstat()(if they exist). For complete details about thestat(),fstat()andlstat()calls, consult the documentation for your system.

Changed in version 3.4:The stat module is backed by a C implementation.

Thestatmodule defines the following functions to test for specific file types:

stat.S_ISDIR(mode)

Return non-zero if the mode is from a directory.

stat.S_ISCHR(mode)

Return non-zero if the mode is from a character special device file.

stat.S_ISBLK(mode)

Return non-zero if the mode is from a block special device file.

stat.S_ISREG(mode)

Return non-zero if the mode is from a regular file.

stat.S_ISFIFO(mode)

Return non-zero if the mode is from a FIFO (named pipe).

stat.S_ISLNK(mode)

Return non-zero if the mode is from a symbolic link.

stat.S_ISSOCK(mode)

Return non-zero if the mode is from a socket.

stat.S_ISDOOR(mode)

Return non-zero if the mode is from a door.Added in version 3.4.

stat.S_ISPORT(mode)

Return non-zero if the mode is from an event port.Added in version 3.4.

stat.S_ISWHT(mode)

Return non-zero if the mode is from a whiteout.Added in version 3.4.

Two additional functions are defined for more general manipulation of the file's mode:

stat.S_IMODE(mode)

Return the portion of the file's mode that can be set byos.chmod()—that is, the file's permission bits, plus the sticky bit, set-group-id, and set-user-id bits (on systems that support them).

stat.S_IFMT(mode)

Return the portion of the file's mode that describes the file type (used by theS_IS*()functions above).

Normally, you would use theos.path.is*()functions for testing the type of a file; the functions here are useful when you are doing multiple tests of the same file and wish to avoid the overhead of thestat()system call for each test. These are also useful when checking for information about a file that isn't handled byos.path, like the tests for block and character devices.

Example:

```
import os, sys
from stat import *

def walktree(top, callback):
    '''recursively descend the directory tree rooted at top,
    calling the callback function for each regular file'''

    for f in os.listdir(top):
        pathname = os.path.join(top, f)
        mode = os.lstat(pathname).st_mode
        if S_ISDIR(mode):
            # It's a directory, recurse into it
            walktree(pathname, callback)
        elif S_ISREG(mode):
            # It's a file, call the callback function
            callback(pathname)
        else:
            # Unknown file type, print a message
            print('Skipping %s' % pathname)

def visitfile(file):
    print('visiting', file)

if __name__ == '__main__':
    walktree(sys.argv[1], visitfile)
```

An additional utility function is provided to convert a file's mode in a human readable string:

stat.filemode(mode)

Convert a file's mode to a string of the form '-rwxrwxrwx'. Added in version 3.3. Changed in version 3.4: The function supports `S_IFDOOR`, `S_IFPORT` and `S_IFWHT`.

All the variables below are simply symbolic indexes into the 10-tuple returned by `os.stat()`, `os.fstat()` or `os.lstat()`.

stat.ST_MODE

Inode protection mode.

stat.ST_INO

Inode number.

stat.ST_DEV

Device inode resides on.

stat.ST_NLINK

Number of links to the inode.

stat.ST_UID

User id of the owner.

stat.ST_GID

Group id of the owner.

stat.ST_SIZE

Size in bytes of a plain file; amount of data waiting on some special files.

stat.ST_ATIME

Time of last access.

stat.ST_MTIME

Time of last modification.

stat.ST_CTIME

The “ctime” as reported by the operating system. On some systems (like Unix) is the time of the last metadata change, and, on others (like Windows), is the creation time (see platform documentation for details).

The interpretation of “file size” changes according to the file type. For plain files this is the size of the file in bytes. For FIFOs and sockets under most flavors of Unix (including Linux in particular), the “size” is the number of bytes waiting to be read at the time of the call to `os.stat()`, `os.fstat()`, or `os.lstat()`; this can sometimes be useful, especially for polling one of these special files after a non-blocking open. The meaning of the size field for other character and block devices varies more, depending on the implementation of the underlying system call.

The variables below define the flags used in the `ST_MODE` field.

Use of the functions above is more portable than use of the first set of flags:

stat.S_IFSOCK

Socket.

stat.S_IFLNK

Symbolic link.

stat.S_IFREG

Regular file.

stat.S_IFBLK

Block device.

stat.S_IFDIR

Directory.

stat.S_IFCHR

Character device.

stat.S_IFIFO

FIFO.

stat.S_IFDOOR

Door. Added in version 3.4.

stat.S_IFPORT

Event port. Added in version 3.4.

stat.S_IFWHT

Whiteout. Added in version 3.4.

■ Nota: `S_IFDOOR`, `S_IFPORT` or `S_IFWHT` are defined as 0 when the platform does not have support for the file types.

The following flags can also be used in the `mode` argument of `os.chmod()`:

stat.S_ISUID

Set UID bit.

stat.S_ISGID

Set-group-ID bit. This bit has several special uses. For a directory it indicates that BSD semantics is to be used for that directory: files created there inherit their group ID from the directory, not from the effective group ID of the creating process, and directories created there will also get the `S_ISGID` bit set. For a file that does not have the group execution bit (`S_IXGRP`) set, the set-group-ID bit indicates mandatory file/record locking (see also `S_ENFMT`).

stat.S_ISVTX

Sticky bit. When this bit is set on a directory it means that a file in that directory can be renamed or deleted only by the owner of the file, by the owner of the directory, or by a privileged process.

stat.S_IRWXU

Mask for file owner permissions.

stat.S_IRUSR

Owner has read permission.

stat.S_IWUSR

Owner has write permission.

stat.S_IXUSR

Owner has execute permission.

stat.S_IRWXG

Mask for group permissions.

stat.S_IRGRP

Group has read permission.

stat.S_IWGRP

Group has write permission.

stat.S_IXGRP

Group has execute permission.

stat.S_IRWXO

Mask for permissions for others (not in group).

stat.S_IROTH

Others have read permission.

stat.S_IWOTH

Others have write permission.

stat.S_IXOTH

Others have execute permission.

stat.S_ENFMT

System V file locking enforcement. This flag is shared with `S_ISGID`: file/record locking is enforced on files that do not have the group execution bit (`S_IXGRP`) set.

stat.S_IREAD

Unix V7 synonym for `S_IRUSR`.

stat.S_IWRITE

Unix V7 synonym for S_IWUSR.

stat.S_IEXEC

Unix V7 synonym for S_IXUSR.

The following flags can be used in the flags argument of `os.chflags()`:

stat.UF_SETTABLE

All user settable flags. Added in version 3.13.

stat.UF_NODUMP

Do not dump the file.

stat.UF_IMMUTABLE

The file may not be changed.

stat.UF_APPEND

The file may only be appended to.

stat.UF_OPAQUE

The directory is opaque when viewed through a union stack.

stat.UF_NOUNLINK

The file may not be renamed or deleted.

stat.UF_COMPRESSED

The file is stored compressed (macOS 10.6+).

stat.UF_TRACKED

Used for handling document IDs (macOS). Added in version 3.13.

stat.UF_DATAVAULT

The file needs an entitlement for reading or writing (macOS 10.13+). Added in version 3.13.

stat.UF_HIDDEN

The file should not be displayed in a GUI (macOS 10.5+).

stat.SF_SETTABLE

All super-user changeable flags. Added in version 3.13.

stat.SF_SUPPORTED

All super-user supported flags. Availability: macOS. Added in version 3.13.

stat.SF_SYNTHETIC

All super-user read-only synthetic flags. Availability: macOS. Added in version 3.13.

stat.SF_ARCHIVED

The file may be archived.

stat.SF_IMMUTABLE

The file may not be changed.

stat.SF_APPEND

The file may only be appended to.

stat.SF_RESTRICTED

The file needs an entitlement to write to (macOS 10.13+)Added in version 3.13.

stat.SF_NOUNLINK

The file may not be renamed or deleted.

stat.SF_SNAPSHOT

The file is a snapshot file.

stat.SF_FIRMLINK

The file is a firmlink (macOS 10.15+)Added in version 3.13.

stat.SF_DATALESS

The file is a dataless object (macOS 10.15+)Added in version 3.13.

See the *BSD or macOS systems man pagechflags(2)for more information.

On Windows, the following file attribute constants are available for use when testing bits in thest_file_attributesmember returned byos.stat(). See theWindows API documentationfor more detail on the meaning of these constants.

stat.FILE_ATTRIBUTE_ARCHIVE

stat.FILE_ATTRIBUTE_COMPRESSED

stat.FILE_ATTRIBUTE_DEVICE

stat.FILE_ATTRIBUTE_DIRECTORY

stat.FILE_ATTRIBUTE_ENCRYPTED

stat.FILE_ATTRIBUTE_HIDDEN

stat.FILE_ATTRIBUTE_INTEGRITY_STREAM

stat.FILE_ATTRIBUTE_NORMAL

stat.FILE_ATTRIBUTE_NOT_CONTENT_INDEXED

stat.FILE_ATTRIBUTE_NO_SCRUB_DATA

stat.FILE_ATTRIBUTE_OFFLINE

stat.FILE_ATTRIBUTE_READONLY

stat.FILE_ATTRIBUTE_REPARSE_POINT

stat.FILE_ATTRIBUTE_SPARSE_FILE

stat.FILE_ATTRIBUTE_SYSTEM

stat.FILE_ATTRIBUTE_TEMPORARY

stat.FILE_ATTRIBUTE_VIRTUAL

Added in version 3.5.

On Windows, the following constants are available for comparing against thest_reparse_tagmember returned byos.lstat(). These are well-known constants, but are not an exhaustive list.

stat.IO_REPARSE_TAG_SYMLINK

stat.IO_REPARSE_TAG_MOUNT_POINT

stat.IO_REPARSE_TAG_APPEXECLINK

Added in version 3.8.

filecmp— File and Directory Comparisons

Source code: `Lib/filecmp.py`

The `filecmp` module defines functions to compare files and directories, with various optional time/correctness trade-offs. For comparing files, see also the `diff` module.

The `filecmp` module defines the following functions:

`filecmp.cmp(f1,f2,shallow=True)`

Compare the files named `f1` and `f2`, returning `True` if they seem equal, `False` otherwise. If `shallow` is `True` and the `os.stat()` signatures (file type, size, and modification time) of both files are identical, the files are taken to be equal. Otherwise, the files are treated as different if their sizes or contents differ. Note that no external programs are called from this function, giving it portability and efficiency. This function uses a cache for past comparisons and the results, with cache entries invalidated

`filecmp.cmpfiles(dir1,dir2,common,shallow=True)`

Compare the files in the two directories `dir1` and `dir2` whose names are given by `common`. Returns three lists of file names: `match`, `mismatch`, `errors`. `match` contains the list of files that match, `mismatch` contains the names of those that don't, and `errors` lists the names of files which could not be compared. Files are listed in `errors` if they don't exist in one of the directories, the user lacks permission to read them or if the comparison could not be done for some other reason. The `shallow` parameter has the same me

`filecmp.clear_cache()`

Clear the `filecmp` cache. This may be useful if a file is compared so quickly after it is modified that it is within the `mtime` resolution of the underlying filesystem. Added in version 3.4.

The `dircmp` class

`class filecmp.dircmp(a,b,ignore=None,hide=None,*,shallow=True)`

`report()`

`report_partial_closure()`

`report_full_closure()`

`left`

`right`

`left_list`

`right_list`

`common`

`left_only`

`right_only`

`common_dirs`

`common_files`

`common_funny`

`same_files`

diff_files

funny_files

subdirs

Construct a new directory comparison object, to compare the directories `a` and `b`. `ignore` is a list of names to ignore, and defaults to `filecmp.DEFAULT_IGNORES`. `hide` is a list of names to hide, and defaults to `[os.curdir, os.pardir]`. The `DirCmp` class compares files by doing shallow comparisons as described for `filecmp.cmp()` by default using the `shallow` parameter. Changed in version 3.13: Added the `shallow` parameter. The `DirCmp` class provides the following methods: `report()` Print (to `sys.stdout`) a comparison between `a` and `b`. `report()`

Print (to `sys.stdout`) a comparison between `a` and `b`.

Print a comparison between `a` and `b` and common immediate subdirectories.

Print a comparison between `a` and `b` and common subdirectories (recursively).

The directory `a`.

The directory `b`.

Files and subdirectories in `a`, filtered by `hide` and `ignore`.

Files and subdirectories in `b`, filtered by `hide` and `ignore`.

Files and subdirectories in both `a` and `b`.

Files and subdirectories only in `a`.

Files and subdirectories only in `b`.

Subdirectories in both `a` and `b`.

Files in both `a` and `b`.

Names in both `a` and `b`, such that the type differs between the directories, or names for which `os.stat()` reports an error.

Files which are identical in both `a` and `b`, using the class's file comparison operator.

Files which are in both `a` and `b`, whose contents differ according to the class's file comparison operator.

Files which are in both `a` and `b`, but could not be compared.

A dictionary mapping names in `common_dir` to `DirCmp` instances (or `MyDirCmp` instances if this instance is of type `MyDirCmp`, a subclass of `DirCmp`). Changed in version 3.10: Previously entries were always `DirCmp` instances. Now entries are the same type as `self`, if `self` is a subclass of `DirCmp`.

filecmp.DEFAULT_IGNORES

Added in version 3.4. List of directories ignored by `DirCmp` by default.

Here is a simplified example of using the `subdirs` attribute to search recursively through two directories to show common different files:

```
>>> from filecmp import dircmp
>>> def print_diff_files(dcmp):
...     for name in dcmp.diff_files:
...         print("diff_file %s found in %s and %s" % (name, dcmp.left,
...             dcmp.right))
...     for sub_dcmp in dcmp.subdirs.values():
...         print_diff_files(sub_dcmp)
...
>>> dcmp = dircmp('dir1', 'dir2')
>>> print_diff_files(dcmp)
```

tempfile— Generate temporary files and directories

Source code: `Lib/tempfile.py`

This module creates temporary files and directories. It works on all supported platforms. `TemporaryFile`, `NamedTemporaryFile`, `TemporaryDirectory`, and `SpooledTemporaryFile` are high-level interfaces which provide automatic cleanup and can be used as context managers. `mkstemp()` and `mkdtemp()` are lower-level functions which require manual cleanup.

All the user-callable functions and constructors take additional arguments which allow direct control over the location and name of temporary files and directories. File names used by this module include a string of random characters which allows those files to be securely created in shared temporary directories. To maintain backward compatibility, the argument order is somewhat odd; it is recommended to use keyword arguments for clarity.

The module defines the following user-callable items:

`tempfile.TemporaryFile(mode='w+b', buffering=-1, encoding=None, newline=None, suffix=None, prefix=None, dir=None, *, errors=None)`

Return a file-like object that can be used as a temporary storage area. The file is created securely, using the same rules as `mkstemp()`. It will be destroyed as soon as it is closed (including an implicit close when the object is garbage collected). Under Unix, the directory entry for the file is either not created at all or is removed immediately after the file is created. Other platforms do not support this; your code should not rely on a temporary file created using this function having or not

`tempfile.NamedTemporaryFile(mode='w+b', buffering=-1, encoding=None, newline=None, suffix=None, prefix=None, dir=None, delete=True, *, errors=None, delete_on_close=True)`

This function operates exactly as `TemporaryFile()` does, except the following differences: This function returns a file that is guaranteed to have a visible name in the file system. To manage the named file, it extends the parameters of `TemporaryFile()` with `delete` and `delete_on_close` parameters that determine whether and how the named file should be automatically deleted. The returned object is always a file-like object whose `file` attribute is the underlying true file object. This file-like object can be used i

`tempfile.SpooledTemporaryFile(max_size=0, mode='w+b', buffering=-1, encoding=None, newline=None, suffix=None, prefix=None, dir=None, *, errors=None)`

`rollover()`

This class operates exactly as `TemporaryFile()` does, except that data is spooled in memory until the file size exceeds `max_size`, or until the file's `fileno()` method is called, at which point the contents are written to disk and operation proceeds as with `TemporaryFile()`. `rollover()` The resulting file has one additional method, `rollover()`, which causes the file to roll over to an on-disk file regardless of its size. The returned object is a file-like object whose `file` attribute is either an `io.BytesIO` or `io.TextIOWrapper`.

The resulting file has one additional method, `rollover()`, which causes the file to roll over to an on-disk file regardless of its size.

`tempfile.TemporaryDirectory(suffix=None, prefix=None, dir=None, ignore_cleanup_errors=False, *, delete=True)`

`name`

`cleanup()`

This class securely creates a temporary directory using the same rules as `mkdtemp()`. The resulting object can be used as a context manager (see Examples). On completion of the context or destruction of the temporary

directory object, the newly created temporary directory and all its contents are removed from the filesystem. The directory name can be retrieved from the `name` attribute of the returned object. When the returned object is used as a context manager, `name` will be assigned to the target

The directory name can be retrieved from the `name` attribute of the returned object. When the returned object is used as a context manager, `name` will be assigned to the target of the `as` clause in the `with` statement, if there is one.

The directory can be explicitly cleaned up by calling the `cleanup()` method. If `ignore_cleanup_errors` is `True`, any unhandled exceptions during explicit or implicit cleanup (such as a `PermissionError` removing open files on Windows) will be ignored, and the remaining removable items deleted on a “best-effort” basis. Otherwise, errors will be raised in whatever context cleanup occurs (the `cleanup()` call, exiting the context manager, when the object is garbage-collected or during interpreter shutdown).

`tempfile.mkstemp(suffix=None, prefix=None, dir=None, text=False)`

Creates a temporary file in the most secure manner possible. There are no race conditions in the file's creation, assuming that the platform properly implements the `os.O_EXCL` flag for `os.open()`. The file is readable and writable only by the creating user ID. If the platform uses permission bits to indicate whether a file is executable, the file is executable by no one. The file descriptor is not inherited by child processes. Unlike `TemporaryFile()`, the user of `mkstemp()` is responsible for deleting the

`tempfile.mkdtemp(suffix=None, prefix=None, dir=None)`

Creates a temporary directory in the most secure manner possible. There are no race conditions in the directory's creation. The directory is readable, writable, and searchable only by the creating user ID. The user of `mkdtemp()` is responsible for deleting the temporary directory and its contents when done with it. The `prefix`, `suffix`, and `dir` arguments are the same as for `mkstemp()`. `mkdtemp()` returns the absolute pathname of the new directory. Raises an `audit` event `tempfile.mkdtemp` with argument `fullpath`. Cha

`tempfile.gettempdir()`

Return the name of the directory used for temporary files. This defines the default value for the `dir` argument to all functions in this module. Python searches a standard list of directories to find one which the calling user can create files in. The list is: The directory named by the `TMPDIR` environment variable. The directory named by the `TEMP` environment variable. The directory named by the `TMP` environment variable. A platform-specific location: On Windows, the directories `C:\TEMP`, `C:\TMP`, `\TEMP`, and `\TMP`, in

`tempfile.gettempdirb()`

Same as `gettempdir()` but the return value is in bytes. Added in version 3.5.

`tempfile.gettempprefix()`

Return the filename prefix used to create temporary files. This does not contain the directory component.

`tempfile.gettempprefixb()`

Same as `gettempprefix()` but the return value is in bytes. Added in version 3.5.

The module uses a global variable to store the name of the directory used for temporary files returned by `gettempdir()`. It can be set directly to override the selection process, but this is discouraged. All functions in this module take a `dir` argument which can be used to specify the directory. This is the recommended approach that does not surprise other unsuspecting code by changing global API behavior.

`tempfile.tempdir`

When set to a value other than `None`, this variable defines the default value for the `dir` argument to the functions defined in this module, including its type, bytes or str. It cannot be a path-like object. If `tempdir` is `None` (the default) at any call to any of the above functions except `gettempprefix()` it is initialized following the algorithm described

in gettempdir(). Note Beware that if you set tempdir to a bytes value, there is a nasty side effect: The global default return type of mkstemp() and mkdtemp() changes

Examples

Here are some examples of typical usage of the tempfile module:

```
>>> import tempfile

# create a temporary file and write some data to it
>>> fp = tempfile.TemporaryFile()
>>> fp.write(b'Hello world!')
# read data from file
>>> fp.seek(0)
>>> fp.read()
b'Hello world!'
# close the file, it will be removed
>>> fp.close()

# create a temporary file using a context manager
>>> with tempfile.TemporaryFile() as fp:
...     fp.write(b'Hello world!')
...     fp.seek(0)
...     fp.read()
b'Hello world!'
>>>
# file is now closed and removed

# create a temporary file using a context manager
# close the file, use the name to open the file again
>>> with tempfile.NamedTemporaryFile(delete_on_close=False) as fp:
...     fp.write(b'Hello world!')
...     fp.close()
... # the file is closed, but not removed
... # open the file again by using its name
...     with open(fp.name, mode='rb') as f:
...         f.read()
b'Hello world!'
>>>
# file is now removed

# create a temporary directory using the context manager
>>> with tempfile.TemporaryDirectory() as tmpdirname:
...     print('created temporary directory', tmpdirname)
>>>
# directory and contents have been removed
```

Deprecated functions and variables

A historical way to create temporary files was to first generate a file name with the mktemp() function and then create a file using this name. Unfortunately this is not secure, because a different process may create a file with this name in the time between the call to mktemp() and the subsequent attempt to create the file by the first process. The solution is to combine the two steps and create the file immediately. This approach is used by mkstemp() and the other functions described above.

tempfile.mktemp(suffix="", prefix="tmp", dir=None)

Deprecated since version 2.3: Use mkstemp() instead. Return an absolute pathname of a file that did not exist at the time the call is made. The prefix, suffix, and dir arguments are similar to those of mkstemp(), except that bytes

file names, suffix=None and prefix=None are not supported. Warning Use of this function may introduce a security hole in your program. By the time you get around to doing anything with the file name it returns, someone else may have beaten you to the punch. mktemp() usage can be repl

glob— Unix style pathname pattern expansion

Source code: `Lib/glob.py`

The `glob` module finds pathnames using pattern matching rules similar to the Unix shell. No tilde expansion is done, but `*`, `?`, and character ranges expressed with `[]` will be correctly matched. This is done by using the `os.scandir()` and `fnmatch.fnmatch()` functions in concert, and not by actually invoking a subshell.

■ **Nota:** Note The pathnames are returned in no particular order. If you need a specific order, sort the results.

Files beginning with a dot (.) can only be matched by patterns that also start with a dot, unlike `fnmatch.fnmatch()` or `pathlib.Path.glob()`. For tilde and shell variable expansion, use `os.path.expanduser()` and `os.path.expandvars()`.

For a literal match, wrap the meta-characters in brackets. For example, `'[?]'` matches the character `'?'`.

The `glob` module defines the following functions:

`glob.glob(pathname, *, root_dir=None, dir_fd=None, recursive=False, include_hidden=False)`

Return a possibly empty list of path names that match `pathname`, which must be a string containing a path specification. `pathname` can be either absolute (like `/usr/src/Python-1.5/Makefile`) or relative (like `../Tools/*/*.gif`), and can contain shell-style wildcards. Broken symlinks are included in the results (as in the shell). Whether or not the results are sorted depends on the file system. If a file that satisfies conditions is removed or added during the call of this function, whether a path nam

`glob.iglob(pathname, *, root_dir=None, dir_fd=None, recursive=False, include_hidden=False)`

Return an iterator which yields the same values as `glob()` without actually storing them all simultaneously. Raises an `auditing event` `glob` with arguments `pathname, recursive`. Raises an `auditing event` `glob/2` with arguments `pathname, recursive, root_dir, dir_fd`. Note This function may return duplicate path names if `pathname` contains multiple `"**"` patterns and `recursive` is true. Changed in version 3.5: Support for recursive globs using `"**"`. Changed in version 3.10: Added the `root_dir` and `dir_fd` parameters. Changed in ver

`glob.escape(pathname)`

Escape all special characters (`'?', '*'` and `'['`). This is useful if you want to match an arbitrary literal string that may have special characters in it. Special characters in drive/UNC sharepoints are not escaped, e.g. on Windows `escape('//?/c:/Quovadis?.txt')` returns `//?/c:/Quovadis[?].txt`. Added in version 3.4.

`glob.translate(pathname, *, recursive=False, include_hidden=False, seps=None)`

Convert the given path specification to a regular expression for use with `re.match()`. The path specification can contain shell-style wildcards. For example: `>>> import glob, re >>>>> regex = glob.translate('**/*.txt', recursive=True, include_hidden=True) >>> regex '(?s:(?:(.+)/)?[^\/*\\.\txt)\z' >>> reobj = re.compile(regex) >>> reobj.match('foo/bar/baz.txt') <re.Match object; span=(0, 15), match='foo/bar/baz.txt' >` Path separators and segments are meaningful to this function, unlike `fnmatch.translate()`. By default wildc

Examples

Consider a directory containing the following files: `1.gif`, `2.txt`, `card.gif` and a subdirectory `sub` which contains only the file `3.txt`. `glob()` will produce the following results. Notice how any leading components of the path are preserved.

```
>>> import glob
>>> glob.glob('./[0-9].*')
['./1.gif', './2.txt']
>>> glob.glob('*.gif')
```

```
['1.gif', 'card.gif']
>>> glob.glob('?.gif')
['1.gif']
>>> glob.glob('**/*.txt', recursive=True)
['2.txt', 'sub/3.txt']
>>> glob.glob('./**/', recursive=True)
['./', './sub/']
```

If the directory contains files starting with `.` they won't be matched by default. For example, consider a directory containing `card.gif` and `.card.gif`:

```
>>> import glob
>>> glob.glob('*.gif')
['card.gif']
>>> glob.glob('.*')
['.card.gif']
```

See also

The `fnmatch` module offers shell-style filename (not path) expansion.

See also

The `pathlib` module offers high-level path objects.

fnmatch— Unix filename pattern matching

Source code: `Lib/fnmatch.py`

This module provides support for Unix shell-style wildcards, which are not the same as regular expressions (which are documented in `re` module). The special characters used in shell-style wildcards are:

Pattern	Meaning
<code>*</code>	matches everything
<code>?</code>	matches any single character
<code>[seq]</code>	matches any character in seq
<code>[!seq]</code>	matches any character not in seq

For a literal match, wrap the meta-characters in brackets. For example, `'[?]'` matches the character `'?'`.

Note that the filename separator (`'/'` on Unix) is not special to this module. See module `glob` for pathname expansion (`glob.glob()` or `glob.globfilter()` to match pathname segments). Similarly, filenames starting with a period are not special for this module, and are matched by the `*` and `?` patterns.

Unless stated otherwise, “filename string” and “pattern string” either refer to `str` or `ISO-8859-1` encoded `bytes` objects. Note that the functions documented below do not allow to mix `bytes` pattern with `str` filename, and vice-versa.

Finally, note that `functools.lru_cache()` with a `maxsize` of 32768 is used to cache the (typed) compiled regex patterns in the following functions: `fnmatch()`, `fnmatchcase()`, `filter()`, `filterfalse()`.

fnmatch.fnmatch(name, pat)

Test whether the filename string `name` matches the pattern string `pat`, returning `True` or `False`. Both parameters are case-normalized using `os.path.normcase()`. `fnmatchcase()` can be used to perform a case-sensitive comparison, regardless of whether that's standard for the operating system. This example will print all file names in the current directory with the extension `.txt`:

```
import fnmatch
import os
for file in os.listdir('.'):
    if fnmatch.fnmatch(file, '*.txt'):
        print(file)
```

fnmatch.fnmatchcase(name, pat)

Test whether the filename string `name` matches the pattern string `pat`, returning `True` or `False`; the comparison is case-sensitive and does not apply `os.path.normcase()`.

fnmatch.filter(names, pat)

Construct a list from those elements of the iterable of filename strings `names` that match the pattern string `pat`. It is the same as `[n for n in names if fnmatch(n, pat)]`, but implemented more efficiently.

fnmatch.filterfalse(names, pat)

Construct a list from those elements of the iterable of filename strings `names` that do not match the pattern string `pat`. It is the same as `[n for n in names if not fnmatch(n, pat)]`, but implemented more efficiently. Added in version 3.14.

fnmatch.translate(pat)

Return the shell-style pattern `pat` converted to a regular expression for using with `re.match()`. The pattern is expected to be `str`. Example:

```
>>> import fnmatch, re
>>> regex = fnmatch.translate('*.txt')
>>> reobj = re.compile(regex)
>>> reobj.match('foobar.txt')
<re.Match object; span=(0, 10), match='foobar.txt'>
```

See also

Module glob

Unix shell-style path expansion.

linecache— Random access to text lines

Source code: `Lib/linecache.py`

The `linecache` module allows one to get any line from a Python source file, while attempting to optimize internally, using a cache, the common case where many lines are read from a single file. This is used by the `traceback` module to retrieve source lines for inclusion in the formatted traceback.

The `tokenize.open()` function is used to open files. This function uses `tokenize.detect_encoding()` to get the encoding of the file; in the absence of an encoding token, the file encoding defaults to UTF-8.

The `linecache` module defines the following functions:

linecache.getline(filename, lineno, module_globals=None)

Get line `lineno` from file named `filename`. This function will never raise an exception — it will return "" on errors (the terminating newline character will be included for lines that are found). If `filename` indicates a frozen module (starting with '<frozen>'), the function will attempt to get the real file name from `module_globals['__file__']` if `module_globals` is not `None`. If a file named `filename` is not found, the function first checks for a PEP 302 `__loader__` in `module_globals`. If there is such a loader and it defi

linecache.clearcache()

Clear the cache. Use this function if you no longer need lines from files previously read using `getline()`.

linecache.checkcache(filename=None)

Check the cache for validity. Use this function if files in the cache may have changed on disk, and you require the updated version. If `filename` is omitted, it will check all the entries in the cache.

linecache.lazycache(filename, module_globals)

Capture enough detail about a non-file-based module to permit getting its lines later via `getline()` even if `module_globals` is `None` in the later call. This avoids doing I/O until a line is actually needed, without having to carry the module globals around indefinitely. Added in version 3.5.

Example:

```
>>> import linecache
>>> linecache.getline(linecache.__file__, 8)
'import sys\n'
```

shutil— High-level file operations

Source code: `Lib/shutil.py`

The `shutil` module offers a number of high-level operations on files and collections of files. In particular, functions are provided which support file copying and removal. For operations on individual files, see also `os` module.

■■ Advertencia: Warning Even the higher-level file copying functions (`shutil.copy()`, `shutil.copy2()`) cannot copy all file metadata. On POSIX platforms, this means that file owner and group are lost as well as ACLs. On Mac OS, the resource fork and other metadata are not used. This means that resources will be lost and file type and creator codes will not be correct. On Windows, file owners, ACLs and alternate data streams are not copied.

Directory and files operations

`shutil.copyfileobj(fsrc, fdst[, length])`

Copy the contents of the file-like object `fsrc` to the file-like object `fdst`. The integer `length`, if given, is the buffer size. In particular, a negative `length` value means to copy the data without looping over the source data in chunks; by default the data is read in chunks to avoid uncontrolled memory consumption. Note that if the current file position of the `fsrc` object is not 0, only the contents from the current file position to the end of the file will be copied. `copyfileobj()` will not guarantee that the

`shutil.copyfile(src, dst, *, follow_symlinks=True)`

Copy the contents (no metadata) of the file named `src` to a file named `dst` and return `dst` in the most efficient way possible. `src` and `dst` are path-like objects or path names given as strings. `dst` must be the complete target file name; look at `copy()` for a copy that accepts a target directory path. If `src` and `dst` specify the same file, `SameFileError` is raised. The destination location must be writable; otherwise, an `OSError` exception will be raised. If `dst` already exists, it will be replaced. Special files such as character

exceptions `shutil.SpecialFileError`

This exception is raised when `copyfile()` or `copytree()` attempt to copy a named pipe. Added in version 2.7.

exceptions `shutil.SameFileError`

This exception is raised if source and destination in `copyfile()` are the same file. Added in version 3.4.

`shutil.copymode(src, dst, *, follow_symlinks=True)`

Copy the permission bits from `src` to `dst`. The file contents, owner, and group are unaffected. `src` and `dst` are path-like objects or path names given as strings. If `follow_symlinks` is false, and both `src` and `dst` are symbolic links, `copymode()` will attempt to modify the mode of `dst` itself (rather than the file it points to). This functionality is not available on every platform; please see `copystat()` for more information. If `copymode()` cannot modify symbolic links on the local platform, and it is asked to do so, it will

`shutil.copystat(src, dst, *, follow_symlinks=True)`

Copy the permission bits, last access time, last modification time, and flags from `src` to `dst`. On Linux, `copystat()` also copies the “extended attributes” where possible. The file contents, owner, and group are unaffected. `src` and `dst` are path-like objects or path names given as strings. If `follow_symlinks` is false, and `src` and `dst` both refer to symbolic links, `copystat()` will operate on the symbolic links themselves rather than the files the symbolic links refer to—reading the information from the `src` symbolic link

`shutil.copy(src, dst, *, follow_symlinks=True)`

Copies the file `src` to the file or directory `dst`. `src` and `dst` should be path-like objects or strings. If `dst` specifies a directory, the file will be copied into `dst` using the base filename from `src`. If `dst` specifies a file that already exists, it

will be replaced. Returns the path to the newly created file. If `follow_symlinks` is false, and `src` is a symbolic link, `dst` will be created as a symbolic link. If `follow_symlinks` is true and `src` is a symbolic link, `dst` will be a copy of the file `src` refers to. `copy()` copies the file `d`

shutil.copy2(src,dst,*,follow_symlinks=True)

Identical to `copy()` except that `copy2()` also attempts to preserve file metadata. When `follow_symlinks` is false, and `src` is a symbolic link, `copy2()` attempts to copy all metadata from the `src` symbolic link to the newly created `dst` symbolic link. However, this functionality is not available on all platforms. On platforms where some or all of this functionality is unavailable, `copy2()` will preserve all the metadata it can; `copy2()` never raises an exception because it cannot preserve file metadata. `copy2()` uses `copystat()`

shutil.ignore_patterns(*patterns)

This factory function creates a function that can be used as a callable for `copytree()`'s `ignore` argument, ignoring files and directories that match one of the glob-style `patterns` provided. See the example below.

shutil.copytree(src,dst,symlinks=False,ignore=None,copy_function=copy2,ignore_dangling_symlinks=False,dirs_exist_ok=False)

Recursively copy an entire directory tree rooted at `src` to a directory named `dst` and return the destination directory. All intermediate directories needed to contain `dst` will also be created by default. Permissions and times of directories are copied with `copystat()`, individual files are copied using `copy2()`. If `symlinks` is true, symbolic links in the source tree are represented as symbolic links in the new tree and the metadata of the original links will be copied as far as the platform allows; if false or

shutil.rmtree(path,ignore_errors=False,onerror=None,*,onexc=None,dir_fd=None)

rmtree.avoids_symlink_attacks

Delete an entire directory tree; `path` must point to a directory (but not a symbolic link to a directory). If `ignore_errors` is true, errors resulting from failed removals will be ignored; if false or omitted, such errors are handled by calling a handler specified by `onexc` or `onerror`, if both are omitted, exceptions are propagated to the caller. This function can support `paths` relative to directory descriptors. Note: On platforms that support the necessary `fd`-based functions a symlink attack resistant version

Indicates whether the current platform and implementation provides a symlink attack resistant version of `rmtree()`. Currently this is only true for platforms supporting `fd`-based directory access functions. Added in version 3.3.

shutil.move(src,dst,copy_function=copy2)

Recursively move a file or directory (`src`) to another location and return the destination. If `dst` is an existing directory or a symlink to a directory, then `src` is moved inside that directory. The destination path in that directory must not already exist. If `dst` already exists but is not a directory, it may be overwritten depending on `os.rename()` semantics. If the destination is on the current filesystem, then `os.rename()` is used. Otherwise, `src` is copied to the destination using `copy_function` and then removed.

shutil.disk_usage(path)

Return disk usage statistics about the given `path` as a named tuple with the attributes `total`, `used` and `free`, which are the amount of total, used and free space, in bytes. `path` may be a file or a directory. Note: On Unix filesystems, `path` must point to a path within a mounted filesystem partition. On those platforms, CPython doesn't attempt to retrieve disk usage information from non-mounted filesystems. Added in version 3.3. Changed in version 3.8: On Windows, `path` can now be a file or directory. Availability: Unix,

shutil.chown(path,user=None,group=None,*,dir_fd=None,follow_symlinks=True)

Change owner `user` and/or `group` of the given `path`. `user` can be a system user name or a uid; the same applies to `group`. At least one argument is required. See also `os.chown()`, the underlying function. Raises an `auditing`

eventsutil.chownwith argumentspath,user,group.Availability: Unix.Added in version 3.3.Changed in version 3.13:Addeddir_fdandfollow_symlinksparameters.

shutil.which(cmd,mode=os.F_OK|os.X_OK,path=None)

Return the path to an executable which would be run if the given`cmd`was called. If `nocmdwould be called, returnNone.modeis a permission mask passed toos.access(), by default determining if the file exists and is executable.pathis a “PATHstring” specifying the directories to look in, delimited byos.pathsep. When nopathis specified, thePATHenvironment variable is read fromos.environ, falling back toos.defpathif it is not set.Ifcmdcontains a directory component,which()only checks the specified path`

exceptionsutil.Error

This exception collects exceptions that are raised during a multi-file operation. For`copytree()`, the exception argument is a list of 3-tuples (`srcname,dstname,exception`).

Platform-dependent efficient copy operations

Starting from Python 3.8, all functions involving a file copy (`copyfile()`,`copy()`,`copy2()`,`copytree()`, and`move()`) may use platform-specific “fast-copy” syscalls in order to copy the file more efficiently (see**po-33671**). “fast-copy” means that the copying operation occurs within the kernel, avoiding the use of userspace buffers in Python as in “`outfd.write(infd.read())`”.

On macOS`copyfile`is used to copy the file content (not metadata).

On Linux`os.copy_file_range()`or`os.sendfile()`is used.

On Solaris`os.sendfile()`is used.

On Windows`shutil.copyfile()`uses a bigger default buffer size (1 MiB instead of 64 KiB) and a `memoryview()`-based variant of `shutil.copyfileobj()`is used.

If the fast-copy operation fails and no data was written in the destination file then `shutil` will silently fall back to less efficient`copyfileobj()`function internally.

Changed in version 3.8.

Changed in version 3.14:Solaris now uses`os.sendfile()`.

Changed in version 3.14:Copy-on-write or server-side copy may be used internally via`os.copy_file_range()`on supported Linux filesystems.

copytree example

An example that uses the`ignore_patterns()`helper:

```
from shutil import copytree, ignore_patterns

copytree(source, destination, ignore=ignore_patterns('*.pyc', 'tmp*'))
```

This will copy everything except`.pyc`files and files or directories whose name starts with`tmp`.

Another example that uses the`ignore`argument to add a logging call:

```
from shutil import copytree
import logging

def _logpath(path, names):
    logging.info('Working in %s', path)
    return [] # nothing will be ignored

copytree(source, destination, ignore=_logpath)
```

rmtree example

This example shows how to remove a directory tree on Windows where some of the files have their read-only bit set. It uses the `onexc` callback to clear the read-only bit and reattempt the remove. Any subsequent failure will propagate.

```
import os, stat
import shutil

def remove_readonly(func, path, _):
    "Clear the readonly bit and reattempt the removal"
    os.chmod(path, stat.S_IWRITE)
    func(path)

shutil.rmtree(directory, onexc=remove_readonly)
```

Archiving operations

Added in version 3.2.

Changed in version 3.5: Added support for the `xz` tar format.

High-level utilities to create and read compressed and archived files are also provided. They rely on the `zipfile` and `tarfile` modules.

`shutil.make_archive(base_name, format[, root_dir[, base_dir[, verbose[, dry_run[, owner[, group[, logger]]]]]])`

Create an archive file (such as zip or tar) and return its name. `base_name` is the name of the file to create, including the path, minus any format-specific extension. `format` is the archive format: one of "zip" (if the `zip` module is available), "tar", "gztar" (if the `gzip` module is available), "bztar" (if the `bz2` module is available), "xztar" (if the `lzma` module is available), or "zstdtar" (if the `zstd` module is available). `root_dir` is a directory that will be the root directory of the archive, all

`shutil.get_archive_formats()`

Return a list of supported formats for archiving. Each element of the returned sequence is a tuple (name, description). By default `shutil` provides these formats: `zip`: ZIP file (if the `zip` module is available). `tar`: Uncompressed tar file. Uses POSIX.1-2001 pax format for new archives. `gztar`: gzip'ed tar-file (if the `gzip` module is available). `bztar`: bzip2'ed tar-file (if the `bz2` module is available). `xztar`: xz'ed tar-file (if the `lzma` module is available). `zstdtar`: Zstandard compressed tar-file (if the `zstd` module is available).

`shutil.register_archive_format(name, function[, extra_args[, description]])`

Register an archiver for the format `name`. `function` is the callable that will be used to unpack archives. The callable will receive the `base_name` of the file to create, followed by the `base_dir` (which defaults to `os.getcwd()`) to start archiving from. Further arguments are passed as keyword arguments: `owner`, `group`, `dry_run` and `logger` (as passed in `make_archive()`). If `function` has the custom attribute `supports_root_dir` set to `True`, the `root_dir` argument is passed as a keyword argument. Otherwise the current working

`shutil.unregister_archive_format(name)`

Remove the archive format `name` from the list of supported formats.

`shutil.unpack_archive(filename[, extract_dir[, format[, filter]])`

Unpack an archive. `filename` is the full path of the archive. `extract_dir` is the name of the target directory where the archive is unpacked. If not provided, the current working directory is used. `format` is the archive format: one of "zip", "tar", "gztar", "bztar", "xztar", or "zstdtar". Or any other format registered with `register_unpack_format()`. If not provided, `unpack_archive()` will use the archive file name extension and see if an unpacker was registered for that extension. In case none is found,

shutil.register_unpack_format(name,extensions,function[,extra_args[,description]])

Registers an unpack format. `name` is the name of the format and `extensions` is a list of extensions corresponding to the format, like `.zip` for Zip files. `function` is the callable that will be used to unpack archives. The callable will receive: the path of the archive, as a positional argument; the directory the archive must be extracted to, as a positional argument; possibly a `filterkeyword` argument, if it was given to `unpack_archive()`; additional keyword arguments, specified by `extra_args` as a sequence of (`name`, `v`

shutil.unregister_unpack_format(name)

Unregister an unpack format. `name` is the name of the format.

shutil.get_unpack_formats()

Return a list of all registered formats for unpacking. Each element of the returned sequence is a tuple (`name`, `extensions`, `description`). By default `shutil` provides these formats: `zip`: ZIP file (unpacking compressed files works only if the corresponding module is available). `tar`: uncompressed tar file. `gztar`: gzip'ed tar-file (if the `zlib` module is available). `bztar`: bzip2'ed tar-file (if the `bz2` module is available). `xztar`: xz'ed tar-file (if the `lzma` module is available). `zstdtar`: Zstandard compressed tar-file (

Archiving example

In this example, we create a gzip'ed tar-file archive containing all files found in the `.ssh` directory of the user:

```
>>> from shutil import make_archive
>>> import os
>>> archive_name = os.path.expanduser(os.path.join('~', 'myarchive'))
>>> root_dir = os.path.expanduser(os.path.join('~', '.ssh'))
>>> make_archive(archive_name, 'gztar', root_dir)
'/Users/tarek/myarchive.tar.gz'
```

The resulting archive contains:

```
$ tar -tzvf /Users/tarek/myarchive.tar.gz
drwx----- tarek/staff      0 2010-02-01 16:23:40 ./
-rw-r--r-- tarek/staff    609 2008-06-09 13:26:54 ./authorized_keys
-rwxr-xr-x tarek/staff     65 2008-06-09 13:26:54 ./config
-rwx----- tarek/staff    668 2008-06-09 13:26:54 ./id_dsa
-rwxr-xr-x tarek/staff    609 2008-06-09 13:26:54 ./id_dsa.pub
-rw----- tarek/staff   1675 2008-06-09 13:26:54 ./id_rsa
-rw-r--r-- tarek/staff    397 2008-06-09 13:26:54 ./id_rsa.pub
-rw-r--r-- tarek/staff  37192 2010-02-06 18:23:10 ./known_hosts
```

Archiving example with base_dir

In this example, similar to the one above, we show how to use `make_archive()`, but this time with the usage of `base_dir`. We now have the following directory structure:

```
$ tree tmp
tmp
├── root
│   ├── structure
│   │   ├── content
│   │   │   ├── please_add.txt
│   │   │   └── do_not_add.txt
```

In the final archive, `please_add.txt` should be included, but `do_not_add.txt` should not. Therefore we use the following:

```
>>> from shutil import make_archive
>>> import os
>>> archive_name = os.path.expanduser(os.path.join('~', 'myarchive'))
>>> make_archive(
...     archive_name,
```

```
...     'tar',
...     root_dir='tmp/root',
...     base_dir='structure/content',
... )
'/Users/tarek/myarchive.tar'
```

Listing the files in the resulting archive gives us:

```
$ python -m tarfile -l /Users/tarek/myarchive.tar
structure/content/
structure/content/please_add.txt
```

Querying the size of the output terminal

shutil.get_terminal_size(fallback=(columns,lines))

Get the size of the terminal window. For each of the two dimensions, the environment variable, `COLUMNS` and `LINES` respectively, is checked. If the variable is defined and the value is a positive integer, it is used. When `COLUMNS` or `LINES` is not defined, which is the common case, the terminal connected to `sys.__stdout__` is queried by invoking `os.get_terminal_size()`. If the terminal size cannot be successfully queried, either because the system doesn't support querying, or because we are not connected to a termi

Data Persistence

The modules described in this chapter support storing Python data in a persistent form on disk.

The `pickle` and `marshal` modules can turn many Python data types into a stream of bytes and then recreate the objects from the bytes. The various DBM-related modules support a family of hash-based file formats that store a mapping of strings to other strings.

The list of modules described in this chapter is:

- `pickle`— Python object serialization
Relationship to other Python modules
Comparison with `marshal`
Comparison with `json`
Data stream format
Module Interface
What can be pickled and unpickled?
Pickling Class Instances
Persistence of External Objects
Dispatch Tables
Handling Stateful Objects
Custom Reduction for Types, Functions, and Other Objects
Out-of-band Buffers
Provider API
Consumer API
Example
Restricting Globals
- `copyreg`— Register `pickle` support functions
Example
- `shelve`— Python object persistence
Restrictions
Example
- `marshal`— Internal Python object serialization
- `dbm`— Interfaces to Unix “databases”
`dbm.sqlite3`— SQLite backend for `dbm`
`dbm.gnu`— GNU database manager
`dbm.ndbm`— New Database Manager
`dbm.dumb`— Portable DBM implementation
- `sqlite3`— DB-API 2.0 interface for SQLite databases
Tutorial
Reference
Module functions
Module constants
Connection objects
Cursor objects
Row objects
Blob objects
Prepare Protocol objects
Exceptions
SQLite and Python types
Default adapters and converters (deprecated)
Command-line interface
How-to guides
How to use placeholders to bind values in SQL queries
How to adapt custom Python types to SQLite values
How to wr

pickle— Python object serialization

Source code: `Lib/pickle.py`

The `pickle` module implements binary protocols for serializing and de-serializing a Python object structure. “Pickling” is the process whereby a Python object hierarchy is converted into a byte stream, and “unpickling” is the inverse operation, whereby a byte stream (from a binary file or bytes-like object) is converted back into an object hierarchy. Pickling (and unpickling) is alternatively known as “serialization”, “marshalling,”[1] or “flattening”; however, to avoid confusion, the terms used here are “pickling” and “unpickling”.

■■ Advertencia: Warning The `pickle` module is not secure. Only unpickle data you trust. It is possible to construct malicious pickle data which will execute arbitrary code during unpickling. Never unpickle data that could have come from an untrusted source, or that could have been tampered with. Consider signing data with `hmac` if you need to ensure that it has not been tampered with. Safer serialization formats such as `json` may be more appropriate if you are processing untrusted data. See [Comparison with json](#).

Relationship to other Python modules

Comparison with marshal

Python has a more primitive serialization module called `marshal`, but in general `pickle` should always be the preferred way to serialize Python objects. `marshal` exists primarily to support Python's `.pyc` files.

The `pickle` module differs from `marshal` in several significant ways:

- `marshal` cannot be used to serialize user-defined classes and their instances. `pickle` can save and restore class instances transparently, however the class definition must be importable and live in the same module as when the object was stored.
- The `marshal` serialization format is not guaranteed to be portable across Python versions. Because its primary job in life is to support `.pyc` files, the Python implementers reserve the right to change the serialization format in non-backwards compatible ways should the need arise. The `pickle` serialization format is guaranteed to be backwards compatible across Python releases provided a compatible pickle

Comparison with json

There are fundamental differences between the pickle protocols and JSON (JavaScript Object Notation):

- JSON is a text serialization format (it outputs unicode text, although most of the time it is then encoded to utf-8), while pickle is a binary serialization format;
- JSON is human-readable, while pickle is not;
- JSON is interoperable and widely used outside of the Python ecosystem, while pickle is Python-specific;
- JSON, by default, can only represent a subset of the Python built-in types, and no custom classes; pickle can represent an extremely large number of Python types (many of them automatically, by clever usage of Python's introspection facilities; complex cases can be tackled by implementing specific object APIs);
- Unlike pickle, deserializing untrusted JSON does not in itself create an arbitrary code execution vulnerability.

See also

The `json` module: a standard library module allowing JSON serialization and deserialization.

Data stream format

The data format used by pickle is Python-specific. This has the advantage that there are no restrictions imposed by external standards such as JSON (which can't represent pointer sharing); however it means that non-Python programs may not be able to reconstruct pickled Python objects.

By default, the pickled data format uses a relatively compact binary representation. If you need optimal size characteristics, you can efficiently compress pickled data.

The module `pickletools` contains tools for analyzing data streams generated by pickle. `pickletools` source code has extensive comments about opcodes used by pickle protocols.

There are currently 6 different protocols which can be used for pickling. The higher the protocol used, the more recent the version of Python needed to read the pickle produced.

- Protocol version 0 is the original "human-readable" protocol and is backwards compatible with earlier versions of Python.
- Protocol version 1 is an old binary format which is also compatible with earlier versions of Python.
- Protocol version 2 was introduced in Python 2.3. It provides much more efficient pickling of new-style classes. Refer to PEP 307 for information about improvements brought by protocol 2.
- Protocol version 3 was added in Python 3.0. It has explicit support for bytes objects and cannot be unpickled by Python 2.x. This was the default protocol in Python 3.0–3.7.
- Protocol version 4 was added in Python 3.4. It adds support for very large objects, pickling more kinds of objects, and some data format optimizations. This was the default protocol in Python 3.8–3.13. Refer to PEP 3154 for information about improvements brought by protocol 4.
- Protocol version 5 was added in Python 3.8. It adds support for out-of-band data and speedup for in-band data. It is the default protocol starting with Python 3.14. Refer to PEP 574 for information about improvements brought by protocol 5.

■ **Note:** Note that `Serialization` is a more primitive notion than persistence; although `pickle` reads and writes file objects, it does not handle the issue of naming persistent objects, nor the (even more complicated) issue of concurrent access to persistent objects. The `pickle` module can transform a complex object into a byte stream and it can transform the byte stream into an object with the same internal structure. Perhaps the most obvious thing to do with these byte streams is to write them onto a file, but it is also conceivable to send them across a network or store them in a database. The `shelve` module provides a

Module Interface

To serialize an object hierarchy, you simply call the `dumps()` function. Similarly, to de-serialize a data stream, you call the `loads()` function. However, if you want more control over serialization and de-serialization, you can create a `Pickler` or an `Unpickler` object, respectively.

The `pickle` module provides the following constants:

`pickle.HIGHEST_PROTOCOL`

An integer, the highest protocol version available. This value can be passed as a protocol value to functions `dump()` and `dumps()` as well as the `Pickler` constructor.

`pickle.DEFAULT_PROTOCOL`

An integer, the default protocol version used for pickling. May be less than `HIGHEST_PROTOCOL`. Currently the default protocol is 5, introduced in Python 3.8 and incompatible with previous versions. This version introduces support for out-of-band buffers, where PEP 3118-compatible data can be transmitted separately from the main pickle stream. Changed in version 3.0: The default protocol is 3. Changed in version 3.8: The default protocol is 4. Changed in version 3.14: The default protocol is 5.

The `pickle` module provides the following functions to make the pickling process more convenient:

`pickle.dump(obj, file, protocol=None, *, fix_imports=True, buffer_callback=None)`

Write the pickled representation of the object `obj` to the open file object `file`. This is equivalent to `Pickler(file, protocol).dump(obj)`. Arguments `file`, `protocol`, `fix_imports` and `buffer_callback` have the same meaning as in the `Pickler` constructor. Changed in version 3.8: The `buffer_callback` argument was added.

`pickle.dumps(obj, protocol=None, *, fix_imports=True, buffer_callback=None)`

Return the pickled representation of the object `obj` as a bytes object, instead of writing it to a file. Arguments `protocol`, `fix_imports` and `buffer_callback` have the same meaning as in the `Pickler` constructor. Changed in version 3.8: The `buffer_callback` argument was added.

`pickle.load(file, *, fix_imports=True, encoding='ASCII', errors='strict', buffers=None)`

Read the pickled representation of an object from the open file object `file` and return the reconstituted object hierarchy specified therein. This is equivalent to `Unpickler(file).load()`. The protocol version of the pickle is detected automatically, so no protocol argument is needed. Bytes past the pickled representation of the object are ignored. Arguments `file`, `fix_imports`, `encoding`, `errors`, `strict` and `buffers` have the same meaning as in the `Unpickler` constructor. Changed in version 3.8: The `buffers` argument was

`pickle.loads(data, *, fix_imports=True, encoding='ASCII', errors='strict', buffers=None)`

Return the reconstituted object hierarchy of the pickled representation `data` of an object. `data` must be a bytes-like object. The protocol version of the pickle is detected automatically, so no protocol argument is needed. Bytes past the pickled representation of the object are ignored. Arguments `fix_imports`, `encoding`, `errors`, `strict` and `buffers` have the same meaning as in the `Unpickler` constructor. Changed in version 3.8: The `buffers` argument was added.

The `pickle` module defines three exceptions:

`exception pickle.PickleError`

Common base class for the other pickling exceptions. It inherits from `Exception`.

`exception pickle.PicklingError`

Error raised when an unpicklable object is encountered by `Pickler`. It inherits from `PickleError`. Refer to `What can be pickled and unpickled?` to learn what kinds of objects can be pickled.

`exception pickle.UnpicklingError`

Error raised when there is a problem unpickling an object, such as a data corruption or a security violation. It inherits from `PickleError`. Note that other exceptions may also be raised during unpickling, including (but not necessarily limited to) `AttributeError`, `EOFError`, `ImportError`, and `IndexError`.

The `pickle` module exports three classes, `Pickler`, `Unpickler` and `PickleBuffer`:

`class pickle.Pickler(file, protocol=None, *, fix_imports=True, buffer_callback=None)`

`dump(obj)`

`persistent_id(obj)`

`dispatch_table`

`reducer_override(obj)`

`fast`

`clear_memo()`

This takes a binary file for writing a pickle data stream. The optional `protocol` argument, an integer, tells the pickler to use the given protocol; supported protocols are 0 to `HIGHEST_PROTOCOL`. If not specified, the default is `DEFAULT_PROTOCOL`. If a negative number is specified, `HIGHEST_PROTOCOL` is selected. The `file` argument must have a `write()` method that accepts a single bytes argument. It can thus be an on-disk file opened for binary writing, an `io.BytesIO` instance, or any other custom object that me

Write the pickled representation of `obj` to the open file object given in the constructor.

Do nothing by default. This exists so a subclass can override it. If `persistent_id()` returns `None`, `obj` is pickled as usual. Any other value causes `Pickler` to emit the returned value as a persistent ID for `obj`. The meaning of this persistent ID should be defined by `Unpickler.persistent_load()`. Note that the value returned by `persistent_id()` cannot itself have a persistent ID. See [Persistence of External Objects](#) for details and examples of uses. Changed in version 3.13: Add the default implementation of this m

A pickler object's dispatch table is a registry of reduction functions of the kind which can be declared using `copyreg.pickle()`. It is a mapping whose keys are classes and whose values are reduction functions. A reduction function takes a single argument of the associated class and should conform to the same interface as a `__reduce__()` method. By default, a pickler object will not have a `dispatch_table` attribute, and it will instead use the global dispatch table managed by the `copyreg` module. However, t

Special reducer that can be defined in `Pickler` subclasses. This method has priority over any reducer in the `dispatch_table`. It should conform to the same interface as a `__reduce__()` method, and can optionally return `NotImplemented` to fallback on `dispatch_table`-registered reducers to pickle `obj`. For a detailed example, see [Custom Reduction for Types, Functions, and Other Objects](#). Added in version 3.8.

Deprecated. Enable fast mode if set to a true value. The fast mode disables the usage of memo, therefore speeding the pickling process by not generating superfluous PUT opcodes. It should not be used with self-referential objects, doing otherwise will cause `Pickler` to recurse infinitely. Use `pickletools.optimize()` if you need more compact pickles.

Clears the pickler's "memo". The memo is the data structure that remembers which objects the pickler has already seen, so that shared or recursive objects are pickled by reference and not by value. This method is useful when re-using picklers.

`class pickle.Unpickler(file, *, fix_imports=True, encoding='ASCII', errors='strict', buffers=None)`

`load()`

`persistent_load(pid)`

`find_class(module, name)`

This takes a binary file for reading a pickle data stream. The protocol version of the pickle is detected automatically, so no protocol argument is needed. The argument `file` must have three methods, a `read()` method that takes an integer argument, a `readinto()` method that takes a buffer argument and a `readline()` method that requires no arguments, as in the `io.BufferedReader` interface. Thus `file` can be an on-disk file opened for binary reading, an `io.BufferedReader`, or any other custom object that meets th

Read the pickled representation of an object from the open file object given in the constructor, and return the reconstituted object hierarchy specified therein. Bytes past the pickled representation of the object are ignored.

Raise an `UnpicklingError` by default. If defined, `persistent_load()` should return the object specified by the persistent ID `pid`. If an invalid persistent ID is encountered, an `UnpicklingError` should be raised. See [Persistence of External Objects](#) for details and examples of uses. Changed in version 3.13: Add the default implementation of this method in the C implementation of `Unpickler`.

Import `module` if necessary and return the object called `name` from it, where the `module` and `name` arguments are `str` objects. Note, unlike its name suggests, `find_class()` is also used for finding functions. Subclasses may override this to gain control over what type of objects and how they can be loaded, potentially reducing security

risks. Refer to `Restricting Globals` for details. Raises an `auditing event` `pickle.find_class` with arguments `module`, `name`.

`class pickle.PickleBuffer(buffer)`

`raw()`

`release()`

A wrapper for a buffer representing picklable data. `buffer` must be a buffer-providing object, such as a bytes-like object or a N-dimensional array. `PickleBuffer` is itself a buffer provider, therefore it is possible to pass it to other APIs expecting a buffer-providing object, such as `memoryview`. `PickleBuffer` objects can only be serialized using pickle protocol 5 or higher. They are eligible for out-of-band serialization. Added in version 3.8. `raw()` Return a `memoryview` of the memory area underlying this buffer.

Return a `memoryview` of the memory area underlying this buffer. The returned object is a one-dimensional, C-contiguous `memoryview` with format `B` (unsigned bytes). `BufferError` is raised if the buffer is neither C- nor Fortran-contiguous.

Release the underlying buffer exposed by the `PickleBuffer` object.

What can be pickled and unpickled?

The following types can be pickled:

- built-in constants (`None`, `True`, `False`, `Ellipsis`, and `NotImplemented`);
- integers, floating-point numbers, complex numbers;
- strings, bytes, bytearrays;
- tuples, lists, sets, and dictionaries containing only picklable objects;
- functions (built-in and user-defined) accessible from the top level of a module (using `def`, not `lambda`);
- classes accessible from the top level of a module;
- instances of such classes for which the result of calling `__getstate__()` is picklable (see section `Pickling Class Instances` for details).

Attempts to pickle unpicklable objects will raise the `PicklingError` exception; when this happens, an unspecified number of bytes may have already been written to the underlying file. Trying to pickle a highly recursive data structure may exceed the maximum recursion depth, a `RecursionError` will be raised in this case. You can carefully raise this limit with `sys.setrecursionlimit()`.

Note that functions (built-in and user-defined) are pickled by fully qualified name, not by value.^[2] This means that only the function name is pickled, along with the name of the containing module and classes. Neither the function's code, nor any of its function attributes are pickled. Thus the defining module must be importable in the unpickling environment, and the module must contain the named object, otherwise an exception will be raised.^[3]

Similarly, classes are pickled by fully qualified name, so the same restrictions in the unpickling environment apply. Note that none of the class's code or data is pickled, so in the following example the class attribute `attr` is not restored in the unpickling environment:

```
class Foo:
    attr = 'A class attribute'

picklestring = pickle.dumps(Foo)
```

These restrictions are why picklable functions and classes must be defined at the top level of a module.

Similarly, when class instances are pickled, their class's code and data are not pickled along with them. Only the instance data are pickled. This is done on purpose, so you can fix bugs in a class or add methods to the class and still load objects that were created with an earlier version of the class. If you plan to have long-lived objects that will see many versions of a class, it may be worthwhile to put a version number in the objects so that suitable conversions can be made by the class's `__setstate__()` method.

Pickling Class Instances

In this section, we describe the general mechanisms available to you to define, customize, and control how class instances are pickled and unpickled.

In most cases, no additional code is needed to make instances picklable. By default, pickle will retrieve the class and the attributes of an instance via introspection. When a class instance is unpickled, its `__init__()` method is usually not invoked. The default behaviour first creates an uninitialized instance and then restores the saved attributes. The following code shows an implementation of this behaviour:

```
def save(obj):
    return (obj.__class__, obj.__dict__)

def restore(cls, attributes):
    obj = cls.__new__(cls)
    obj.__dict__.update(attributes)
    return obj
```

Classes can alter the default behaviour by providing one or several special methods:

object.__getnewargs_ex__()

In protocols 2 and newer, classes that implement the `__getnewargs_ex__()` method can dictate the values passed to the `__new__()` method upon unpickling. The method must return a pair `(args,kwargs)` where `args` is a tuple of positional arguments and `kwargs` a dictionary of named arguments for constructing the object. Those will be passed to the `__new__()` method upon unpickling. You should implement this method if the `__new__()` method of your class requires keyword-only arguments. Otherwise, it is recommended for

object.__getnewargs__()

This method serves a similar purpose as `__getnewargs_ex__()`, but supports only positional arguments. It must return a tuple of arguments `args` which will be passed to the `__new__()` method upon unpickling. `__getnewargs__()` will not be called if `__getnewargs_ex__()` is defined. Changed in version 3.6: Before Python 3.6, `__getnewargs__()` was called instead of `__getnewargs_ex__()` in protocols 2 and 3.

object.__getstate__()

Classes can further influence how their instances are pickled by overriding the method `__getstate__()`. It is called and the returned object is pickled as the contents for the instance, instead of a default state. There are several cases: For a class that has no instance `__dict__` and no `__slots__`, the default state is `None`. For a class that has an instance `__dict__` and no `__slots__`, the default state is `self.__dict__`. For a class that has an instance `__dict__` and `__slots__`, the default state is a tuple consist

object.__setstate__(state)

Upon unpickling, if the class defines `__setstate__()`, it is called with the unpickled state. In that case, there is no requirement for the state object to be a dictionary. Otherwise, the pickled state must be a dictionary and its items are assigned to the new instance's dictionary. Note if `__reduce__()` returns a state with value `None` at pickling, the `__setstate__()` method will not be called upon unpickling.

Refer to the section [Handling Stateful Objects](#) for more information about how to use the methods `__getstate__()` and `__setstate__()`.

■ Nota: NoteAt unpickling time, some methods like `__getattr__()`, `__getattribute__()`, or `__setattr__()` may be called upon the instance. In case those methods rely on some internal invariant being true, the type should implement `__new__()` to establish such an invariant, as `__init__()` is not called when unpickling an instance.

As we shall see, pickle does not use directly the methods described above. In fact, these methods are part of the copy protocol which implements the `__reduce__()` special method. The copy protocol provides a unified interface for retrieving the data necessary for pickling and copying objects.[4]

Although powerful, implementing `__reduce__()` directly in your classes is error prone. For this reason, class designers should use the high-level interface (i.e., `__getnewargs_ex__()`, `__getstate__()` and `__setstate__()`) whenever possible. We will show, however, cases where using `__reduce__()` is the only option or leads to more efficient pickling or both.

object.__reduce__()

The interface is currently defined as follows. The `__reduce__()` method takes no argument and shall return either a string or preferably a tuple (the returned object is often referred to as the “reduce value”). If a string is returned, the string should be interpreted as the name of a global variable. It should be the object’s local name relative to its module; the pickle module searches the module namespace to determine the object’s module. This behaviour is typically useful for singletons. When

object.__reduce_ex__(protocol)

Alternatively, a `__reduce_ex__()` method may be defined. The only difference is this method should take a single integer argument, the protocol version. When defined, pickle will prefer it over the `__reduce__()` method. In addition, `__reduce__()` automatically becomes a synonym for the extended version. The main use for this method is to provide backwards-compatible reduce values for older Python releases.

Persistence of External Objects

For the benefit of object persistence, the pickle module supports the notion of a reference to an object outside the pickled data stream. Such objects are referenced by a persistent ID, which should be either a string of alphanumeric characters (for protocol 0)[5] or just an arbitrary object (for any newer protocol).

The resolution of such persistent IDs is not defined by the pickle module; it will delegate this resolution to the user-defined methods on the pickler and unpickler, `persistent_id()` and `persistent_load()` respectively.

To pickle objects that have an external persistent ID, the pickler must have a custom `persistent_id()` method that takes an object as an argument and returns either `None` or the persistent ID for that object. When `None` is returned, the pickler simply pickles the object as normal. When a persistent ID string is returned, the pickler will pickle that object, along with a marker so that the unpickler will recognize it as a persistent ID.

To unpickle external objects, the unpickler must have a custom `persistent_load()` method that takes a persistent ID object and returns the referenced object.

Here is a comprehensive example presenting how persistent ID can be used to pickle external objects by reference.

```
# Simple example presenting how persistent ID can be used to pickle
# external objects by reference.
```

```
import pickle
import sqlite3
from collections import namedtuple
```

```
# Simple class representing a record in our database.
MemoRecord = namedtuple("MemoRecord", "key, task")
```

```
class DBPickler(pickle.Pickler):
    def persistent_id(self, obj):
```

```

# Instead of pickling MemoRecord as a regular class instance, we emit a
# persistent ID.
if isinstance(obj, MemoRecord):
    # Here, our persistent ID is simply a tuple, containing a tag and a
    # key, which refers to a specific record in the database.
    return ("MemoRecord", obj.key)
else:
    # If obj does not have a persistent ID, return None. This means obj
    # needs to be pickled as usual.
    return None

class DBUnpickler(pickle.Unpickler):

    def __init__(self, file, connection):
        super().__init__(file)
        self.connection = connection

    def persistent_load(self, pid):
        # This method is invoked whenever a persistent ID is encountered.
        # Here, pid is the tuple returned by DBPickler.
        cursor = self.connection.cursor()
        type_tag, key_id = pid
        if type_tag == "MemoRecord":
            # Fetch the referenced record from the database and return it.
            cursor.execute("SELECT * FROM memos WHERE key=?", (str(key_id),))
            key, task = cursor.fetchone()
            return MemoRecord(key, task)
        else:
            # Always raises an error if you cannot return the correct object.
            # Otherwise, the unpickler will think None is the object referenced
            # by the persistent ID.
            raise pickle.UnpicklingError("unsupported persistent object")

def main():
    import io
    import pprint

    # Initialize and populate our database.
    conn = sqlite3.connect(":memory:")
    cursor = conn.cursor()
    cursor.execute("CREATE TABLE memos(key INTEGER PRIMARY KEY, task TEXT)")
    tasks = (
        'give food to fish',
        'prepare group meeting',
        'fight with a zebra',
    )
    for task in tasks:
        cursor.execute("INSERT INTO memos VALUES(NULL, ?)", (task,))

    # Fetch the records to be pickled.
    cursor.execute("SELECT * FROM memos")
    memos = [MemoRecord(key, task) for key, task in cursor]
    # Save the records using our custom DBPickler.
    file = io.BytesIO()
    DBPickler(file).dump(memos)

    print("Pickled records:")
    pprint.pprint(memos)

    # Update a record, just for good measure.
    cursor.execute("UPDATE memos SET task='learn italian' WHERE key=1")

    # Load the records from the pickle data stream.

```



```
file.seek(0)
memos = DBUnpickler(file, conn).load()
```

Dispatch Tables

If one wants to customize pickling of some classes without disturbing any other code which depends on pickling, then one can create a pickler with a private dispatch table.

The global dispatch table managed by the `copyreg` module is available as `copyreg.dispatch_table`. Therefore, one may choose to use a modified copy of `copyreg.dispatch_table` as a private dispatch table.

For example

```
f = io.BytesIO()
p = pickle.Pickler(f)
p.dispatch_table = copyreg.dispatch_table.copy()
p.dispatch_table[SomeClass] = reduce_SomeClass
```

creates an instance of `pickle.Pickler` with a private dispatch table which handles the `SomeClass` class specially. Alternatively, the code

```
class MyPickler(pickle.Pickler):
    dispatch_table = copyreg.dispatch_table.copy()
    dispatch_table[SomeClass] = reduce_SomeClass
f = io.BytesIO()
p = MyPickler(f)
```

does the same but all instances of `MyPickler` will by default share the private dispatch table. On the other hand, the code

```
copyreg.pickle(SomeClass, reduce_SomeClass)
f = io.BytesIO()
p = pickle.Pickler(f)
```

modifies the global dispatch table shared by all users of the `copyreg` module.

Handling Stateful Objects

Here's an example that shows how to modify pickling behavior for a class. The `TextReader` class below opens a text file, and returns the line number and line contents each time its `readline()` method is called. If a `TextReader` instance is pickled, all attributes except the file object member are saved. When the instance is unpickled, the file is reopened, and reading resumes from the last location. The `__setstate__()` and `__getstate__()` methods are used to implement this behavior.

```
class TextReader:
    """Print and number lines in a text file."""

    def __init__(self, filename):
        self.filename = filename
        self.file = open(filename)
        self.lineno = 0

    def readline(self):
        self.lineno += 1
        line = self.file.readline()
        if not line:
            return None
        if line.endswith('\n'):
            line = line[:-1]
        return "%i: %s" % (self.lineno, line)

    def __getstate__(self):
        # Copy the object's state from self.__dict__ which contains
        # all our instance attributes. Always use the dict.copy()
        # method to avoid modifying the original state.
```

```

        state = self.__dict__.copy()
        # Remove the unpicklable entries.
        del state['file']
        return state

    def __setstate__(self, state):
        # Restore instance attributes (i.e., filename and lineno).
        self.__dict__.update(state)
        # Restore the previously opened file's state. To do so, we need to
        # reopen it and read from it until the line count is restored.
        file = open(self.filename)
        for _ in range(self.lineno):
            file.readline()
        # Finally, save the file.
        self.file = file

```

A sample usage might be something like this:

```

>>> reader = TextReader("hello.txt")
>>> reader.readline()
'1: Hello world!'
>>> reader.readline()
'2: I am line number two.'
>>> new_reader = pickle.loads(pickle.dumps(reader))
>>> new_reader.readline()
'3: Goodbye!'

```

Custom Reduction for Types, Functions, and Other Objects

Added in version 3.8.

Sometimes, `dispatch_table` may not be flexible enough. In particular we may want to customize pickling based on another criterion than the object's type, or we may want to customize the pickling of functions and classes.

For those cases, it is possible to subclass from the `Pickler` class and implement a `reducer_override()` method. This method can return an arbitrary reduction tuple (see `__reduce__()`). It can alternatively return `NotImplemented` to fallback to the traditional behavior.

If both the `dispatch_table` and `reducer_override()` are defined, then `reducer_override()` method takes priority.

■ **Nota:** Note for performance reasons, `reducer_override()` may not be called for the following objects: `None`, `True`, `False`, and exact instances of `int`, `float`, `bytes`, `str`, `dict`, `set`, `frozenset`, `list` and `tuple`.

Here is a simple example where we allow pickling and reconstructing a given class:

```

import io
import pickle

class MyClass:
    my_attribute = 1

class MyPickler(pickle.Pickler):
    def reducer_override(self, obj):
        """Custom reducer for MyClass."""
        if getattr(obj, "__name__", None) == "MyClass":
            return type, (obj.__name__, obj.__bases__,
                          {'my_attribute': obj.my_attribute})
        else:
            # For any other object, fallback to usual reduction
            return NotImplemented

f = io.BytesIO()
p = MyPickler(f)
p.dump(MyClass)

```

```
del MyClass

unpickled_class = pickle.loads(f.getvalue())

assert isinstance(unpickled_class, type)
assert unpickled_class.__name__ == "MyClass"
assert unpickled_class.my_attribute == 1
```

Out-of-band Buffers

Added in version 3.8.

In some contexts, the `pickle` module is used to transfer massive amounts of data. Therefore, it can be important to minimize the number of memory copies, to preserve performance and resource consumption. However, normal operation of the `pickle` module, as it transforms a graph-like structure of objects into a sequential stream of bytes, intrinsically involves copying data to and from the pickle stream.

This constraint can be eschewed if both the provider (the implementation of the object types to be transferred) and the consumer (the implementation of the communications system) support the out-of-band transfer facilities provided by pickle protocol 5 and higher.

Provider API

The large data objects to be pickled must implement a `__reduce_ex__()` method specialized for protocol 5 and higher, which returns a `PickleBuffer` instance (instead of e.g. `bytes` object) for any large data.

A `PickleBuffer` object signals that the underlying buffer is eligible for out-of-band data transfer. Those objects remain compatible with normal usage of the `pickle` module. However, consumers can also opt-in to tell pickle that they will handle those buffers by themselves.

Consumer API

A communications system can enable custom handling of the `PickleBuffer` objects generated when serializing an object graph.

On the sending side, it needs to pass a `buffer_callback` argument to `Pickler` (or to `dump()` or `dumps()` function), which will be called with each `PickleBuffer` generated while pickling the object graph. Buffers accumulated by the `buffer_callback` will not see their data copied into the pickle stream, only a cheap marker will be inserted.

On the receiving side, it needs to pass a `buffers` argument to `Unpickler` (or to `load()` or `loads()` function), which is an iterable of the buffers which were passed to `buffer_callback`. That iterable should produce buffers in the same order as they were passed to `buffer_callback`. Those buffers will provide the data expected by the reconstructors of the objects whose pickling produced the original `PickleBuffer` objects.

Between the sending side and the receiving side, the communications system is free to implement its own transfer mechanism for out-of-band buffers. Potential optimizations include the use of shared memory or datatype-dependent compression.

Example

Here is a trivial example where we implement a `bytearray` subclass able to participate in out-of-band buffer pickling:

```
class ZeroCopyByteArray(bytearray):

    def __reduce_ex__(self, protocol):
        if protocol >= 5:
            return type(self)._reconstruct, (PickleBuffer(self),), None
        else:
            # PickleBuffer is forbidden with pickle protocols <= 4.
```

```

        return type(self)._reconstruct, (bytearray(self),)

    @classmethod
    def _reconstruct(cls, obj):
        with memoryview(obj) as m:
            # Get a handle over the original buffer object
            obj = m.obj
            if type(obj) is cls:
                # Original buffer object is a ZeroCopyByteArray, return it
                # as-is.
                return obj
            else:
                return cls(obj)

```

The reconstructor (the `_reconstruct` class method) returns the buffer's providing object if it has the right type. This is an easy way to simulate zero-copy behaviour on this toy example.

On the consumer side, we can pickle those objects the usual way, which when unserialized will give us a copy of the original object:

```

b = ZeroCopyByteArray(b"abc")
data = pickle.dumps(b, protocol=5)
new_b = pickle.loads(data)
print(b == new_b) # True
print(b is new_b) # False: a copy was made

```

But if we pass `abuffer_callback` and then give back the accumulated buffers when unserializing, we are able to get back the original object:

```

b = ZeroCopyByteArray(b"abc")
buffers = []
data = pickle.dumps(b, protocol=5, buffer_callback=buffers.append)
new_b = pickle.loads(data, buffers=buffers)
print(b == new_b) # True
print(b is new_b) # True: no copy was made

```

This example is limited by the fact that `bytearray` allocates its own memory: you cannot create a `bytearray` instance that is backed by another object's memory. However, third-party datatypes such as NumPy arrays do not have this limitation, and allow use of zero-copy pickling (or making as few copies as possible) when transferring between distinct processes or systems.

See also

PEP 574– Pickle protocol 5 with out-of-band data

Restricting Globals

By default, unpickling will import any class or function that it finds in the pickle data. For many applications, this behaviour is unacceptable as it permits the unpickler to import and invoke arbitrary code. Just consider what this hand-crafted pickle data stream does when loaded:

```

>>> import pickle
>>> pickle.loads(b"cos\nsystem\n(S'echo hello world'\ntr.")
hello world
0

```

In this example, the unpickler imports the `os.system()` function and then apply the string argument “echo hello world”. Although this example is inoffensive, it is not difficult to imagine one that could damage your system.

For this reason, you may want to control what gets unpickled by customizing `Unpickler.find_class()`. Unlike its name suggests, `Unpickler.find_class()` is called whenever a global (i.e., a class or a function) is requested. Thus it is possible to either completely forbid globals or restrict them to a safe subset.

Here is an example of an unpickler allowing only few safe classes from the `builtins` module to be loaded:

```
import builtins
import io
import pickle

safe_builtins = {
    'range',
    'complex',
    'set',
    'frozenset',
    'slice',
}

class RestrictedUnpickler(pickle.Unpickler):

    def find_class(self, module, name):
        # Only allow safe classes from builtins.
        if module == "builtins" and name in safe_builtins:
            return getattr(builtins, name)
        # Forbid everything else.
        raise pickle.UnpicklingError("global '%s.%s' is forbidden" %
                                      (module, name))

def restricted_loads(s):
    """Helper function analogous to pickle.loads()."""
    return RestrictedUnpickler(io.BytesIO(s)).load()
```

A sample usage of our unpickler working as intended:

```
>>> restricted_loads(pickle.dumps([1, 2, range(15)]))
[1, 2, range(0, 15)]
>>> restricted_loads(b'cos\nsystem\n(S'echo hello world'\ntr.')')
Traceback (most recent call last):
...
pickle.UnpicklingError: global 'os.system' is forbidden
>>> restricted_loads(b'cbuiltins\neval\n'
...                  b'(S'getattr(__import__("os"), "system")'
...                  b'("echo hello world")'\ntr.')')
Traceback (most recent call last):
...
pickle.UnpicklingError: global 'builtins.eval' is forbidden
```

As our examples shows, you have to be careful with what you allow to be unpickled. Therefore if security is a concern, you may want to consider alternatives such as the marshalling API in `xmlrpc.client` or third-party solutions.

Performance

Recent versions of the pickle protocol (from protocol 2 and upwards) feature efficient binary encodings for several common features and built-in types. Also, the `pickle` module has a transparent optimizer written in C.

Examples

For the simplest code, use the `dump()` and `load()` functions.

```
import pickle

# An arbitrary collection of objects supported by pickle.
data = {
    'a': [1, 2.0, 3+4j],
```

```

    'b': ("character string", b"byte string"),
    'c': {None, True, False}
}

with open('data.pickle', 'wb') as f:
    # Pickle the 'data' dictionary using the highest protocol available.
    pickle.dump(data, f, pickle.HIGHEST_PROTOCOL)

```

The following example reads the resulting pickled data.

```

import pickle

with open('data.pickle', 'rb') as f:
    # The protocol version used is detected automatically, so we do not
    # have to specify it.
    data = pickle.load(f)

```

Command-line interface

The `pickle` module can be invoked as a script from the command line, it will display contents of the pickle files. However, when the pickle file that you want to examine comes from an untrusted source, `-mpickletools` is a safer option because it does not execute pickle bytecode, see `pickletools` CLI usage.

```
python -m pickle pickle_file [pickle_file ...]
```

The following option is accepted:

pickle_file

A pickle file to read, or to indicate reading from standard input.

See also

Module `copyreg`

Module `pickletools`

Module `shelve`

Module `copy`

Module `marshal`

Pickle interface constructor registration for extension types.

Tools for working with and analyzing pickled data.

Indexed databases of objects; uses `pickle`.

Shallow and deep object copying.

High-performance serialization of built-in types.

Footnotes

Don't confuse this with the `marshal` module

This is why `lambda` functions cannot be pickled: all `lambda` functions share the same name: `<lambda>`.

The exception raised will likely be an `ImportError` or an `AttributeError` but it could be something else.

The `copy` module uses this protocol for shallow and deep copying operations.

The limitation on alphanumeric characters is due to the fact that persistent IDs in protocol 0 are delimited by the newline character. Therefore if any kind of newline characters occurs in persistent IDs, the resulting pickled data will become unreadable.

copyreg— Register picklesupport functions

Source code: `Lib/copyreg.py`

The `copyreg` module offers a way to define functions used while pickling specific objects.

The `pickle` and `copy` modules use those functions when pickling/copying those objects. The module provides configuration information about object constructors which are not classes. Such constructors may be factory functions or class instances.

`copyreg.constructor(object)`

Declares `object` to be a valid constructor. If `object` is not callable (and hence not valid as a constructor), raises `TypeError`.

`copyreg.pickle(type,function,constructor_ob=None)`

Declares that `function` should be used as a “reduction” function for objects of type `type`. `function` must return either a string or a tuple containing between two and six elements. See the `dispatch_table` for more details on the interface of `function`. The `constructor_ob` parameter is a legacy feature and is now ignored, but if passed it must be a callable. Note that the `dispatch_table` attribute of a pickler object or subclass of `Pickler` can also be used for declaring reduction functions.

Example

The example below would like to show how to register a pickle function and how it will be used:

```
>>> import copyreg, copy, pickle
>>> class C:
...     def __init__(self, a):
...         self.a = a
...
>>> def pickle_c(c):
...     print("pickling a C instance...")
...     return C, (c.a,)
...
>>> copyreg.pickle(C, pickle_c)
>>> c = C(1)
>>> d = copy.copy(c)
pickling a C instance...
>>> p = pickle.dumps(c)
pickling a C instance...
```

shelve— Python object persistence

Source code: `Lib/shelve.py`

A “shelf” is a persistent, dictionary-like object. The difference with “dbm” databases is that the values (not the keys!) in a shelf can be essentially arbitrary Python objects — anything that the `pickle` module can handle. This includes most class instances, recursive data types, and objects containing lots of shared sub-objects. The keys are ordinary strings.

`shelve.open(filename, flag='c', protocol=None, writeback=False)`

Open a persistent dictionary. The filename specified is the base filename for the underlying database. As a side-effect, an extension may be added to the filename and more than one file may be created. By default, the underlying database file is opened for reading and writing. The optional `flag` parameter has the same interpretation as the `flag` parameter of `dbm.open()`. By default, pickles created with `pickle.DEFAULT_PROTOCOL` are used to serialize values. The version of the pickle protocol can be spe

■■ Advertencia: WarningBecause the `shelve` module is backed by `pickle`, it is insecure to load a shelf from an untrusted source. Like with `pickle`, loading a shelf can execute arbitrary code.

Shelf objects support most of the methods and operations supported by dictionaries (except copying, constructors and operators `|` and `=`). This eases the transition from dictionary based scripts to those requiring persistent storage.

Two additional methods are supported:

`Shelf.sync()`

Write back all entries in the cache if the shelf was opened with `writeback` set to `True`. Also empty the cache and synchronize the persistent dictionary on disk, if feasible. This is called automatically when the shelf is closed with `close()`.

`Shelf.close()`

Synchronize and close the persistent dict object. Operations on a closed shelf will fail with a `ValueError`.

See also

Persistent dictionary recipe with widely supported storage formats and having the speed of native dictionaries.

Restrictions

- The choice of which database package will be used (such as `dbm`, `ndbm` or `dbm.gnu`) depends on which interface is available. Therefore it is not safe to open the database directly using `dbm`. The database is also (unfortunately) subject to the limitations of `dbm`, if it is used — this means that (the pickled representation of) the objects stored in the database should be fairly small, and in rare cases key
- The `shelve` module does not support concurrent read/write access to shelved objects. (Multiple simultaneous read accesses are safe.) When a program has a shelf open for writing, no other program should have it open for reading or writing. Unix file locking can be used to solve this, but this differs across Unix versions and requires knowledge about the database implementation used.
- On macOS `dbm.ndbm` can silently corrupt the database file on updates, which can cause hard crashes when trying to read from the database.

`class shelve.Shelf(dict, protocol=None, writeback=False, keyencoding='utf-8')`

A subclass of `collections.abc.MutableMapping` which stores pickled values in the dict object. By default, pickles created with `pickle.DEFAULT_PROTOCOL` are used to serialize values. The version of the pickle protocol can be specified with the `protocol` parameter. See the `pickle` documentation for a discussion of the pickle protocols. If the `writeback` parameter is `True`, the object will hold a cache of all entries accessed and write them back to the dict at sync and close times. This allows natural operations on mut

`class shelve.BsdDbShelf(dict, protocol=None, writeback=False, keyencoding='utf-8')`

A subclass of `Shelf` which exposes `first()`, `next()`, `previous()`, `last()` and `set_location()` methods. These are available in the third-party `bsddb` module from `pybsddb` but not in other database modules. The dict object passed to the constructor must support those methods. This is generally accomplished by calling one of `bsddb.hashopen()`, `bsddb.btopen()` or `bsddb.rnopen()`. The optional `protocol`, `writeback`, and `keyencoding` parameters have the same interpretation as for the `Shelf` class.

`class shelve.DbfilenameShelf(filename, flag='c', protocol=None, writeback=False)`

A subclass of `Shelf` which accepts a filename instead of a dict-like object. The underlying file will be opened using `dbm.open()`. By default, the file will be created and opened for both read and write. The optional `flag` parameter has the same interpretation as for the `open()` function. The optional `protocol` and `writeback` parameters have the same interpretation as for the `Shelf` class.

Example

To summarize the interface (key is a string, data is an arbitrary object):

```
import shelve

d = shelve.open(filename) # open -- file may get suffix added by low-level
                           # library

d[key] = data              # store data at key (overwrites old data if
                           # using an existing key)
data = d[key]              # retrieve a COPY of data at key (raise KeyError
                           # if no such key)
del d[key]                 # delete data stored at key (raises KeyError
                           # if no such key)

flag = key in d             # true if the key exists
klist = list(d.keys())      # a list of all existing keys (slow!)

# as d was opened WITHOUT writeback=True, beware:
d['xx'] = [0, 1, 2]         # this works as expected, but...
d['xx'].append(3)           # *this doesn't!* -- d['xx'] is STILL [0, 1, 2]!

# having opened d without writeback=True, you need to code carefully:
temp = d['xx']              # extracts the copy
temp.append(5)              # mutates the copy
d['xx'] = temp              # stores the copy right back, to persist it

# or, d=shelve.open(filename, writeback=True) would let you just code
# d['xx'].append(5) and have it work as expected, BUT it would also
# consume more memory and make the d.close() operation slower.

d.close()                  # close it
```

See also

Module `dbm`

Module `pickle`

Generic interface to dbm-style databases.

Object serialization used byshelve.

marshal— Internal Python object serialization

This module contains functions that can read and write Python values in a binary format. The format is specific to Python, but independent of machine architecture issues (e.g., you can write a Python value to a file on a PC, transport the file to a Mac, and read it back there). Details of the format are undocumented on purpose; it may change between Python versions (although it rarely does).[1]

This is not a general “persistence” module. For general persistence and transfer of Python objects through RPC calls, see the `pickle` and `shelve` modules. The `marshal` module exists mainly to support reading and writing the “pseudo-compiled” code for Python modules of `.pyc` files. Therefore, the Python maintainers reserve the right to modify the `marshal` format in backward incompatible ways should the need arise. The format of code objects is not compatible between Python versions, even if the version of the format is the same. De-serializing a code object in the incorrect Python version has undefined behavior. If you’re serializing and de-serializing Python objects, use the `pickle` module instead – the performance is comparable, version independence is guaranteed, and `pickle` supports a substantially wider range of objects than `marshal`.

■■ Advertencia: WarningThe `marshal` module is not intended to be secure against erroneous or maliciously constructed data. Never unmarshal data received from an untrusted or unauthenticated source.

There are functions that read/write files as well as functions operating on bytes-like objects.

Not all Python object types are supported; in general, only objects whose value is independent from a particular invocation of Python can be written and read by this module. The following types are supported:

- Numeric types: `int`, `bool`, `float`, `complex`.
- Strings (`str`) and `bytes`. Bytes-like objects like `bytearray` are marshalled as `bytes`.
- Containers: `tuple`, `list`, `set`, `frozenset`, and (since version 5), `slice`. It should be understood that these are supported only if the values contained therein are themselves supported. Recursive containers are supported since version 3.
- The singletons `None`, `Ellipsis` and `StopIteration`.
- code objects, if `allow_code` is `true`. See note above about version dependence.

Changed in version 3.4:

- Added format version 3, which supports marshalling recursive lists, sets and dictionaries.
- Added format version 4, which supports efficient representations of short strings.

Changed in version 3.14: Added format version 5, which allows marshalling slices.

The module defines these functions:

`marshal.dump(value, file, version=version, *, allow_code=True)`

Write the value on the open file. The value must be a supported type. The file must be a writeable binary file. If the value has (or contains an object that has) an unsupported type, a `ValueError` exception is raised — but garbage data will also be written to the file. The object will not be properly read back by `load()`. Code objects are only supported if `allow_code` is `true`. The `version` argument indicates the data format that `dump` should use (see below). Raises an auditing event `marshal.dump` with arguments `value`

`marshal.load(file, *, allow_code=True)`

Read one value from the open file and return it. If no valid value is read (e.g. because the data has a different Python version’s incompatible `marshal` format), raise `EOFError`, `ValueError` or `TypeError`. Code objects are only supported if `allow_code` is `true`. The file must be a readable binary file. Raises an auditing event `marshal.load` with no arguments. Note if an object containing an unsupported type was marshalled with `dump()`, `load()` will

substitute None for the unmarshallable type. Changed in version 3.10: This can

marshal.dumps(value, version=version, *, allow_code=True)

Return the bytes object that would be written to a file by `dump(value, file)`. The value must be a supported type. Raise a `ValueError` exception if value has (or contains an object that has) an unsupported type. Code objects are only supported if `allow_code` is true. The `version` argument indicates the data format that `dumps` should use (see below). Raises an auditing event `marshal.dumps` with arguments `value, version`. Changed in version 3.13: Added the `allow_code` parameter.

marshal.loads(bytes, *, allow_code=True)

Convert the bytes-like object to a value. If no valid value is found, raise `EOFError`, `ValueError` or `TypeError`. Code objects are only supported if `allow_code` is true. Extra bytes in the input are ignored. Raises an auditing event `marshal.loads` with argument `bytes`. Changed in version 3.10: This call used to raise a `code.__new__` audit event for each code object. Now it raises a single `marshal.load` event for the entire load operation. Changed in version 3.13: Added the `allow_code` parameter.

In addition, the following constants are defined:

marshal.version

Indicates the format that the module uses. Version 0 is the historical first version; subsequent versions add new features. Generally, a new version becomes the default when it is introduced.

Version	Available since	New features
1	Python 2.4	Sharing interned strings
2	Python 2.5	Binary representation of floats
3	Python 3.4	Support for object instancing and recursion
4	Python 3.4	Efficient representation of short strings
5	Python 3.14	Support for slice objects

Footnotes

The name of this module stems from a bit of terminology used by the designers of Modula-3 (amongst others), who use the term “marshalling” for shipping of data around in a self-contained form. Strictly speaking, “to marshal” means to convert some data from internal to external form (in an RPC buffer for instance) and “unmarshalling” for the reverse process.

dbm— Interfaces to Unix “databases”

Source code: `Lib/dbm/__init__.py`

dbm is a generic interface to variants of the DBM database:

- `dbm.sqlite3`
- `dbm.gnu`
- `dbm.ndbm`

If none of these modules are installed, the slow-but-simple implementation in `moduledbm.dumb` will be used. There is a third party interface to the Oracle Berkeley DB.

exception `dbm.error`

A tuple containing the exceptions that can be raised by each of the supported modules, with a unique exception also named `dbm.error` as the first item — the latter is used when `dbm.error` is raised.

`dbm.whichdb(filename)`

This function attempts to guess which of the several simple database modules available — `dbm.sqlite3`, `dbm.gnu`, `dbm.ndbm`, or `dbm.dumb` — should be used to open a given file. Return one of the following values: `None` if the file can't be opened because it's unreadable or doesn't exist; the empty string (`''`) if the file's format can't be guessed; a string containing the required module name, such as `'dbm.ndbm'` or `'dbm.gnu'`. Changed in version 3.11: `filename` accepts a path-like object.

`dbm.open(file, flag='r', mode=0o666)`

Parameters:

Open a database and return the corresponding database object. Parameters: `file` (path-like object) — The database file to open. If the database file already exists, the `whichdb()` function is used to determine its type and the appropriate module is used; if it does not exist, the first submodule listed above that can be imported is used. `flag` (str) — `'r'` (default): Open existing database for reading only. `'w'`: Open existing database for reading and writing. `'c'`: Open database for reading and writing, creating

`file` (path-like object) — The database file to open. If the database file already exists, the `whichdb()` function is used to determine its type and the appropriate module is used; if it does not exist, the first submodule listed above that can be imported is used. `flag` (str) — `'r'` (default): Open existing database for reading only. `'w'`: Open existing database for reading and writing. `'c'`: Open database for reading and writing, creating it if it doesn't exist. `'n'`: Always create a new, empty database, open fo

The object returned by `open()` supports the basic functionality of mutable mappings; keys and their corresponding values can be stored, retrieved, and deleted, and iteration, the `in` operator and methods `keys()`, `get()`, `setdefault()` and `clear()` are available. The `keys()` method returns a list instead of a view object. The `setdefault()` method requires two arguments.

Key and values are always stored as bytes. This means that when strings are used they are implicitly converted to the default encoding before being stored.

These objects also support being used in a `with` statement, which will automatically close them when done.

Changed in version 3.2: `get()` and `setdefault()` methods are now available for all dbm backends.

Changed in version 3.4: Added native support for the context management protocol to the objects returned by `open()`.

Changed in version 3.8: Deleting a key from a read-only database raises a database module specific exception instead of `KeyError`.

Changed in version 3.13: `clear()` methods are now available for all dbm backends.

The following example records some hostnames and a corresponding title, and then prints out the contents of the database:

```
import dbm

# Open database, creating it if necessary.
with dbm.open('cache', 'c') as db:

    # Record some values
    db[b'hello'] = b'there'
    db['www.python.org'] = 'Python Website'
    db['www.cnn.com'] = 'Cable News Network'

    # Note that the keys are considered bytes now.
    assert db[b'www.python.org'] == b'Python Website'
    # Notice how the value is now in bytes.
    assert db['www.cnn.com'] == b'Cable News Network'

    # Often-used methods of the dict interface work too.
    print(db.get('python.org', b'not present'))

    # Storing a non-string key or value will raise an exception (most
    # likely a TypeError).
    db['www.yahoo.com'] = 4

# db is automatically closed when leaving the with statement.
```

See also

Moduleshelve

Persistence module which stores non-string data.

The individual submodules are described in the following sections.

dbm.sqlite3— SQLite backend for dbm

Added in version 3.13.

Source code: `Lib/dbm/sqlite3.py`

This module uses the standard library `sqlite3` module to provide an SQLite backend for the `dbm` module. The files created by `dbm.sqlite3` can thus be opened by `sqlite3`, or any other SQLite browser, including the SQLite CLI.

Availability: not WASI.

This module does not work or is not available on WebAssembly. See [WebAssembly platforms](#) for more information.

`dbm.sqlite3.open(filename, /, flag='r', mode=0o666)`

Parameters:

`sqlite3.close()`

Open an SQLite database. Parameters: `filename` (path-like object) – The path to the database to be opened. `flag` (str) – 'r' (default): Open existing database for reading only. 'w': Open existing database for reading and writing. 'c': Open database for reading and writing, creating it if it doesn't exist. 'n': Always create a new, empty database, open for reading and writing. `mode` – The Unix file access mode of the file (default: octal 0o666), used only when the database has to be created. The returned database

filename(path-like object) – The path to the database to be opened.flag(str) –'r'(default): Open existing database for reading only.'w': Open existing database for reading and writing.'c': Open database for reading and writing, creating it if it doesn't exist.'n': Always create a new, empty database, open for reading and writing.mode– The Unix file access mode of the file (default: octal0o666), used only when the database has to be created.

Close the SQLite database.

dbm.gnu— GNU database manager

Source code:Lib/dbm/gnu.py

The dbm.gnumodule provides an interface to the GDBM library, similar to the dbm.ndbm module, but with additional functionality like crash tolerance.

■ Nota: NoteThe file formats created by dbm.gnu and dbm.ndbm are incompatible and can not be used interchangeably.

Availability: not Android, not iOS, not WASI.

This module is not supported on mobile platforms or WebAssembly platforms.

Availability: Unix.

exception dbm.gnu.error

Raised on dbm.gnu-specific errors, such as I/O errors.KeyError is raised for general mapping errors like specifying an incorrect key.

dbm.gnu.open_flags

A string of characters the flag parameter of open() supports.

dbm.gnu.open(filename,flag='r',mode=0o666,)

Parameters:

Raises:

gdbm.close()

gdbm.firstkey()

gdbm.nextkey(key)

gdbm.reorganize()

gdbm.sync()

Open a GDBM database and return a gdbmobject.Parameters:filename(path-like object) – The database file to open.flag(str) –'r'(default): Open existing database for reading only.'w': Open existing database for reading and writing.'c': Open database for reading and writing, creating it if it doesn't exist.'n': Always create a new, empty database, open for reading and writing.The following additional characters may be appended to control how the database is opened:'f': Open the database in fast mode.

filename(path-like object) – The database file to open.flag(str) –'r'(default): Open existing database for reading only.'w': Open existing database for reading and writing.'c': Open database for reading and writing, creating it if it doesn't exist.'n': Always create a new, empty database, open for reading and writing.The following additional characters may be appended to control how the database is opened:'f': Open the database in fast mode. Writes to the database will not be synchronized.'s': S

error– If an invalid flag argument is passed.

Close the GDBM database.

It's possible to loop over every key in the database using this method and the `nextkey()` method. The traversal is ordered by GDBM's internal hash values, and won't be sorted by the key values. This method returns the starting key.

Returns the key that follows `key` in the traversal. The following code prints every key in the database `db`, without having to create a list in memory that contains them all: `k=db.firstkey()while k is not None:print(k)k=db.nextkey(k)`

If you have carried out a lot of deletions and would like to shrink the space used by the GDBM file, this routine will reorganize the database. `gdbmobjects` will not shorten the length of a database file except by using this reorganization; otherwise, deleted file space will be kept and reused as new (key, value) pairs are added.

When the database has been opened in fast mode, this method forces any unwritten data to be written to the disk.

dbm.ndbm— New Database Manager

Source code: `Lib/dbm/ndbm.py`

The `dbm.ndbm` module provides an interface to the NDBM library. This module can be used with the “classic” NDBM interface or the GDBM compatibility interface.

■ Nota: Note The file formats created by `dbm.gnu` and `dbm.ndbm` are incompatible and can not be used interchangeably.

■■ Advertencia: Warning The NDBM library shipped as part of macOS has an undocumented limitation on the size of values, which can result in corrupted database files when storing values larger than this limit. Reading such corrupted files can result in a hard crash (segmentation fault).

Availability: not Android, not iOS, not WASI.

This module is not supported on mobile platforms or WebAssembly platforms.

Availability: Unix.

exception `dbm.ndbm.error`

Raised on `dbm.ndbm`-specific errors, such as I/O errors. `KeyError` is raised for general mapping errors like specifying an incorrect key.

`dbm.ndbm.library`

Name of the NDBM implementation library used.

`dbm.ndbm.open(filename, flag='r', mode=0o666, /)`

Parameters:

`ndbm.close()`

Open an NDBM database and return an `ndbmobject`. Parameters: `filename` (path-like object) – The basename of the database file (without the `.dir` or `.pag` extensions). `flag` (str) – 'r' (default): Open existing database for reading only. 'w': Open existing database for reading and writing. 'c': Open database for reading and writing, creating it if it doesn't exist. 'n': Always create a new, empty database, open for reading and writing. `mode` (int) – The Unix file access mode of the file (default: `octal0o666`), used only

`filename` (path-like object) – The basename of the database file (without the `.dir` or `.pag` extensions). `flag` (str) – 'r' (default): Open existing database for reading only. 'w': Open existing database for reading and writing. 'c': Open database for reading and writing, creating it if it doesn't exist. 'n': Always create a new, empty database, open for reading and writing. `mode` (int) – The Unix file access mode of the file (default: `octal0o666`), used only when the database has to be created.

Close the NDBM database.

dbm.dumb— Portable DBM implementation

Source code: `Lib/dbm/dumb.py`

■ Nota: Note that `dbm.dumbmodule` is intended as a last resort fallback for `dbmmodule` when a more robust module is not available. `dbm.dumbmodule` is not written for speed and is not nearly as heavily used as the other database modules.

`dbm.dumbmodule` provides a persistentdict-like interface which is written entirely in Python. Unlike otherdbmbackends, such as `dbm.gnu`, no external library is required.

`dbm.dumbmodule` defines the following:

exceptiondbm.dumb.error

Raised ondbm.dumb-specific errors, such as I/O errors.`KeyError` is raised for general mapping errors like specifying an incorrect key.

dbm.dumb.open(filename,flag='c',mode=0o666)

Parameters:

dumbdbm.close()

dumbdbm.sync()

Open `adbm.dumbdatabase`. Parameters: `filename`—The basename of the database file (without extensions). A new database creates the following files: `filename.dat` `filename.dir` `flag(str)` —'r': Open existing database for reading only. 'w': Open existing database for reading and writing. 'c'(default): Open database for reading and writing, creating it if it doesn't exist. 'n': Always create a new, empty database, open for reading and writing. `mode(int)` — The Unix file access mode of the file (default: octal0o666)

`filename`—The basename of the database file (without extensions). A new database creates the following files: `filename.dat` `filename.dir` `flag(str)` —'r': Open existing database for reading only. 'w': Open existing database for reading and writing. 'c'(default): Open database for reading and writing, creating it if it doesn't exist. 'n': Always create a new, empty database, open for reading and writing. `mode(int)` — The Unix file access mode of the file (default: octal0o666), used only when the database has

Close the database.

Synchronize the on-disk directory and data files. This method is called by `theshelve.Shelf.sync()` method.

sqlite3— DB-API 2.0 interface for SQLite databases

Source code: `Lib/sqlite3/`

SQLite is a C library that provides a lightweight disk-based database that doesn't require a separate server process and allows accessing the database using a nonstandard variant of the SQL query language. Some applications can use SQLite for internal data storage. It's also possible to prototype an application using SQLite and then port the code to a larger database such as PostgreSQL or Oracle.

The `sqlite3` module was written by Gerhard Häring. It provides an SQL interface compliant with the DB-API 2.0 specification described by PEP 249, and requires the third-party `SQLite` library.

This is an optional module. If it is missing from your copy of CPython, look for documentation from your distributor (that is, whoever provided Python to you). If you are the distributor, see [Requirements for optional modules](#).

This document includes four main sections:

- [Tutorial](#) teaches how to use the `sqlite3` module.
- [Reference](#) describes the classes and functions this module defines.
- [How-to guides](#) details how to handle specific tasks.
- [Explanation](#) provides in-depth background on transaction control.

See also

<https://www.sqlite.org>

<https://www.w3schools.com/sql/>

PEP 249- Database API Specification 2.0

The SQLite web page; the documentation describes the syntax and the available data types for the supported SQL dialect.

[Tutorial](#), reference and examples for learning SQL syntax.

PEP written by Marc-André Lemburg.

Tutorial

In this tutorial, you will create a database of Monty Python movies using basic `sqlite3` functionality. It assumes a fundamental understanding of database concepts, including [cursors](#) and [transactions](#).

First, we need to create a new database and open a database connection to allow `sqlite3` to work with it. Call `sqlite3.connect()` to create a connection to the database `tutorial.db` in the current working directory, implicitly creating it if it does not exist:

```
import sqlite3
con = sqlite3.connect("tutorial.db")
```

The returned `Connection` object `con` represents the connection to the on-disk database.

In order to execute SQL statements and fetch results from SQL queries, we will need to use a database cursor. Call `con.cursor()` to create the `Cursor`:

```
cur = con.cursor()
```

Now that we've got a database connection and a cursor, we can create a database table with columns for title, release year, and review score. For simplicity, we can just use column names in the table declaration – thanks to the flexible typing feature of SQLite, specifying the data types is optional. Execute the `CREATE TABLE` statement by calling `cur.execute(...)`:

```
cur.execute("CREATE TABLE movie(title, year, score)")
```

We can verify that the new table has been created by querying the `sqlite_master` built-in to SQLite, which should now contain an entry for the table definition (see [The Schema Table](#) for details). Execute that query by calling `cur.execute(...)`, assign the result to `res`, and call `res.fetchone()` to fetch the resulting row:

```
>>> res = cur.execute("SELECT name FROM sqlite_master")
>>> res.fetchone()
('movie',)
```

We can see that the table has been created, as the query returns a tuple containing the table's name. If we query `sqlite_master` for a non-existent table `spam`, `res.fetchone()` will return `None`:

```
>>> res = cur.execute("SELECT name FROM sqlite_master WHERE name='spam'")
>>> res.fetchone() is None
True
```

Now, add two rows of data supplied as SQL literals by executing an `INSERT` statement, once again by calling `cur.execute(...)`:

```
cur.execute("""
    INSERT INTO movie VALUES
        ('Monty Python and the Holy Grail', 1975, 8.2),
        ('And Now for Something Completely Different', 1971, 7.5)
""")
```

The `INSERT` statement implicitly opens a transaction, which needs to be committed before changes are saved in the database (see [Transaction control](#) for details). Call `con.commit()` on the connection object to commit the transaction:

```
con.commit()
```

We can verify that the data was inserted correctly by executing a `SELECT` query. Use the now-familiar `cur.execute(...)` to assign the result to `res`, and call `res.fetchall()` to return all resulting rows:

```
>>> res = cur.execute("SELECT score FROM movie")
>>> res.fetchall()
[(8.2,), (7.5,)]
```

The result is a list of two tuples, one per row, each containing that row's score value.

Now, insert three more rows by calling `cur.executemany(...)`:

```
data = [
    ("Monty Python Live at the Hollywood Bowl", 1982, 7.9),
    ("Monty Python's The Meaning of Life", 1983, 7.5),
    ("Monty Python's Life of Brian", 1979, 8.0),
]
cur.executemany("INSERT INTO movie VALUES(?, ?, ?)", data)
con.commit() # Remember to commit the transaction after executing INSERT.
```

Notice that `?` placeholders are used to bind data to the query. Always use placeholders instead of string formatting to bind Python values to SQL statements, to avoid SQL injection attacks (see [How to use placeholders to bind values in SQL queries](#) for more details).

We can verify that the new rows were inserted by executing a `SELECT` query, this time iterating over the results of the query:

```
>>> for row in cur.execute("SELECT year, title FROM movie ORDER BY year"):
...     print(row)
```

```
(1971, 'And Now for Something Completely Different')
(1975, 'Monty Python and the Holy Grail')
(1979, 'Monty Python's Life of Brian')
(1982, 'Monty Python Live at the Hollywood Bowl')
(1983, 'Monty Python's The Meaning of Life')
```

Each row is a two-item tuple of (year, title), matching the columns selected in the query.

Finally, verify that the database has been written to disk by calling `con.close()` to close the existing connection, opening a new one, creating a new cursor, then querying the database:

```
>>> con.close()
>>> new_con = sqlite3.connect("tutorial.db")
>>> new_cur = new_con.cursor()
>>> res = new_cur.execute("SELECT title, year FROM movie ORDER BY score DESC")
>>> title, year = res.fetchone()
>>> print(f'The highest scoring Monty Python movie is {title!r}, released in {year!r}')
The highest scoring Monty Python movie is 'Monty Python and the Holy Grail', released in 1975
>>> new_con.close()
```

You've now created an SQLite database using the `sqlite3` module, inserted data and retrieved values from it in multiple ways.

See also

- [How-to guides](#) for further reading:
 - [How to use placeholders to bind values in SQL queries](#)
 - [How to adapt custom Python types to SQLite values](#)
 - [How to convert SQLite values to custom Python types](#)
 - [How to use the connection context manager](#)
 - [How to create and use row factories](#)
- [Explanation](#) for in-depth background on transaction control.

Reference

Module functions

sqlite3.connect(database, timeout=5.0, detect_types=0, isolation_level='DEFERRED', check_same_thread=True, factory=sqlite3.Connection, cached_statements=128, uri=False, *, autoccommit=sqlite3.LEGACY_TRANSACTION_CONTROL)

Parameters:

Return type:

Open a connection to an SQLite database. Parameters: `database` (path-like object) – The path to the database file to be opened. You can pass `":memory:"` to create an SQLite database existing only in memory, and open a connection to it. `timeout` (float) – How many seconds the connection should wait before raising an `OperationalError` when a table is locked. If another connection opens a transaction to modify a table, that table will be locked until the transaction is committed. Default five seconds. `detect_type`

`database` (path-like object) – The path to the database file to be opened. You can pass `":memory:"` to create an SQLite database existing only in memory, and open a connection to it. `timeout` (float) – How many seconds the connection should wait before raising an `OperationalError` when a table is locked. If another connection opens a transaction to modify a table, that table will be locked until the transaction is committed. Default five seconds. `detect_types` (int) – Control whether and how data types not native

Connection

sqlite3.complete_statement(statement)

Return `True` if the string `statement` appears to contain one or more complete SQL statements. No syntactic verification or parsing of any kind is performed, other than checking that there are no unclosed string literals and

the statement is terminated by a semicolon. For example: `>>>sqlite3.complete_statement("SELECT foo FROM bar;")` True `>>>sqlite3.complete_statement("SELECT foo")` False This function may be useful during command-line input to determine if the entered text seems to form a complete SQL statement

sqlite3.enable_callback_tracebacks(flag, /)

Enable or disable callback tracebacks. By default you will not get any tracebacks in user-defined functions, aggregates, converters, authorizer callbacks etc. If you want to debug them, you can call this function with `flag` set to `True`. Afterwards, you will get tracebacks from callbacks on `sys.stderr`. Use `False` to disable the feature again. Note: Errors in user-defined function callbacks are logged as unraisable exceptions. Use `anunraisablehookhandler` for introspection of the failed callback.

sqlite3.register_adapter(type, adapter, /)

Register an `adapter` callable to adapt the Python `type` into an SQLite type. The adapter is called with a Python object of `type` as its sole argument, and must return a value of `atype` that SQLite natively understands.

sqlite3.register_converter(type_name, converter, /)

Register the `converter` callable to convert SQLite objects of `type_name` into a Python object of a specific type. The converter is invoked for all SQLite values of `type_name`; it is passed a `bytes` object and should return an object of the desired Python type. Consult the parameter `detect_types` of `connect()` for information regarding how type detection works. Note: `type_name` and the name of the type in your query are matched case-insensitively.

Module constants

sqlite3.LEGACY_TRANSACTION_CONTROL

Set `autocommit` to this constant to select old style (pre-Python 3.12) transaction control behaviour. See `Transaction control via the isolation_level attribute` for more information.

sqlite3.PARSE_DECLTYPES

Pass this flag value to the `detect_types` parameter of `connect()` to look up a converter function using the declared types for each column. The types are declared when the database table is created. `sqlite3` will look up a converter function using the first word of the declared type as the converter dictionary key. For example: `CREATE TABLE test (i integer primary key, !willlookupaconverternamed"integer" p point, !willlookupaconverternamed"point" n number(10) !willlookupaconverternamed"number")` This flag may be combine

sqlite3.PARSE_COLNAMES

Pass this flag value to the `detect_types` parameter of `connect()` to look up a converter function by using the type name, parsed from the query column name, as the converter dictionary key. The query column name must be wrapped in double quotes ("") and the type name must be wrapped in square brackets ([]). `SELECT MAX(p) as "p [point]" FROM test; !willlookupconverter"point"` This flag may be combined with `PARSE_DECLTYPES` using the `|` (bitwise or) operator.

sqlite3.SQLITE_OK

sqlite3.SQLITE_DENY

sqlite3.SQLITE_IGNORE

Flags that should be returned by the `authorizer_callback` callable passed to `Connection.set_authorizer()`, to indicate whether: Access is allowed (`SQLITE_OK`), The SQL statement should be aborted with an error (`SQLITE_DENY`) The column should be treated as a `NULL` value (`SQLITE_IGNORE`)

sqlite3.apilevel

String constant stating the supported DB-API level. Required by the DB-API. Hard-coded to "2.0".

sqlite3.paramstyle

String constant stating the type of parameter marker formatting expected by the `sqlite3` module. Required by the DB-API. Hard-coded to "qmark". Note that the named DB-API parameter style is also supported.

sqlite3.sqlite_version

Version number of the runtime SQLite library as a string.

sqlite3.sqlite_version_info

Version number of the runtime SQLite library as a tuple of integers.

sqlite3.threadafety

Integer constant required by the DB-API 2.0, stating the level of thread safety the `sqlite3` module supports. This attribute is set based on the default threading mode the underlying SQLite library is compiled with. The SQLite threading modes are: Single-thread: In this mode, all mutexes are disabled and SQLite is unsafe to use in more than a single thread at once. Multi-thread: In this mode, SQLite can be safely used by multiple threads provided that no single database connection is used simultaneously.

sqlite3.SQLITE_DBCONFIG_DEFENSIVE

sqlite3.SQLITE_DBCONFIG_DQS_DDL

sqlite3.SQLITE_DBCONFIG_DQS_DML

sqlite3.SQLITE_DBCONFIG_ENABLE_FKEY

sqlite3.SQLITE_DBCONFIG_ENABLE_FTS3_TOKENIZER

sqlite3.SQLITE_DBCONFIG_ENABLE_LOAD_EXTENSION

sqlite3.SQLITE_DBCONFIG_ENABLE_QPSG

sqlite3.SQLITE_DBCONFIG_ENABLE_TRIGGER

sqlite3.SQLITE_DBCONFIG_ENABLE_VIEW

sqlite3.SQLITE_DBCONFIG_LEGACY_ALTER_TABLE

sqlite3.SQLITE_DBCONFIG_LEGACY_FILE_FORMAT

sqlite3.SQLITE_DBCONFIG_NO_CKPT_ON_CLOSE

sqlite3.SQLITE_DBCONFIG_RESET_DATABASE

sqlite3.SQLITE_DBCONFIG_TRIGGER_EQP

sqlite3.SQLITE_DBCONFIG_TRUSTED_SCHEMA

sqlite3.SQLITE_DBCONFIG_WRITABLE_SCHEMA

https://www.sqlite.org/c3ref/c_dbconfig_defensive.html

These constants are used for the `Connection.setconfig()` and `getconfig()` methods. The availability of these constants varies depending on the version of SQLite Python was compiled with. Added in version 3.12. See also https://www.sqlite.org/c3ref/c_dbconfig_defensive.html SQLite docs: Database Connection Configuration Options

SQLite docs: Database Connection Configuration Options

Deprecated since version 3.12, removed in version 3.14: The `version` and `version_info` constants.

Connection objects

class sqlite3.Connection

cursor(factory=Cursor)

blobopen(table,column,rowid,/,*,readonly=False,name='main')

Parameters:

Raises:

Return type:

commit()

rollback()

close()

execute(sql,parameters=(),/)

executemany(sql,parameters,/)

executescript(sql_script,/)

create_function(name,narg,func,*,deterministic=False)

Parameters:

create_aggregate(name,n_arg,aggregate_class)

Parameters:

create_window_function(name,num_params,aggregate_class,/)

Parameters:

Raises:

create_collation(name,callable,/)

interrupt()

set_authorizer(authorizer_callback)

set_progress_handler(progress_handler,n)

set_trace_callback(trace_callback)

enable_load_extension(enabled,/)

load_extension(path,/,*,entrypoint=None)

Parameters:

iterdump(*,filter=None)

Parameters:

backup(target,*,pages=-1,progress=None,name='main',sleep=0.250)

Parameters:

getlimit(category,/)

Parameters:

Return type:

Raises:

setlimit(category,limit,/)

Parameters:

Return type:

Raises:

getConfig(op,/)

Parameters:

Return type:

setconfig(op,enable=True,/)

Parameters:

serialize(*,name='main')

Parameters:

Return type:

deserialize(data,/,*,name='main')

Parameters:

Raises:

autocommit

in_transaction

isolation_level

row_factory

text_factory

total_changes

Each open SQLite database is represented by a `Connection` object, which is created using `sqlite3.connect()`. Their main purpose is creating `Cursor` objects, and `Transaction` control. See also `How to use connection shortcut methods`. `How to use the connection context manager`. Changed in version 3.13: A `ResourceWarning` is emitted if `close()` is not called before a `Connection` object is deleted. An SQLite database connection has the following attributes and methods: `cursor(factory=Cursor)` Create and return a `Cursor` object. The c

Create and return a `Cursor` object. The cursor method accepts a single optional parameter `factory`. If supplied, this must be a callable returning an instance of `Cursor` or its subclasses.

Open a `Blob` handle to an existing BLOB. Parameters: `table(str)` – The name of the table where the blob is located. `column(str)` – The name of the column where the blob is located. `rowid(int)` – The row id where the blob is located. `readonly(bool)` – Set to `True` if the blob should be opened without write permissions. Defaults to `False`. `name(str)` – The name of the database where the blob is located. Defaults to `"main"`. Raises: `OperationalError`– When trying to open a blob in a `WITHOUTROWID` table. Return type: `Blob`

`table(str)` – The name of the table where the blob is located. `column(str)` – The name of the column where the blob is located. `rowid(int)` – The row id where the blob is located. `readonly(bool)` – Set to `True` if the blob should be opened without write permissions. Defaults to `False`. `name(str)` – The name of the database where the blob is located. Defaults to `"main"`.

`OperationalError`– When trying to open a blob in a `WITHOUTROWID` table.

`Blob`

Commit any pending transaction to the database. If `autocommit` is `True`, or there is no open transaction, this method does nothing. If `autocommit` is `False`, a new transaction is implicitly opened if a pending transaction was

committed by this method.

Roll back to the start of any pending transaction. If `autocommit=True`, or there is no open transaction, this method does nothing. If `autocommit=False`, a new transaction is implicitly opened if a pending transaction was rolled back by this method.

Close the database connection. If `autocommit=False`, any pending transaction is implicitly rolled back. If `autocommit=True` or `LEGACY_TRANSACTION_CONTROL`, no implicit transaction control is executed. Make sure to call `commit()` before closing to avoid losing pending changes.

Create a new `Cursor` object and call `execute()` on it with the given SQL and parameters. Return the new cursor object.

Create a new `Cursor` object and call `executemany()` on it with the given SQL and parameters. Return the new cursor object.

Create a new `Cursor` object and call `executescript()` on it with the given SQL script. Return the new cursor object.

Create or remove a user-defined SQL function. Parameters: `name(str)` – The name of the SQL function. `narg(int)` – The number of arguments the SQL function can accept. If -1, it may take any number of arguments. `func(callback|None)` – A callable that is called when the SQL function is invoked. The callable must return a type natively supported by SQLite. Set to `None` to remove an existing SQL function. `deterministic(bool)` – If `True`, the created SQL function is marked as deterministic, which allows SQLite to perform

`name(str)` – The name of the SQL function. `narg(int)` – The number of arguments the SQL function can accept. If -1, it may take any number of arguments. `func(callback|None)` – A callable that is called when the SQL function is invoked. The callable must return a type natively supported by SQLite. Set to `None` to remove an existing SQL function. `deterministic(bool)` – If `True`, the created SQL function is marked as deterministic, which allows SQLite to perform additional optimizations.

Create or remove a user-defined SQL aggregate function. Parameters: `name(str)` – The name of the SQL aggregate function. `n_arg(int)` – The number of arguments the SQL aggregate function can accept. If -1, it may take any number of arguments. `aggregate_class(class|None)` – A class must implement the following methods: `step()`: Add a row to the aggregate. `finalize()`: Return the final result of the aggregate as a type natively supported by SQLite. The number of arguments that the `step()` method must accept is controlled by `n_arg`. Set to `None` to remove an existing SQL aggregate function.

`name(str)` – The name of the SQL aggregate function. `n_arg(int)` – The number of arguments the SQL aggregate function can accept. If -1, it may take any number of arguments. `aggregate_class(class|None)` – A class must implement the following methods: `step()`: Add a row to the aggregate. `finalize()`: Return the final result of the aggregate as a type natively supported by SQLite. The number of arguments that the `step()` method must accept is controlled by `n_arg`. Set to `None` to remove an existing SQL aggregate function.

Create or remove a user-defined aggregate window function. Parameters: `name(str)` – The name of the SQL aggregate window function to create or remove. `num_params(int)` – The number of arguments the SQL aggregate window function can accept. If -1, it may take any number of arguments. `aggregate_class(class|None)` – A class that must implement the following methods: `step()`: Add a row to the current window. `value()`: Return the current value of the aggregate. `inverse()`: Remove a row from the current window. `finalize()`: Return the final result of the aggregate as a type natively supported by SQLite.

`name(str)` – The name of the SQL aggregate window function to create or remove. `num_params(int)` – The number of arguments the SQL aggregate window function can accept. If -1, it may take any number of arguments. `aggregate_class(class|None)` – A class that must implement the following methods: `step()`: Add a row to the current window. `value()`: Return the current value of the aggregate. `inverse()`: Remove a row from the current window. `finalize()`: Return the final result of the aggregate as a type natively supported by SQLite.

`NotSupportedError` – If used with a version of SQLite older than 3.25.0, which does not support aggregate window functions.

Create a collation named `name` using the collating function `callable`. `callable` is passed two string arguments, and it should return an integer: 1 if the first is ordered higher than the second, -1 if the first is ordered lower than the second, 0 if they are ordered equal. The following example shows a reverse sorting collation:

```
def collate_reverse(string1, string2):
    if string1 == string2:
        return 0
    elif string1 < string2:
        return 1
    else:
        return -1
con = sqlite3.connect(":memory:")
con.create_collation("reverse", collate_reverse)
cur = con.cursor()
```

Call this method from a different thread to abort any queries that might be executing on the connection. Aborted queries will raise an `OperationalError`.

Register `callable` `authorizer_callback` to be invoked for each attempt to access a column of a table in the database. The callback should return one of `SQLITE_OK`, `SQLITE_DENY`, or `SQLITE_IGNORE` to signal how access to the column should be handled by the underlying SQLite library. The first argument to the callback signifies what kind of operation is to be authorized. The second and third argument will be arguments or `None` depending on the first argument. The 4th argument is the name of the database ("main",

Register `callable` `progress_handler` to be invoked for every instruction of the SQLite virtual machine. This is useful if you want to get called from SQLite during long-running operations, for example to update a GUI. If you want to clear any previously installed progress handler, call the method with `None` for `progress_handler`. Returning a non-zero value from the handler function will terminate the currently executing query and cause it to raise a `DatabaseError` exception. Changed in version 3.13: Passing `prog`

Register `callable` `trace_callback` to be invoked for each SQL statement that is actually executed by the SQLite backend. The only argument passed to the callback is the statement (as `str`) that is being executed. The return value of the callback is ignored. Note that the backend does not only run statements passed to the `Cursor.execute()` methods. Other sources include the transaction management of the `sqlite3` module and the execution of triggers defined in the current database. Passing `None` as a `trace_callback` will

Enable the SQLite engine to load SQLite extensions from shared libraries if `enabled` is `True`; else, disallow loading SQLite extensions. SQLite extensions can define new functions, aggregates or whole new virtual table implementations. One well-known extension is the fulltext-search extension distributed with SQLite. Note the `sqlite3` module is not built with loadable extension support by default, because some platforms (notably macOS) have SQLite libraries which are compiled without this feature. To get

Load an SQLite extension from a shared library. Enable extension loading with `enable_load_extension()` before calling this method. Parameters: `path(str)` – The path to the SQLite extension. `entrypoint(str|None)` – Entry point name. If `None` (the default), SQLite will come up with an entry point name of its own; see the SQLite docs [Loading an Extension](#) for details. Raises an `auditing` event `sqlite3.load_extension` with arguments `connection`, `path`. Added in version 3.2. Changed in version 3.10: Added the `sqlite3.load_exten`

`path(str)` – The path to the SQLite extension. `entrypoint(str|None)` – Entry point name. If `None` (the default), SQLite will come up with an entry point name of its own; see the SQLite docs [Loading an Extension](#) for details.

Return an iterator to dump the database as SQL source code. Useful when saving an in-memory database for later restoration. Similar to the `dump` command in the `sqlite3` shell. Parameters: `filter(str|None)` – An optional LIKE pattern for database objects to dump, e.g. `prefix_%`. If `None` (the default), all database objects will be included. Example: # Convert file `example.db` to SQL dump file `dump.sql`

```
con = sqlite3.connect('example.db')
with open('dump.sql', 'w') as f:
    for line in con.iterdump():
        f.write('%s\n' % line)
con.close()
```

`filter(str|None)` – An optional LIKE pattern for database objects to dump, e.g. `prefix_%`. If `None` (the default), all database objects will be included.

Create a backup of an SQLite database. Works even if the database is being accessed by other clients or concurrently by the same connection. Parameters: `target(Connection)` – The database connection to save the backup to. `pages(int)` – The number of pages to copy at a time. If equal to or less than 0, the entire database is copied in a single step. Defaults to -1. `progress(callback|None)` – If set to a callable, it is invoked with three integer arguments for every backup iteration: the status of the last it

`target(Connection)` – The database connection to save the backup to.
`pages(int)` – The number of pages to copy at a time. If equal to or less than 0, the entire database is copied in a single step. Defaults to 1.
`progress(callback|None)` – If set to a callable, it is invoked with three integer arguments for every backup iteration: the status of the last iteration, the remaining number of pages still to be copied, and the total number of pages. Defaults to None.
`name(str)` – The name of the database to back up.

Get a connection runtime limit.
Parameters: `category(int)` – The SQLite limit category to be queried.
Return type: `int`
Raises: `ProgrammingError` – If category is not recognised by the underlying SQLite library.
Example, query the maximum length of an SQL statement for `Connection` (the default is 1000000000):
>>> `con.getlimit(sqlite3.SQLITE_LIMIT_SQL_LENGTH)`
1000000000
Added in version 3.11.

`category(int)` – The SQLite limit category to be queried.

`int`

`ProgrammingError` – If category is not recognised by the underlying SQLite library.

Set a connection runtime limit. Attempts to increase a limit above its hard upper bound are silently truncated to the hard upper bound. Regardless of whether or not the limit was changed, the prior value of the limit is returned.
Parameters: `category(int)` – The SQLite limit category to be set.
`limit(int)` – The value of the new limit. If negative, the current limit is unchanged.
Return type: `int`
Raises: `ProgrammingError` – If category is not recognised by the underlying SQLite library.
Example, limit the number of bytes in an SQL statement:

`category(int)` – The SQLite limit category to be set.
`limit(int)` – The value of the new limit. If negative, the current limit is unchanged.

`int`

`ProgrammingError` – If category is not recognised by the underlying SQLite library.

Query a boolean connection configuration option.
Parameters: `op(int)` – ASQLITE_DBCONFIG code.
Return type: `bool`
Added in version 3.12.

`op(int)` – ASQLITE_DBCONFIG code.

`bool`

Set a boolean connection configuration option.
Parameters: `op(int)` – ASQLITE_DBCONFIG code.
`enable(bool)` – True if the configuration option should be enabled (default); False if it should be disabled.
Added in version 3.12.

`op(int)` – ASQLITE_DBCONFIG code.
`enable(bool)` – True if the configuration option should be enabled (default); False if it should be disabled.

Serialize a database into a bytes object. For an ordinary on-disk database file, the serialization is just a copy of the disk file. For an in-memory database or a “temp” database, the serialization is the same sequence of bytes which would be written to disk if that database were backed up to disk.
Parameters: `name(str)` – The database name to be serialized. Defaults to “main”.
Return type: `bytes`
Note: This method is only available if the underlying SQLite library has the `serialize` API.
Added in version 3.12.

`name(str)` – The database name to be serialized. Defaults to “main”.

`bytes`

Deserialize a serialized database into a Connection. This method causes the database connection to disconnect from the database name, and reopen it as an in-memory database based on the serialization contained in data.
Parameters: `data(bytes)` – A serialized database.
`name(str)` – The database name to deserialize into. Defaults to “main”.
Raises: `OperationalError` – If the database connection is currently involved in a read transaction or a backup operation.
`DatabaseError` – If data does not contain a valid SQLite data.

`data(bytes)` – A serialized database.
`name(str)` – The database name to deserialize into. Defaults to “main”.

OperationalError– If the database connection is currently involved in a read transaction or a backup operation.
DatabaseError– If data does not contain a valid SQLite database.
OverflowError– If `len(data)` is larger than $2^{63}-1$.

This attribute controls PEP 249-compliant transaction behaviour. `autocommit` has three allowed values: `False`: Select PEP 249-compliant transaction behaviour, implying that `sqlite3` ensures a transaction is always open. `Usecommit()` and `rollback()` to close transactions. This is the recommended value of `autocommit`. `True`: Use SQLite's `autocommit` mode. `commit()` and `rollback()` have no effect in this mode. **LEGACY_TRANSACTION_CONTROL**: Pre-Python 3.12 (non-PEP 249-compliant) transaction control. See `isolation_level` for more details.

This read-only attribute corresponds to the low-level SQLite `autocommit` mode. `True` if a transaction is active (there are uncommitted changes), `False` otherwise. Added in version 3.2.

Controls the legacy transaction handling mode of `sqlite3`. If set to `None`, transactions are never implicitly opened. If set to one of `"DEFERRED"`, `"IMMEDIATE"`, or `"EXCLUSIVE"`, corresponding to the underlying SQLite transaction behaviour, implicit transaction management is performed. If not overridden by the `isolation_level` parameter of `connect()`, the default is `""`, which is an alias for `"DEFERRED"`. Note: Using `autocommit` to control transaction handling is recommended over using `isolation_level`. `isolation_level` has no effect.

The initial `row_factory` for `Cursor` objects created from this connection. Assigning to this attribute does not affect the `row_factory` of existing cursors belonging to this connection, only new ones. Is `None` by default, meaning each row is returned as a tuple. See [How to create and use row factories](#) for more details.

A callable that accepts a `bytes` parameter and returns a text representation of it. The callable is invoked for SQLite values with the `TEXT` data type. By default, this attribute is set to `str`. See [How to handle non-UTF-8 text encodings](#) for more details.

Return the total number of database rows that have been modified, inserted, or deleted since the database connection was opened.

Cursor objects

A `Cursor` object represents a database cursor which is used to execute SQL statements, and manage the context of a fetch operation. Cursors are created using `Connection.cursor()`, or by using any of the connection shortcut methods.

Cursor objects are iterators, meaning that if you execute a `SELECT` query, you can simply iterate over the cursor to fetch the resulting rows:

```
for row in cur.execute("SELECT t FROM data"):
    print(row)
```

`class sqlite3.Cursor`

`execute(sql, parameters=(),/)`

Parameters:

Raises:

`executemany(sql, parameters,/)`

Parameters:

Raises:

`executescript(sql_script,/)`

`fetchone()`

fetchmany(size=cursor.arraysize)

fetchall()

close()

setinputsizes(sizes, /)

setoutputsize(size, column=None, /)

arraysize

connection

description

lastrowid

rowcount

row_factory

ACursorinstance has the following attributes and methods.**execute(sql, parameters=(), /)**Execute a single SQL statement, optionally binding Python values using placeholders. **Parameters:** sql(str) – A single SQL statement. **parameters(dict|sequence)** – Python values to bind to placeholders in sql. Adictif named placeholders are used. Asequenceif unnamed placeholders are used. SeeHow to use placeholders to bind values in SQL queries. **Raises:** ProgrammingError– When sql contains more than one SQL statement. When named

Execute a single SQL statement, optionally binding Python values using placeholders. **Parameters:** sql(str) – A single SQL statement. **parameters(dict|sequence)** – Python values to bind to placeholders in sql. Adictif named placeholders are used. Asequenceif unnamed placeholders are used. SeeHow to use placeholders to bind values in SQL queries. **Raises:** ProgrammingError– When sql contains more than one SQL statement. When named placeholders are used and parameters is a sequence instead of adict. Ifautocommit is LEG

sql(str) – A single SQL statement. **parameters(dict|sequence)** – Python values to bind to placeholders in sql. Adictif named placeholders are used. Asequenceif unnamed placeholders are used. SeeHow to use placeholders to bind values in SQL queries.

ProgrammingError– When sql contains more than one SQL statement. When named placeholders are used and parameters is a sequence instead of adict.

For every item in parameters, repeatedly execute the parameterized DML SQL statements sql. Uses the same implicit transaction handling as execute(). **Parameters:** sql(str) – A single SQL DML statement. **parameters(iterable)** – An iterable of parameters to bind with the placeholders in sql. SeeHow to use placeholders to bind values in SQL queries. **Raises:** ProgrammingError– When sql contains more than one SQL statement or is not a DML statement, When named placeholders are used and the items in parameters are sequences in

sql(str) – A single SQL DML statement. **parameters(iterable)** – An iterable of parameters to bind with the placeholders in sql. SeeHow to use placeholders to bind values in SQL queries.

ProgrammingError– When sql contains more than one SQL statement or is not a DML statement, When named placeholders are used and the items in parameters are sequences instead of dicts.

Execute the SQL statements in sql_script. If the autocommit is LEGACY_TRANSACTION_CONTROL and there is a pending transaction, an implicit COMMIT statement is executed first. No other implicit transaction control is performed; any transaction control must be added to sql_script. sql_script must be a string. Example: # cur is an sqlite3.Cursor object cur.executescript("""BEGIN; CREATE TABLE person(firstname, lastname, age); CREATE TABLE book(title, author, published); CREATE TABLE publisher(name, address); COMMIT;""

If row_factory is None, return the next row query result set as a tuple. Else, pass it to the row factory and return its result. Return None if no more data is available.

Return the next set of rows of a query result as a list. Return an empty list if no more rows are available. The number of rows to fetch per call is specified by the `size` parameter. If `size` is not given, `arraysize` determines the number of rows to be fetched. If fewer than `size` rows are available, as many rows as are available are returned. Note there are performance considerations involved with the `size` parameter. For optimal performance, it is usually best to use the `arraysize` attribute. If the `size` parameter

Return all (remaining) rows of a query result as a list. Return an empty list if no rows are available. Note that the `arraysize` attribute can affect the performance of this operation.

Close the cursor now (rather than whenever `__del__` is called). The cursor will be unusable from this point forward; a `ProgrammingError` exception will be raised if any operation is attempted with the cursor.

Required by the DB-API. Does nothing in `sqlite3`.

Required by the DB-API. Does nothing in `sqlite3`.

Read/write attribute that controls the number of rows returned by `fetchmany()`. The default value is 1 which means a single row would be fetched per call. Changed in version 3.14.1: Negative values are rejected by raising `ValueError`.

Read-only attribute that provides the `SQLite` database connection belonging to the cursor. A `Cursor` object created by calling `con.cursor()` will have a `connection` attribute that refers to `con`:

```
>>> con = sqlite3.connect(":memory:")
>>> cur = con.cursor()
>>> cur.connection == con
True
>>> con.close()
```

Read-only attribute that provides the column names of the last query. To remain compatible with the Python DB API, it returns a 7-tuple for each column where the last six items of each tuple are `None`. It is set for `SELECT` statements without any matching rows as well.

Read-only attribute that provides the row id of the last inserted row. It is only updated after successful `INSERT` or `REPLACE` statements using the `execute()` method. For other statements, after `executemany()` or `executescript()`, or if the insertion failed, the value of `lastrowid` is left unchanged. The initial value of `lastrowid` is `None`. Note Inserts into `WITHOUTROWID` tables are not recorded. Changed in version 3.6: Added support for the `REPLACE` statement.

Read-only attribute that provides the number of modified rows for `INSERT`, `UPDATE`, `DELETE`, and `REPLACE` statements; is -1 for other statements, including CTE queries. It is only updated by the `execute()` and `executemany()` methods, after the statement has run to completion. This means that any resulting rows must be fetched in order for `rowcount` to be updated.

Control how a row fetched from this `Cursor` is represented. If `None`, a row is represented as a tuple. Can be set to the included `sqlite3.Row`; or a callable that accepts two arguments, a `Cursor` object and the tuple of row values, and returns a custom object representing an `SQLite` row. Defaults to what `Connection.row_factory` was set to when the `Cursor` was created. Assigning to this attribute does not affect `Connection.row_factory` of the parent connection. See [How to create and use row factories](#) for more details.

Row objects

class `sqlite3.Row`

`keys()`

A `Row` instance serves as a highly optimized `row_factory` for `Connection` objects. It supports iteration, equality testing, `len()`, and mapping access by column name and index. Two `Row` objects compare equal if they have identical column names and values. See [How to create and use row factories](#) for more details. `keys()` Return a list of column names as strings. Immediately after a query, it is the first member of each tuple in `Cursor.description`. Changed in version 3.5: Added support of slicing.

Return a list of column names as strings. Immediately after a query, it is the first member of each tuple in `Cursor.description`.

Blob objects

class sqlite3.Blob

close()

read(length=-1, /)

write(data, /)

tell()

seek(offset, origin=os.SEEK_SET, /)

Added in version 3.11. A `blob` instance is a file-like object that can read and write data in an SQLite BLOB. Call `len(blob)` to get the size (number of bytes) of the blob. Use `indices` and `slices` for direct access to the blob data. Use the `Blob` as a context manager to ensure that the blob handle is closed after use. `con = sqlite3.connect(":memory:")`
`con.execute("CREATE TABLE test(blob_col BLOB)")`
`con.execute("INSERT INTO test(blob_col) VALUES(zeroblob(13))")`
Write to our blob, using two write operations:
`with con.blob`

Close the blob. The blob will be unusable from this point onward. An `Error` (or subclass) exception will be raised if any further operation is attempted with the blob.

Read `length` bytes of data from the blob at the current offset position. If the end of the blob is reached, the data up to EOF will be returned. When `length` is not specified, or is negative, `read()` will read until the end of the blob.

Write `data` to the blob at the current offset. This function cannot change the blob length. Writing beyond the end of the blob will raise `ValueError`.

Return the current access position of the blob.

Set the current access position of the blob to `offset`. The `origin` argument defaults to `os.SEEK_SET` (absolute blob positioning). Other values for `origin` are `os.SEEK_CUR` (seek relative to the current position) and `os.SEEK_END` (seek relative to the blob's end).

PrepareProtocol objects

class sqlite3.PrepareProtocol

The `PrepareProtocol` type's single purpose is to act as a PEP 246 style adaption protocol for objects that can adapt themselves to native SQLite types.

Exceptions

The exception hierarchy is defined by the DB-API 2.0 (PEP 249).

exceptions sqlite3.Warning

This exception is not currently raised by the `sqlite3` module, but may be raised by applications using `sqlite3`, for example if a user-defined function truncates data while inserting. `Warning` is a subclass of `Exception`.

exceptions sqlite3.Error

sqlite_errorcode

sqlite_errname

The base class of the other exceptions in this module. Use this to catch all errors with one single `except` statement. `Error` is a subclass of `Exception`. If the exception originated from within the SQLite library, the following two attributes are added to the exception: `sqlite_errorcode` The numeric error code from the SQLite API. Added in version 3.11. `sqlite_errname` The symbolic name of the numeric error code from the SQLite API. Added in version 3.11.

The numeric error code from the SQLite API. Added in version 3.11.

The symbolic name of the numeric error code from the SQLite API Added in version 3.11.

exceptions.sqlite3.InterfaceError

Exception raised for misuse of the low-level SQLite C API. In other words, if this exception is raised, it probably indicates a bug in the sqlite3 module. InterfaceError is a subclass of Error.

exceptions.sqlite3.DatabaseError

Exception raised for errors that are related to the database. This serves as the base exception for several types of database errors. It is only raised implicitly through the specialised subclasses. DatabaseError is a subclass of Error.

exceptions.sqlite3.DataError

Exception raised for errors caused by problems with the processed data, like numeric values out of range, and strings which are too long. DataError is a subclass of DatabaseError.

exceptions.sqlite3.OperationalError

Exception raised for errors that are related to the database's operation, and not necessarily under the control of the programmer. For example, the database path is not found, or a transaction could not be processed. OperationalError is a subclass of DatabaseError.

exceptions.sqlite3.IntegrityError

Exception raised when the relational integrity of the database is affected, e.g. a foreign key check fails. It is a subclass of DatabaseError.

exceptions.sqlite3.InternalError

Exception raised when SQLite encounters an internal error. If this is raised, it may indicate that there is a problem with the runtime SQLite library. InternalError is a subclass of DatabaseError.

exceptions.sqlite3.ProgrammingError

Exception raised for sqlite3 API programming errors, for example supplying the wrong number of bindings to a query, or trying to operate on a closed Connection. ProgrammingError is a subclass of DatabaseError.

exceptions.sqlite3.NotSupportedError

Exception raised in case a method or database API is not supported by the underlying SQLite library. For example, setting deterministic to True in create_function(), if the underlying SQLite library does not support deterministic functions. NotSupportedError is a subclass of DatabaseError.

SQLite and Python types

SQLite natively supports the following types: NULL, INTEGER, REAL, TEXT, BLOB.

The following Python types can thus be sent to SQLite without any problem:

Python type		SQLite type		None		NULL		int		INTEGER		float		REAL		str		TEXT		bytes		BLOB
-------------	--	-------------	--	------	--	------	--	-----	--	---------	--	-------	--	------	--	-----	--	------	--	-------	--	------

This is how SQLite types are converted to Python types by default:

SQLite type		Python type		NULL		None		INTEGER		int		REAL		float		TEXT		depends on	text_factory		,	str		by default		BLOB		bytes
-------------	--	-------------	--	------	--	------	--	---------	--	-----	--	------	--	-------	--	------	--	------------	--------------	--	---	-----	--	------------	--	------	--	-------

The type system of the sqlite3 module is extensible in two ways: you can store additional Python types in an SQLite database via object adapters, and you can let the sqlite3 module convert SQLite types to Python types via converters.

Default adapters and converters (deprecated)

■ Nota: NoteThe default adapters and converters are deprecated as of Python 3.12. Instead, use theAdapter and converter recipesand tailor them to your needs.

The deprecated default adapters and converters consist of:

- An adapter fordatetime.dateobjects tostringsinISO 8601format.
- An adapter fordatetime.datetimeobjects to strings in ISO 8601 format.
- A converter fordeclared“date” types todatetime.dateobjects.
- A converter for declared “timestamp” types todatetime.datetimeobjects. Fractional parts will be truncated to 6 digits (microsecond precision).

■ Nota: NoteThe default “timestamp” converter ignores UTC offsets in the database and always returns a naivedatetime.datetimeobject. To preserve UTC offsets in timestamps, either leave converters disabled, or register an offset-aware converter withregister_converter().

Deprecated since version 3.12.

Command-line interface

Thesqlite3module can be invoked as a script, using the interpreter’s-mswitch, in order to provide a simple SQLite shell. The argument signature is as follows:

```
python -m sqlite3 [-h] [-v] [filename] [sql]
```

Type.quit or CTRL-D to exit the shell.

-h,--help

Print CLI help.

-v,--version

Print underlying SQLite library version.

Added in version 3.12.

How-to guides

How to use placeholders to bind values in SQL queries

SQL operations usually need to use values from Python variables. However, beware of using Python’s string operations to assemble queries, as they are vulnerable toSQL injection attacks. For example, an attacker can simply close the single quote and injectORTRUEto select all rows:

```
>>> # Never do this -- insecure!
>>> symbol = input()
' OR TRUE; --
>>> sql = "SELECT * FROM stocks WHERE symbol = '%s'" % symbol
>>> print(sql)
SELECT * FROM stocks WHERE symbol = '' OR TRUE; --
>>> cur.execute(sql)
```

Instead, use the DB-API’s parameter substitution. To insert a variable into a query string, use a placeholder in the string, and substitute the actual values into the query by providing them as atupleof values to the second argument of the cursor’sexecute()method.

An SQL statement may use one of two kinds of placeholders: question marks (qmark style) or named placeholders (named style). For the qmark style,parametersmust be asequencewhose length must match the number of placeholders, or aProgrammingErroris raised. For the named style,parametersmust be an instance of adict(or a subclass), which must contain keys for all named parameters; any extra items are ignored. Here’s an

example of both styles:

```
con = sqlite3.connect(":memory:")
cur = con.execute("CREATE TABLE lang(name, first_appeared)")

# This is the named style used with executemany():
data = (
    {"name": "C", "year": 1972},
    {"name": "Fortran", "year": 1957},
    {"name": "Python", "year": 1991},
    {"name": "Go", "year": 2009},
)
cur.executemany("INSERT INTO lang VALUES(:name, :year)", data)

# This is the qmark style used in a SELECT query:
params = (1972,)
cur.execute("SELECT * FROM lang WHERE first_appeared = ?", params)
print(cur.fetchall())
con.close()
```

■ Nota: NotePEP 249numeric placeholders arenotsupported. If used, they will be interpreted as named placeholders.

How to adapt custom Python types to SQLite values

SQLite supports only a limited set of data types natively. To store custom Python types in SQLite databases, adapt them to one of the Python types SQLite natively understands.

There are two ways to adapt Python objects to SQLite types: letting your object adapt itself, or using an adapter callable. The latter will take precedence above the former. For a library that exports a custom type, it may make sense to enable that type to adapt itself. As an application developer, it may make more sense to take direct control by registering custom adapter functions.

How to write adaptable objects

Suppose we have a `Point` class that represents a pair of coordinates, `x` and `y`, in a Cartesian coordinate system. The coordinate pair will be stored as a text string in the database, using a semicolon to separate the coordinates. This can be implemented by adding a `__conform__(self, protocol)` method which returns the adapted value. The object passed to `protocol` will be of type `PrepareProtocol`.

```
class Point:
    def __init__(self, x, y):
        self.x, self.y = x, y

    def __conform__(self, protocol):
        if protocol is sqlite3.PrepareProtocol:
            return f"{self.x};{self.y}"

con = sqlite3.connect(":memory:")
cur = con.cursor()

cur.execute("SELECT ?", (Point(4.0, -3.2),))
print(cur.fetchone()[0])
con.close()
```

How to register adapter callables

The other possibility is to create a function that converts the Python object to an SQLite-compatible type. This function can then be registered using `register_adapter()`.

```
class Point:
    def __init__(self, x, y):
        self.x, self.y = x, y
```

```
def adapt_point(point):
    return f"{point.x};{point.y}"

sqlite3.register_adapter(Point, adapt_point)

con = sqlite3.connect(":memory:")
cur = con.cursor()

cur.execute("SELECT ?", (Point(1.0, 2.5),))
print(cur.fetchone()[0])
con.close()
```

How to convert SQLite values to custom Python types

Writing an adapter lets you convert from custom Python types to SQLite values. To be able to convert from SQLite values to custom Python types, we use converters.

Let's go back to the `Point` class. We stored the x and y coordinates separated via semicolons as strings in SQLite.

First, we'll define a converter function that accepts the string as a parameter and constructs a `Point` object from it.

■ **Note:** `NoteConverter` functions are always passed a bytes object, no matter the underlying SQLite data type.

```
def convert_point(s):
    x, y = map(float, s.split(b";"))
    return Point(x, y)
```

We now need to tell SQLite when it should convert a given SQLite value. This is done when connecting to a database, using the `detect_types` parameter of `connect()`. There are three options:

- Implicit: `setdetect_type` to `PARSE_DECLTYPES`
- Explicit: `setdetect_type` to `PARSE_COLNAMES`
- Both: `setdetect_type` to `sqlite3.PARSE_DECLTYPES|sqlite3.PARSE_COLNAMES`. Column names take precedence over declared types.

The following example illustrates the implicit and explicit approaches:

```
class Point:
    def __init__(self, x, y):
        self.x, self.y = x, y

    def __repr__(self):
        return f"Point({self.x}, {self.y})"

def adapt_point(point):
    return f"{point.x};{point.y}"

def convert_point(s):
    x, y = list(map(float, s.split(b";")))
    return Point(x, y)

# Register the adapter and converter
sqlite3.register_adapter(Point, adapt_point)
sqlite3.register_converter("point", convert_point)

# 1) Parse using declared types
p = Point(4.0, -3.2)
con = sqlite3.connect(":memory:", detect_types=sqlite3.PARSE_DECLTYPES)
cur = con.execute("CREATE TABLE test(p point)")

cur.execute("INSERT INTO test(p) VALUES(?)", (p,))
cur.execute("SELECT p FROM test")
print("with declared types:", cur.fetchone()[0])
```

```

cur.close()
con.close()

# 2) Parse using column names
con = sqlite3.connect(":memory:", detect_types=sqlite3.PARSE_COLNAMES)
cur = con.execute("CREATE TABLE test(p)")

cur.execute("INSERT INTO test(p) VALUES(?)", (p,))
cur.execute('SELECT p AS "p [point]" FROM test')
print("with column names:", cur.fetchone()[0])
cur.close()
con.close()

```

Adapter and converter recipes

This section shows recipes for common adapters and converters.

```

import datetime as dt
import sqlite3

def adapt_date_iso(val):
    """Adapt datetime.date to ISO 8601 date."""
    return val.isoformat()

def adapt_datetime_iso(val):
    """Adapt datetime.datetime to timezone-naive ISO 8601 date."""
    return val.replace(tzinfo=None).isoformat()

def adapt_datetime_epoch(val):
    """Adapt datetime.datetime to Unix timestamp."""
    return int(val.timestamp())

sqlite3.register_adapter(dt.date, adapt_date_iso)
sqlite3.register_adapter(dt.datetime, adapt_datetime_iso)
sqlite3.register_adapter(dt.datetime, adapt_datetime_epoch)

def convert_date(val):
    """Convert ISO 8601 date to datetime.date object."""
    return dt.date.fromisoformat(val.decode())

def convert_datetime(val):
    """Convert ISO 8601 datetime to datetime.datetime object."""
    return dt.datetime.fromisoformat(val.decode())

def convert_timestamp(val):
    """Convert Unix epoch timestamp to datetime.datetime object."""
    return dt.datetime.fromtimestamp(int(val))

sqlite3.register_converter("date", convert_date)
sqlite3.register_converter("datetime", convert_datetime)
sqlite3.register_converter("timestamp", convert_timestamp)

```

How to use connection shortcut methods

Using the `execute()`, `executemany()`, and `executescript()` methods of the `Connection` class, your code can be written more concisely because you don't have to create the (often superfluous) `Cursor` objects explicitly. Instead, the `Cursor` objects are created implicitly and these shortcut methods return the cursor objects. This way, you can execute a `SELECT` statement and iterate over it directly using only a single call on the `Connection` object.

```

# Create and fill the table.
con = sqlite3.connect(":memory:")
con.execute("CREATE TABLE lang(name, first_appeared)")
data = [
    ("C++", 1985),
    ("Objective-C", 1984),

```

```

]
con.executemany("INSERT INTO lang(name, first_appeared) VALUES(?, ?)", data)

# Print the table contents
for row in con.execute("SELECT name, first_appeared FROM lang"):
    print(row)

print("I just deleted", con.execute("DELETE FROM lang").rowcount, "rows")

# close() is not a shortcut method and it's not called automatically;
# the connection object should be closed manually
con.close()

```

How to use the connection context manager

AConnectionObject can be used as a context manager that automatically commits or rolls back open transactions when leaving the body of the context manager. If the body of the with statement finishes without exceptions, the transaction is committed. If this commit fails, or if the body of the with statement raises an uncaught exception, the transaction is rolled back. If autocommits is False, a new transaction is implicitly opened after committing or rolling back.

If there is no open transaction upon leaving the body of the with statement, or if autocommits is True, the context manager does nothing.

■ **Nota:** NoteThe context manager neither implicitly opens a new transaction nor closes the connection. If you need a closing context manager, consider using contextlib.closing().

```

con = sqlite3.connect(":memory:")
con.execute("CREATE TABLE lang(id INTEGER PRIMARY KEY, name VARCHAR UNIQUE)")

# Successful, con.commit() is called automatically afterwards
with con:
    con.execute("INSERT INTO lang(name) VALUES(?)", ("Python",))

# con.rollback() is called after the with block finishes with an exception,
# the exception is still raised and must be caught
try:
    with con:
        con.execute("INSERT INTO lang(name) VALUES(?)", ("Python",))
except sqlite3.IntegrityError:
    print("couldn't add Python twice")

# Connection object used as context manager only commits or rollbacks transactions,
# so the connection object should be closed manually
con.close()

```

How to work with SQLite URIs

Some useful URI tricks include:

- Open a database in read-only mode:

```

>>> con = sqlite3.connect("file:tutorial.db?mode=ro", uri=True)
>>> con.execute("CREATE TABLE readonly(data)")
Traceback (most recent call last):
OperationalError: attempt to write a readonly database
>>> con.close()

```

- Do not implicitly create a new database file if it does not already exist; will raise OperationalError if unable to create a new file:

```

>>> con = sqlite3.connect("file:nosuchdb.db?mode=rw", uri=True)
Traceback (most recent call last):
OperationalError: unable to open database file

```

- Create a shared named in-memory database:

```
db = "file:mem1?mode=memory&cache=shared"
con1 = sqlite3.connect(db, uri=True)
con2 = sqlite3.connect(db, uri=True)
with con1:
    con1.execute("CREATE TABLE shared(data)")
    con1.execute("INSERT INTO shared VALUES(28)")
res = con2.execute("SELECT data FROM shared")
assert res.fetchone() == (28,)

con1.close()
con2.close()
```

More information about this feature, including a list of parameters, can be found in the [SQLite URI documentation](#).

How to create and use row factories

By default, `sqlite3` represents each row as a tuple. If a tuple does not suit your needs, you can use the `sqlite3.Row` class or a custom `row_factory`.

While `row_factory` exists as an attribute both on the `Cursor` and the `Connection`, it is recommended to set `Connection.row_factory`, so all cursors created from the connection will use the same row factory.

`Row` provides indexed and case-insensitive named access to columns, with minimal memory overhead and performance impact over a tuple. To use `Row` as a row factory, assign it to the `row_factory` attribute:

```
>>> con = sqlite3.connect(":memory:")
>>> con.row_factory = sqlite3.Row
```

Queries now return `Row` objects:

```
>>> res = con.execute("SELECT 'Earth' AS name, 6378 AS radius")
>>> row = res.fetchone()
>>> row.keys()
['name', 'radius']
>>> row[0]          # Access by index.
'Earth'
>>> row["name"]     # Access by name.
'Earth'
>>> row["RADIUS"]   # Column names are case-insensitive.
6378
>>> con.close()
```

■ **Nota:** Note the `FROM` clause can be omitted in the `SELECT` statement, as in the above example. In such cases, `SQLite` returns a single row with columns defined by expressions, e.g. literals, with the given aliases `expr AS alias`.

You can create a custom `row_factory` that returns each row as a dict, with column names mapped to values:

```
def dict_factory(cursor, row):
    fields = [column[0] for column in cursor.description]
    return {key: value for key, value in zip(fields, row)}
```

Using it, queries now return a dict instead of a tuple:

```
>>> con = sqlite3.connect(":memory:")
>>> con.row_factory = dict_factory
>>> for row in con.execute("SELECT 1 AS a, 2 AS b"):
...     print(row)
{'a': 1, 'b': 2}
>>> con.close()
```

The following row factory returns a named tuple:

```
from collections import namedtuple
```

```
def namedtuple_factory(cursor, row):
    fields = [column[0] for column in cursor.description]
    cls = namedtuple("Row", fields)
    return cls._make(row)
```

`namedtuple_factory()` can be used as follows:

```
>>> con = sqlite3.connect(":memory:")
>>> con.row_factory = namedtuple_factory
>>> cur = con.execute("SELECT 1 AS a, 2 AS b")
>>> row = cur.fetchone()
>>> row
Row(a=1, b=2)
>>> row[0] # Indexed access.
1
>>> row.b # Attribute access.
2
>>> con.close()
```

With some adjustments, the above recipe can be adapted to use `dataclass`, or any other custom class, instead of `namedtuple`.

How to handle non-UTF-8 text encodings

By default, `sqlite3` uses `str` to adapt SQLite values with the `TEXT` data type. This works well for UTF-8 encoded text, but it might fail for other encodings and invalid UTF-8. You can use a custom `text_factory` to handle such cases.

Because of SQLite's flexible typing, it is not uncommon to encounter table columns with the `TEXT` data type containing non-UTF-8 encodings, or even arbitrary data. To demonstrate, let's assume we have a database with ISO-8859-2 (Latin-2) encoded text, for example a table of Czech-English dictionary entries. Assuming we now have a `Connection` instance `con` connected to this database, we can decode the Latin-2 encoded text using `this_text_factory`:

```
con.text_factory = lambda data: str(data, encoding="latin2")
```

For invalid UTF-8 or arbitrary data in stored `TEXT` table columns, you can use the following technique, borrowed from the `Unicode HOWTO`:

```
con.text_factory = lambda data: str(data, errors="surrogateescape")
```

■ **Nota:** Note that the `sqlite3` module API does not support strings containing surrogates.

See also

[Unicode HOWTO](#)

Explanation

Transaction control

`sqlite3` offers multiple methods of controlling whether, when and how database transactions are opened and closed. Transaction control via the `autocommit` attribute is recommended, while Transaction control via the `isolation_level` attribute retains the pre-Python 3.12 behaviour.

Transaction control via the `autocommit` attribute

The recommended way of controlling transaction behaviour is through the `Connection.autocommit` attribute, which should preferably be set using the `autocommit` parameter of `connect()`.

It is suggested to set `autocommit` to `False`, which implies PEP 249-compliant transaction control. This means:

- `sqlite3` ensures that a transaction is always open, so `connect()`, `Connection.commit()`, and `Connection.rollback()` will implicitly open a new transaction (immediately after closing the pending one, for the latter two). `sqlite3` uses `BEGIN DEFERRED` statements when opening transactions.
- Transactions should be committed explicitly using `commit()`.
- Transactions should be rolled back explicitly using `rollback()`.
- An implicit rollback is performed if the database is `close()`-ed with pending changes.

Set `autocommit` to `True` to enable SQLite's `autocommit` mode. In this mode, `Connection.commit()` and `Connection.rollback()` have no effect. Note that SQLite's `autocommit` mode is distinct from the PEP 249-compliant `Connection.autocommit` attribute; use `Connection.in_transaction` to query the low-level SQLite `autocommit` mode.

Set `autocommit` to `LEGACY_TRANSACTION_CONTROL` to leave transaction control behaviour to the `Connection.isolation_level` attribute. See [Transaction control via the `isolation\_level` attribute](#) for more information.

Transaction control via the `isolation_level` attribute

■ **Note:** The recommended way of controlling transactions is via the `autocommit` attribute. See [Transaction control via the `autocommit` attribute](#).

If `Connection.autocommit` is set to `LEGACY_TRANSACTION_CONTROL` (the default), transaction behaviour is controlled using the `Connection.isolation_level` attribute. Otherwise, `isolation_level` has no effect.

If the connection attribute `isolation_level` is not `None`, new transactions are implicitly opened before `execute()` and `executemany()` executes `INSERT`, `UPDATE`, `DELETE`, or `REPLACE` statements; for other statements, no implicit transaction handling is performed. Use the `commit()` and `rollback()` methods to respectively commit and roll back pending transactions. You can choose the underlying SQLite transaction behaviour— that is, whether and what type of `BEGIN` statements `sqlite3` implicitly executes – via the `isolation_level` attribute.

If `isolation_level` is set to `None`, no transactions are implicitly opened at all. This leaves the underlying SQLite library in `autocommit` mode, but also allows the user to perform their own transaction handling using explicit SQL statements. The underlying SQLite library `autocommit` mode can be queried using the `in_transaction` attribute.

The `executescript()` method implicitly commits any pending transaction before execution of the given SQL script, regardless of the value of `isolation_level`.

Changed in version 3.6: `sqlite3` used to implicitly commit an open transaction before DDL statements. This is no longer the case.

Changed in version 3.12: The recommended way of controlling transactions is now via the `autocommit` attribute.

Data Compression and Archiving

The modules described in this chapter support data compression with the `zlib`, `gzip`, `bzip2`, `lzma`, and `zstd` algorithms, and the creation of ZIP- and tar-format archives. See also Archiving operations provided by the `shutil` module.

- `Thecompressionpackage`
- `compression.zstd`— Compression compatible with the Zstandard formatExceptionsReading and writing compressed filesCompressing and decompressing data in memoryZstandard dictionariesAdvanced parameter controlMiscellaneousExamples
- `zlib`— Compression compatible with `gzip`
- `gzip`— Support for `gzip` filesExamples of usageCommand-line interfaceCommand-line options
- `bz2`— Support for `bzip2` compression(De)compression of filesIncremental (de)compressionOne-shot (de)compressionExamples of usage
- `lzma`— Compression using the LZMA algorithmReading and writing compressed filesCompressing and decompressing data in memoryMiscellaneousSpecifying custom filter chainsExamples
- `zipfile`— Work with ZIP archivesZipFile objectsPath objectsPyZipFile objectsZipInfo objectsCommand-line interfaceCommand-line optionsDecompression pitfallsFrom file itselfFile system limitationsResources limitationsInterruptionDefault behaviors of extraction
- `tarfile`— Read and write tar archive filesTarFile ObjectsTarInfo ObjectsExtraction filtersDefault named filtersFilter errorsHints for further verificationSupporting older Python versionsStateful extraction filter exampleCommand-Line InterfaceCommand-line optionsExamplesReading examplesWriting examplesSupported tar formatsUnicode issues

Thecompressionpackage

Added in version 3.14.

Thecompressionpackage contains the canonical compression modules containing interfaces to several different compression algorithms. Some of these modules have historically been available as separate modules; those will continue to be available under their original names for compatibility reasons, and will not be removed without a deprecation cycle. The use of modules incompressionis encouraged where practical.

- `compression.bz2`– Re-exports**bz2**
- `compression.gzip`– Re-exports**gzip**
- `compression.lzma`– Re-exports**lzma**
- `compression.zlib`– Re-exports**zlib**
- `compression.zstd`– Wrapper for the Zstandard compression library

compression.zstd— Compression compatible with the Zstandard format

Added in version 3.14.

Source code: `Lib/compression/zstd/__init__.py`

This module provides classes and functions for compressing and decompressing data using the Zstandard (orzstd) compression algorithm. The `zstd` manual describes Zstandard as “a fast lossless compression algorithm, targeting real-time compression scenarios at zlib-level and better compression ratios.” Also included is a file interface that supports reading and writing the contents of `.zst` files created by the `zstd` utility, as well as raw zstd compressed streams.

The `compression.zstd` module contains:

- The `open()` function and `ZstdFile` class for reading and writing compressed files.
- The `ZstdCompressor` and `ZstdDecompressor` classes for incremental (de)compression.
- The `compress()` and `decompress()` functions for one-shot (de)compression.
- The `train_dict()` and `finalize_dict()` functions and the `ZstdDict` class to train and manage Zstandard dictionaries.
- The `CompressionParameter`, `DecompressionParameter`, and `Strategy` classes for setting advanced (de)compression parameters.

This is an optional module. If it is missing from your copy of CPython, look for documentation from your distributor (that is, whoever provided Python to you). If you are the distributor, see [Requirements for optional modules](#).

Exceptions

`compression.zstd.ZstdError`

This exception is raised when an error occurs during compression or decompression, or while initializing the (de)compressor state.

Reading and writing compressed files

`compression.zstd.open(file,/,mode='rb',*,level=None,options=None,zstd_dict=None,encoding=None,errors=None,newline=None)`

Open a Zstandard-compressed file in binary or text mode, returning a file object. The `file` argument can be either a file name (given as a `str`, `bytes` or path-like object), in which case the named file is opened, or it can be an existing file object to read from or write to. The `mode` argument can be either `'rb'` for reading (default), `'wb'` for overwriting, `'ab'` for appending, or `'xb'` for exclusive creation. These can equivalently be given as `'r'`, `'w'`, `'a'`, and `'x'` respectively. You may also open in text mode with `'rt'`, `'wt'`

`class compression.zstd.ZstdFile(file,/,mode='rb',*,level=None,options=None,zstd_dict=None)`

`peek(size=-1)`

`mode`

`name`

Open a Zstandard-compressed file in binary mode. `ZstdFile` can wrap an already-open file object, or operate directly on a named file. The `file` argument specifies either the file object to wrap, or the name of the file to open (as `str`, `bytes` or path-like object). If wrapping an existing file object, the wrapped file will not be closed when the `ZstdFile` is closed. The `mode` argument can be either `'rb'` for reading (default), `'wb'` for overwriting, `'xb'` for exclusive creation, or `'ab'` for appending. These can equivalently

Return buffered data without advancing the file position. At least one byte of data will be returned, unless EOF has been reached. The exact number of bytes returned is unspecified (the `size` argument is ignored). Note While calling `peek()` does not change the file position of the `ZstdFile`, it may change the position of the underlying file object (for example, if the `ZstdFile` was constructed by passing a file object for `file`).

`'rb'` for reading and `'wb'` for writing.

The name of the Zstandard file. Equivalent to the `name` attribute of the underlying file object.

Compressing and decompressing data in memory

`compression.zstd.compress(data, level=None, options=None, zstd_dict=None)`

Compress `data` (a bytes-like object), returning the compressed data as a bytes object. The `level` argument is an integer controlling the level of compression. `level` is an alternative to setting `CompressionParameter.compression_level` in `options`. Use `bounds()` on `compression_level` to get the values that can be passed for `level`. If advanced compression options are needed, the `level` argument must be omitted and in the `options` dictionary the `CompressionParameter.compression_level` parameter should be set. The `options` argument is a

`compression.zstd.decompress(data, zstd_dict=None, options=None)`

Decompress `data` (a bytes-like object), returning the uncompressed data as a bytes object. The `options` argument is a Python dictionary containing advanced decompression parameters. The valid keys and values for compression parameters are documented as part of the `DecompressionParameter` documentation. The `zstd_dict` argument is an instance of `ZstdDict` containing trained data used during compression. This must be the same Zstandard dictionary used during compression. If `data` is the concatenation of multiple distinct

`class compression.zstd.ZstdCompressor(level=None, options=None, zstd_dict=None)`

`compress(data, mode=ZstdCompressor.CONTINUE)`

`flush(mode=ZstdCompressor.FLUSH_FRAME)`

`set_pledged_input_size(size)`

`CONTINUE`

`FLUSH_BLOCK`

`FLUSH_FRAME`

`last_mode`

Create a compressor object, which can be used to compress data incrementally. For a more convenient way of compressing a single chunk of data, see the module-level function `compress()`. The `level` argument is an integer controlling the level of compression. `level` is an alternative to setting `CompressionParameter.compression_level` in `options`. Use `bounds()` on `compression_level` to get the values that can be passed for `level`. If advanced compression options are needed, the `level` argument must be omitted and in the `opti`

Compressdata(abytes-like object), returning abytesobject with compressed data if possible, or otherwise an emptybytesobject. Some ofdatamay be buffered internally, for use in later calls tocompress()andflush(). The returned data should be concatenated with the output of any previous calls tocompress().Themodeargument is aZstdCompressorattribute, eitherCONTINUE,FLUSH_BLOCK, orFLUSH_FRAME.When all data has been provided to the compressor, call theflush()method to finish the compression process. If

Finish the compression process, returning abytesobject containing any data stored in the compressor's internal buffers.The modeargument is aZstdCompressorattribute, eitherFLUSH_BLOCK, orFLUSH_FRAME.

Specify the amount of uncompressed datasizethat will be provided for the next frame.sizewill be written into the frame header of the next frame unlessCompressionParameter.content_size_flagisFalseor0. A size of0means that the frame is empty. IfsizeisNone, the frame header will omit the frame size. Frames that include the uncompressed data size require less memory to decompress, especially at higher compression levels.If last_modeis notFLUSH_FRAME, aValueErroris raised as the compressor is not at t

Collect more data for compression, which may or may not generate output immediately. This mode optimizes the compression ratio by maximizing the amount of data per block and frame.

Complete and write a block to the data stream. The data returned so far can be immediately decompressed. Past data can still be referenced in future blocks generated by calls tocompress(), improving compression.

Complete and write out a frame. Future data provided tocompress()will be written into a new frame andcannotreference past data.

The last mode passed to eithercompress()orflush(). The value can be one ofCONTINUE,FLUSH_BLOCK, orFLUSH_FRAME. The initial value isFLUSH_FRAME, signifying that the compressor is at the start of a new frame.

classcompression.zstd.ZstdDecompressor(zstd_dict=None,options=None)

decompress(data,max_length=-1)

eof

unused_data

needs_input

Create a decompressor object, which can be used to decompress data incrementally.For a more convenient way of decompressing an entire compressed stream at once, see the module-level functiondecompress().Theoptionsargument is a Python dictionary containing advanced decompression parameters. The valid keys and values for compression parameters are documented as part of theDecompressionParameterdocumentation.Thezstd_dictargument is an instance ofZstdDictcontaining trained data used during compressi

Decompressdata(abytes-like object), returning uncompressed data as bytes. Some ofdatamay be buffered internally, for use in later calls todecompress(). The returned data should be concatenated with the output of any previous calls todecompress().Ifmax_lengthis non-negative, the method returns at mostmax_lengthbytes of decompressed data. If this limit is reached and further output can be produced, thenneeds_inputattribute will be set toFalse. In this case, the next call todecompress()may provideda

Trueif the end-of-stream marker has been reached.

Data found after the end of the compressed stream.Before the end of the stream is reached, this will beb".

Falseif thedecompress()method can provide more decompressed data before requiring new compressed input.

Zstandard dictionaries

compression.zstd.train_dict(samples,dict_size)

Train a Zstandard dictionary, returning a `ZstdDict` instance. Zstandard dictionaries enable more efficient compression of smaller sizes of data, which is traditionally difficult to compress due to less repetition. If you are compressing multiple similar groups of data (such as similar files), Zstandard dictionaries can improve compression ratios and speed significantly. The `samples` argument (an iterable of `bytes` objects), is the population of samples used to train the Zstandard dictionary. The `dict_size` argument

compression.zstd.finalize_dict(zstd_dict,/,samples,dict_size,level)

An advanced function for converting a “raw content” Zstandard dictionary into a regular Zstandard dictionary. “Raw content” dictionaries are a sequence of bytes that do not need to follow the structure of a normal Zstandard dictionary. The `zstd_dict` argument is a `ZstdDict` instance with the `dict_content` containing the raw dictionary contents. The `samples` argument (an iterable of `bytes` objects), contains sample data for generating the Zstandard dictionary. The `dict_size` argument, an integer, is the maximum size

class compression.zstd.ZstdDict(dict_content,/,*,is_raw=False)

dict_content

dict_id

as_digested_dict

as_undigested_dict

A wrapper around Zstandard dictionaries. Dictionaries can be used to improve the compression of many small chunks of data. Use `train_dict()` if you need to train a new dictionary from sample data. The `dict_content` argument (a `bytes`-like object), is the already trained dictionary information. The `is_raw` argument, a boolean, is an advanced parameter controlling the meaning of `dict_content`. `True` means `dict_content` is a “raw content” dictionary, without any format restrictions. `False` means `dict_content` is an ordinary

The content of the Zstandard dictionary, a `bytes` object. It's the same as the `dict_content` argument in the `__init__` method. It can be used with other programs, such as the `zstdCLI` program.

Identifier of the Zstandard dictionary, a non-negative int value. Non-zero means the dictionary is ordinary, created by Zstandard functions and following the Zstandard format. 0 means a “raw content” dictionary, free of any format restriction, used for advanced users. Note The meaning of 0 for `ZstdDict.dict_id` is different from the `dictionary_id` attribute to the `get_frame_info()` function.

Load as a digested dictionary.

Load as an undigested dictionary.

Advanced parameter control

class compression.zstd.CompressionParameter

bounds()

compression_level

window_log

hash_log

chain_log

search_log

min_match

target_length

strategy

enable_long_distance_matching

ldm_hash_log

ldm_min_match

ldm_bucket_size_log

ldm_hash_rate_log

content_size_flag

checksum_flag

dict_id_flag

nb_workers

job_size

overlap_log

An `IntEnum` containing the advanced compression parameter keys that can be used when compressing data. The `bounds()` method can be used on any attribute to get the valid values for that parameter. Parameters are optional; any omitted parameter will have its value selected automatically. Example getting the lower and upper bound of `compression_level`: `lower, upper = CompressionParameter.compression_level.bounds()` Example setting the `window_log` to the maximum size: `_lower, upper = CompressionParameter.window_log.bounds()`

Return the tuple of int bounds, (lower, upper), of a compression parameter. This method should be called on the attribute you wish to retrieve the bounds of. For example, to get the valid values for `compression_level`, one may check the result of `CompressionParameter.compression_level.bounds()`. Both the lower and upper bounds are inclusive.

A high-level means of setting other compression parameters that affect the speed and ratio of compressing data. Regular compression levels are greater than 0. Values greater than 20 are considered “ultra” compression and require more memory than other levels. Negative values can be used to trade off faster compression for worse compression ratios. Setting the level to zero uses `COMPRESSION_LEVEL_DEFAULT`.

Maximum allowed back-reference distance the compressor can use when compressing data, expressed as power of two, $1 \leq \text{window_log} < 2^{\text{window_log}}$ bytes. This parameter greatly influences the memory usage of compression. Higher values require more memory but gain better compression values. A value of zero causes the value to be selected automatically.

Size of the initial probe table, as a power of two. The resulting memory usage is $1 \leq (\text{hash_log} + 2) < 2^{\text{hash_log} + 2}$ bytes. Larger tables improve compression ratio of strategies $\leq \text{dfast}$, and improve compression speed of strategies $> \text{dfast}$. A value of zero causes the value to be selected automatically.

Size of the multi-probe search table, as a power of two. The resulting memory usage is $1 \leq (\text{chain_log} + 2) < 2^{\text{chain_log} + 2}$ bytes. Larger tables result in better and slower compression. This parameter has no effect for the `fast` strategy. It's still useful when using `dfast` strategy, in which case it defines a secondary probe table. A value of zero causes the value to be selected automatically.

Number of search attempts, as a power of two. More attempts result in better and slower compression. This parameter is useless for `fast` and `dfast` strategies. A value of zero causes the value to be selected automatically.

Minimum size of searched matches. Larger values increase compression and decompression speed, but decrease ratio. Note that `Zstandard` can still find matches of smaller size, it just tweaks its search algorithm to look for this size and larger. For all strategies $< \text{btopt}$, the effective minimum is 4; for all strategies $> \text{fast}$, the

effective maximum is 6. A value of zero causes the value to be selected automatically.

The impact of this field depends on the selected strategy. For strategies `btopt`, `btultra` and `btultra2`, the value is the length of a match considered “good enough” to stop searching. Larger values make compression ratios better, but compresses slower. For strategy `fast`, it is the distance between match sampling. Larger values make compression faster, but with a worse compression ratio. A value of zero causes the value to be selected automatically.

The higher the value of selected strategy, the more complex the compression technique used by `zstd`, resulting in higher compression ratios but slower compression. See also `Strategy`.

Long distance matching can be used to improve compression for large inputs by finding large matches at greater distances. It increases memory usage and window size. `True` or `1` enable long distance matching while `False` or `0` disable it. Enabling this parameter increases `default_window_log` to 128 MiB except when expressly set to a different value. This setting is enabled by default if `window_log` $\geq$ 128 MiB and the compression strategy $\geq$ `btopt` (compression level 16+).

Size of the table for long distance matching, as a power of two. Larger values increase memory usage and compression ratio, but decrease compression speed. A value of zero causes the value to be selected automatically.

Minimum match size for long distance matcher. Larger or too small values can often decrease the compression ratio. A value of zero causes the value to be selected automatically.

Log size of each bucket in the long distance matcher hash table for collision resolution. Larger values improve collision resolution but decrease compression speed. A value of zero causes the value to be selected automatically.

Frequency of inserting/looking up entries into the long distance matcher hash table. Larger values improve compression speed. Deviating far from the default value will likely result in a compression ratio decrease. A value of zero causes the value to be selected automatically.

Write the size of the data to be compressed into the Zstandard frame header when known prior to compressing. This flag only takes effect under the following scenarios: Calling `compress()` for one-shot compression. Providing all of the data to be compressed in the frame in a single `ZstdCompressor.compress()` call, with the `ZstdCompressor.FLUSH_FRAME` mode. Calling `ZstdCompressor.set_pledged_input_size()` with the exact amount of data that will be provided to the compressor prior to any calls to `ZstdCompressor.comp`.

A four-byte checksum using `XXHash64` of the uncompressed content is written at the end of each frame. Zstandard’s decompression code verifies the checksum. If there is a mismatch a `ZstdError` exception is raised. `True` or `1` enable checksum generation while `False` or `0` disable it.

When compressing with a `ZstdDict`, the dictionary’s ID is written into the frame header. `True` or `1` enable storing the dictionary ID while `False` or `0` disable it.

Select how many threads will be spawned to compress in parallel. When `nb_workers` $>$ 0, enables multi-threaded compression, a value of 1 means “one-thread multi-threaded mode”. More workers improve speed, but also increase memory usage and slightly reduce compression ratio. A value of zero disables multi-threading.

Size of a compression job, in bytes. This value is enforced only when `nb_workers` $\geq$ 1. Each compression job is completed in parallel, so this value can indirectly impact the number of active threads. A value of zero causes the value to be selected automatically.

Sets how much data is reloaded from previous jobs (threads) for new jobs to be used by the look behind window during compression. This value is only used when `nb_workers` $\geq$ 1. Acceptable values vary from 0 to 9. 0 means dynamically set the overlap amount. 1 means no overlap. 9 means use a full window size from the previous job. Each increment halves/doubles the overlap size. “8” means an overlap of `window_size/2`, “7” means an

overlap of `window_size/4`, etc.

class `compression.zstd.DecompressionParameter`

`bounds()`

`window_log_max`

An `IntEnum` containing the advanced decompression parameter keys that can be used when decompressing data. Parameters are optional; any omitted parameter will have its value selected automatically. The `bounds()` method can be used on any attribute to get the valid values for that parameter. Example setting the `window_log_max` to the maximum size: `data=compress(b'Some very long buffer of bytes...')_lower,upper=DecompressionParameter.window_log_max.bounds()options={DecompressionParameter.window_log_max:upper}`

Return the tuple of int bounds, (lower, upper), of a decompression parameter. This method should be called on the attribute you wish to retrieve the bounds of. Both the lower and upper bounds are inclusive.

The base-two logarithm of the maximum size of the window used during decompression. This can be useful to limit the amount of memory used when decompressing data. A larger maximum window size leads to faster decompression. A value of zero causes the value to be selected automatically.

class `compression.zstd.Strategy`

`fast`

`dfast`

`greedy`

`lazy`

`lazy2`

`btlazy2`

`btopt`

`btultra`

`btultra2`

An `IntEnum` containing strategies for compression. Higher-numbered strategies correspond to more complex and slower compression. Note the values of attributes of `Strategy` are not necessarily stable across `zstd` versions. Only the ordering of the attributes may be relied upon. The attributes are listed below in order. The following strategies are available: `fast` `dfast` `greedy` `lazy` `lazy2` `btlazy2` `btopt` `btultra` `btultra2`

Miscellaneous

`compression.zstd.get_frame_info(frame_buffer)`

Retrieve a `FrameInfo` object containing metadata about a Zstandard frame. Frames contain metadata related to the compressed data they hold.

class `compression.zstd.FrameInfo`

`decompressed_size`

`dictionary_id`

Metadata related to a Zstandard frame.
`decompressed_size` The size of the decompressed contents of the frame.
`dictionary_id` An integer representing the Zstandard dictionary ID needed for decompressing the frame. 0 means the dictionary ID was not recorded in the frame header. This may mean that a Zstandard

dictionary is not needed, or that the ID of a required dictionary was not recorded.

The size of the decompressed contents of the frame.

An integer representing the Zstandard dictionary ID needed for decompressing the frame. 0 means the dictionary ID was not recorded in the frame header. This may mean that a Zstandard dictionary is not needed, or that the ID of a required dictionary was not recorded.

compression.zstd.COMPRESSION_LEVEL_DEFAULT

The default compression level for Zstandard:3.

compression.zstd.zstd_version_info

Version number of the runtime zstd library as a tuple of integers (major, minor, release).

Examples

Reading in a compressed file:

```
from compression import zstd

with zstd.open("file.zst") as f:
    file_content = f.read()
```

Creating a compressed file:

```
from compression import zstd

data = b"Insert Data Here"
with zstd.open("file.zst", "w") as f:
    f.write(data)
```

Compressing data in memory:

```
from compression import zstd

data_in = b"Insert Data Here"
data_out = zstd.compress(data_in)
```

Incremental compression:

```
from compression import zstd

comp = zstd.ZstdCompressor()
out1 = comp.compress(b"Some data\n")
out2 = comp.compress(b"Another piece of data\n")
out3 = comp.compress(b"Even more data\n")
out4 = comp.flush()
# Concatenate all the partial results:
result = b"".join([out1, out2, out3, out4])
```

Writing compressed data to an already-open file:

```
from compression import zstd

with open("myfile", "wb") as f:
    f.write(b"This data will not be compressed\n")
    with zstd.open(f, "w") as zstf:
        zstf.write(b"This *will* be compressed\n")
    f.write(b"Not compressed\n")
```

Creating a compressed file using compression parameters:

```
from compression import zstd
```

```
options = {  
    zstd.CompressionParameter.checksum_flag: 1  
}  
with zstd.open("file.zst", "w", options=options) as f:  
    f.write(b"Mind if I squeeze in?")
```

zlib— Compression compatible with gzip

For applications that require data compression, the functions in this module allow compression and decompression, using the `zlib` library.

This is an optional module. If it is missing from your copy of CPython, look for documentation from your distributor (that is, whoever provided Python to you). If you are the distributor, see [Requirements for optional modules](#).

`zlib`'s functions have many options and often need to be used in a particular order. This documentation doesn't attempt to cover all of the permutations; consult the `zlib` manual for authoritative information.

For reading and writing `.gz` files see the `gzip` module.

The available exception and functions in this module are:

exception `zlib.error`

Exception raised on compression and decompression errors.

`zlib.adler32(data[,value])`

Computes an Adler-32 checksum of `data`. (An Adler-32 checksum is almost as reliable as a CRC32 but can be computed much more quickly.) The result is an unsigned 32-bit integer. If `value` is present, it is used as the starting value of the checksum; otherwise, a default value of 1 is used. Passing in `value` allows computing a running checksum over the concatenation of several inputs. The algorithm is not cryptographically strong, and should not be used for authentication or digital signatures. Since

`zlib.compress(data[,level=Z_DEFAULT_COMPRESSION,wbits=MAX_WBITS])`

Compresses the bytes in `data`, returning a bytes object containing compressed data. `level` is an integer from 0 to 9 or -1 controlling the level of compression; See `Z_BEST_SPEED(1)`, `Z_BEST_COMPRESSION(9)`, `Z_NO_COMPRESSION(0)`, and the default, `Z_DEFAULT_COMPRESSION(-1)` for more information about these values. The `wbits` argument controls the size of the history buffer (or the “window size”) used when compressing data, and whether a header and trailer is included in the output. It can take several ranges of values,

`zlib.compressobj(level=Z_DEFAULT_COMPRESSION,method=DEFLATED,wbits=MAX_WBITS,memLevel=DEF_MEM_LEVEL,strategy=Z_DEFAULT_STRATEGY[,zdict])`

Returns a compression object, to be used for compressing data streams that won't fit into memory at once. `level` is the compression level – an integer from 0 to 9 or -1.

See `Z_BEST_SPEED(1)`, `Z_BEST_COMPRESSION(9)`, `Z_NO_COMPRESSION(0)`, and the default, `Z_DEFAULT_COMPRESSION(-1)` for more information about these values. `method` is the compression algorithm. Currently, the only supported value is `DEFLATED`. The `wbits` parameter controls the size of the history buffer (or the “window size”), and what header and trailer follow.

`zlib.crc32(data[,value])`

Computes a CRC (Cyclic Redundancy Check) checksum of `data`. The result is an unsigned 32-bit integer. If `value` is present, it is used as the starting value of the checksum; otherwise, a default value of 0 is used. Passing in `value` allows computing a running checksum over the concatenation of several inputs. The algorithm is not cryptographically strong, and should not be used for authentication or digital signatures. Since the algorithm is designed for use as a checksum algorithm, it is not suitable for

`zlib.decompress(data[,wbits=MAX_WBITS,bufsize=DEF_BUF_SIZE])`

Decompresses the bytes in `data`, returning a bytes object containing the uncompressed data. The `wbits` parameter depends on the format of `data`, and is discussed further below. If `bufsize` is given, it is used as

the initial size of the output buffer. Raises the `error` exception if any error occurs. The `wbits` parameter controls the size of the history buffer (or “window size”), and what header and trailer format is expected. It is similar to the parameter for `compressobj()`, but accepts more ranges of values: +8

zlib.decompressobj(wbits=MAX_WBITS[,zdict])

Returns a decompression object, to be used for decompressing data streams that won’t fit into memory at once. The `wbits` parameter controls the size of the history buffer (or the “window size”), and what header and trailer format is expected. It has the same meaning as described for `decompress()`. The `zdict` parameter specifies a predefined compression dictionary. If provided, this must be the same dictionary as was used by the compressor that produced the data that is to be decompressed. Note if `zdict` is a

Compression objects support the following methods:

Compress.compress(data)

Compress `data`, returning a bytes object containing compressed data for at least part of the data in `data`. This data should be concatenated to the output produced by any preceding calls to the `compress()` method. Some input may be kept in internal buffers for later processing.

Compress.flush([mode])

All pending input is processed, and a bytes object containing the remaining compressed output is returned. `mode` can be selected from the constants `Z_NO_FLUSH`, `Z_PARTIAL_FLUSH`, `Z_SYNC_FLUSH`, `Z_FULL_FLUSH`, `Z_BLOCK`, or `Z_FINISH`, defaulting to `Z_FINISH`. Except `Z_FINISH`, all constants allow compressing further bytestrings of data, while `Z_FINISH` finishes the compressed stream and prevents compressing any more data. After calling `flush()` with `mode` set to `Z_FINISH`, the `compress()` method cannot be called again; the only

Compress.copy()

Returns a copy of the compression object. This can be used to efficiently compress a set of data that share a common initial prefix.

Changed in version 3.8: Added `copy.copy()` and `copy.deepcopy()` support to compression objects.

Decompression objects support the following methods and attributes:

Decompress.unused_data

A bytes object which contains any bytes past the end of the compressed data. That is, this remains `b''` until the last byte that contains compression data is available. If the whole bytestring turned out to contain compressed data, this is `b''`, an empty bytes object.

Decompress.unconsumed_tail

A bytes object that contains any data that was not consumed by the last `decompress()` call because it exceeded the limit for the uncompressed data buffer. This data has not yet been seen by the `zlib` machinery, so you must feed it (possibly with further data concatenated to it) back to a subsequent `decompress()` method call in order to get correct output.

Decompress.eof

A boolean indicating whether the end of the compressed data stream has been reached. This makes it possible to distinguish between a properly formed compressed stream, and an incomplete or truncated one. Added in version 3.3.

Decompress.decompress(data,max_length=0)

Decompress `data`, returning a bytes object containing the uncompressed data corresponding to at least part of the data in `string`. This data should be concatenated to the output produced by any preceding calls to the `decompress()` method. Some of the input data may be preserved in internal buffers for later processing. If the

optional parameter `max_length` is non-zero then the return value will be no longer than `max_length`. This may mean that not all of the compressed input can be processed; and `unconsume`

Decompress.flush([length])

All pending input is processed, and a bytes object containing the remaining uncompressed output is returned. After calling `flush()`, the `decompress()` method cannot be called again; the only realistic action is to delete the object. The optional parameter `length` sets the initial size of the output buffer.

Decompress.copy()

Returns a copy of the decompression object. This can be used to save the state of the decompressor midway through the data stream in order to speed up random seeks into the stream at a future point.

Changed in version 3.8: Added `copy.copy()` and `copy.deepcopy()` support to decompression objects.

The following constants are available to configure compression and decompression behavior:

zlib.DEFLATED

The deflate compression method.

zlib.MAX_WBITS

The maximum window size, expressed as a power of 2. For example, if `MAX_WBITS` is 15 it results in a window size of 32 KiB.

zlib.DEF_MEM_LEVEL

The default memory level for compression objects.

zlib.DEF_BUF_SIZE

The default buffer size for decompression operations.

zlib.Z_NO_COMPRESSION

Compression level 0; no compression. Added in version 3.6.

zlib.Z_BEST_SPEED

Compression level 1; fastest and produces the least compression.

zlib.Z_BEST_COMPRESSION

Compression level 9; slowest and produces the most compression.

zlib.Z_DEFAULT_COMPRESSION

Default compression level (-1); a compromise between speed and compression. Currently equivalent to compression level 6.

zlib.Z_DEFAULT_STRATEGY

Default compression strategy, for normal data.

zlib.Z_FILTERED

Compression strategy for data produced by a filter (or predictor).

zlib.Z_HUFFMAN_ONLY

Compression strategy that forces Huffman coding only.

zlib.Z_RLE

Compression strategy that limits match distances to one (run-length encoding). This constant is only available if Python was compiled with zlib 1.2.0.1 or greater. Added in version 3.6.

zlib.Z_FIXED

Compression strategy that prevents the use of dynamic Huffman codes. This constant is only available if Python was compiled with zlib 1.2.2.2 or greater. Added in version 3.6.

zlib.Z_NO_FLUSH

Flush mode0. No special flushing behavior. Added in version 3.6.

zlib.Z_PARTIAL_FLUSH

Flush mode1. Flush as much output as possible.

zlib.Z_SYNC_FLUSH

Flush mode2. All output is flushed and the output is aligned to a byte boundary.

zlib.Z_FULL_FLUSH

Flush mode3. All output is flushed and the compression state is reset.

zlib.Z_FINISH

Flush mode4. All pending input is processed, no more input is expected.

zlib.Z_BLOCK

Flush mode5. A deflate block is completed and emitted. This constant is only available if Python was compiled with zlib 1.2.2.2 or greater. Added in version 3.6.

zlib.Z_TREES

Flush mode6, for inflate operations. Instructs inflate to return when it gets to the next deflate block boundary. This constant is only available if Python was compiled with zlib 1.2.3.4 or greater. Added in version 3.6.

Information about the version of the zlib library in use is available through the following constants:

zlib.ZLIB_VERSION

The version string of the zlib library that was used for building the module. This may be different from the zlib library actually used at runtime, which is available as `ZLIB_RUNTIME_VERSION`.

zlib.ZLIB_RUNTIME_VERSION

The version string of the zlib library actually loaded by the interpreter. Added in version 3.3.

zlib.ZLIBNG_VERSION

The version string of the zlib-ng library that was used for building the module if zlib-ng was used. When present, the `ZLIB_VERSION` and `ZLIB_RUNTIME_VERSION` constants reflect the version of the zlib API provided by zlib-ng. If zlib-ng was not used to build the module, this constant will be absent. Added in version 3.14.

See also

Module `gzip`

<https://www.zlib.net>

<https://www.zlib.net/manual.html>

Reading and writing `gzip`-format files.

The zlib library home page.

The zlib manual explains the semantics and usage of the library's many functions.

In case gzip (de)compression is a bottleneck, the `python-isal` package speeds up (de)compression with a mostly compatible API.

gzip— Support for gzip files

Source code: `Lib/gzip.py`

This module provides a simple interface to compress and decompress files just like the GNU `gzip` and `gunzip` programs would.

This is an optional module. If it is missing from your copy of CPython, look for documentation from your distributor (that is, whoever provided Python to you). If you are the distributor, see [Requirements for optional modules](#).

The data compression is provided by the `zlib` module.

The `gzip` module provides the `GzipFile` class, as well as the `open()`, `compress()` and `decompress()` convenience functions. The `GzipFile` class reads and writes `gzip`-format files, automatically compressing or decompressing the data so that it looks like an ordinary file object.

Note that additional file formats which can be decompressed by the `gzip` and `gunzip` programs, such as those produced by `compress` and `pack`, are not supported by this module.

The module defines the following items:

`gzip.open(filename, mode='rb', compresslevel=9, encoding=None, errors=None, newline=None)`

Open a `gzip`-compressed file in binary or text mode, returning a file object. The `filename` argument can be an actual filename (a `str` or `bytes` object), or an existing file object to read from or write to. The `mode` argument can be any of `r`, `rb`, `a`, `ab`, `w`, `wb`, `x` or `xb` for binary mode, or `rt`, `at`, `wt`, or `xt` for text mode. The default is `rb`. The `compresslevel` argument is an integer from 0 to 9, as for the `GzipFile` constructor. For binary mode, this function is equivalent to the `GzipFile` constructor: `GzipFile(file`

`exception gzip.BadGzipFile`

An exception raised for invalid `gzip` files. It inherits from `IOError`. `EOFError` and `zlib.error` can also be raised for invalid `gzip` files. Added in version 3.8.

`class gzip.GzipFile(filename=None, mode=None, compresslevel=9, fileobj=None, mtime=None)`

`peek(n)`

`mode`

`mtime`

`name`

Constructor for the `GzipFile` class, which simulates most of the methods of a file object, with the exception of the `truncate()` method. At least one of `fileobj` and `filename` must be given a non-trivial value. The new class instance is based on `fileobj`, which can be a regular file, an `io.BytesIO` object, or any other object which simulates a file. It defaults to `None`, in which case `filename` is opened to provide a file object. When `fileobj` is not `None`, the `filename` argument is only used to be included in the `gzip` file header.

Read uncompressed bytes without advancing the file position. The number of bytes returned may be more or less than requested. Note that while calling `peek()` does not change the file position of the `GzipFile`, it may change the position of the underlying file object (e.g. if the `GzipFile` was constructed with the `fileobj` parameter). Added in version 3.2.

`'rb'` for reading and `'wb'` for writing. Changed in version 3.13: In previous versions it was an integer 1 or 2.

When decompressing, this attribute is set to the last timestamp in the most recently read header. It is an integer, holding the number of seconds since the Unix epoch (00:00:00 UTC, January 1, 1970). The initial value before reading any headers is `None`.

The path to the gzip file on disk, as a string. Equivalent to the output of `os.fspath()` on the original input path, with no other normalization, resolution or expansion.

gzip.compress(data, compresslevel=9, *, mtime=0)

Compress the data, returning a bytes object containing the compressed data. `compresslevel` and `mtime` have the same meaning as in the `GzipFile` constructor above, but `mtime` defaults to 0 for reproducible output. Added in version 3.2. Changed in version 3.8: Added the `mtime` parameter for reproducible output. Changed in version 3.11: Speed is improved by compressing all data at once instead of in a streamed fashion. Calls with `mtime` set to 0 are delegated to `zlib.compress()` for better speed. In this situation the output may

gzip.decompress(data)

Decompress the data, returning a bytes object containing the uncompressed data. This function is capable of decompressing multi-member gzip data (multiple gzip blocks concatenated together). When the data is certain to contain only one member the `zlib.decompress()` function with `wbits` set to 31 is faster. Added in version 3.2. Changed in version 3.11: Speed is improved by decompressing members at once in memory instead of in a streamed fashion.

Examples of usage

Example of how to read a compressed file:

```
import gzip
with gzip.open('/home/joe/file.txt.gz', 'rb') as f:
    file_content = f.read()
```

Example of how to create a compressed GZIP file:

```
import gzip
content = b"Lots of content here"
with gzip.open('/home/joe/file.txt.gz', 'wb') as f:
    f.write(content)
```

Example of how to GZIP compress an existing file:

```
import gzip
import shutil
with open('/home/joe/file.txt', 'rb') as f_in:
    with gzip.open('/home/joe/file.txt.gz', 'wb') as f_out:
        shutil.copyfileobj(f_in, f_out)
```

Example of how to GZIP compress a binary string:

```
import gzip
s_in = b"Lots of content here"
s_out = gzip.compress(s_in)
```

See also

Module `zlib`

The basic data compression module needed to support the gzip file format.

In case gzip (de)compression is a bottleneck, the `python-isal` package speeds up (de)compression with a mostly compatible API.

Command-line interface

The `gzip` module provides a simple command line interface to compress or decompress files.

Once executed the `gzip` module keeps the input file(s).

Changed in version 3.8: Add a new command line interface with a usage. By default, when you will execute the CLI, the default compression level is 6.

Command-line options

file

If file is not specified, read from `sys.stdin`.

--fast

Indicates the fastest compression method (less compression).

--best

Indicates the slowest compression method (best compression).

-d, --decompress

Decompress the given file.

-h, --help

Show the help message.

bz2— Support for bzip2 compression

Source code: `Lib/bz2.py`

This module provides a comprehensive interface for compressing and decompressing data using the bzip2 compression algorithm.

The `bz2` module contains:

- The `open()` function and `BZ2File` class for reading and writing compressed files.
- The `BZ2Compressor` and `BZ2Decompressor` classes for incremental (de)compression.
- The `compress()` and `decompress()` functions for one-shot (de)compression.

This is an optional module. If it is missing from your copy of CPython, look for documentation from your distributor (that is, whoever provided Python to you). If you are the distributor, see [Requirements](#) for optional modules.

(De)compression of files

`bz2.open(filename, mode='rb', compresslevel=9, encoding=None, errors=None, newline=None)`

Open a bzip2-compressed file in binary or text mode, returning a file object. As with the constructor for `BZ2File`, the `filename` argument can be an actual filename (a `str` or `bytes` object), or an existing file object to read from or write to. The `mode` argument can be any of `'r'`, `'rb'`, `'w'`, `'wb'`, `'x'`, `'xb'`, `'a'` or `'ab'` for binary mode, or `'rt'`, `'wt'`, `'xt'`, or `'at'` for text mode. The default is `'rb'`. The `compresslevel` argument is an integer from 1 to 9, as for the `BZ2File` constructor. For binary mode, this function is equivalent to

`class bz2.BZ2File(filename, mode='r', *, compresslevel=9)`

`peek([n])`

`fileno()`

`readable()`

`seekable()`

`writable()`

`read1(size=-1)`

`readinto(b)`

`mode`

`name`

Open a bzip2-compressed file in binary mode. If `filename` is a `str` or `bytes` object, open the named file directly. Otherwise, `filename` should be a file object, which will be used to read or write the compressed data. The `mode` argument can be either `'r'` for reading (default), `'w'` for overwriting, `'x'` for exclusive creation, or `'a'` for appending. These can equivalently be given as `'rb'`, `'wb'`, `'xb'` and `'ab'` respectively. If `filename` is a file object (rather than an actual file name), a mode of `'w'` does not truncate the file, and is

Return buffered data without advancing the file position. At least one byte of data will be returned (unless at EOF). The exact number of bytes returned is unspecified. Note While calling `peek()` does not change the file position of the `BZ2File`, it may change the position of the underlying file object (e.g. if the `BZ2File` was constructed by passing a file object for `filename`). Added in version 3.3.

Return the file descriptor for the underlying file. Added in version 3.3.

Return whether the file was opened for reading. Added in version 3.3.

Return whether the file supports seeking. Added in version 3.3.

Return whether the file was opened for writing. Added in version 3.3.

Read up to `size` uncompressed bytes, while trying to avoid making multiple reads from the underlying stream. Reads up to a buffer's worth of data if `size` is negative. Returns `None` if the file is at EOF. Added in version 3.3.

Read bytes into `buf`. Returns the number of bytes read (0 for EOF). Added in version 3.3.

'rb' for reading and 'wb' for writing. Added in version 3.13.

The `bzip2` file name. Equivalent to the `name` attribute of the underlying file object. Added in version 3.13.

Incremental (de)compression

class bz2.BZ2Compressor(`compresslevel=9`)

compress(`data`)

flush()

Create a new compressor object. This object may be used to compress data incrementally. For one-shot compression, use the `compress()` function instead. `compresslevel`, if given, must be an integer between 1 and 9. The default is 9. `compress(data)` Provide data to the compressor object. Returns a chunk of compressed data if possible, or an empty byte string otherwise. When you have finished providing data to the compressor, call the `flush()` method to finish the compression process. `flush()` Finish the compression

Provide data to the compressor object. Returns a chunk of compressed data if possible, or an empty byte string otherwise. When you have finished providing data to the compressor, call the `flush()` method to finish the compression process.

Finish the compression process. Returns the compressed data left in internal buffers. The compressor object may not be used after this method has been called.

class bz2.BZ2Decompressor

decompress(`data`, `max_length=-1`)

eof

unused_data

needs_input

Create a new decompressor object. This object may be used to decompress data incrementally. For one-shot compression, use the `decompress()` function instead. Note This class does not transparently handle inputs containing multiple compressed streams, unlike `decompress()` and `BZ2File`. If you need to decompress a multi-stream input with `BZ2Decompressor`, you must use a new decompressor for each stream. `decompress(data, max_length=-1)` Decompress data (a bytes-like object), returning uncompressed data as bytes. Some

`Decompress(data)` (a bytes-like object), returning uncompressed data as bytes. Some of data may be buffered internally, for use in later calls to `decompress()`. The returned data should be concatenated with the output of any previous calls to `decompress()`. If `max_length` is nonnegative, returns at most `max_length` bytes of decompressed data. If this limit is reached and further output can be produced, the `needs_input` attribute will be set to `False`. In this case, the next call to `decompress()` may provide data as `sb"to ob`

`True` if the end-of-stream marker has been reached. Added in version 3.3.

Data found after the end of the compressed stream. If this attribute is accessed before the end of the stream has been reached, its value will be b''.

False if the decompress() method can provide more decompressed data before requiring new uncompressed input. Added in version 3.5.

One-shot (de)compression

bz2.compress(data, compresslevel=9)

Compress data, a bytes-like object. compresslevel, if given, must be an integer between 1 and 9. The default is 9. For incremental compression, use a BZ2Compressor instead.

bz2.decompress(data)

Decompress data, a bytes-like object. If data is the concatenation of multiple compressed streams, decompress all of the streams. For incremental decompression, use a BZ2Decompressor instead. Changed in version 3.3: Support for multi-stream inputs was added.

Examples of usage

Below are some examples of typical usage of the bz2 module.

Using compress() and decompress() to demonstrate round-trip compression:

```
>>> import bz2
>>> data = b"""\
... Donec rhoncus quis sapien sit amet molestie. Fusce scelerisque vel augue
... nec ullamcorper. Nam rutrum pretium placerat. Aliquam vel tristique lorem,
... sit amet cursus ante. In interdum laoreet mi, sit amet ultrices purus
... pulvinar a. Nam gravida euismod magna, non varius justo tincidunt feugiat.
... Aliquam pharetra lacus non risus vehicula rutrum. Maecenas aliquam leo
... felis. Pellentesque semper nunc sit amet nibh ullamcorper, ac elementum
... dolor luctus. Curabitur lacinia mi ornare consectetur vestibulum."""
>>> c = bz2.compress(data)
>>> len(data) / len(c) # Data compression ratio
1.513595166163142
>>> d = bz2.decompress(c)
>>> data == d # Check equality to original object after round-trip
True
```

Using BZ2Compressor for incremental compression:

```
>>> import bz2
>>> def gen_data(chunks=10, chunksize=1000):
...     """Yield incremental blocks of chunksize bytes."""
...     for _ in range(chunks):
...         yield b"z" * chunksize
...
>>> comp = bz2.BZ2Compressor()
>>> out = b""
>>> for chunk in gen_data():
...     # Provide data to the compressor object
...     out = out + comp.compress(chunk)
...
>>> # Finish the compression process. Call this once you have
>>> # finished providing data to the compressor.
>>> out = out + comp.flush()
```

The example above uses a very “nonrandom” stream of data (a stream of b“z” chunks). Random data tends to compress poorly, while ordered, repetitive data usually yields a high compression ratio.

Writing and reading a bzip2-compressed file in binary mode:

```
>>> import bz2
>>> data = b"""\
... Donec rhoncus quis sapien sit amet molestie. Fusce scelerisque vel augue
... nec ullamcorper. Nam rutrum pretium placerat. Aliquam vel tristique lorem,
... sit amet cursus ante. In interdum laoreet mi, sit amet ultrices purus
... pulvinar a. Nam gravida euismod magna, non varius justo tincidunt feugiat.
... Aliquam pharetra lacus non risus vehicula rutrum. Maecenas aliquam leo
... felis. Pellentesque semper nunc sit amet nibh ullamcorper, ac elementum
... dolor luctus. Curabitur lacinia mi ornare consectetur vestibulum."""
>>> with bz2.open("myfile.bz2", "wb") as f:
...     # Write compressed data to file
...     unused = f.write(data)
...
>>> with bz2.open("myfile.bz2", "rb") as f:
...     # Decompress data from file
...     content = f.read()
...
>>> content == data # Check equality to original object after round-trip
True
```

Izma— Compression using the LZMA algorithm

Added in version 3.3.

Source code: `Lib/izma.py`

This module provides classes and convenience functions for compressing and decompressing data using the LZMA compression algorithm. Also included is a file interface supporting the .xz and legacy .lzma file formats used by the xz utility, as well as raw compressed streams.

The interface provided by this module is very similar to that of the bz2 module. Note that `LZMAFile` and `bz2.BZ2File` are not thread-safe, so if you need to use a single `LZMAFile` instance from multiple threads, it is necessary to protect it with a lock.

This is an optional module. If it is missing from your copy of CPython, look for documentation from your distributor (that is, whoever provided Python to you). If you are the distributor, see Requirements for optional modules.

exception `Izma.LZMAError`

This exception is raised when an error occurs during compression or decompression, or while initializing the compressor/decompressor state.

Reading and writing compressed files

`Izma.open(filename, mode='rb', *, format=None, check=-1, preset=None, filters=None, encoding=None, errors=None, newline=None)`

Open an LZMA-compressed file in binary or text mode, returning a file object. The `filename` argument can be either an actual file name (given as a `str`, `bytes` or path-like object), in which case the named file is opened, or it can be an existing file object to read from or write to. The `mode` argument can be any of `"r"`, `"rb"`, `"w"`, `"wb"`, `"x"`, `"xb"`, `"a"` or `"ab"` for binary mode, or `"rt"`, `"wt"`, `"xt"`, or `"at"` for text mode. The default is `"rb"`. When opening a file for reading, the `format` and `filters` arguments have the same meanings as

`class Izma.LZMAFile(filename=None, mode='r', *, format=None, check=-1, preset=None, filters=None)`

`peek(size=-1)`

`mode`

`name`

Open an LZMA-compressed file in binary mode. An `LZMAFile` can wrap an already-open file object, or operate directly on a named file. The `filename` argument specifies either the file object to wrap, or the name of the file to open (as a `str`, `bytes` or path-like object). When wrapping an existing file object, the wrapped file will not be closed when the `LZMAFile` is closed. The `mode` argument can be either `"r"` for reading (default), `"w"` for overwriting, `"x"` for exclusive creation, or `"a"` for appending. These can equivalently

Return buffered data without advancing the file position. At least one byte of data will be returned, unless EOF has been reached. The exact number of bytes returned is unspecified (the `size` argument is ignored). Note that while calling `peek()` does not change the file position of the `LZMAFile`, it may change the position of the underlying file object (e.g. if the `LZMAFile` was constructed by passing a file object for `filename`).

`'rb'` for reading and `'wb'` for writing. Added in version 3.13.

The `Izma` file name. Equivalent to the `name` attribute of the underlying file object. Added in version 3.13.

Compressing and decompressing data in memory

classlzma.LZMACompressor(format=FORMAT_XZ,check=-1,preset=None,filters=None)

FORMAT_XZ: The.xzcontainer format.

FORMAT_ALONE: The legacy.lzmacontainer format.

FORMAT_RAW: A raw data stream, not using any container format.

compress(data)

flush()

Create a compressor object, which can be used to compress data incrementally. For a more convenient way of compressing a single chunk of data, see `compress()`. The `format` argument specifies what container format should be used. Possible values are: **FORMAT_XZ**: The .xz container format. This is the default format. **FORMAT_ALONE**: The legacy .lzma container format. This format is more limited than .xz – it does not support integrity checks or multiple filters. **FORMAT_RAW**: A raw data stream, not using any container fo

This is the default format.

This format is more limited than .xz – it does not support integrity checks or multiple filters.

This format specifier does not support integrity checks, and requires that you always specify a custom filter chain (for both compression and decompression). Additionally, data compressed in this manner cannot be decompressed using **FORMAT_AUTO** (see `LZMADecompressor`).

`Compressdata(abytesobject)`, returning `abytesobject` containing compressed data for at least part of the input. Some of `data` may be buffered internally, for use in later calls to `compress()` and `flush()`. The returned data should be concatenated with the output of any previous calls to `compress()`.

Finish the compression process, returning `abytesobject` containing any data stored in the compressor's internal buffers. The compressor cannot be used after this method has been called.

classlzma.LZMADecompressor(format=FORMAT_AUTO,memlimit=None,filters=None)

decompress(data,max_length=-1)

check

eof

unused_data

needs_input

Create a decompressor object, which can be used to decompress data incrementally. For a more convenient way of decompressing an entire compressed stream at once, see `decompress()`. The `format` argument specifies the container format that should be used. The default is **FORMAT_AUTO**, which can decompress both .xz and .lzma files. Other possible values are **FORMAT_XZ**, **FORMAT_ALONE**, and **FORMAT_RAW**. The `memlimit` argument specifies a limit (in bytes) on the amount of memory that the decompressor can use. When this argume

`Decompressdata(abytes-like object)`, returning uncompressed data as bytes. Some of `data` may be buffered internally, for use in later calls to `decompress()`. The returned data should be concatenated with the output of any previous calls to `decompress()`. If `max_length` is nonnegative, returns at most `max_length` bytes of decompressed data. If this limit is reached and further output can be produced, the `needs_input` attribute will be set to `False`. In this case, the next call to `decompress()` may provide `data` as `b"to ob`

The ID of the integrity check used by the input stream. This may be **CHECK_UNKNOWN** until enough of the input has been decoded to determine what integrity check it uses.

True if the end-of-stream marker has been reached.

Data found after the end of the compressed stream. Before the end of the stream is reached, this will be "".

False if the `decompress()` method can provide more decompressed data before requiring new uncompressed input. Added in version 3.5.

`lzma.compress(data, format=FORMAT_XZ, check=-1, preset=None, filters=None)`

Compress `data` (a bytes object), returning the compressed data as a bytes object. See `LZMACompressor` above for a description of the `format`, `check`, `preset` and `filters` arguments.

`lzma.decompress(data, format=FORMAT_AUTO, memlimit=None, filters=None)`

Decompress `data` (a bytes object), returning the uncompressed data as a bytes object. If `data` is the concatenation of multiple distinct compressed streams, decompress all of these streams, and return the concatenation of the results. See `LZMADecompressor` above for a description of the `format`, `memlimit` and `filters` arguments.

Miscellaneous

`lzma.is_check_supported(check)`

Return True if the given integrity check is supported on this system. `CHECK_NONE` and `CHECK_CRC32` are always supported. `CHECK_CRC64` and `CHECK_SHA256` may be unavailable if you are using a version of `liblzma` that was compiled with a limited feature set.

Specifying custom filter chains

A filter chain specifier is a sequence of dictionaries, where each dictionary contains the ID and options for a single filter. Each dictionary must contain the key "id", and may contain additional keys to specify filter-dependent options. Valid filter IDs are as follows:

- Compression filters: `FILTER_LZMA1` (for use with `FORMAT_ALONE`) `FILTER_LZMA2` (for use with `FORMAT_XZ` and `FORMAT_RAW`)

- Delta filter: `FILTER_DELTA`

- Branch-Call-Jump (BCJ)

filters: `FILTER_X86` `FILTER_IA64` `FILTER_ARM` `FILTER_ARMTHUMB` `FILTER_POWERPC` `FILTER_SPARC`

A filter chain can consist of up to 4 filters, and cannot be empty. The last filter in the chain must be a compression filter, and any other filters must be delta or BCJ filters.

Compression filters support the following options (specified as additional entries in the dictionary representing the filter):

- `preset`: A compression preset to use as a source of default values for options that are not specified explicitly.
- `dict_size`: Dictionary size in bytes. This should be between 4 KiB and 1.5 GiB (inclusive).
- `lc`: Number of literal context bits.
- `lp`: Number of literal position bits. The sum `lc+lp` must be at most 4.
- `pb`: Number of position bits; must be at most 4.
- `mode`: `MODE_FAST` or `MODE_NORMAL`.
- `nice_len`: What should be considered a "nice length" for a match. This should be 273 or less.
- `mf`: What match finder to use – `MF_HC3`, `MF_HC4`, `MF_BT2`, `MF_BT3`, or `MF_BT4`.

- depth: Maximum search depth used by match finder. 0 (default) means to select automatically based on other filter options.

The delta filter stores the differences between bytes, producing more repetitive input for the compressor in certain circumstances. It supports one option, `dist`. This indicates the distance between bytes to be subtracted. The default is 1, i.e. take the differences between adjacent bytes.

The BCJ filters are intended to be applied to machine code. They convert relative branches, calls and jumps in the code to use absolute addressing, with the aim of increasing the redundancy that can be exploited by the compressor. These filters support one option, `start_offset`. This specifies the address that should be mapped to the beginning of the input data. The default is 0.

Examples

Reading in a compressed file:

```
import lzma
with lzma.open("file.xz") as f:
    file_content = f.read()
```

Creating a compressed file:

```
import lzma
data = b"Insert Data Here"
with lzma.open("file.xz", "w") as f:
    f.write(data)
```

Compressing data in memory:

```
import lzma
data_in = b"Insert Data Here"
data_out = lzma.compress(data_in)
```

Incremental compression:

```
import lzma
lzc = lzma.LZMACompressor()
out1 = lzc.compress(b"Some data\n")
out2 = lzc.compress(b"Another piece of data\n")
out3 = lzc.compress(b"Even more data\n")
out4 = lzc.flush()
# Concatenate all the partial results:
result = b"".join([out1, out2, out3, out4])
```

Writing compressed data to an already-open file:

```
import lzma
with open("file.xz", "wb") as f:
    f.write(b"This data will not be compressed\n")
    with lzma.open(f, "w") as lzf:
        lzf.write(b"This *will* be compressed\n")
    f.write(b"Not compressed\n")
```

Creating a compressed file using a custom filter chain:

```
import lzma
my_filters = [
    {"id": lzma.FILTER_DELTA, "dist": 5},
    {"id": lzma.FILTER_LZMA2, "preset": 7 | lzma.PRESET_EXTREME},
]
with lzma.open("file.xz", "w", filters=my_filters) as f:
    f.write(b"blah blah blah")
```

zipfile— Work with ZIP archives

Source code:Lib/zipfile/

The ZIP file format is a common archive and compression standard. This module provides tools to create, read, write, append, and list a ZIP file. Any advanced use of this module will require an understanding of the format, as defined in PKZIP Application Note.

This module does not handle multipart ZIP files. It can handle ZIP files that use the ZIP64 extensions (that is ZIP files that are more than 4 GiB in size). It supports decryption of encrypted files in ZIP archives, but it cannot create an encrypted file. Decryption is extremely slow as it is implemented in native Python rather than C.

Handling compressed archives requires optional modules such as `zlib`, `bz2`, `lzma`, and `compression.zstd`. If any of them are missing from your copy of CPython, look for documentation from your distributor (that is, whoever provided Python to you). If you are the distributor, see `Requirements` for optional modules.

The module defines the following items:

exception `zipfile.BadZipFile`

The error raised for bad ZIP files. Added in version 3.2.

exception `zipfile.BadZipfile`

Alias of `BadZipFile`, for compatibility with older Python versions. Deprecated since version 3.2.

exception `zipfile.LargeZipFile`

The error raised when a ZIP file would require ZIP64 functionality but that has not been enabled.

class `zipfile.ZipFile`

The class for reading and writing ZIP files. See section `ZipFile objects` for constructor details.

class `zipfile.Path`

Class that implements a subset of the interface provided by `pathlib.Path`, including the full `importlib.resources.abc.Traversable` interface. Added in version 3.8.

class `zipfile.PyZipFile`

Class for creating ZIP archives containing Python libraries.

class `zipfile.ZipInfo(filename='NoName', date_time=(1980,1,1,0,0,0))`

`_for_archive(archive)`

Class used to represent information about a member of an archive. Instances of this class are returned by the `getinfo()` and `infolist()` methods of `ZipFile` objects. Most users of the `zipfile` module will not need to create these, but only use those created by this module. `filename` should be the full name of the archive member, and `date_time` should be a tuple containing six fields which describe the time of the last modification to the file; the fields are described in section `ZipInfo objects`. Changed in version

Resolve the `date_time`, compression attributes, and external attributes to suitable defaults as used by `ZipFile.writestr()`. Returns self for chaining. Added in version 3.14.

`zipfile.is_zipfile(filename)`

Returns `True` if `filename` is a valid ZIP file based on its magic number, otherwise returns `False`. `filename` may be a file or file-like object too. Changed in version 3.1: Support for file and file-like objects.

`zipfile.ZIP_STORED`

The numeric constant for an uncompressed archive member.

zipfile.ZIP_DEFLATED

The numeric constant for the usual ZIP compression method. This requires the `zlib` module.

zipfile.ZIP_BZIP2

The numeric constant for the BZIP2 compression method. This requires the `bz2` module. Added in version 3.3.

zipfile.ZIP_LZMA

The numeric constant for the LZMA compression method. This requires the `lzma` module. Added in version 3.3.

zipfile.ZIP_ZSTANDARD

The numeric constant for Zstandard compression. This requires the `compression.zstd` module. Note in APPNOTE 6.3.7, the method ID 20 was assigned to Zstandard compression. This was changed in APPNOTE 6.3.8 to method ID 93 to avoid conflicts, with method ID 20 being deprecated. For compatibility, the `zipfile` module reads both method IDs but will only write data with method ID 93. Added in version 3.14.

■ Nota: Note The ZIP file format specification has included support for bzip2 compression since 2001, for LZMA compression since 2006, and Zstandard compression since 2020. However, some tools (including older Python releases) do not support these compression methods, and may either refuse to process the ZIP file altogether, or fail to extract individual files.

See also

PKZIP Application Note

Info-ZIP Home Page

Documentation on the ZIP file format by Phil Katz, the creator of the format and algorithms used.

Information about the Info-ZIP project's ZIP archive programs and development libraries.

ZipFile objects

class `zipfile.ZipFile(file, mode='r', compression=ZIP_STORED, allowZip64=True, compresslevel=None, *, strict_timestamps=True, metadata_encoding=None)`

Open a ZIP file, where `file` can be a path to a file (a string), a file-like object or a path-like object. The `mode` parameter should be 'r' to read an existing file, 'w' to truncate and write a new file, 'a' to append to an existing file, or 'x' to exclusively create and write a new file. If `mode` is 'x' and `file` refers to an existing file, a `FileExistsError` will be raised. If `mode` is 'a' and `file` refers to an existing ZIP file, then additional files are added to it. If `file` does not refer to a ZIP file, then a new ZIP archive

ZipFile.close()

Close the archive file. You must call `close()` before exiting your program or essential records will not be written.

ZipFile.getinfo(name)

Return a `ZipInfo` object with information about the archive member `name`. Calling `getinfo()` for a name not currently contained in the archive will raise a `KeyError`.

ZipFile.infolist()

Return a list containing a `ZipInfo` object for each member of the archive. The objects are in the same order as their entries in the actual ZIP file on disk if an existing archive was opened.

ZipFile.namelist()

Return a list of archive members by name.

ZipFile.open(name,mode='r',pwd=None,*,force_zip64=False)

Access a member of the archive as a binary file-like object. `name` can be either the name of a file within the archive or a `ZipInfo` object. The `mode` parameter, if included, must be 'r' (the default) or 'w'. `pwd` is the password used to decrypt encrypted ZIP files as a bytes object. `open()` is also a context manager and therefore supports the `with` statement: `with ZipFile('spam.zip') as myzip: with myzip.open('eggs.txt') as myfile: print(myfile.read())` With mode 'r' the file-like object (`ZipExtFile`) is read-only and provides the

ZipFile.extract(member,path=None,pwd=None)

Extract a member from the archive to the current working directory; `member` must be its full name or a `ZipInfo` object. Its file information is extracted as accurately as possible. `path` specifies a different directory to extract to. `member` can be a filename or a `ZipInfo` object. `pwd` is the password used for encrypted files as a bytes object. Returns the normalized path created (a directory or new file). Note if a member filename is an absolute path, a drive/UNC share point and leading (back)slashes will be stripped

ZipFile.extractall(path=None,members=None,pwd=None)

Extract all members from the archive to the current working directory. `path` specifies a different directory to extract to. `members` is optional and must be a subset of the list returned by `namelist()`. `pwd` is the password used for encrypted files as a bytes object. **Warning** Never extract archives from untrusted sources without prior inspection. It is possible that files are created outside of `path`, e.g. members that have absolute filenames starting with "/" or filenames with two dots "..". This module attempts to

ZipFile.printdir()

Print a table of contents for the archive to `sys.stdout`.

ZipFile.setpassword(pwd)

Set `pwd` (a bytes object) as default password to extract encrypted files.

ZipFile.read(name,pwd=None)

Return the bytes of the `filename` in the archive. `name` is the name of the file in the archive, or a `ZipInfo` object. The archive must be open for read or append. `pwd` is the password used for encrypted files as a bytes object and, if specified, overrides the default password set with `setpassword()`. Calling `read()` on a `ZipFile` that uses a compression method other than `ZIP_STORED`, `ZIP_DEFLATED`, `ZIP_BZIP2`, `ZIP_LZMA`, or `ZIP_ZSTD` will raise a `NotImplementedError`. An error will also be raised if the corresponding com

ZipFile.testzip()

Read all the files in the archive and check their CRC's and file headers. Return the name of the first bad file, or else return `None`. Changed in version 3.6: Calling `testzip()` on a closed `ZipFile` will raise a `ValueError`. Previously, a `RuntimeError` was raised.

ZipFile.write(filename,arcname=None,compress_type=None,compresslevel=None)

Write the file named `filename` to the archive, giving it the archive name `arcname` (by default, this will be the same as `filename`, but without a drive letter and with leading path separators removed). If given, `compress_type` overrides the value given for the `compression` parameter to the constructor for the new entry. Similarly, `compresslevel` will override the constructor if given. The archive must be open with mode 'w', 'x' or 'a'. Note The ZIP file standard historically did not specify a metadata encoding, but s

ZipFile.writestr(zinfo_or_arcname,data,compress_type=None,compresslevel=None)

Write a file into the archive. The contents is `data`, which may be either a `str` or a bytes instance; if it is a `str`, it is encoded as UTF-8 first. `zinfo_or_arcname` is either the file name it will be given in the archive, or a `ZipInfo` instance. If it's an instance, at least the filename, date, and time must be given. If it's a name, the date and time is set to the current date and time. The archive must be opened with mode 'w', 'x' or 'a'. If

given, compress_type overrides the value given for the compression parameter

ZipFile.mkdir(zinfo_or_directory, mode=511)

Create a directory inside the archive. If `zinfo_or_directory` is a string, a directory is created inside the archive with the mode that is specified in the `mode` argument. If, however, `zinfo_or_directory` is a `ZipInfo` instance then the `mode` argument is ignored. The archive must be opened with mode 'w', 'x' or 'a'. Added in version 3.11.

The following data attributes are also available:

ZipFile.filename

Name of the ZIP file.

ZipFile.debug

The level of debug output to use. This may be set from 0 (the default, no output) to 3 (the most output). Debugging information is written to `sys.stdout`.

ZipFile.comment

The comment associated with the ZIP file as a bytes object. If assigning a comment to a `ZipFile` instance created with mode 'w', 'x' or 'a', it should be no longer than 65535 bytes. Comments longer than this will be truncated.

Path objects

class zipfile.Path(root, at="")

Construct a `Path` object from a `root` (which may be a `ZipFile` instance or a file suitable for passing to the `ZipFile` constructor). `at` specifies the location of this `Path` within the zip file, e.g. 'dir/file.txt', 'dir/', or ''. Defaults to the empty string, indicating the root. Note: The `Path` class does not sanitize filenames within the ZIP archive. Unlike the `ZipFile.extract()` and `ZipFile.extractall()` methods, it is the caller's responsibility to validate or sanitize filenames to prevent path traversal vulnerabilities.

`Path` objects expose the following features of `pathlib.Path` objects:

`Path` objects are traversable using the `/` operator or `joinpath`.

Path.name

The final path component.

Path.open(mode='r', *, pwd, **)

Invoke `ZipFile.open()` on the current path. Allows opening for read or write, text or binary through supported modes: 'r', 'w', 'rb', 'wb'. Positional and keyword arguments are passed through to `io.TextIOWrapper` when opened as text and ignored otherwise. `pwd` is the `pwd` parameter to `ZipFile.open()`. Changed in version 3.9: Added support for text and binary modes for `open`. Default mode is now text. Changed in version 3.11.2: The `encoding` parameter can be supplied as a positional argument without causing a `TypeError`.

Path.iterdir()

Enumerate the children of the current directory.

Path.is_dir()

Return `True` if the current context references a directory.

Path.is_file()

Return `True` if the current context references a file.

Path.is_symlink()

Return True if the current context references a symbolic link. Added in version 3.12. Changed in version 3.13: Previously, `is_symlink` would unconditionally return False.

Path.exists()

Return True if the current context references a file or directory in the zip file.

Path.suffix

The last dot-separated portion of the final component, if any. This is commonly called the file extension. Added in version 3.11: Added `Path.suffix` property.

Path.stem

The final path component, without its suffix. Added in version 3.11: Added `Path.stem` property.

Path.suffixes

A list of the path's suffixes, commonly called file extensions. Added in version 3.11: Added `Path.suffixes` property.

Path.read_text(*, **)

Read the current file as unicode text. Positional and keyword arguments are passed through `io.TextIOWrapper` (except `buffer`, which is implied by the context). Changed in version 3.11.2: The `encoding` parameter can be supplied as a positional argument without causing a `TypeError`. As it could in 3.9. Code needing to be compatible with unpatched 3.10 and 3.11 versions must pass `allio.TextIOWrapper` arguments, `encoding` included, as keywords.

Path.read_bytes()

Read the current file as bytes.

Path.joinpath(*other)

Return a new `Path` object with each of the `other` arguments joined. The following are equivalent: `>>> Path(...).joinpath('child').joinpath('grandchild') >>> Path(...).joinpath('child', 'grandchild') >>> Path(...)/'child'/'grandchild'` Change in version 3.10: Prior to 3.10, `joinpath` was undocumented and accepted exactly one parameter.

The `zip` project provides backports of the latest path object functionality to older Pythons. Use `zip.Path` in place of `zipfile.Path` for early access to changes.

PyZipFile objects

The `PyZipFile` constructor takes the same parameters as the `ZipFile` constructor, and one additional parameter, `optimize`.

`class zipfile.PyZipFile(file, mode='r', compression=ZIP_STORED, allowZip64=True, optimize=-1)`

`writepy(pathname, basename="", filterfunc=None)`

Changed in version 3.2: Added the `optimize` parameter. Changed in version 3.4: ZIP64 extensions are enabled by default. Instances have one method in addition to those of `ZipFile` objects: `writepy(pathname, basename="", filterfunc=None)` Search for files `*.py` and add the corresponding file to the archive. If the `optimize` parameter to `PyZipFile` was not given or -1, the corresponding file is a `*.pyc` file, compiling if necessary. If the `optimize` parameter to `PyZipFile` was 0, 1 or 2, only files with that optimization level (see `compile`)

Search for files `*.py` and add the corresponding file to the archive. If the `optimize` parameter to `PyZipFile` was not given or -1, the corresponding file is a `*.pyc` file, compiling if necessary. If the `optimize` parameter to `PyZipFile` was 0, 1 or 2, only files with that optimization level (see `compile()`) are added to the archive, compiling if

necessary. If `pathname` is a file, the filename must end with `.py`, and just the (corresponding `*.pyc`) file is added at the top level (no path information). If `pathname` is a file that do

ZipInfo objects

Instances of the `ZipInfo` class are returned by the `getinfo()` and `infolist()` methods of `ZipFile` objects. Each object stores information about a single member of the ZIP archive.

There is one class method to make a `ZipInfo` instance for a filesystem file:

classmethod ZipInfo.from_file(filename, arcname=None, *, strict_timestamps=True)

Construct a `ZipInfo` instance for a file on the filesystem, in preparation for adding it to a zip file. `filename` should be the path to a file or directory on the filesystem. If `arcname` is specified, it is used as the name within the archive. If `arcname` is not specified, the name will be the same as `filename`, but with any drive letter and leading path separators removed. The `strict_timestamps` argument, when set to `False`, allows to zip files older than 1980-01-01 at the cost of setting the timestamp to 1980-01-01

Instances have the following methods and attributes:

ZipInfo.is_dir()

Return `True` if this archive member is a directory. This uses the entry's name: directories should always end with `/`. Added in version 3.6.

ZipInfo.filename

Name of the file in the archive.

ZipInfo.date_time

The time and date of the last modification to the archive member. This is a tuple of six values representing the “last [modified] file time” and “last [modified] file date” fields from the ZIP file's central directory. The tuple contains: `Index` `Value` `0` Year (≥ 1980) `1` Month (one-based) `2` Day of month (one-based) `3` Hours (zero-based) `4` Minutes (zero-based) `5` Seconds (zero-based) Note The ZIP format supports multiple timestamp fields in different locations (central directory, extra fields for NTFS/UNIX systems,

ZipInfo.compress_type

Type of compression for the archive member.

ZipInfo.comment

Comment for the individual archive member as a bytes object.

ZipInfo.extra

Expansion field data. The PKZIP Application Note contains some comments on the internal structure of the data contained in this bytes object.

ZipInfo.create_system

System which created ZIP archive.

ZipInfo.create_version

PKZIP version which created ZIP archive.

ZipInfo.extract_version

PKZIP version needed to extract archive.

ZipInfo.reserved

Must be zero.

ZipInfo.flag_bits

ZIP flag bits.

ZipInfo.volume

Volume number of file header.

ZipInfo.internal_attr

Internal attributes.

ZipInfo.external_attr

External file attributes.

ZipInfo.header_offset

Byte offset to the file header.

ZipInfo.CRC

CRC-32 of the uncompressed file.

ZipInfo.compress_size

Size of the compressed data.

ZipInfo.file_size

Size of the uncompressed file.

Command-line interface

The `zipfile` module provides a simple command-line interface to interact with ZIP archives.

If you want to create a new ZIP archive, specify its name after the `-c` option and then list the filename(s) that should be included:

```
$ python -m zipfile -c monty.zip spam.txt eggs.txt
```

Passing a directory is also acceptable:

```
$ python -m zipfile -c monty.zip life-of-brian_1979/
```

If you want to extract a ZIP archive into the specified directory, use the `-e` option:

```
$ python -m zipfile -e monty.zip target-dir/
```

For a list of the files in a ZIP archive, use the `-l` option:

```
$ python -m zipfile -l monty.zip
```

Command-line options

-l<zipfile>

--list<zipfile>

List files in a zipfile.

-c<zipfile><source1>...<sourceN>

--create<zipfile><source1>...<sourceN>

Create zipfile from source files.

-e<zipfile><output_dir>

--extract<zipfile><output_dir>

Extract zipfile into target directory.

-t<zipfile>

--test<zipfile>

Test whether the zipfile is valid or not.

--metadata-encoding<encoding>

Specify encoding of member names for -l, -e and -t. Added in version 3.11.

Decompression pitfalls

The extraction in zipfile module might fail due to some pitfalls listed below.

From file itself

Decompression may fail due to incorrect password / CRC checksum / ZIP format or unsupported compression method / decryption.

File system limitations

Exceeding limitations on different file systems can cause decompression failed. Such as allowable characters in the directory entries, length of the file name, length of the pathname, size of a single file, and number of files, etc.

Resources limitations

The lack of memory or disk volume would lead to decompression failed. For example, decompression bombs (aka ZIP bomb) apply to zipfile library that can cause disk volume exhaustion.

Interruption

Interruption during the decompression, such as pressing control-C or killing the decompression process may result in incomplete decompression of the archive.

Default behaviors of extraction

Not knowing the default extraction behaviors can cause unexpected decompression results. For example, when extracting the same archive twice, it overwrites files without asking.

tarfile— Read and write tar archive files

Source code: `Lib/tarfile.py`

The `tarfile` module makes it possible to read and write tar archives, including those using gzip, bz2 and lzma compression. Use the `zipfile` module to read or write .zip files, or the higher-level functions in `shutil`.

Some facts and figures:

- reads and writes gzip, bz2, compression.zstd, and lzma compressed archives if the respective modules are available. If any of these optional modules are missing from your copy of CPython, look for documentation from your distributor (that is, whoever provided Python to you). If you are the distributor, see Requirements for optional modules.
- read/write support for the POSIX.1-1988 (ustar) format.
- read/write support for the GNU tar format including longname and longlink extensions, read-only support for all variants of the sparse extension including restoration of sparse files.
- read/write support for the POSIX.1-2001 (pax) format.
- handles directories, regular files, hardlinks, symbolic links, fifos, character devices and block devices and is able to acquire and restore file information like timestamp, access permissions and owner.

Changed in version 3.3: Added support for lzma compression.

Changed in version 3.12: Archives are extracted using a filter, which makes it possible to either limit surprising/dangerous features, or to acknowledge that they are expected and the archive is fully trusted.

Changed in version 3.14: Set the default extraction filter to `data`, which disallows some dangerous features such as links to absolute paths or paths outside of the destination. Previously, the filter strategy was equivalent to `fully_trusted`.

Changed in version 3.14: Added support for Zstandard compression using `compression.zstd`.

`tarfile.open(name=None, mode='r', fileobj=None, bufsize=10240, **kwargs)`

Return a `TarFile` object for the path `name`. For detailed information on `TarFile` objects and the keyword arguments that are allowed, see `TarFile` Objects. `mode` has to be a string of the form `'filemode[:compression]'`, it defaults to `'r'`. Here is a full list of mode combinations: `mode` `action` `'r'` or `'r:*` Open for reading with transparent compression (recommended). `'r'` Open for reading exclusively without compression. `'r:gz'` Open for reading with gzip compression. `'r:bz2'` Open for reading with bz2 compression. `'r:xz'`

`class tarfile.TarFile`

Class for reading and writing tar archives. Do not use this class directly: use `tarfile.open()` instead. See `TarFile` Objects.

`tarfile.is_tarfile(name)`

Return `True` if `name` is a tar archive file, that the `tarfile` module can read. `name` may be a str, file, or file-like object. Changed in version 3.9: Support for file and file-like objects.

The `tarfile` module defines the following exceptions:

`exception tarfile.TarError`

Base class for all `tarfile` exceptions.

`exception tarfile.ReadError`

Is raised when a tar archive is opened, that either cannot be handled by the `tarfile` module or is somehow invalid.

exceptiontarfile.CompressionError

Is raised when a compression method is not supported or when the data cannot be decoded properly.

exceptiontarfile.StreamError

Is raised for the limitations that are typical for stream-like `TarFile` objects.

exceptiontarfile.ExtractError

Is raised for non-fatal errors when using `TarFile.extract()`, but only if `TarFile.errorlevel==2`.

exceptiontarfile.HeaderError

Is raised by `TarInfo.frombuf()` if the buffer it gets is invalid.

exceptiontarfile.FilterError

tarinfo

Base class for members refused by filters. `tarinfo` information about the member that the filter refused to extract, as `TarInfo`.

Information about the member that the filter refused to extract, as `TarInfo`.

exceptiontarfile.AbsolutePathError

Raised to refuse extracting a member with an absolute path.

exceptiontarfile.OutsideDestinationError

Raised to refuse extracting a member outside the destination directory.

exceptiontarfile.SpecialFileError

Raised to refuse extracting a special file (e.g. a device or pipe).

exceptiontarfile.AbsoluteLinkError

Raised to refuse extracting a symbolic link with an absolute path.

exceptiontarfile.LinkOutsideDestinationError

Raised to refuse extracting a symbolic link pointing outside the destination directory.

exceptiontarfile.LinkFallbackError

Raised to refuse emulating a link (hard or symbolic) by extracting another archive member, when that member would be rejected by the filter location. The exception that was raised to reject the replacement member is available as `BaseException.__context__`. Added in version 3.14.

The following constants are available at the module level:

tarfile.ENCODING

The default character encoding: 'utf-8' on Windows, the value returned by `sys.getfilesystemencoding()` otherwise.

tarfile.REGTYPE

tarfile.AREGTYPE

A regular filetype.

tarfile.LNKTYPE

A link (inside `tarfile`) type.

tarfile.SYMTYPE

A symbolic linktype.

tarfile.CHRTYPE

A character special devicetype.

tarfile.BLKTYPE

A block special devicetype.

tarfile.DIRTYPE

A directorytype.

tarfile.FIFOTYPE

A FIFO special devicetype.

tarfile.CONTTYPE

A contiguous filetype.

tarfile.GNUTYPE_LONGNAME

A GNU tar longnametype.

tarfile.GNUTYPE_LONGLINK

A GNU tar longlinktype.

tarfile.GNUTYPE_SPARSE

A GNU tar sparse filetype.

Each of the following constants defines a tar archive format that the `tarfile` module is able to create. See [section Supported tar formats](#) for details.

tarfile.USTAR_FORMAT

POSIX.1-1988 (ustar) format.

tarfile.GNU_FORMAT

GNU tar format.

tarfile.PAX_FORMAT

POSIX.1-2001 (pax) format.

tarfile.DEFAULT_FORMAT

The default format for creating archives. This is currently `PAX_FORMAT`. Changed in version 3.8: The default format for new archives was changed to `PAX_FORMAT` from `GNU_FORMAT`.

See also

Module `zipfile`**Archiving operations****GNU tar manual, Basic Tar Format**

Documentation of the `zipfile` standard module.

Documentation of the higher-level archiving facilities provided by the standard `shutil` module.

Documentation for tar archive files, including GNU tar extensions.

TarFile Objects

The `TarFile` object provides an interface to a tar archive. A tar archive is a sequence of blocks. An archive member (a stored file) is made up of a header block followed by data blocks. It is possible to store a file in a tar archive several times. Each archive member is represented by a `TarInfo` object, see `TarInfo` Objects for details.

A `TarFile` object can be used as a context manager in a `with` statement. It will automatically be closed when the block is completed. Please note that in the event of an exception an archive opened for writing will not be finalized; only the internally used file object will be closed. See the `Examples` section for a use case.

Added in version 3.2: Added support for the context management protocol.

`class tarfile.TarFile(name=None, mode='r', fileobj=None, format=DEFAULT_FORMAT, tarinfo=TarInfo, dereference=False, ignore_zeros=False, encoding=ENCODING, errors='surrogateescape', pax_headers=None, debug=0, errorlevel=1, stream=False)`

All following arguments are optional and can be accessed as instance attributes as well. `name` is the pathname of the archive. `name` may be a path-like object. It can be omitted if `fileobj` is given. In this case, the file object's `name` attribute is used if it exists. `mode` is either 'r' to read from an existing archive, 'a' to append data to an existing file, 'w' to create a new file overwriting an existing one, or 'x' to create a new file only if it does not already exist. If `fileobj` is given, it is used for reading or

`classmethod TarFile.open(...)`

Alternative constructor. The `tarfile.open()` function is actually a shortcut to this class method.

`TarFile.getmember(name)`

Return a `TarInfo` object for member `name`. If `name` can not be found in the archive, `KeyError` is raised. Note if a member occurs more than once in the archive, its last occurrence is assumed to be the most up-to-date version.

`TarFile.getmembers()`

Return the members of the archive as a list of `TarInfo` objects. The list has the same order as the members in the archive.

`TarFile.getnames()`

Return the members as a list of their names. It has the same order as the list returned by `getmembers()`.

`TarFile.list(verbose=True, *, members=None)`

Print a table of contents to `sys.stdout`. If `verbose` is `False`, only the names of the members are printed. If it is `True`, output similar to that of `ls -l` is produced. If optional `members` is given, it must be a subset of the list returned by `getmembers()`. Changed in version 3.5: Added the `members` parameter.

`TarFile.next()`

Return the next member of the archive as a `TarInfo` object, when `TarFile` is opened for reading. Return `None` if there is no more available.

`TarFile.extractall(path='.', members=None, *, numeric_owner=False, filter=None)`

Extract all members from the archive to the current working directory or directory `path`. If optional `members` is given, it must be a subset of the list returned by `getmembers()`. Directory information like owner, modification time and permissions are set after all members have been extracted. This is done to work around two problems: A directory's modification time is reset each time a file is created in it. And, if a directory's permissions do not allow writing, extracting files to it will fail. If `num`

`TarFile.extract(member, path="", set_attrs=True, *, numeric_owner=False, filter=None)`

Extract a member from the archive to the current working directory, using its full name. Its file information is extracted as accurately as possible. `member` may be a filename or a `TarInfo` object. You can specify a different directory using `path`. `path` may be a path-like object. File attributes (owner, mtime, mode) are set unless `set_attrs` is false. Then numeric `owner` and filter arguments are the same as for `extractall()`. Note The `extract()` method does not take care of several extraction issues. In most cases you should

TarFile.extractfile(member)

Extract a member from the archive as a file object. `member` may be a filename or a `TarInfo` object. If `member` is a regular file or a link, an `io.BufferedReader` object is returned. For all other existing members, `None` is returned. If `member` does not appear in the archive, `KeyError` is raised. Changed in version 3.3: Return an `io.BufferedReader` object. Changed in version 3.13: The returned `io.BufferedReader` object has the `mode` attribute which is always equal to `'rb'`.

TarFile.errorlevel:int

If `errorlevel` is 0, errors are ignored when using `TarFile.extract()` and `TarFile.extractall()`. Nevertheless, they appear as error messages in the debug output when `debug` is greater than 0. If 1 (the default), all fatal errors are raised as `OSError` or `FilterError` exceptions. If 2, all non-fatal errors are raised as `TarError` exceptions as well. Some exceptions, e.g. ones caused by wrong argument types or data corruption, are always raised. Custom extraction filters should raise `FilterError` for fatal errors and `ExtractError` for non

TarFile.extraction_filter

Added in version 3.12. The extraction filter used as a default for the `filter` argument of `extract()` and `extractall()`. The attribute may be `None` or a callable. String names are not allowed for this attribute, unlike the `filter` argument to `extract()`. If `extraction_filter` is `None` (the default), extraction methods will use the `datafilter` by default. The attribute may be set on instances or overridden in subclasses. It also is possible to set it on the `TarFile` class itself to set a global default, although, since it affects

TarFile.add(name, arcname=None, recursive=True, *, filter=None)

Add the `filename` to the archive. `name` may be any type of file (directory, fifo, symbolic link, etc.). If given, `arcname` specifies an alternative name for the file in the archive. Directories are added recursively by default. This can be avoided by setting `recursive` to `False`. Recursion adds entries in sorted order. If `filter` is given, it should be a function that takes a `TarInfo` object argument and returns the changed `TarInfo` object. If it instead returns `None` the `TarInfo` object will be excluded from the archive.

TarFile.addfile(tarinfo, fileobj=None)

Add the `TarInfo` object `tarinfo` to the archive. If `tarinfo` represents a non zero-size regular file, the `fileobj` argument should be a binary file, and `tarinfo.size` bytes are read from it and added to the archive. You can create `TarInfo` objects directly, or by using `gettarinfo()`. Changed in version 3.13: `fileobj` must be given for non-zero-sized regular files.

TarFile.gettarinfo(name=None, arcname=None, fileobj=None)

Create a `TarInfo` object from the result of `os.stat()` or equivalent on an existing file. The file is either named by `name`, or specified as a file object `fileobj` with a file descriptor. `name` may be a path-like object. If given, `arcname` specifies an alternative name for the file in the archive, otherwise, the name is taken from `fileobj.name` attribute, or the `name` argument. The name should be a text string. You can modify some of the `TarInfo`'s attributes before you add it using `addfile()`. If the file object is not

TarFile.close()

Close the `TarFile`. In write mode, two finishing zero blocks are appended to the archive.

TarFile.pax_headers:dict

A dictionary containing key-value pairs of pax global headers.

TarInfo Objects

A `TarInfo` object represents one member in a `TarFile`. Aside from storing all required attributes of a file (like file type, size, time, permissions, owner etc.), it provides some useful methods to determine its type. It does not contain the file's data itself.

`TarInfo` objects are returned by `TarFile`'s methods `getmember()`, `getmembers()` and `gettarinfo()`.

Modifying the objects returned by `getmember()` or `getmembers()` will affect all subsequent operations on the archive. For cases where this is unwanted, you can use `copy.copy()` or call the `replace()` method to create a modified copy in one step.

Several attributes can be set to `None` to indicate that a piece of metadata is unused or unknown. Different `TarInfo` methods handle `None` differently:

- The `extract()` or `extractall()` methods will ignore the corresponding metadata, leaving it set to a default.
- `addfile()` will fail.
- `list()` will print a placeholder string.

class `tarfile.TarInfo(name="")`

Create a `TarInfo` object.

class method `TarInfo.frombuf(buf, encoding, errors)`

Create and return a `TarInfo` object from string buffer `buf`. Raises `HeaderError` if the buffer is invalid.

class method `TarInfo.fromtarfile(tarfile)`

Read the next member from the `TarFile` object `tarfile` and return it as a `TarInfo` object.

`TarInfo.tobuf(format=DEFAULT_FORMAT, encoding=ENCODING, errors='surrogateescape')`

Create a string buffer from a `TarInfo` object. For information on the arguments see the constructor of the `TarFile` class. Changed in version 3.2: Use 'surrogateescape' as the default for the `errors` argument.

A `TarInfo` object has the following public data attributes:

`TarInfo.name:str`

Name of the archive member.

`TarInfo.size:int`

Size in bytes.

`TarInfo.mtime:int|float`

Time of last modification in seconds since the epoch, as `ino.stat_result.st_mtime`. Changed in version 3.12: Can be set to `None` for `extract()` and `extractall()`, causing extraction to skip applying this attribute.

`TarInfo.mode:int`

Permission bits, as for `os.chmod()`. Changed in version 3.12: Can be set to `None` for `extract()` and `extractall()`, causing extraction to skip applying this attribute.

`TarInfo.type`

File type. `type` is usually one of these constants: `REGTYPE`, `AREGTYPE`, `LNKTYPE`, `SYMTYPE`, `DIRTYPE`, `FIFOTYPE`, `CONTTYPE`, `CHRTYPE`, `BLKTYPE`, `GNUTYPE_SPARSE`. To determine the type of a `TarInfo` object more conveniently, use the `is*()` methods below.

TarInfo.linkname:str

Name of the target file name, which is only present inTarInfoobjects of typeLNKTYPEandSYMTYPE.For symbolic links (SYMTYPE), thelinknameis relative to the directory that contains the link. For hard links (LNKTYPE), thelinknameis relative to the root of the archive.

TarInfo.uid:int

User ID of the user who originally stored this member.Changed in version 3.12:Can be set toNoneforextract()andextractall(), causing extraction to skip applying this attribute.

TarInfo.gid:int

Group ID of the user who originally stored this member.Changed in version 3.12:Can be set toNoneforextract()andextractall(), causing extraction to skip applying this attribute.

TarInfo.uname:str

User name.Changed in version 3.12:Can be set toNoneforextract()andextractall(), causing extraction to skip applying this attribute.

TarInfo.gname:str

Group name.Changed in version 3.12:Can be set toNoneforextract()andextractall(), causing extraction to skip applying this attribute.

TarInfo.chksum:int

Header checksum.

TarInfo.devmajor:int

Device major number.

TarInfo.devminor:int

Device minor number.

TarInfo.offset:int

The tar header starts here.

TarInfo.offset_data:int

The file's data starts here.

TarInfo.sparse

Sparse member information.

TarInfo.pax_headers:dict

A dictionary containing key-value pairs of an associated pax extended header.

TarInfo.replace(name=...,mtime=...,mode=...,linkname=...,uid=...,gid=...,uname=...,gname=...,deep=True)

Added in version 3.12.Return anewcopy of theTarInfoobject with the given attributes changed. For example, to return aTarInfowith the group name set to'staff', use:new_tarinfo=old_tarinfo.replace(gname='staff')By default, a deep copy is made. Ifdeepis false, the copy is shallow, i.e.pax_headersand any custom attributes are shared with the originalTarInfoobject.

ATarInfoobject also provides some convenient query methods:

TarInfo.isfile()

ReturnTrueif theTarInfoobject is a regular file.

TarInfo.isreg()

Same as `isfile()`.

TarInfo.isdir()

Return `True` if it is a directory.

TarInfo.issym()

Return `True` if it is a symbolic link.

TarInfo.islnk()

Return `True` if it is a hard link.

TarInfo.ischr()

Return `True` if it is a character device.

TarInfo.isblk()

Return `True` if it is a block device.

TarInfo.isfifo()

Return `True` if it is a FIFO.

TarInfo.isdev()

Return `True` if it is one of character device, block device or FIFO.

Extraction filters

Added in version 3.12.

The `tar` format is designed to capture all details of a UNIX-like filesystem, which makes it very powerful. Unfortunately, the features make it easy to create tar files that have unintended – and possibly malicious – effects when extracted. For example, extracting a tar file can overwrite arbitrary files in various ways (e.g. by using absolute paths, path components, or symlinks that affect later members).

In most cases, the full functionality is not needed. Therefore, `tarfile` supports extraction filters: a mechanism to limit functionality, and thus mitigate some of the security issues.

■■■ Advertencia: Warning None of the available filters blocks all dangerous archive features. Never extract archives from untrusted sources without prior inspection. See also `Hints` for further verification.

See also

PEP 706

Contains further motivation and rationale behind the design.

The `filter` argument to `TarFile.extract()` or `extractall()` can be:

- the string `'fully_trusted'`: Honor all metadata as specified in the archive. Should be used if the user trusts the archive completely, or implements their own complex verification.
- the string `'tar'`: Honor most `tar`-specific features (i.e. features of UNIX-like filesystems), but block features that are very likely to be surprising or malicious. See `tar_filter()` for details.
- the string `'data'`: Ignore or block most features specific to UNIX-like filesystems. Intended for extracting cross-platform data archives. See `data_filter()` for details.

- `None`(default): Use `TarFile.extraction_filter`. If that is also `None` (the default), the 'data' filter will be used. Changed in version 3.14: The default filter is set to `data`. Previously, the default was equivalent to `fully_trusted`.
- A callable which will be called for each extracted member with a `TarInfo` describing the member and the destination path to where the archive is extracted (i.e. the same path is used for all members): `filter(member:TarInfo,path:str,/)->TarInfo|None`. The callable is called just before each member is extracted, so it can take the current state of the disk into account. It can: return a `TarInfo` object which will

Default named filters

The pre-defined, named filters are available as functions, so they can be reused in custom filters:

`tarfile.fully_trusted_filter(member,path)`

Return member unchanged. This implements the 'fully_trusted' filter.

`tarfile.tar_filter(member,path)`

Implements the 'tar' filter. Strip leading slashes (/ and os.sep) from filenames. Refuse to extract files with absolute paths (in case the name is absolute even after stripping slashes, e.g. `C:/foo` on Windows). This raises `AbsolutePathError`. Refuse to extract files whose absolute path (after following symlinks) would end up outside the destination. This raises `OutsideDestinationError`. Clear high mode bits (`setuid`, `setgid`, `sticky`) and group/other write bits (`S_IWGRP`|`S_IWOTH`). Return the modified `TarInfo` member.

`tarfile.data_filter(member,path)`

Implements the 'data' filter. In addition to what `tar_filter` does: Normalize link targets (`TarInfo.linkname`) using `os.path.normpath()`. Note that this removes internal..components, which may change the meaning of the link if the path in `TarInfo.linkname` traverses symbolic links. Refuse to extract links (hard or soft) that link to absolute paths, or ones that link outside the destination. This raises `AbsoluteLinkError` or `LinkOutsideDestinationError`. Note that such files are refused even on platforms that do not

Filter errors

When a filter refuses to extract a file, it will raise an appropriate exception, a subclass of `FilterError`. This will abort the extraction if `TarFile.errorlevel` is 1 or more. With `errorlevel=0` the error will be logged and the member will be skipped, but extraction will continue.

Hints for further verification

Even with `filter='data'`, `tarfile` is not suited for extracting untrusted files without prior inspection. Among other issues, the pre-defined filters do not prevent denial-of-service attacks. Users should do additional checks.

Here is an incomplete list of things to consider:

- Extract to a new temporary directory to prevent e.g. exploiting pre-existing links, and to make it easier to clean up after a failed extraction.
- Disallow symbolic links if you do not need the functionality.
- When working with untrusted data, use external (e.g. OS-level) limits on disk, memory and CPU usage.
- Check filenames against an allow-list of characters (to filter out control characters, confusables, foreign path separators, and so on).
- Check that filenames have expected extensions (discouraging files that execute when you "click on them", or extension-less files like Windows special device names).
- Limit the number of extracted files, total size of extracted data, filename length (including symlink length), and size of individual files.

- Check for files that would be shadowed on case-insensitive filesystems.

Also note that:

- Tar files may contain multiple versions of the same file. Later ones are expected to overwrite any earlier ones. This feature is crucial to allow updating tape archives, but can be abused maliciously.
- `tarfile` does not protect against issues with “live” data, e.g. an attacker tinkering with the destination (or source) directory while extraction (or archiving) is in progress.

Supporting older Python versions

Extraction filters were added to Python 3.12, but may be backported to older versions as security updates. To check whether the feature is available, use e.g. `hasattr(tarfile, 'data_filter')` rather than checking the Python version.

The following examples show how to support Python versions with and without the feature. Note that setting `extraction_filter` will affect any subsequent operations.

- Fully trusted archive: `my_tarfile.extraction_filter=(lambda member, path: member)my_tarfile.extractall()`
- Use the 'data' filter if available, but revert to Python 3.11 behavior ('fully_trusted') if this feature is not available: `my_tarfile.extraction_filter=getattr(tarfile, 'data_filter', (lambda member, path: member))my_tarfile.extractall()`
- Use the 'data' filter; fail if it is not available: `my_tarfile.extractall(filter=tarfile.data_filter)or:my_tarfile.extraction_filter=tarfile.data_filtermy_tarfile.extractall()`
- Use the 'data' filter; warn if it is not available: `ifhasattr(tarfile, 'data_filter'):my_tarfile.extractall(filter='data')else:## remove this when no longer neededwarn_the_user('Extracting may be unsafe; consider updating Python')my_tarfile.extractall()`

Stateful extraction filter example

While `tarfile`'s extraction methods take a simple filter callable, custom filters may be more complex objects with an internal state. It may be useful to write these as context managers, to be used like this:

```
with StatefulFilter() as filter_func:
    tar.extractall(path, filter=filter_func)
```

Such a filter can be written as, for example:

```
class StatefulFilter:
    def __init__(self):
        self.file_count = 0

    def __enter__(self):
        return self

    def __call__(self, member, path):
        self.file_count += 1
        return member

    def __exit__(self, *exc_info):
        print(f'{self.file_count} files extracted')
```

Command-Line Interface

Added in version 3.4.

The `tarfile` module provides a simple command-line interface to interact with tar archives.

If you want to create a new tar archive, specify its name after the -c option and then list the filename(s) that should be included:

```
$ python -m tarfile -c monty.tar spam.txt eggs.txt
```

Passing a directory is also acceptable:

```
$ python -m tarfile -c monty.tar life-of-brian_1979/
```

If you want to extract a tar archive into the current directory, use the -e option:

```
$ python -m tarfile -e monty.tar
```

You can also extract a tar archive into a different directory by passing the directory's name:

```
$ python -m tarfile -e monty.tar other-dir/
```

For a list of the files in a tar archive, use the -l option:

```
$ python -m tarfile -l monty.tar
```

Command-line options

-l<tarfile>

--list<tarfile>

List files in a tarfile.

-c<tarfile><source1>...<sourceN>

--create<tarfile><source1>...<sourceN>

Create tarfile from source files.

-e<tarfile>[<output_dir>]

--extract<tarfile>[<output_dir>]

Extract tarfile into the current directory if <output_dir> is not specified.

-t<tarfile>

--test<tarfile>

Test whether the tarfile is valid or not.

-v,--verbose

Verbose output.

--filter<filtername>

Specifies the filter for --extract. See Extraction filters for details. Only string names are accepted (that is, fully_trusted, tar, and data).

Examples

Reading examples

How to extract an entire tar archive to the current working directory:

```
import tarfile
tar = tarfile.open("sample.tar.gz")
tar.extractall(filter='data')
tar.close()
```

How to extract a subset of a tar archive with `TarFile.extractall()` using a generator function instead of a list:

```
import os
import tarfile

def py_files(members):
    for tarinfo in members:
        if os.path.splitext(tarinfo.name)[1] == ".py":
            yield tarinfo

tar = tarfile.open("sample.tar.gz")
tar.extractall(members=py_files(tar))
tar.close()
```

How to read a gzip compressed tar archive and display some member information:

```
import tarfile
tar = tarfile.open("sample.tar.gz", "r:gz")
for tarinfo in tar:
    print(tarinfo.name, "is", tarinfo.size, "bytes in size and is ", end="")
    if tarinfo.isreg():
        print("a regular file.")
    elif tarinfo.isdir():
        print("a directory.")
    else:
        print("something else.")
tar.close()
```

Writing examples

How to create an uncompressed tar archive from a list of filenames:

```
import tarfile
tar = tarfile.open("sample.tar", "w")
for name in ["foo", "bar", "quux"]:
    tar.add(name)
tar.close()
```

The same example using the `with` statement:

```
import tarfile
with tarfile.open("sample.tar", "w") as tar:
    for name in ["foo", "bar", "quux"]:
        tar.add(name)
```

How to create and write an archive to stdout using `sys.stdout.buffer` in the `fileobj` parameter in `TarFile.add()`:

```
import sys
import tarfile
with tarfile.open("sample.tar.gz", "w|gz", fileobj=sys.stdout.buffer) as tar:
    for name in ["foo", "bar", "quux"]:
        tar.add(name)
```

How to create an archive and reset the user information using the `filter` parameter in `TarFile.add()`:

```
import tarfile
def reset(tarinfo):
    tarinfo.uid = tarinfo.gid = 0
    tarinfo.uname = tarinfo.gname = "root"
    return tarinfo
tar = tarfile.open("sample.tar.gz", "w:gz")
tar.add("foo", filter=reset)
tar.close()
```

Supported tar formats

There are three tar formats that can be created with the `tarfile` module:

- The POSIX.1-1988 ustar format (`USTAR_FORMAT`). It supports filenames up to a length of at best 256 characters and linknames up to 100 characters. The maximum file size is 8 GiB. This is an old and limited but widely supported format.
- The GNU tar format (`GNU_FORMAT`). It supports long filenames and linknames, files bigger than 8 GiB and sparse files. It is the de facto standard on GNU/Linux systems. `tarfile` fully supports the GNU tar extensions for long names, sparse file support is read-only.
- The POSIX.1-2001 pax format (`PAX_FORMAT`). It is the most flexible format with virtually no limits. It supports long filenames and linknames, large files and stores pathnames in a portable way. Modern tar implementations, including GNU tar, `bsdtar/libarchive` and `star`, fully support extended pax features; some old or unmaintained libraries may not, but should treat pax archives as if they were in the `ustar` format.

There are some more variants of the tar format which can be read, but not created:

- The ancient V7 format. This is the first tar format from Unix Seventh Edition, storing only regular files and directories. Names must not be longer than 100 characters, there is no user/group name information. Some archives have miscalculated header checksums in case of fields with non-ASCII characters.
- The SunOS tar extended format. This format is a variant of the POSIX.1-2001 pax format, but is not compatible.

Unicode issues

The tar format was originally conceived to make backups on tape drives with the main focus on preserving file system information. Nowadays tar archives are commonly used for file distribution and exchanging archives over networks. One problem of the original format (which is the basis of all other formats) is that there is no concept of supporting different character encodings. For example, an ordinary tar archive created on a UTF-8 system cannot be read correctly on a Latin-1 system if it contains non-ASCII characters. Textual metadata (like filenames, linknames, user/group names) will appear damaged. Unfortunately, there is no way to autodetect the encoding of an archive. The pax format was designed to solve this problem. It stores non-ASCII metadata using the universal character encoding UTF-8.

The details of character conversion in `tarfile` are controlled by the `encoding` and `errors` keyword arguments of the `TarFile` class.

`encoding` defines the character encoding to use for the metadata in the archive. The default value is `sys.getfilesystemencoding()` or `'ascii'` as a fallback. Depending on whether the archive is read or written, the metadata must be either decoded or encoded. If `encoding` is not set appropriately, this conversion may fail.

The `errors` argument defines how characters are treated that cannot be converted. Possible values are listed in section [Error Handlers](#). The default scheme is `'surrogateescape'` which Python also uses for its file system calls, see [File Names](#), [Command Line Arguments](#), and [Environment Variables](#).

For `PAX_FORMAT` archives (the default), `encoding` is generally not needed because all the metadata is stored using UTF-8. `encoding` is only used in the rare cases when binary pax headers are decoded or when strings with surrogate characters are stored.

File Formats

The modules described in this chapter parse various miscellaneous file formats that aren't markup languages and are not related to e-mail.

- [csv](#)— CSV File Reading and Writing [Module Contents](#) [Dialects and Formatting Parameters](#) [Reader Objects](#) [Writer Objects](#) [Examples](#)
- [configparser](#)— Configuration file parser [Quick Start](#) [Supported Datatypes](#) [Fallback Values](#) [Supported INI File Structure](#) [Unnamed Sections](#) [Interpolation of values](#) [Mapping Protocol Access](#) [Customizing Parser Behaviour](#) [Legacy API Examples](#) [ConfigParser Objects](#) [RawConfigParser Objects](#) [Exceptions](#)
- [tomllib](#)— Parse TOML files [Examples](#) [Conversion Table](#)
- [netrc](#)— netrc file processing [netrc Objects](#)
- [plistlib](#)— Generate and parse Apple.plist files [Examples](#)

csv— CSV File Reading and Writing

Source code: `Lib/csv.py`

The so-called CSV (Comma Separated Values) format is the most common import and export format for spreadsheets and databases. CSV format was used for many years prior to attempts to describe the format in a standardized way in RFC 4180. The lack of a well-defined standard means that subtle differences often exist in the data produced and consumed by different applications. These differences can make it annoying to process CSV files from multiple sources. Still, while the delimiters and quoting characters vary, the overall format is similar enough that it is possible to write a single module which can efficiently manipulate such data, hiding the details of reading and writing the data from the programmer.

The `csv` module implements classes to read and write tabular data in CSV format. It allows programmers to say, “write this data in the format preferred by Excel,” or “read data from this file which was generated by Excel,” without knowing the precise details of the CSV format used by Excel. Programmers can also describe the CSV formats understood by other applications or define their own special-purpose CSV formats.

The `csv` module's `reader` and `writer` objects read and write sequences. Programmers can also read and write data in dictionary form using the `DictReader` and `DictWriter` classes.

See also

PEP 305- CSV File API

The Python Enhancement Proposal which proposed this addition to Python.

Module Contents

The `csv` module defines the following functions:

`csv.reader(csvfile,/,dialect='excel',**fmtparams)`

Return a reader object that will process lines from the given `csvfile`. A `csvfile` must be an iterable of strings, each in the reader's defined csv format. A `csvfile` is most commonly a file-like object or list. If `csvfile` is a file object, it should be opened with `newline=""`. [1] An optional `dialect` parameter can be given which is used to define a set of parameters specific to a particular CSV dialect. It may be an instance of a subclass of the `Dialect` class or one of the strings returned by the `list_dialect`

`csv.writer(csvfile,/,dialect='excel',**fmtparams)`

Return a writer object responsible for converting the user's data into delimited strings on the given file-like object. `csvfile` can be any object with a `write()` method. If `csvfile` is a file object, it should be opened with `newline=""` [1]. An optional `dialect` parameter can be given which is used to define a set of parameters specific to a particular CSV dialect. It may be an instance of a subclass of the `Dialect` class or one of the strings returned by the `list_dialects()` function. The other optional `fmtpara`

`csv.register_dialect(name,/,dialect='excel',**fmtparams)`

Associate `dialect` with `name`. `name` must be a string. The dialect can be specified either by passing a sub-class of `Dialect`, or by `fmtparams` keyword arguments, or both, with keyword arguments overriding parameters of the dialect. For full details about dialects and formatting parameters, see section `Dialects and Formatting Parameters`.

`csv.unregister_dialect(name)`

Delete the dialect associated with `name` from the dialect registry. An `Error` is raised if `name` is not a registered dialect name.

csv.get_dialect(name)

Return the dialect associated with `name`. An `Error` is raised if `name` is not a registered dialect name. This function returns an `immutableDialect`.

csv.list_dialects()

Return the names of all registered dialects.

csv.field_size_limit()

csv.field_size_limit(new_limit)

Returns the current maximum field size allowed by the parser. If `new_limit` is given, this becomes the new limit.

The `csv` module defines the following classes:

class csv.DictReader(f, fieldnames=None, restkey=None, restval=None, dialect='excel', *args, **kwargs)

Create an object that operates like a regular reader but maps the information in each row to a dict whose keys are given by the optional `fieldnames` parameter. The `fieldnames` parameter is a sequence. If `fieldnames` is omitted, the values in the first row of `file` will be used as the `fieldnames` and will be omitted from the results. If `fieldnames` is provided, they will be used and the first row will be included in the results. Regardless of how the `fieldnames` are determined, the dictionary preserves their orig

class csv.DictWriter(f, fieldnames, restval="", extrasaction='raise', dialect='excel', *args, **kwargs)

Create an object which operates like a regular writer but maps dictionaries onto output rows. The `fieldnames` parameter is a sequence of keys that identify the order in which values in the dictionary passed to the `writerow()` method are written to file. The optional `restval` parameter specifies the value to be written if the dictionary is missing a key in `fieldnames`. If the dictionary passed to the `writerow()` method contains a key not found in `fieldnames`, the optional `extrasaction` parameter indicates what ac

class csv.Dialect

The `Dialect` class is a container class whose attributes contain information for how to handle double quotes, whitespace, delimiters, etc. Due to the lack of a strict CSV specification, different applications produce subtly different CSV data. `Dialect` instances define how `reader` and `writer` instances behave. All available `Dialect` names are returned by `list_dialects()`, and they can be registered with specific `reader` and `writer` classes through their initializer (`__init__`) functions like this: `import csv with open('stude`

class csv.excel

The `excel` class defines the usual properties of an Excel-generated CSV file. It is registered with the dialect name 'excel'.

class csv.excel_tab

The `excel_tab` class defines the usual properties of an Excel-generated TAB-delimited file. It is registered with the dialect name 'excel-tab'.

class csv.unix_dialect

The `unix_dialect` class defines the usual properties of a CSV file generated on UNIX systems, i.e. using '\n' as line terminator and quoting all fields. It is registered with the dialect name 'unix'. Added in version 3.2.

class csv.Sniffer

sniff(sample, delimiters=None)

has_header(sample)

The `Sniffer` class is used to deduce the format of a CSV file. The `Sniffer` class provides two methods: `sniff(sample, delimiters=None)` Analyze the given `sample` and return a `Dialect` subclass reflecting the

parameters found. If the optional `delimiters` parameter is given, it is interpreted as a string containing possible valid delimiter characters. `has_header(sample)` Analyze the sample text (presumed to be in CSV format) and return `True` if the first row appears to be a series of column headers. Inspecting each column

Analyze the given sample and return a `Dialect` subclass reflecting the parameters found. If the optional `delimiters` parameter is given, it is interpreted as a string containing possible valid delimiter characters.

Analyze the sample text (presumed to be in CSV format) and return `True` if the first row appears to be a series of column headers. Inspecting each column, one of two key criteria will be considered to estimate if the sample contains a header: the second through *n*-th rows contain numeric values the second through *n*-th rows contain strings where at least one value's length differs from that of the putative header of that column. Twenty-one rows after the header are sampled; if more than half of the columns

An example for `Sniffer` use:

```
with open('example.csv', newline='') as csvfile:
    dialect = csv.Sniffer().sniff(csvfile.read(1024))
    csvfile.seek(0)
    reader = csv.reader(csvfile, dialect)
    # ... process CSV file contents here ...
```

The `csv` module defines the following constants:

csv.QUOTE_ALL

Instructs writer objects to quote all fields.

csv.QUOTE_MINIMAL

Instructs writer objects to only quote those fields which contain special characters such as `delimiter`, `quotechar`, `'\r'`, `'\n'` or any of the characters in `lineterminator`.

csv.QUOTE_NONNUMERIC

Instructs writer objects to quote all non-numeric fields. Instructs reader objects to convert all non-quoted fields to `float`. Note some numeric types, such as `bool`, `Fraction`, or `IntEnum`, have a string representation that cannot be converted to `float`. They cannot be read in the `QUOTE_NONNUMERIC` and `QUOTE_STRINGS` modes.

csv.QUOTE_NONE

Instructs writer objects to never quote fields. When the current `delimiter`, `quotechar`, `escapechar`, `'\r'`, `'\n'` or any of the characters in `lineterminator` occurs in output data it is preceded by the current `escapechar` character. If `escapechar` is not set, the writer will raise `Error` if any characters that require escaping are encountered. Set `quotechar` to `None` to prevent its escaping. Instructs reader objects to perform no special processing of quote characters.

csv.QUOTE_NOTNULL

Instructs writer objects to quote all fields which are not `None`. This is similar to `QUOTE_ALL`, except that if a field value is `None` an empty (unquoted) string is written. Instructs reader objects to interpret an empty (unquoted) field as `None` and to otherwise behave as `QUOTE_ALL`. Added in version 3.12.

csv.QUOTE_STRINGS

Instructs writer objects to always place quotes around fields which are strings. This is similar to `QUOTE_NONNUMERIC`, except that if a field value is `None` an empty (unquoted) string is written. Instructs reader objects to interpret an empty (unquoted) string as `None` and to otherwise behave as `QUOTE_NONNUMERIC`. Added in version 3.12.

The `csv` module defines the following exception:

exception csv.Error

Raised by any of the functions when an error is detected.

Dialects and Formatting Parameters

To make it easier to specify the format of input and output records, specific formatting parameters are grouped together into dialects. A dialect is a subclass of the `Dialect` class containing various attributes describing the format of the CSV file. When creating reader or writer objects, the programmer can specify a string or a subclass of the `Dialect` class as the dialect parameter. In addition to, or instead of, the dialect parameter, the programmer can also specify individual formatting parameters, which have the same names as the attributes defined below for the `Dialect` class.

Dialects support the following attributes:

Dialect.delimiter

A one-character string used to separate fields. It defaults to `,`.

Dialect.doublequote

Controls how instances of `quotechar` appearing inside a field should themselves be quoted. When `True`, the character is doubled. When `False`, the `escapechar` is used as a prefix to the `quotechar`. It defaults to `True`. On output, if `doublequote` is `False` and `noescapechar` is set, `Error` is raised if a `quotechar` is found in a field.

Dialect.escapechar

A one-character string used by the writer to escape characters that require escaping: the `delimiter`, the `quotechar`, `'\r'`, `'\n'` and any of the characters in `lineterminator` are escaped if `quoting` is set to `QUOTE_NONE`; the `quotechar` is escaped if `doublequote` is `False`; the `escapechar` itself. On reading, the `escapechar` removes any special meaning from the following character. It defaults to `None`, which disables escaping. Changed in version 3.11: An empty `escapechar` is not allowed.

Dialect.lineterminator

The string used to terminate lines produced by the writer. It defaults to `'\r\n'`. Note that the reader is hard-coded to recognise either `'\r'` or `'\n'` as end-of-line, and ignores `lineterminator`. This behavior may change in the future.

Dialect.quotechar

A one-character string used to quote fields containing special characters, such as the `delimiter` or the `quotechar`, or which contain new-line characters (`'\r'`, `'\n'` or any of the characters in `lineterminator`). It defaults to `"`. Can be set to `None` to prevent escaping `"` if `quoting` is set to `QUOTE_NONE`. Changed in version 3.11: An empty `quotechar` is not allowed.

Dialect.quoting

Controls when quotes should be generated by the writer and recognised by the reader. It can take on any of the `QUOTE_*` constants and defaults to `QUOTE_MINIMAL` if `quotechar` is not `None`, and `QUOTE_NONE` otherwise.

Dialect.skipinitialspace

When `True`, spaces immediately following the `delimiter` are ignored. The default is `False`. When `combiningdelimiter=""` with `skipinitialspace=True`, unquoted empty fields are not allowed.

Dialect.strict

When `True`, raise `exceptionError` on bad CSV input. The default is `False`.

Reader Objects

Reader objects (DictReader instances and objects returned by `thereader()` function) have the following public methods:

csvreader.__next__()

Return the next row of the reader's iterable object as a list (if the object was returned from `reader()`) or a dict (if it is a DictReader instance), parsed according to the current dialect. Usually you should call this as `next(reader)`.

Reader objects have the following public attributes:

csvreader.dialect

A read-only description of the dialect in use by the parser.

csvreader.line_num

The number of lines read from the source iterator. This is not the same as the number of records returned, as records can span multiple lines.

DictReader objects have the following public attribute:

DictReader.fieldnames

If not passed as a parameter when creating the object, this attribute is initialized upon first access or when the first record is read from the file.

Writer Objects

writer objects (DictWriter instances and objects returned by `thewriter()` function) have the following public methods. `row` must be an iterable of strings or numbers for writer objects and a dictionary mapping fieldnames to strings or numbers (by passing them through `str()` first) for DictWriter objects. Note that complex numbers are written out surrounded by parens. This may cause some problems for other programs which read CSV files (assuming they support complex numbers at all).

csvwriter.writerow(row, /)

Write the `row` parameter to the writer's file object, formatted according to the current dialect. Return the return value of the call to the `write` method of the underlying file object. Changed in version 3.5: Added support of arbitrary iterables.

csvwriter.writerows(rows, /)

Write all elements in `rows` (an iterable of row objects as described above) to the writer's file object, formatted according to the current dialect.

Writer objects have the following public attribute:

csvwriter.dialect

A read-only description of the dialect in use by the writer.

DictWriter objects have the following public method:

DictWriter.writeheader()

Write a row with the field names (as specified in the constructor) to the writer's file object, formatted according to the current dialect. Return the return value of the `csvwriter.writerow()` call used internally. Added in version 3.2. Changed in version 3.8: `writeheader()` now also returns the value returned by the `csvwriter.writerow()` method it uses internally.

Examples

The simplest example of reading a CSV file:

```
import csv
with open('some.csv', newline='') as f:
    reader = csv.reader(f)
    for row in reader:
        print(row)
```

Reading a file with an alternate format:

```
import csv
with open('passwd', newline='') as f:
    reader = csv.reader(f, delimiter=':', quoting=csv.QUOTE_NONE)
    for row in reader:
        print(row)
```

The corresponding simplest possible writing example is:

```
import csv
with open('some.csv', 'w', newline='') as f:
    writer = csv.writer(f)
    writer.writerows(someiterable)
```

Since `open()` is used to open a CSV file for reading, the file will by default be decoded into unicode using the system default encoding (see `locale.getencoding()`). To decode a file using a different encoding, use the `encoding` argument of `open`:

```
import csv
with open('some.csv', newline='', encoding='utf-8') as f:
    reader = csv.reader(f)
    for row in reader:
        print(row)
```

The same applies to writing in something other than the system default encoding: specify the encoding argument when opening the output file.

Registering a new dialect:

```
import csv
csv.register_dialect('unixpwd', delimiter=':', quoting=csv.QUOTE_NONE)
with open('passwd', newline='') as f:
    reader = csv.reader(f, 'unixpwd')
```

A slightly more advanced use of the reader — catching and reporting errors:

```
import csv, sys
filename = 'some.csv'
with open(filename, newline='') as f:
    reader = csv.reader(f)
    try:
        for row in reader:
            print(row)
    except csv.Error as e:
        sys.exit(f'file {filename}, line {reader.line_num}: {e}')
```

And while the module doesn't directly support parsing strings, it can easily be done:

```
import csv
for row in csv.reader(['one,two,three']):
    print(row)
```

Footnotes

If `newline=""` is not specified, newlines embedded inside quoted fields will not be interpreted correctly, and on platforms that use `\r\n` line endings on write an extra `\r` will be added. It should always be safe to specify `newline=""`,

since the csv module does its own (universal) newline handling.

configparser— Configuration file parser

Source code: `Lib/configparser.py`

This module provides the `ConfigParser` class which implements a basic configuration language which provides a structure similar to what's found in Microsoft Windows INI files. You can use this to write Python programs which can be customized by end users easily.

■ **Nota:** Note This library does not interpret or write the value-type prefixes used in the Windows Registry extended version of INI syntax.

See also

Module `tomllib`

Module `shlex`

Module `json`

TOML is a well-specified format for application configuration files. It is specifically designed to be an improved version of INI.

Support for creating Unix shell-like mini-languages which can also be used for application configuration files.

The `json` module implements a subset of JavaScript syntax which is sometimes used for configuration, but does not support comments.

Quick Start

Let's take a very basic configuration file that looks like this:

```
[DEFAULT]
ServerAliveInterval = 45
Compression = yes
CompressionLevel = 9
ForwardX11 = yes

[forge.example]
User = hg

[topsecret.server.example]
Port = 50022
ForwardX11 = no
```

The structure of INI files is described in the following section. Essentially, the file consists of sections, each of which contains keys with values. `configparser` classes can read and write such files. Let's start by creating the above configuration file programmatically.

```
>>> import configparser
>>> config = configparser.ConfigParser()
>>> config['DEFAULT'] = {'ServerAliveInterval': '45',
...                    'Compression': 'yes',
...                    'CompressionLevel': '9'}
>>> config['forge.example'] = {}
>>> config['forge.example']['User'] = 'hg'
>>> config['topsecret.server.example'] = {}
>>> topsecret = config['topsecret.server.example']
>>> topsecret['Port'] = '50022'      # mutates the parser
>>> topsecret['ForwardX11'] = 'no'   # same here
>>> config['DEFAULT']['ForwardX11'] = 'yes'
>>> with open('example.ini', 'w') as configfile:
...     config.write(configfile)
```

...

As you can see, we can treat a config parser much like a dictionary. There are differences, outlined later, but the behavior is very close to what you would expect from a dictionary.

Now that we have created and saved a configuration file, let's read it back and explore the data it holds.

```
>>> config = configparser.ConfigParser()
>>> config.sections()
[]
>>> config.read('example.ini')
['example.ini']
>>> config.sections()
['forge.example', 'topsecret.server.example']
>>> 'forge.example' in config
True
>>> 'python.org' in config
False
>>> config['forge.example']['User']
'hg'
>>> config['DEFAULT']['Compression']
'yes'
>>> topsecret = config['topsecret.server.example']
>>> topsecret['ForwardX11']
'no'
>>> topsecret['Port']
'50022'
>>> for key in config['forge.example']:
...     print(key)
user
compressionlevel
serveraliveinterval
compression
forwardx11
>>> config['forge.example']['ForwardX11']
'yes'
```

As we can see above, the API is pretty straightforward. The only bit of magic involves the `DEFAULT` section which provides default values for all other sections[1]. Note also that keys in sections are case-insensitive and stored in lowercase[1].

It is possible to read several configurations into a single `ConfigParser`, where the most recently added configuration has the highest priority. Any conflicting keys are taken from the more recent configuration while the previously existing keys are retained. The example below reads in an `override.ini` file, which will override any conflicting keys from the `example.ini` file.

```
[DEFAULT]
ServerAliveInterval = -1

>>> config_override = configparser.ConfigParser()
>>> config_override['DEFAULT'] = {'ServerAliveInterval': '-1'}
>>> with open('override.ini', 'w') as configfile:
...     config_override.write(configfile)
...
>>> config_override = configparser.ConfigParser()
>>> config_override.read(['example.ini', 'override.ini'])
['example.ini', 'override.ini']
>>> print(config_override.get('DEFAULT', 'ServerAliveInterval'))
-1
```

This behaviour is equivalent to a `ConfigParser.read()` call with several files passed to the `filenames` parameter.

Supported Datatypes

Config parsers do not guess datatypes of values in configuration files, always storing them internally as strings. This means that if you need other datatypes, you should convert on your own:

```
>>> int(topsecret['Port'])
50022
>>> float(topsecret['CompressionLevel'])
9.0
```

Since this task is so common, config parsers provide a range of handy getter methods to handle integers, floats and booleans. The last one is the most interesting because simply passing the value `tobool()` would do no good since `bool('False')` is still `True`. This is why config parsers also provide `getboolean()`. This method is case-insensitive and recognizes Boolean values from 'yes'/'no', 'on'/'off', 'true'/'false' and '1'/'0'[1]. For example:

```
>>> topsecret.getboolean('ForwardX11')
False
>>> config['forge.example'].getboolean('ForwardX11')
True
>>> config.getboolean('forge.example', 'Compression')
True
```

Apart from `getboolean()`, config parsers also provide equivalent `getint()` and `getfloat()` methods. You can register your own converters and customize the provided ones.[1]

Fallback Values

As with a dictionary, you can use a section's `get()` method to provide fallback values:

```
>>> topsecret.get('Port')
'50022'
>>> topsecret.get('CompressionLevel')
'9'
>>> topsecret.get('Cipher')
>>> topsecret.get('Cipher', '3des-cbc')
'3des-cbc'
```

Please note that default values have precedence over fallback values. For instance, in our example the 'CompressionLevel' key was specified only in the 'DEFAULT' section. If we try to get it from the section 'topsecret.server.example', we will always get the default, even if we specify a fallback:

```
>>> topsecret.get('CompressionLevel', '3')
'9'
```

One more thing to be aware of is that the parser-level `get()` method provides a custom, more complex interface, maintained for backwards compatibility. When using this method, a fallback value can be provided via the `fallbackkeyword-only` argument:

```
>>> config.get('forge.example', 'monster',
...           fallback='No such things as monsters')
'No such things as monsters'
```

The same `fallback` argument can be used with the `getint()`, `getfloat()` and `getboolean()` methods, for example:

```
>>> 'BatchMode' in topsecret
False
>>> topsecret.getboolean('BatchMode', fallback=True)
True
>>> config['DEFAULT']['BatchMode'] = 'no'
>>> topsecret.getboolean('BatchMode', fallback=True)
False
```

Supported INI File Structure

A configuration file consists of sections, each led by a[section]header, followed by key/value entries separated by a specific string (=or:by default[1]). By default, section names are case sensitive but keys are not[1]. Leading and trailing whitespace is removed from keys and values. Values can be omitted if the parser is configured to allow it[1], in which case the key/value delimiter may also be left out. Values can also span multiple lines, as long as they are indented deeper than the first line of the value. Depending on the parser's mode, blank lines may be treated as parts of multiline values or ignored.

By default, a valid section name can be any string that does not contain '\n'. To change this, seeConfigParser.SECTCRE.

The first section name may be omitted if the parser is configured to allow an unnamed top level section withallow_unnamed_section=True. In this case, the keys/values may be retrieved byUNNAMED_SECTIONas inconfig[UNNAMED_SECTION].

Configuration files may include comments, prefixed by specific characters (#and;by default[1]). Comments may appear on their own on an otherwise empty line, possibly indented.[1]

For example:

```
[Simple Values]
key=value
spaces in keys=allowed
spaces in values=allowed as well
spaces around the delimiter = obviously
you can also use : to delimit keys from values

[All Values Are Strings]
values like this: 1000000
or this: 3.14159265359
are they treated as numbers? : no
integers, floats and booleans are held as: strings
can use the API to get converted values directly: true

[Multiline Values]
chorus: I'm a lumberjack, and I'm okay
      I sleep all night and I work all day

[No Values]
key_without_value
empty string value here =

[You can use comments]
# like this
; or this

# By default only in an empty line.
# Inline comments can be harmful because they prevent users
# from using the delimiting characters as parts of values.
# That being said, this can be customized.

[Sections Can Be Indented]
    can_values_be_as_well = True
    does_that_mean_anything_special = False
    purpose = formatting for readability
    multiline_values = are
        handled just fine as
        long as they are indented
        deeper than the first line
        of a value
    # Did I mention we can indent comments, too?
```

Unnamed Sections

The name of the first section (or unique) may be omitted and values retrieved by the `UNNAMED_SECTION` attribute.

```
>>> config = """
... option = value
...
... [ Section 2 ]
... another = val
... """
>>> unnamed = configparser.ConfigParser(allow_unnamed_section=True)
>>> unnamed.read_string(config)
>>> unnamed.get(configparser.UNNAMED_SECTION, 'option')
'value'
```

Interpolation of values

On top of the core functionality, `ConfigParser` supports interpolation. This means values can be preprocessed before returning them from `get()` calls.

`class configparser.BasicInterpolation`

The default implementation used by `ConfigParser`. It enables values to contain format strings which refer to other values in the same section, or values in the special default section[1]. Additional default values can be provided on initialization. For

example: `[Paths]home_dir:/Usersmy_dir:%(home_dir)s/lumberjackmy_pictures:%(my_dir)s/Pictures[Escape]#` use a `%%` to escape the `%` sign (`%` is the only character that needs to be escaped): `gain:80%%` In the example above, `ConfigParser` with `interpolation` set to `B`

`class configparser.ExtendedInterpolation`

An alternative handler for interpolation which implements a more advanced syntax, used for instance in `zc.buildout`. Extended interpolation is using `${section:option}` to denote a value from a foreign section. Interpolation can span multiple levels. For convenience, if the `section:part` is omitted, interpolation defaults to the current section (and possibly the default values from the special section). For example, the configuration specified above with basic interpolation, would look like this with `e`

Mapping Protocol Access

Added in version 3.2.

Mapping protocol access is a generic name for functionality that enables using custom objects as if they were dictionaries. In case of `configparser`, the mapping interface implementation is using the `parser['section']['option']` notation.

`parser['section']` in particular returns a proxy for the section's data in the parser. This means that the values are not copied but they are taken from the original parser on demand. What's even more important is that when values are changed on a section proxy, they are actually mutated in the original parser.

`configparser` objects behave as close to actual dictionaries as possible. The mapping interface is complete and adheres to the `MutableMappingABC`. However, there are a few differences that should be taken into account:

- By default, all keys in sections are accessible in a case-insensitive manner[1].
E.g. `for option in parser["section"]` yields only `option` x `form`'ed option key names. This means lowercased keys by default. At the same time, for a section that holds the key 'a', both expressions `return True: "a" in parser["section"]` and `"A" in parser["section"]`

- All sections include `DEFAULTSECT` values as well which means that `clear()` on a section may not leave the section visibly empty. This is because default values cannot be deleted from the section (because technically they are not there). If they are overridden in the section, deleting causes the default value to be visible again. Trying to delete a default value causes a `KeyError`.
- `DEFAULTSECT` cannot be removed from the parser: trying to delete it raises `ValueError`, `parser.clear()` leaves it intact, `parser.popitem()` never returns it.
- `parser.get(section, option, **kwargs)` - the second argument is not a fallback value. Note however that the `section-levelget()` methods are compatible both with the mapping protocol and the classic `configparser` API.
- `parser.items()` is compatible with the mapping protocol (returns a list of `section_name, section_proxy` pairs including the `DEFAULTSECT`). However, this method can also be invoked with arguments: `parser.items(section, raw, vars)`. The latter call returns a list of `option, value` pairs for a specified section, with all interpolations expanded (unless `raw=True` is provided).

The mapping protocol is implemented on top of the existing legacy API so that subclasses overriding the original interface still should have mappings working as expected.

Customizing Parser Behaviour

There are nearly as many INI format variants as there are applications using it. `configparser` goes a long way to provide support for the largest sensible set of INI styles available. The default functionality is mainly dictated by historical background and it's very likely that you will want to customize some of the features.

The most common way to change the way a specific config parser works is to use the `__init__()` options:

- `defaults`, default value: `None` This option accepts a dictionary of key-value pairs which will be initially put in the `DEFAULT` section. This makes for an elegant way to support concise configuration files that don't specify values which are the same as the documented default. Hint: if you want to specify default values for a specific section, use `read_dict()` before you read the actual file.
- `dict_type`, default value: `dict` This option has a major impact on how the mapping protocol will behave and how the written configuration files look. With the standard dictionary, every section is stored in the order they were added to the parser. Same goes for options within sections. An alternative dictionary type can be used for example to sort sections and options on write-back. Please note: there
- `allow_no_value`, default value: `False` Some configuration files are known to include settings without values, but which otherwise conform to the syntax supported by `configparser`. The `allow_no_value` parameter to the constructor can be used to indicate that such values should be accepted:

```
>>>import configparser>>>sample_config="""...[mysqld]...user = mysql...pid-file = /var/run/mysqld/mysqld.pid...skip-exte
```
- `delimiters`, default value: `('=', ':')` Delimiters are substrings that delimit keys from values within a section. The first occurrence of a delimiting substring on a line is considered a delimiter. This means values (but not keys) can contain the delimiters. See also the `space_around_delimiters` argument to `ConfigParser.write()`.
- `comment_prefixes`, default value: `(';', '#')`
- `inline_comment_prefixes`, default value: `None` Comment prefixes are strings that indicate the start of a valid comment within a config file. `comment_prefixes` are used only on otherwise empty lines (optionally indented) whereas `inline_comment_prefixes` can be used after every valid value (e.g. section names, options and empty lines as well). By default inline comments are disabled and `'#'` and `';` are used as p
- `strict`, default value: `True` When set to `True`, the parser will not allow for any section or option duplicates while reading from a single source (using `read_file()`, `read_string()` or `read_dict()`). It is recommended to use strict parsers in new applications. Changed in version 3.2: In previous versions of `configparser` behaviour

matchedstrict=False.

- empty_lines_in_values, default value: TrueIn config parsers, values can span multiple lines as long as they are indented more than the key that holds them. By default parsers also let empty lines to be parts of values. At the same time, keys can be arbitrarily indented themselves to improve readability. In consequence, when configuration files get big and complex, it is easy for the user to lose
- default_section, default value: configparser.DEFAULTSECT(that is:"DEFAULT")The convention of allowing a special section of default values for other sections or interpolation purposes is a powerful concept of this library, letting users create complex declarative configurations. This section is normally called"DEFAULT"but this can be customized to point to any other valid section name. Some typica
- interpolation, default value: configparser.BasicInterpolationInterpolation behaviour may be customized by providing a custom handler through theinterpolationargument.Nonecan be used to turn off interpolation completely,ExtendedInterpolation()provides a more advanced variant inspired byzc.buildout. More on the subject in thededicated documentation section.RawConfigParserhas a default value ofNone.
- converters, default value: not setConfig parsers provide option value getters that perform type conversion. By defaultgetint(),getfloat(), andgetboolean()are implemented. Should other getters be desirable, users may define them in a subclass or pass a dictionary where each key is a name of the converter and each value is a callable implementing said conversion. For instance, passing{'decimal':d

More advanced customization may be achieved by overriding default values of these parser attributes. The defaults are defined on the classes, so they may be overridden by subclasses or by attribute assignment.

ConfigParser.BOOLEAN_STATES

By default when usinggetboolean(), config parsers consider the following valuesTrue:'1','yes','true','on'and the following valuesFalse:'0','no','false','off'. You can override this by specifying a custom dictionary of strings and their Boolean outcomes. For example:>>>custom=configparser.ConfigParser()>>>custom['section1']={'funky':\n ope'}>>>custom['section1'].getboolean('funky')Traceback (most recent call last):...ValueError:Not a boolean:\n ope>>>custom.BOOLEAN_STATES={'sure':True,'nope':False

ConfigParser.optionxform(option)

This method transforms option names on every read, get, or set operation. The default converts the name to lowercase. This also means that when a configuration file gets written, all keys will be lowercase. Override this method if that's unsuitable. For example:>>>config="""...[Section1]...Key = Value.....[Section2]...AnotherKey = Value...""">>>typical=configparser.ConfigParser()>>>typical.read_string(config)>>>list(typical['Section1'].keys())['key']>>>list(typical['Section2'].keys())['anoth

ConfigParser.SECTCRE

A compiled regular expression used to parse section headers. The default matches[section]to the name"section". Whitespace is considered part of the section name, thus[larch]will be read as a section of name"larch". Override this attribute if that's unsuitable. For example:>>>importre>>>config="""...[Section 1]...option = value.....[Section 2]...another = val...""">>>typical=configparser.ConfigParser()>>>typical.read_string(config)>>>typical.sections()['Section 1', 'Section 2 ']>>>cus

Legacy API Examples

Mainly because of backwards compatibility concerns,configparserprovides also a legacy API with explicitget/setmethods. While there are valid use cases for the methods outlined below, mapping protocol access is preferred for new projects. The legacy API is at times more advanced, low-level and downright counterintuitive.

An example of writing to a configuration file:

```
import configparser

config = configparser.RawConfigParser()

# Please note that using RawConfigParser's set functions, you can assign
# non-string values to keys internally, but will receive an error when
# attempting to write to a file or when you get it in non-raw mode. Setting
# values using the mapping protocol or ConfigParser's set() does not allow
# such assignments to take place.
config.add_section('Section1')
config.set('Section1', 'an_int', '15')
config.set('Section1', 'a_bool', 'true')
config.set('Section1', 'a_float', '3.1415')
config.set('Section1', 'baz', 'fun')
config.set('Section1', 'bar', 'Python')
config.set('Section1', 'foo', '%(bar)s is %(baz)s!')

# Writing our configuration file to 'example.cfg'
with open('example.cfg', 'w') as configfile:
    config.write(configfile)
```

An example of reading the configuration file again:

```
import configparser

config = configparser.RawConfigParser()
config.read('example.cfg')

# getfloat() raises an exception if the value is not a float
# getint() and getboolean() also do this for their respective types
a_float = config.getfloat('Section1', 'a_float')
an_int = config.getint('Section1', 'an_int')
print(a_float + an_int)

# Notice that the next output does not interpolate '%(bar)s' or '%(baz)s'.
# This is because we are using a RawConfigParser().
if config.getboolean('Section1', 'a_bool'):
    print(config.get('Section1', 'foo'))
```

To get interpolation, use ConfigParser:

```
import configparser

cfg = configparser.ConfigParser()
cfg.read('example.cfg')

# Set the optional *raw* argument of get() to True if you wish to disable
# interpolation in a single get operation.
print(cfg.get('Section1', 'foo', raw=False)) # -> "Python is fun!"
print(cfg.get('Section1', 'foo', raw=True))  # -> "%(bar)s is %(baz)s!"

# The optional *vars* argument is a dict with members that will take
# precedence in interpolation.
print(cfg.get('Section1', 'foo', vars={'bar': 'Documentation',
                                       'baz': 'evil'}))

# The optional *fallback* argument can be used to provide a fallback value
print(cfg.get('Section1', 'foo'))
# -> "Python is fun!"

print(cfg.get('Section1', 'foo', fallback='Monty is not.'))
# -> "Python is fun!"

print(cfg.get('Section1', 'monster', fallback='No such things as monsters.'))
```



```

        # -> "No such things as monsters."

# A bare print(cfg.get('Section1', 'monster')) would raise NoOptionError
# but we can also use:

print(cfg.get('Section1', 'monster', fallback=None))
    # -> None

```

Default values are available in both types of ConfigParsers. They are used in interpolation if an option used is not defined elsewhere.

```

import configparser

# New instance with 'bar' and 'baz' defaulting to 'Life' and 'hard' each
config = configparser.ConfigParser({'bar': 'Life', 'baz': 'hard'})
config.read('example.cfg')

print(config.get('Section1', 'foo'))      # -> "Python is fun!"
config.remove_option('Section1', 'bar')
config.remove_option('Section1', 'baz')
print(config.get('Section1', 'foo'))      # -> "Life is hard!"

```

ConfigParser Objects

```

class configparser.ConfigParser(defaults=None, dict_type=dict, allow_no_value=False, *, delimiters=('=', ':'),
, comment_prefixes=(';', '#'), inline_comment_prefixes=None, strict=True, empty_lines_in_values=True, default_section=configparser.DEFAULTSECT, interpolation=BasicInterpolation(), converters={}, allow_unna
m

```

defaults()

sections()

add_section(section)

has_section(section)

options(section)

has_option(section, option)

read(filename, encoding=None)

read_file(f, source=None)

read_string(string, source='<string>')

read_dict(dictionary, source='<dict>')

get(section, option, *, raw=False, vars=None[, fallback])

getint(section, option, *, raw=False, vars=None[, fallback])

getfloat(section, option, *, raw=False, vars=None[, fallback])

getboolean(section, option, *, raw=False, vars=None[, fallback])

items(raw=False, vars=None)

items(section, raw=False, vars=None)

set(section, option, value)

write(fileobject, space_around_delimiters=True)

remove_option(section,option)

remove_section(section)

optionxform(option)

The main configuration parser. When `defaults` is given, it is initialized into the dictionary of intrinsic defaults. When `dict_type` is given, it will be used to create the dictionary objects for the list of sections, for the options within a section, and for the default values. When `delimiters` is given, it is used as the set of substrings that divide keys from values. When `comment_prefixes` is given, it will be used as the set of substrings that prefix comments in otherwise empty lines. Comments can be

Return a dictionary containing the instance-wide defaults.

Return a list of the sections available; the default section is not included in the list.

Add a section named `section` to the instance. If a section by the given name already exists, `DuplicateSectionError` is raised. If the default section name is passed, `ValueError` is raised. The name of the section must be a string; if not, `TypeError` is raised. Changed in version 3.2: Non-string section names raise `TypeError`.

Indicates whether the named section is present in the configuration. The default section is not acknowledged.

Return a list of options available in the specified section.

If the given section exists, and contains the given option, return `True`; otherwise return `False`. If the specified section is `None` or an empty string, `DEFAULT` is assumed.

Attempt to read and parse an iterable of filenames, returning a list of filenames which were successfully parsed. If `filenames` is a string, a bytes object or a path-like object, it is treated as a single filename. If a file named `filename` cannot be opened, that file will be ignored. This is designed so that you can specify an iterable of potential configuration file locations (for example, the current directory, the user's home directory, and some system-wide directory), and all existing configura

Read and parse configuration data from `f` which must be an iterable yielding Unicode strings (for example files opened in text mode). Optional argument `source` specifies the name of the file being read. If not given and `f` has a `name` attribute, that is used for `source`; the default is `'<???'>'`. Added in version 3.2: Replaces `readfp()`.

Parse configuration data from a string. Optional argument `source` specifies a context-specific name of the string passed. If not given, `'<string>'` is used. This should commonly be a filesystem path or a URL. Added in version 3.2.

Load configuration from any object that provides a `dict-like items()` method. Keys are section names, values are dictionaries with keys and values that should be present in the section. If the used dictionary type preserves order, sections and their keys will be added in order. Values are automatically converted to strings. Optional argument `source` specifies a context-specific name of the dictionary passed. If not given, `<dict>` is used. This method can be used to copy state between parsers. Added in ve

Get an option value for the named section. If `vars` is provided, it must be a dictionary. The option is looked up in `vars` (if provided), `section`, and in `DEFAULTSECT` in that order. If the key is not found and `fallback` is provided, it is used as a fallback value. `None` can be provided as a fallback value. All the `'%'` interpolations are expanded in the return values, unless the `raw` argument is true. Values for interpolation keys are looked up in the same manner as the option. Changed in version 3.2: Arguments `raw`, `vars` and `f`

A convenience method which coerces the option in the specified section to an integer. See `get()` for explanation of `raw`, `vars` and `fallback`.

A convenience method which coerces the option in the specified section to a floating-point number. See `get()` for explanation of `raw`, `vars` and `fallback`.

A convenience method which coerces the option in the specified section to a Boolean value. Note that the accepted values for the option are '1', 'yes', 'true', and 'on', which cause this method to return True, and '0', 'no', 'false', and 'off', which cause it to return False. These string values are checked in a case-insensitive manner. Any other value will cause it to raise ValueError. See get() for explanation of raw, vars and fallback.

When section is not given, return a list of section_name, section_proxy pairs, including DEFAULTSECT. Otherwise, return a list of name, value pairs for the options in the given section. Optional arguments have the same meaning as for the get() method. Changed in version 3.8: Items present in vars no longer appear in the result. The previous behaviour mixed actual parser options with variables provided for interpolation.

If the given section exists, set the given option to the specified value; otherwise raise NoSectionError. option and value must be strings; if not, TypeError is raised.

Write a representation of the configuration to the specified file object, which must be opened in text mode (accepting strings). This representation can be parsed by a future read() call. If space_around_delimiters is true, delimiters between keys and values are surrounded by spaces. Changed in version 3.14: Raises InvalidWriteError if this would write a representation which cannot be accurately parsed by a future read() call from this parser.

Remove the specified option from the specified section. If the section does not exist, raise NoSectionError. If the option existed to be removed, return True; otherwise return False.

Remove the specified section from the configuration. If the section in fact existed, return True. Otherwise return False.

Transforms the option name option as found in an input file or as passed in by client code to the form that should be used in the internal structures. The default implementation returns a lower-case version of option; subclasses may override this or client code can set an attribute of this name on instances to affect this behavior. You don't need to subclass the parser to use this method, you can also set it on an instance, to a function that takes a string argument and returns a string. Setting i

configparser.UNNAMED_SECTION

A special object representing a section name used to reference the unnamed section (see Unnamed Sections).

configparser.MAX_INTERPOLATION_DEPTH

The maximum depth for recursive interpolation for get() when the raw parameter is false. This is relevant only when the default interpolation is used.

RawConfigParser Objects

```
class configparser.RawConfigParser(defaults=None, dict_type=dict, allow_no_value=False, *, delimiters=('=', ':'), comment_prefixes=(';', '#'), inline_comment_prefixes=None, strict=True, empty_lines_in_values=True, default_section=configparser.DEFAULTSECT, interpolation=BasicInterpolation(), converters={}, allow_u
n
```

add_section(section)

set(section, option, value)

Legacy variant of the ConfigParser. It has interpolation disabled by default and allows for non-string section names, option names, and values via its unsafe add_section and set methods, as well as the legacy defaults= keyword argument handling. Changed in version 3.2: allow_no_value, delimiters, comment_prefixes, strict, empty_lines_in_values, default_section and interpolation were added. Changed in version 3.5: The converters argument was added. Changed in version 3.8: The default dict_type is dict, since it now prese

Add a section named `section` or `UNNAMED_SECTION` to the instance. If the given section already exists, `DuplicateSectionError` is raised. If the default section name is passed, `ValueError` is raised. If `UNNAMED_SECTION` is passed and support is disabled, `UnnamedSectionDisabledError` is raised. Type of `section` is not checked which lets users create non-string named sections. This behaviour is unsupported and may cause internal errors.

If the given section exists, set the given option to the specified value; otherwise raise `NoSectionError`. While it is possible to use `RawConfigParser` (or `ConfigParser` with `rawparameters` set to `true`) for internal storage of non-string values, full functionality (including interpolation and output to files) can only be achieved using string values. This method lets users assign non-string values to keys internally. This behaviour is unsupported and will cause errors when attempting to write to a file or

Exceptions

`exceptionconfigparser.Error`

Base class for all other `configparser` exceptions.

`exceptionconfigparser.NoSectionError`

Exception raised when a specified section is not found.

`exceptionconfigparser.DuplicateSectionError`

Exception raised if `add_section()` is called with the name of a section that is already present or in strict parsers when a section is found more than once in a single input file, string or dictionary. Changed in version 3.2: Added the optional `source` and `lineno` attributes and parameters to `__init__()`.

`exceptionconfigparser.DuplicateOptionError`

Exception raised by strict parsers if a single option appears twice during reading from a single file, string or dictionary. This catches misspellings and case sensitivity-related errors, e.g. a dictionary may have two keys representing the same case-insensitive configuration key.

`exceptionconfigparser.NoOptionError`

Exception raised when a specified option is not found in the specified section.

`exceptionconfigparser.InterpolationError`

Base class for exceptions raised when problems occur performing string interpolation.

`exceptionconfigparser.InterpolationDepthError`

Exception raised when string interpolation cannot be completed because the number of iterations exceeds `MAX_INTERPOLATION_DEPTH`. Subclass of `InterpolationError`.

`exceptionconfigparser.InterpolationMissingOptionError`

Exception raised when an option referenced from a value does not exist. Subclass of `InterpolationError`.

`exceptionconfigparser.InterpolationSyntaxError`

Exception raised when the source text into which substitutions are made does not conform to the required syntax. Subclass of `InterpolationError`.

`exceptionconfigparser.MissingSectionHeaderError`

Exception raised when attempting to parse a file which has no section headers.

`exceptionconfigparser.ParsingError`

Exception raised when errors occur attempting to parse a file. Changed in version 3.12: The `filename` attribute and `__init__()` constructor argument were removed. They have been available using the `name` source since 3.2.

`exceptionconfigparser.MultilineContinuationError`

Exception raised when a key without a corresponding value is continued with an indented line. Added in version 3.13.

`exceptionconfigparser.UnnamedSectionDisabledError`

Exception raised when attempting to use the `UNNAMED_SECTION` without enabling it. Added in version 3.14.

`exceptionconfigparser.InvalidWriteError`

Exception raised when an attempted `ConfigParser.write()` would not be parsed accurately with a future `ConfigParser.read()` call. Ex: Writing a key beginning with the `ConfigParser.SECTCRE` pattern would parse as a section header when read. Attempting to write this will raise this exception. Added in version 3.14.

Footnotes

Config parsers allow for heavy customization. If you are interested in changing the behaviour outlined by the footnote reference, consult the [Customizing Parser Behaviour](#) section.

tomllib— Parse TOML files

Added in version 3.11.

Source code: `Lib/tomllib`

This module provides an interface for parsing TOML 1.0.0 (Tom's Obvious Minimal Language, <https://toml.io>). This module does not support writing TOML.

See also

The `Tomli-W` package is a TOML writer that can be used in conjunction with this module, providing a write API familiar to users of the standard library `marshal` and `pickle` modules.

See also

The `TOML Kit` package is a style-preserving TOML library with both read and write capability. It is a recommended replacement for this module for editing already existing TOML files.

This module defines the following functions:

tomllib.load(fp, *, parse_float=float)

Read a TOML file. The first argument should be a readable and binary file object. Return an `dict`. Convert TOML types to Python using this `conversion table`. `parse_float` will be called with the string of every TOML float to be decoded. By default, this is equivalent to `float(num_str)`. This can be used to use another datatype or parser for TOML floats (e.g. `decimal.Decimal`). The callable must not return an `dict` or a list, else a `ValueError` is raised. A `TOMLDecodeError` will be raised on an invalid TOML document.

tomllib.loads(s, *, parse_float=float)

Load TOML from a `str` object. Return an `dict`. Convert TOML types to Python using this `conversion table`. The `parse_float` argument has the same meaning as in `load()`. A `TOMLDecodeError` will be raised on an invalid TOML document.

The following exceptions are available:

exception tomllib.TOMLDecodeError(msg, doc, pos)

msg

doc

pos

lineno

colno

Subclass of `ValueError` with the following additional attributes: `msg` The unformatted error message. `doc` The TOML document being parsed. `pos` The index of `doc` where parsing failed. `lineno` The line corresponding to `pos`. `colno` The column corresponding to `pos`. Changed in version 3.14: Added the `msg`, `doc` and `pos` parameters. Added the `msg`, `doc`, `pos`, `lineno` and `colno` attributes. Deprecated since version 3.14: Passing free-form positional arguments is deprecated.

The unformatted error message.

The TOML document being parsed.

The index of `doc` where parsing failed.

The line corresponding to `pos`.

The column corresponding topos.

Examples

Parsing a TOML file:

```
import toml

with open("pyproject.toml", "rb") as f:
    data = toml.load(f)
```

Parsing a TOML string:

```
import toml

toml_str = """
python-version = "3.11.0"
python-implementation = "CPython"
"""

data = toml.loads(toml_str)
```

Conversion Table

TOML	Python	TOML document	dict	string	str	integer	int	float	float
(configurable with	parse_float)	boolean	bool	offset date-time	datetime.datetime (tzinfo attribute set to an instance of datetime.timezone)	local date-time datetime.datetime (tzinfo attribute set to None)	local date datetime.date	local time datetime.time	array list table dict inline table dict array of tables list of dicts

netrc— netrc file processing

Source code: `Lib/netrc.py`

The `netrc` class parses and encapsulates the netrc file format used by the Unix `ftp` program and other FTP clients.

`class netrc.netrc([file])`

An `netrc` instance or subclass instance encapsulates data from a netrc file. The initialization argument, if present, specifies the file to parse. If no argument is given, the file `netrc` in the user's home directory – as determined by `os.path.expanduser()` – will be read. Otherwise, a `FileNotFoundError` exception will be raised. Parse errors will raise `NetrcParseError` with diagnostic information including the file name, line number, and terminating token. If no argument is specified on a POSIX system, the

`exception netrc.NetrcParseError`

msg

filename

lineno

Exception raised by the `netrc` class when syntactical errors are encountered in source text. Instances of this exception provide three interesting attributes: `msg` Textual explanation of the error. `filename` The name of the source file. `lineno` The line number on which the error was found.

Textual explanation of the error.

The name of the source file.

The line number on which the error was found.

netrc Objects

An `netrc` instance has the following methods:

`netrc.authenticators(host)`

Return a 3-tuple `(login, account, password)` of authenticators for `host`. If the netrc file did not contain an entry for the given host, return the tuple associated with the 'default' entry. If neither matching host nor default entry is available, return `None`.

`netrc.__repr__()`

Dump the class data as a string in the format of a netrc file. (This discards comments and may reorder the entries.)

Instances of `netrc` have public instance variables:

`netrc.hosts`

Dictionary mapping host names to `(login, account, password)` tuples. The 'default' entry, if any, is represented as a pseudo-host by that name.

`netrc.macros`

Dictionary mapping macro names to string lists.

plistlib— Generate and parse Apple.plistfiles

Source code: `Lib/plistlib.py`

This module provides an interface for reading and writing the “property list” files used by Apple, primarily on macOS and iOS. This module supports both binary and XML plist files.

The property list (.plist) file format is a simple serialization supporting basic object types, like dictionaries, lists, numbers and strings. Usually the top level object is a dictionary.

To write out and to parse a plist file, use `thedump()` and `load()` functions.

To work with plist data in bytes or string objects, use `dumps()` and `loads()`.

Values can be strings, integers, floats, booleans, tuples, lists, dictionaries (but only with string keys), bytes, bytearray or datetime.datetime objects.

Changed in version 3.4: New API, old API deprecated. Support for binary format plists added.

Changed in version 3.8: Support added for reading and writing UID tokens in binary plists as used by `NSKeyedArchiver` and `NSKeyedUnarchiver`.

Changed in version 3.9: Old API removed.

See also

PList manual page

Apple's documentation of the file format.

This module defines the following functions:

plistlib.load(fp, *, fmt=None, dict_type=dict, aware_datetime=False)

Read a plist file. `fp` should be a readable and binary file object. Return the unpacked root object (which usually is a dictionary). The `fmt` is the format of the file and the following values are valid: `None`: Autodetect the file format `FMT_XML`: XML file format `FMT_BINARY`: Binary plist format The `dict_type` is the type used for dictionaries that are read from the plist file. When `aware_datetime` is true, fields with type `datetime.datetime` will be created as aware object, with tzinfo as `datetime.UTC`. XML data for the `FMT_`

plistlib.loads(data, *, fmt=None, dict_type=dict, aware_datetime=False)

Load a plist from a bytes or string object. See `load()` for an explanation of the keyword arguments. Added in version 3.4. Changed in version 3.13: `data` can be a string when `fmt` equals `FMT_XML`.

plistlib.dump(value, fp, *, fmt=FMT_XML, sort_keys=True, skipkeys=False, aware_datetime=False)

Write `value` to a plist file. `fp` should be a writable, binary file object. The `fmt` argument specifies the format of the plist file and can be one of the following values: `FMT_XML`: XML formatted plist file `FMT_BINARY`: Binary formatted plist file When `sort_keys` is true (the default) the keys for dictionaries will be written to the plist in sorted order, otherwise they will be written in the iteration order of the dictionary. When `skipkeys` is false (the default) the function raises `TypeError` when a key of a dictionary

plistlib.dumps(value, *, fmt=FMT_XML, sort_keys=True, skipkeys=False, aware_datetime=False)

Return `value` as a plist-formatted bytes object. See the documentation for `dump()` for an explanation of the keyword arguments of this function. Added in version 3.4.

The following classes are available:

class plistlib.UID(data)

data

Wraps anint. This is used when reading or writing NSKeyedArchiver encoded data, which contains UID (see PList manual).dataInt value of the UID. It must be in the range $0 \leq \text{data} < 2^{64}$. Added in version 3.8.

Int value of the UID. It must be in the range $0 \leq \text{data} < 2^{64}$.

The following constants are available:

plistlib.FMT_XML

The XML format for plist files. Added in version 3.4.

plistlib.FMT_BINARY

The binary format for plist files. Added in version 3.4.

The module defines the following exceptions:

exception plistlib.InvalidFileException

Raised when a file cannot be parsed. Added in version 3.4.

Examples

Generating a plist:

```
import datetime as dt
import plistlib

pl = dict(
    aString = "Doodah",
    aList = ["A", "B", 12, 32.1, [1, 2, 3]],
    aFloat = 0.1,
    anInt = 728,
    aDict = dict(
        anotherString = "<hello & hi there!>",
        aThirdString = "M\xe4ssig, Ma\xdf",
        aTrueValue = True,
        aFalseValue = False,
    ),
    someData = b"<binary gunk>",
    someMoreData = b"<lots of binary gunk>" * 10,
    aDate = dt.datetime.now()
)
print(plistlib.dumps(pl).decode())
```

Parsing a plist:

```
import plistlib

plist = b'<plist version="1.0">
<dict>
  <key>foo</key>
  <string>bar</string>
</dict>
</plist>'
pl = plistlib.loads(plist)
print(pl["foo"])
```

Cryptographic Services

The modules described in this chapter implement various algorithms of a cryptographic nature. They are available at the discretion of the installation. Here's an overview:

- **hashlib**— Secure hashes and message digests
Hash algorithmsUsageConstructorsAttributesHash ObjectsSHAKE variable length digestsFile hashingKey derivationBLAKE2Creating hash objectsConstantsExamplesSimple hashingUsing different digest sizesKeyed hashingRandomized hashingPersonalizationTree modeCredits
- **hmac**— Keyed-Hashing for Message Authentication
- **secrets**— Generate secure random numbers for managing secretsRandom numbersGenerating tokensHow many bytes should tokens use?Other functionsRecipes and best practices

hashlib— Secure hashes and message digests

Source code: `Lib/hashlib.py`

This module implements a common interface to many different hash algorithms. Included are the FIPS secure hash algorithms SHA224, SHA256, SHA384, SHA512, (defined in the FIPS 180-4 standard), the SHA-3 series (defined in the FIPS 202 standard) as well as the legacy algorithms SHA1 (formerly part of FIPS) and the MD5 algorithm (defined in internetRFC 1321).

■ **Nota:** Not if you want the `adler32` or `crc32` hash functions, they are available in the `zlib` module.

Hash algorithms

There is one constructor method named for each type of hash. All return a hash object with the same simple interface. For example: use `sha256()` to create a SHA-256 hash object. You can now feed this object with bytes-like objects (normally bytes) using the `update` method. At any point you can ask it for the digest of the concatenation of the data fed to it so far using the `digest()` or `hexdigest()` methods.

To allow multithreading, the Python GIL is released while computing a hash supplied more than 2047 bytes of data at once in its constructor or `update` method.

Constructors for hash algorithms that are always present in this module are `sha1()`, `sha224()`, `sha256()`, `sha384()`, `sha512()`, `sha3_224()`, `sha3_256()`, `sha3_384()`, `sha3_512()`, `shake_128()`, `shake_256()`, `blake2b()`, and `blake2s()`. `md5()` is normally available as well, though it may be missing or blocked if you are using a rare “FIPS compliant” build of Python. These correspond to `algorithms_guaranteed`.

Additional algorithms may also be available if your Python distribution’s `hashlib` was linked against a build of OpenSSL that provides others. Others are not guaranteed available on all installations and will only be accessible by name via `new()`. See `algorithms_available`.

■■ **Advertencia:** Warning Some algorithms have known hash collision weaknesses (including MD5 and SHA1). Refer to [Attacks on cryptographic hash algorithms](#) and the [hashlib-seealso](#) section at the end of this document.

Added in version 3.6: SHA3 (Keccak) and SHAKE

constructor `sha3_224()`, `sha3_256()`, `sha3_384()`, `sha3_512()`, `shake_128()`, `shake_256()` were added. `blake2b()` and `blake2s()` were added.

Changed in version 3.9: All hashlib constructors take a keyword-only argument `usedforsecurity` with default value `True`. A false value allows the use of insecure and blocked hashing algorithms in restricted environments. `False` indicates that the hashing algorithm is not used in a security context, e.g. as a non-cryptographic one-way compression function.

Changed in version 3.9: Hashlib now uses SHA3 and SHAKE from OpenSSL if it provides it.

Changed in version 3.12: For any of the MD5, SHA1, SHA2, or SHA3 algorithms that the linked OpenSSL does not provide we fall back to a verified implementation from the `HACL*` project.

Usage

To obtain the digest of the byte string `b'Nobody inspects the spammish repetition'`:

```
>>> import hashlib
>>> m = hashlib.sha256()
>>> m.update(b'Nobody inspects')
>>> m.update(b' the spammish repetition')
>>> m.digest()
```

```
b'\x03\x1e\xdd}Ae\x15\x93\xc5\xfe\\\x00o\xa5u+7\xfd\xdf\xf7\xbcN\x84:\xa6\xaf\x0c\x95\x0fK\x94\x06'
>>> m.hexdigest()
'031edd7d41651593c5fe5c006fa5752b37fddff7bc4e843aa6af0c950f4b9406'
```

More condensed:

```
>>> hashlib.sha256(b"Nobody inspects the spammish repetition").hexdigest()
'031edd7d41651593c5fe5c006fa5752b37fddff7bc4e843aa6af0c950f4b9406'
```

Constructors

hashlib.new(name,[data,]*,usedforsecurity=True)

Is a generic constructor that takes the stringnameof the desired algorithm as its first parameter. It also exists to allow access to the above listed hashes as well as any other algorithms that your OpenSSL library may offer.

Usingnew()with an algorithm name:

```
>>> h = hashlib.new('sha256')
>>> h.update(b"Nobody inspects the spammish repetition")
>>> h.hexdigest()
'031edd7d41651593c5fe5c006fa5752b37fddff7bc4e843aa6af0c950f4b9406'
```

hashlib.md5([data,]*,usedforsecurity=True)

hashlib.sha1([data,]*,usedforsecurity=True)

hashlib.sha224([data,]*,usedforsecurity=True)

hashlib.sha256([data,]*,usedforsecurity=True)

hashlib.sha384([data,]*,usedforsecurity=True)

hashlib.sha512([data,]*,usedforsecurity=True)

hashlib.sha3_224([data,]*,usedforsecurity=True)

hashlib.sha3_256([data,]*,usedforsecurity=True)

hashlib.sha3_384([data,]*,usedforsecurity=True)

hashlib.sha3_512([data,]*,usedforsecurity=True)

Named constructors such as these are faster than passing an algorithm name tonew().

Attributes

Hashlib provides the following constant module attributes:

hashlib.algorithms_guaranteed

A set containing the names of the hash algorithms guaranteed to be supported by this module on all platforms. Note that 'md5' is in this list despite some upstream vendors offering an odd "FIPS compliant" Python build that excludes it.Added in version 3.2.

hashlib.algorithms_available

A set containing the names of the hash algorithms that are available in the running Python interpreter. These names will be recognized when passed tonew().algorithms_guaranteedwill always be a subset. The same algorithm may appear multiple times in this set under different names (thanks to OpenSSL).Added in version 3.2.

Hash Objects

The following values are provided as constant attributes of the hash objects returned by the constructors:

hash.digest_size

The size of the resulting hash in bytes.

hash.block_size

The internal block size of the hash algorithm in bytes.

A hash object has the following attributes:

hash.name

The canonical name of this hash, always lowercase and always suitable as a parameter to `new()` to create another hash of this type. Changed in version 3.4: The name attribute has been present in CPython since its inception, but until Python 3.4 was not formally specified, so may not exist on some platforms.

A hash object has the following methods:

hash.update(data)

Update the hash object with the bytes-like object. Repeated calls are equivalent to a single call with the concatenation of all the arguments: `m.update(a);m.update(b)` is equivalent to `m.update(a+b)`.

hash.digest()

Return the digest of the data passed to the `update()` method so far. This is a bytes object of size `digest_size` which may contain bytes in the whole range from 0 to 255.

hash.hexdigest()

Like `digest()` except the digest is returned as a string object of double length, containing only hexadecimal digits. This may be used to exchange the value safely in email or other non-binary environments.

hash.copy()

Return a copy ("clone") of the hash object. This can be used to efficiently compute the digests of data sharing a common initial substring.

SHAKE variable length digests

hashlib.shake_128([data,]*,usedforsecurity=True)

hashlib.shake_256([data,]*,usedforsecurity=True)

The `shake_128()` and `shake_256()` algorithms provide variable length digests with `length_in_bits//2` up to 128 or 256 bits of security. As such, their digest methods require a length. Maximum length is not limited by the SHAKE algorithm.

shake.digest(length)

Return the digest of the data passed to the `update()` method so far. This is a bytes object of size `length` which may contain bytes in the whole range from 0 to 255.

shake.hexdigest(length)

Like `digest()` except the digest is returned as a string object of double length, containing only hexadecimal digits. This may be used to exchange the value in email or other non-binary environments.

Example use:

```
>>> h = hashlib.shake_256(b'Nobody inspects the spammish repetition')
>>> h.hexdigest(20)
'44709d6fcb83d92a76dcb0b668c98e1b1d3dafe7'
```

File hashing

The hashlib module provides a helper function for efficient hashing of a file or file-like object.

hashlib.file_digest(fileobj,digest,/,)

Return a digest object that has been updated with contents of file object. fileobj must be a file-like object opened for reading in binary mode. It accepts file objects from builtin open(), BytesIO instances, SocketIO objects from socket.socket.makefile(), and similar. fileobj must be opened in blocking mode, otherwise a BlockingIOError may be raised. The function may bypass Python's I/O and use the file descriptor from fileno() directly. fileobj must be assumed to be in an unknown state after this function r

Key derivation

Key derivation and key stretching algorithms are designed for secure password hashing. Naive algorithms such as `sha1(password)` are not resistant against brute-force attacks. A good password hashing function must be tunable, slow, and include a salt.

hashlib.pbkdf2_hmac(hash_name,password,salt,iterations,dklen=None)

The function provides PKCS#5 password-based key derivation function 2. It uses HMAC as pseudorandom function. The `stringhash_name` is the desired name of the hash digest algorithm for HMAC, e.g. 'sha1' or 'sha256'. `password` and `salt` are interpreted as buffers of bytes. Applications and libraries should limit `password` to a sensible length (e.g. 1024). `salt` should be about 16 or more bytes from a proper source, e.g. `os.urandom()`. The number of iterations should be chosen based on the hash algorithm and computing

```
hashlib.scrypt(password,*,salt,n,r,p,maxmem=0,dklen=64)
```

The function provides script password-based key derivation function as defined in RFC 7914. `password` and `salt` must be byte-like objects. Applications and libraries should limit `password` to a sensible length (e.g. 1024). `salt` should be about 16 or more bytes from a proper source, e.g. `os.urandom()`. `n` is the CPU/Memory cost factor, `r` the block size, `p` parallelization factor and `maxmem` limits memory (OpenSSL 1.1.0 defaults to 32 MiB). `dklen` is the length of the derived key in bytes. Added in version 3.6.

BLAKE2

BLAKE2 is a cryptographic hash function defined in RFC 7693 that comes in two flavors:

- BLAKE2b, optimized for 64-bit platforms and produces digests of any size between 1 and 64 bytes,
- BLAKE2s, optimized for 8- to 32-bit platforms and produces digests of any size between 1 and 32 bytes.

BLAKE2 supports keyed mode (a faster and simpler replacement for HMAC), salted hashing, personalization, and tree hashing.

Hash objects from this module follow the API of standard library's `hashlib` objects.

Creating hash objects

New hash objects are created by calling constructor functions:

```
hashlib.blake2b(data=b"*,digest_size=64,key=b",salt=b",person=b",fanout=1,depth=1,leaf_size=0,node_offset=0,node_depth=0,inner_size=0,last_node=False,usedforsecurity=True)
```

```
hashlib.blake2s(data=b"*,digest_size=32,key=b",salt=b",person=b",fanout=1,depth=1,leaf_size=0,node_offset=0,node_depth=0,inner_size=0,last_node=False,usedforsecurity=True)
```

These functions return the corresponding hash objects for calculating BLAKE2b or BLAKE2s. They optionally take these general parameters:

- data: initial chunk of data to hash, which must be bytes-like object. It can be passed only as positional argument.
- digest_size: size of output digest in bytes.
- key: key for keyed hashing (up to 64 bytes for BLAKE2b, up to 32 bytes for BLAKE2s).
- salt: salt for randomized hashing (up to 16 bytes for BLAKE2b, up to 8 bytes for BLAKE2s).
- person: personalization string (up to 16 bytes for BLAKE2b, up to 8 bytes for BLAKE2s).

The following table shows limits for general parameters (in bytes):

Hash	digest_size	len(key)	len(salt)	len(person)	BLAKE2b	64	64	16	16	BLAKE2s
	32	32	8	8						

■ Nota: Note BLAKE2 specification defines constant lengths for salt and personalization parameters, however, for convenience, this implementation accepts byte strings of any size up to the specified length. If the length of the parameter is less than specified, it is padded with zeros, thus, for example, `b'salt'` and `b'salt\x00'` is the same value. (This is not the case for key.)

These sizes are available as module constants described below.

Constructor functions also accept the following tree hashing parameters:

- fanout: fanout (0 to 255, 0 if unlimited, 1 in sequential mode).
- depth: maximal depth of tree (1 to 255, 255 if unlimited, 1 in sequential mode).
- leaf_size: maximal byte length of leaf (0 to $2^{32}-1$, 0 if unlimited or in sequential mode).
- node_offset: node offset (0 to $2^{64}-1$ for BLAKE2b, 0 to $2^{48}-1$ for BLAKE2s, 0 for the first, leftmost, leaf, or in sequential mode).
- node_depth: node depth (0 to 255, 0 for leaves, or in sequential mode).
- inner_size: inner digest size (0 to 64 for BLAKE2b, 0 to 32 for BLAKE2s, 0 in sequential mode).
- last_node: boolean indicating whether the processed node is the last one (False for sequential mode).

See section 2.10 in BLAKE2 specification for comprehensive review of tree hashing.

Constants

blake2b.SALT_SIZE

blake2s.SALT_SIZE

Salt length (maximum length accepted by constructors).

blake2b.PERSON_SIZE

blake2s.PERSON_SIZE

Personalization string length (maximum length accepted by constructors).

blake2b.MAX_KEY_SIZE

blake2s.MAX_KEY_SIZE

Maximum key size.

blake2b.MAX_DIGEST_SIZE

blake2s.MAX_DIGEST_SIZE

Maximum digest size that the hash function can output.

Examples

Simple hashing

To calculate hash of some data, you should first construct a hash object by calling the appropriate constructor function (`blake2b()` or `blake2s()`), then update it with the data by calling `update()` on the object, and, finally, get the digest out of the object by calling `digest()` (or `hexdigest()` for hex-encoded string).

```
>>> from hashlib import blake2b
>>> h = blake2b()
>>> h.update(b'Hello world')
>>> h.hexdigest()
'6ff843ba685842aa82031d3f53c48b66326df7639a63d128974c5c14f31a0f33343a8c65551134ed1ae0f2b0dd2bb495dc81039e3eeb0a'
```

As a shortcut, you can pass the first chunk of data to update directly to the constructor as the positional argument:

```
>>> from hashlib import blake2b
>>> blake2b(b'Hello world').hexdigest()
'6ff843ba685842aa82031d3f53c48b66326df7639a63d128974c5c14f31a0f33343a8c65551134ed1ae0f2b0dd2bb495dc81039e3eeb0a'
```

You can call `hash.update()` as many times as you need to iteratively update the hash:

```
>>> from hashlib import blake2b
>>> items = [b'Hello', b' ', b'world']
>>> h = blake2b()
>>> for item in items:
...     h.update(item)
...
>>> h.hexdigest()
'6ff843ba685842aa82031d3f53c48b66326df7639a63d128974c5c14f31a0f33343a8c65551134ed1ae0f2b0dd2bb495dc81039e3eeb0a'
```

Using different digest sizes

BLAKE2 has configurable size of digests up to 64 bytes for BLAKE2b and up to 32 bytes for BLAKE2s. For example, to replace SHA-1 with BLAKE2b without changing the size of output, we can tell BLAKE2b to produce 20-byte digests:

```
>>> from hashlib import blake2b
>>> h = blake2b(digest_size=20)
>>> h.update(b'Replacing SHA1 with the more secure function')
>>> h.hexdigest()
'd24f26cf8de66472d58d4e1b1774b4c9158b1f4c'
>>> h.digest_size
20
>>> len(h.digest())
20
```

Hash objects with different digest sizes have completely different outputs (shorter hashes are not prefixes of longer hashes); BLAKE2b and BLAKE2s produce different outputs even if the output length is the same:

```
>>> from hashlib import blake2b, blake2s
>>> blake2b(digest_size=10).hexdigest()
'6fa1d8fcfd719046d762'
>>> blake2b(digest_size=11).hexdigest()
'eb6ec15daf9546254f0809'
>>> blake2s(digest_size=10).hexdigest()
```

```
'1bf21a98c78a1c376ae9'
>>> blake2s(digest_size=11).hexdigest()
'567004bf96e4a25773ebf4'
```

Keyed hashing

Keyed hashing can be used for authentication as a faster and simpler replacement for Hash-based message authentication code (HMAC). BLAKE2 can be securely used in prefix-MAC mode thanks to the indistinguishability property inherited from BLAKE.

This example shows how to get a (hex-encoded) 128-bit authentication code for message `b'message data'` with key `b'pseudorandomkey'`:

```
>>> from hashlib import blake2b
>>> h = blake2b(key=b'pseudorandom key', digest_size=16)
>>> h.update(b'message data')
>>> h.hexdigest()
'3d363ff7401e02026f4a4687d4863ced'
```

As a practical example, a web application can symmetrically sign cookies sent to users and later verify them to make sure they weren't tampered with:

```
>>> from hashlib import blake2b
>>> from hmac import compare_digest
>>>
>>> SECRET_KEY = b'pseudorandomly generated server secret key'
>>> AUTH_SIZE = 16
>>>
>>> def sign(cookie):
...     h = blake2b(digest_size=AUTH_SIZE, key=SECRET_KEY)
...     h.update(cookie)
...     return h.hexdigest().encode('utf-8')
>>>
>>> def verify(cookie, sig):
...     good_sig = sign(cookie)
...     return compare_digest(good_sig, sig)
>>>
>>> cookie = b'user-alice'
>>> sig = sign(cookie)
>>> print("{0},{1}".format(cookie.decode('utf-8'), sig))
user-alice,b'43b3c982cf697e0c5ab22172d1ca7421'
>>> verify(cookie, sig)
True
>>> verify(b'user-bob', sig)
False
>>> verify(cookie, b'0102030405060708090a0b0c0d0e0f00')
False
```

Even though there's a native keyed hashing mode, BLAKE2 can, of course, be used in HMAC construction with `hmac` module:

```
>>> import hmac, hashlib
>>> m = hmac.new(b'secret key', digestmod=hashlib.blake2s)
>>> m.update(b'message')
>>> m.hexdigest()
'e3c8102868d28b5ff85fc35dda07329970d1a01e273c37481326fe0c861c8142'
```

Randomized hashing

By setting `salt` parameter users can introduce randomization to the hash function. Randomized hashing is useful for protecting against collision attacks on the hash function used in digital signatures.

Randomized hashing is designed for situations where one party, the message preparer, generates all or part of a message to be signed by a second party, the message signer. If the message preparer is able to find

cryptographic hash function collisions (i.e., two messages producing the same hash value), then they might prepare meaningful versions of the message that would produce the same hash value and digital signature, but with different results (e.g., transferring \$1,000,000 to an account, rather than \$10). Cryptographic hash functions have been designed with collision resistance as a major goal, but the current concentration on attacking cryptographic hash functions may result in a given cryptographic hash function providing less collision resistance than expected. Randomized hashing offers the signer additional protection by reducing the likelihood that a preparer can generate two or more messages that ultimately yield the same hash value during the digital signature generation process — even if it is practical to find collisions for the hash function. However, the use of randomized hashing may reduce the amount of security provided by a digital signature when all portions of the message are prepared by the signer.

(NIST SP-800-106 “Randomized Hashing for Digital Signatures”)

In BLAKE2 the salt is processed as a one-time input to the hash function during initialization, rather than as an input to each compression function.

■■ Advertencia: WarningSalted hashing(or just hashing) with BLAKE2 or any other general-purpose cryptographic hash function, such as SHA-256, is not suitable for hashing passwords. SeeBLAKE2 FAQfor more information.

```
>>> import os
>>> from hashlib import blake2b
>>> msg = b'some message'
>>> # Calculate the first hash with a random salt.
>>> salt1 = os.urandom(blake2b.SALT_SIZE)
>>> h1 = blake2b(salt=salt1)
>>> h1.update(msg)
>>> # Calculate the second hash with a different random salt.
>>> salt2 = os.urandom(blake2b.SALT_SIZE)
>>> h2 = blake2b(salt=salt2)
>>> h2.update(msg)
>>> # The digests are different.
>>> h1.digest() != h2.digest()
True
```

Personalization

Sometimes it is useful to force hash function to produce different digests for the same input for different purposes. Quoting the authors of the Skein hash function:

We recommend that all application designers seriously consider doing this; we have seen many protocols where a hash that is computed in one part of the protocol can be used in an entirely different part because two hash computations were done on similar or related data, and the attacker can force the application to make the hash inputs the same. Personalizing each hash function used in the protocol summarily stops this type of attack.

(The Skein Hash Function Family, p. 21)

BLAKE2 can be personalized by passing bytes to the `person` argument:

```
>>> from hashlib import blake2b
>>> FILES_HASH_PERSON = b'MyApp Files Hash'
>>> BLOCK_HASH_PERSON = b'MyApp Block Hash'
>>> h = blake2b(digest_size=32, person=FILES_HASH_PERSON)
>>> h.update(b'the same content')
>>> h.hexdigest()
'20d9cd024d4fb086aae819a1432dd2466de12947831b75c5a30cf2676095d3b4'
>>> h = blake2b(digest_size=32, person=BLOCK_HASH_PERSON)
>>> h.update(b'the same content')
>>> h.hexdigest()
'cf68fb5761b9c44e7878bfb2c4c9aea52264a80b75005e65619778de59f383a3'
```

Personalization together with the keyed mode can also be used to derive different keys from a single one.

```
>>> from hashlib import blake2s
>>> from base64 import b64decode, b64encode
>>> orig_key = b64decode(b'Rm5EPJai72qcK3RGPw3vPNfZy50ZothY+kHY6h21KM=')
>>> enc_key = blake2s(key=orig_key, person=b'kEncrypt').digest()
>>> mac_key = blake2s(key=orig_key, person=b'kMAC').digest()
>>> print(b64encode(enc_key).decode('utf-8'))
rbPb15S/Z9t+agffno5wuhB77VbRi6F9Iv2qIXU7WHw=
>>> print(b64encode(mac_key).decode('utf-8'))
G9GtHFE1YluXY1zWPLYk1e/nWfu0WSEb0KRcjhDeP/o=
```

Tree mode

Here's an example of hashing a minimal tree with two leaf nodes:

```
  10
 /  \
00  01
```

This example uses 64-byte internal digests, and returns the 32-byte final digest:

```
>>> from hashlib import blake2b
>>>
>>> FANOUT = 2
>>> DEPTH = 2
>>> LEAF_SIZE = 4096
>>> INNER_SIZE = 64
>>>
>>> buf = bytearray(6000)
>>>
>>> # Left leaf
... h00 = blake2b(buf[0:LEAF_SIZE], fanout=FANOUT, depth=DEPTH,
...               leaf_size=LEAF_SIZE, inner_size=INNER_SIZE,
...               node_offset=0, node_depth=0, last_node=False)
>>> # Right leaf
... h01 = blake2b(buf[LEAF_SIZE:], fanout=FANOUT, depth=DEPTH,
...               leaf_size=LEAF_SIZE, inner_size=INNER_SIZE,
...               node_offset=1, node_depth=0, last_node=True)
>>> # Root node
... h10 = blake2b(digest_size=32, fanout=FANOUT, depth=DEPTH,
...               leaf_size=LEAF_SIZE, inner_size=INNER_SIZE,
...               node_offset=0, node_depth=1, last_node=True)
>>> h10.update(h00.digest())
>>> h10.update(h01.digest())
>>> h10.hexdigest()
'3ad2a9b37c6070e374c7a8c508fe20ca86b6ed54e286e93a0318e95e881db5aa'
```

Credits

BLAKE2 was designed by Jean-Philippe Aumasson, Samuel Neves, Zooko Wilcox-O'Hearn, and Christian Winnerlein based on SHA-3 finalist BLAKE created by Jean-Philippe Aumasson, Luca Henzen, Willi Meier, and Raphael C.-W. Phan.

It uses core algorithm from ChaCha cipher designed by Daniel J. Bernstein.

The stdlib implementation is based on `pyblake2` module. It was written by Dmitry Chestnykh based on C implementation written by Samuel Neves. The documentation was copied from `pyblake2` and written by Dmitry Chestnykh.

The C code was partly rewritten for Python by Christian Heimes.

The following public domain dedication applies for both C hash function implementation, extension code, and this documentation:

To the extent possible under law, the author(s) have dedicated all copyright and related and neighboring rights to this software to the public domain worldwide. This software is distributed without any warranty.

You should have received a copy of the CC0 Public Domain Dedication along with this software. If not, see <https://creativecommons.org/publicdomain/zero/1.0/>.

The following people have helped with development or contributed their changes to the project and the public domain according to the Creative Commons Public Domain Dedication 1.0 Universal:

- Alexandr Sokolovskiy

See also

Modulehmac

Modulebase64

<https://nvlpubs.nist.gov/nistpubs/fips/nist.fips.180-4.pdf>

<https://csrc.nist.gov/pubs/fips/202/final>

<https://www.blake2.net/>

https://en.wikipedia.org/wiki/Cryptographic_hash_function

<https://www.ietf.org/rfc/rfc8018.txt>

<https://nvlpubs.nist.gov/nistpubs/Legacy/SP/nistspecialpublication800-132.pdf>

A module to generate message authentication codes using hashes.

Another way to encode binary hashes for non-binary environments.

The FIPS 180-4 publication on Secure Hash Algorithms.

The FIPS 202 publication on the SHA-3 Standard.

Official BLAKE2 website.

Wikipedia article with information on which algorithms have known issues and what that means regarding their use.

PKCS #5: Password-Based Cryptography Specification Version 2.1

NIST Recommendation for Password-Based Key Derivation.

hmac— Keyed-Hashing for Message Authentication

Source code: `Lib/hmac.py`

This module implements the HMAC algorithm as described by RFC 2104. The interface allows to use any hash function with a fixed digest size. In particular, extendable output functions such as SHAKE-128 or SHAKE-256 cannot be used with HMAC.

hmac.new(key,msg=None,digestmod)

Return a new hmac object. `key` is a bytes or bytearray object giving the secret key. If `msg` is present, the method `update(msg)` is made. `digestmod` is the digest name, digest constructor or module for the HMAC object to use. It may be any name suitable to `hashlib.new()`. Despite its argument position, it is required. Changed in version 3.4: `Parameter key` can be a bytes or bytearray object. `Parameter msg` can be of any type supported by `hashlib`. `Parameter digestmod` can be the name of a hash algorithm. Changed in

hmac.digest(key,msg,digest)

Return digest of `msg` for given secret key and digest. The function is equivalent to `HMAC(key,msg,digest).digest()`, but uses an optimized C or inline implementation, which is faster for messages that fit into memory. The parameters `key`, `msg`, and `digest` have the same meaning as in `new()`. CPython implementation detail, the optimized C implementation is only used when `digest` is a string and name of a digest algorithm, which is supported by OpenSSL. Added in version 3.7.

class hmac.HMAC

An HMAC object has the following methods:

HMAC.update(msg)

Update the hmac object with `msg`. Repeated calls are equivalent to a single call with the concatenation of all the arguments: `m.update(a);m.update(b)` is equivalent to `m.update(a+b)`. Changed in version 3.4: `Parameter msg` can be of any type supported by `hashlib`.

HMAC.digest()

Return the digest of the bytes passed to the `update()` method so far. This bytes object will be the same length as the `digest_size` of the digest given to the constructor. It may contain non-ASCII bytes, including NUL bytes. **Warning** When comparing the output of `digest()` to an externally supplied digest during a verification routine, it is recommended to use the `compare_digest()` function instead of the `==` operator to reduce the vulnerability to timing attacks.

HMAC.hexdigest()

Like `digest()` except the digest is returned as a string twice the length containing only hexadecimal digits. This may be used to exchange the value safely in email or other non-binary environments. **Warning** When comparing the output of `hexdigest()` to an externally supplied digest during a verification routine, it is recommended to use the `compare_digest()` function instead of the `==` operator to reduce the vulnerability to timing attacks.

HMAC.copy()

Return a copy ("clone") of the hmac object. This can be used to efficiently compute the digests of strings that share a common initial substring.

A hash object has the following attributes:

HMAC.digest_size

The size of the resulting HMAC digest in bytes.

HMAC.block_size

The internal block size of the hash algorithm in bytes. Added in version 3.4.

HMAC.name

The canonical name of this HMAC, always lowercase, e.g. `hmac-md5`. Added in version 3.4.

Changed in version 3.10: Removed the undocumented attributes `HMAC.digest_cons`, `HMAC.inner`, and `HMAC.outer`.

This module also provides the following helper function:

hmac.compare_digest(a,b)

Return `a==b`. This function uses an approach designed to prevent timing analysis by avoiding content-based short circuiting behaviour, making it appropriate for cryptography. `a` and `b` must both be of the same type: either `str` (ASCII only, as e.g. returned by `HMAC.hexdigest()`), or a bytes-like object. Note if `a` and `b` are of different lengths, or if an error occurs, a timing attack could theoretically reveal information about the types and lengths of `a` and `b`—but not their values. Added in version 3.3. Changed in version 3.10.

See also

Module `hashlib`

The Python module providing secure hash functions.

secrets— Generate secure random numbers for managing secrets

Added in version 3.6.

Source code: `Lib/secrets.py`

The `secrets` module is used for generating cryptographically strong random numbers suitable for managing data such as passwords, account authentication, security tokens, and related secrets.

In particular, `secrets` should be used in preference to the default pseudo-random number generator in the `random` module, which is designed for modelling and simulation, not security or cryptography.

See also

PEP 506

Random numbers

The `secrets` module provides access to the most secure source of randomness that your operating system provides.

`secrets.SystemRandom`

A class for generating random numbers using the highest-quality sources provided by the operating system. See `random.SystemRandom` for additional details.

`secrets.choice(seq)`

Return a randomly chosen element from a non-empty sequence.

`secrets.randbelow(exclusive_upper_bound)`

Return a random int in the range `[0, exclusive_upper_bound)`.

`secrets.randbits(k)`

Return a non-negative int with `k` random bits.

Generating tokens

The `secrets` module provides functions for generating secure tokens, suitable for applications such as password resets, hard-to-guess URLs, and similar.

`secrets.token_bytes(nbytes=None)`

Return a random byte string containing `nbytes` number of bytes. If `nbytes` is not specified or `None`, `DEFAULT_ENTROPY` is used instead. >>> `token_bytes(16)`
`b'\xebr\x17D*\txae\xd4\xe3S\xb6\xe2\xebP1\x8b'`

`secrets.token_hex(nbytes=None)`

Return a random text string, in hexadecimal. The string has `nbytes` random bytes, each byte converted to two hex digits. If `nbytes` is not specified or `None`, `DEFAULT_ENTROPY` is used instead. >>> `token_hex(16)`
`'f9bf78b9a18ce6d46a0cd2b0b86df9da'`

`secrets.token_urlsafe(nbytes=None)`

Return a random URL-safe text string, containing `n` bytes random bytes. The text is Base64 encoded, so on average each byte results in approximately 1.3 characters. If `n` bytes is not specified or `None`, `DEFAULT_ENTROPY` is used instead. >>> `token_urlsafe(16)` 'Drmhze6EPcv0fN_81Bj-nA'

How many bytes should tokens use?

To be secure against brute-force attacks, tokens need to have sufficient randomness. Unfortunately, what is considered sufficient will necessarily increase as computers get more powerful and able to make more guesses in a shorter period. As of 2015, it is believed that 32 bytes (256 bits) of randomness is sufficient for the typical use-case expected for the `secrets` module.

For those who want to manage their own token length, you can explicitly specify how much randomness is used for tokens by giving an argument to the various `token_*` functions. That argument is taken as the number of bytes of randomness to use.

Otherwise, if no argument is provided, or if the argument is `None`, the `token_*` functions use `DEFAULT_ENTROPY` instead.

`secrets.DEFAULT_ENTROPY`

Default number of bytes of randomness used by the `token_*` functions. The exact value is subject to change at any time, including during maintenance releases.

Other functions

`secrets.compare_digest(a,b)`

Return `True` if strings or bytes-like objects `a` and `b` are equal, otherwise `False`, using a “constant-time compare” to reduce the risk of timing attacks. See `mac.compare_digest()` for additional details.

Recipes and best practices

This section shows recipes and best practices for using `secrets` to manage a basic level of security.

Generate an eight-character alphanumeric password:

```
import string
import secrets
alphabet = string.ascii_letters + string.digits
password = ''.join(secrets.choice(alphabet) for i in range(8))
```

■ **Nota:** Note Applications should not store passwords in a recoverable format, whether plain text or encrypted. They should be salted and hashed using a cryptographically strong one-way (irreversible) hash function.

Generate a ten-character alphanumeric password with at least one lowercase character, at least one uppercase character, and at least three digits:

```
import string
import secrets
alphabet = string.ascii_letters + string.digits
while True:
    password = ''.join(secrets.choice(alphabet) for i in range(10))
    if (any(c.islower() for c in password)
        and any(c.isupper() for c in password)
        and sum(c.isdigit() for c in password) >= 3):
        break
```

Generate an XKCD-style passphrase:

```
import secrets
```

```
# On standard Linux systems, use a convenient dictionary file.  
# Other platforms may need to provide their own word-list.  
with open('/usr/share/dict/words') as f:  
    words = [word.strip() for word in f]  
    password = ' '.join(secrets.choice(words) for i in range(4))
```

Generate a hard-to-guess temporary URL containing a security token suitable for password recovery applications:

```
import secrets  
url = 'https://example.com/reset=' + secrets.token_urlsafe()
```

Generic Operating System Services

The modules described in this chapter provide interfaces to operating system features that are available on (almost) all operating systems, such as files and a clock. The interfaces are generally modeled after the Unix or C interfaces, but they are available on most other systems as well. Here's an overview:

- **os**— Miscellaneous operating system interfacesFile Names, Command Line Arguments, and Environment VariablesPython UTF-8 ModeProcess ParametersFile Object CreationFile Descriptor OperationsQuerying the size of a terminalInheritance of File DescriptorsFiles and DirectoriesTimer File DescriptorsLinux extended attributesProcess ManagementInterface to the schedulerMiscellaneous System InformationRandom
- **io**— Core tools for working with streamsOverviewText I/OBinary I/ORaw I/OText EncodingOpt-in EncodingWarningHigh-level Module InterfaceClass hierarchyI/O Base ClassesRaw File I/OBuffered StreamsText I/OStatic TypingPerformanceBinary I/OText I/OMulti-threadingReentrancy
- **time**— Time access and conversionsFunctionsClock ID ConstantsTimezone Constants
- **logging**— Logging facility for PythonLogger ObjectsLogging LevelsHandler ObjectsFormatter ObjectsFilter ObjectsLogRecord ObjectsLogRecord attributesLoggerAdapter ObjectsThread SafetyModule-Level FunctionsModule-Level AttributesIntegration with the warnings module
- **logging.config**— Logging configurationConfiguration functionsSecurity considerationsConfiguration dictionary schemaDictionary Schema DetailsIncremental ConfigurationObject connectionsUser-defined objectsHandler configuration orderAccess to external objectsAccess to internal objectsImport resolution and custom importersConfiguring QueueHandler and QueueListenerConfiguration file format
- **logging.handlers**— Logging handlersStreamHandlerFileHandlerNullHandlerWatchedFileHandlerBaseRotatingHandlerRotatingFileHandlerTimedRotatingFileHandlerSocketHandlerDatagramHandlerSysLogHandlerNTEventLogHandlerSMTPHandlerMemoryHandlerHTTPHandlerQueueHandlerQueueListener
- **platform**— Access to underlying platform's identifying dataCross platformJava platformWindows platformmacOS platformiOS platformUnix platformsLinux platformsAndroid platformCommand-line usage
- **errno**— Standard errno system symbols
- **ctypes**— A foreign function library for Pythonctypes tutorialLoading dynamic link librariesAccessing functions from loaded dllsCalling functionsFundamental data typesCalling functions, continuedCalling variadic functionsCalling functions with your own custom data typesSpecifying the required argument types (function prototypes)Return typesPassing pointers (or: passing parameters by reference)Struct

os— Miscellaneous operating system interfaces

Source code: `Lib/os.py`

This module provides a portable way of using operating system dependent functionality. If you just want to read or write a file see `open()`, if you want to manipulate paths, see `os.path` module, and if you want to read all the lines in all the files on the command line see `fileinput` module. For creating temporary files and directories see `tempfile` module, and for high-level file and directory handling see `shutil` module.

Notes on the availability of these functions:

- The design of all built-in operating system dependent modules of Python is such that as long as the same functionality is available, it uses the same interface; for example, the function `os.stat(path)` returns stat information about `path` in the same format (which happens to have originated with the POSIX interface).
- Extensions peculiar to a particular operating system are also available through `os` module, but using them is of course a threat to portability.
- All functions accepting path or file names accept both bytes and string objects, and result in an object of the same type, if a path or file name is returned.
- On VxWorks, `os.popen`, `os.fork`, `os.execv` and `os.spawn*p*` are not supported.
- On WebAssembly platforms, Android and iOS, large parts of `os` module are not available or behave differently. APIs related to processes (e.g. `fork()`, `execve()`) and resources (e.g. `nice()`) are not available. Others like `getuid()` and `getpid()` are emulated or stubs. WebAssembly platforms also lack support for signals (e.g. `kill()`, `wait()`).

■ Nota: Note All functions in this module raise `OSError` (or subclasses thereof) in the case of invalid or inaccessible file names and paths, or other arguments that have the correct type, but are not accepted by the operating system.

exception `os.error`

An alias for the built-in `OSError` exception.

`os.name`

The name of the operating system dependent module imported. The following names have currently been registered: `'posix'`, `'nt'`, `'java'`. See also `sys.platform` has a finer granularity. `os.uname()` gives system-dependent version information. The `platform` module provides detailed checks for the system's identity.

File Names, Command Line Arguments, and Environment Variables

In Python, file names, command line arguments, and environment variables are represented using the string type. On some systems, decoding these strings to and from bytes is necessary before passing them to the operating system. Python uses the `filesystem` encoding and error handler to perform this conversion (see `sys.getfilesystemencoding()`).

The `filesystem` encoding and error handler are configured at Python startup by the `PyConfig_Read()` function: see `filesystem_encoding` and `filesystem_errors` members of `PyConfig`.

Changed in version 3.1: On some systems, conversion using the file system encoding may fail. In this case, Python uses the `surrogateescape` encoding error handler, which means that undecodable bytes are replaced by a Unicode character `U+DCxx` on decoding, and these are again translated to the original byte on encoding.

The file system encoding must guarantee to successfully decode all bytes below 128. If the file system encoding fails to provide this guarantee, API functions can raise `UnicodeError`.

See also the `locale` encoding.

Python UTF-8 Mode

Added in version 3.7: See PEP 540 for more details.

The Python UTF-8 Mode ignores the `locale` encoding and forces the usage of the UTF-8 encoding:

- Use UTF-8 as the filesystem encoding.
- `sys.getfilesystemencoding()` returns 'utf-8'.
- `locale.getpreferredencoding()` returns 'utf-8' (the `do_setlocale` argument has no effect).
- `sys.stdin`, `sys.stdout`, and `sys.stderr` all use UTF-8 as their text encoding, with the `surrogateescape` error handler being enabled for `sys.stdin` and `sys.stdout` (`sys.stderr` continues to use `backslashreplace` as it does in the default locale-aware mode).
- On Unix, `os.device_encoding()` returns 'utf-8' rather than the device encoding.

Note that the standard stream settings in UTF-8 mode can be overridden by `PYTHONIOENCODING` (just as they can be in the default locale-aware mode).

As a consequence of the changes in those lower level APIs, other higher level APIs also exhibit different default behaviours:

- Command line arguments, environment variables and filenames are decoded to text using the UTF-8 encoding.
- `os.fsdecode()` and `os.fsencode()` use the UTF-8 encoding.
- `open()`, `io.open()`, and `codecs.open()` use the UTF-8 encoding by default. However, they still use the strict error handler by default so that attempting to open a binary file in text mode is likely to raise an exception rather than producing nonsense data.

The Python UTF-8 Mode is enabled if the `LC_CTYPE` locale is `CorPOSIX` at Python startup (see the `PyConfig_Read()` function).

It can be enabled or disabled using the `-Xutf8` command line option and the `PYTHONUTF8` environment variable.

If the `PYTHONUTF8` environment variable is not set at all, then the interpreter defaults to using the current locale settings, unless the current locale is identified as a legacy ASCII-based locale (as described for `PYTHONCOERCECLOCALE`), and locale coercion is either disabled or fails. In such legacy locales, the interpreter will default to enabling UTF-8 mode unless explicitly instructed not to do so.

The Python UTF-8 Mode can only be enabled at the Python startup. Its value can be read from `sys.flags.utf8_mode`.

See also the UTF-8 mode on Windows and the filesystem encoding and error handler.

See also

PEP 686

Python 3.15 will make Python UTF-8 Mode default.

Process Parameters

These functions and data items provide information and operate on the current process and user.

`os.ctermid()`

Return the filename corresponding to the controlling terminal of the process. Availability: Unix, not WASI.

os.environ

A mapping object where keys and values are strings that represent the process environment. For example, `environ["HOME"]` is the pathname of your home directory (on some platforms), and is equivalent to `getenv("HOME")` in C. This mapping is captured the first time the `os` module is imported, typically during Python startup as part of `posixmodule.py`. Changes to the environment made after this time are not reflected in `os.environ`, except for changes made by modifying `os.environ` directly. This mapping may be updated by `os.putenv()` or `os.unsetenv()`.

os.environb

Bytes version of `os.environ`: a mapping object where both keys and values are bytes objects representing the process environment. `os.environ` and `os.environb` are synchronized (modifying `os.environb` updates `os.environ`, and vice versa). `os.environb` is only available if `sys.platform == 'win32'`. Added in version 3.2. Changed in version 3.9: Updated to support PEP 584's merge (`|`) and update (`|=`) operators.

os.reload_environ()

`os.reload_environ()` updates `os.environ` and `os.environb` with any such changes to the current process environment. Warning: This function is not thread-safe. Calling it while the environment is being modified in another thread is an undefined behavior. Reading from `os.environ` or `os.environb` while `os.reload_environ()` is running may result in a segmentation fault.

os.chdir(path)

os.fchdir(fd)

os.getcwd()

These functions are described in [Files and Directories](#).

os.fsencode(filename)

Encode a path-like filename to the filesystem encoding and error handler; return bytes unchanged. `os.fsdecode()` is the reverse function. Added in version 3.2. Changed in version 3.6: Support added to accept objects implementing the `os.PathLike` interface.

os.fsdecode(filename)

Decode a path-like filename from the filesystem encoding and error handler; return str unchanged. `os.fsencode()` is the reverse function. Added in version 3.2. Changed in version 3.6: Support added to accept objects implementing the `os.PathLike` interface.

os.fspath(path)

Return the file system representation of the path. If `str` or `bytes` is passed in, it is returned unchanged. Otherwise `__fspath__()` is called and its value is returned as long as it is a `str` or `bytes` object. In all other cases, `TypeError` is raised. Added in version 3.6.

class os.PathLike

__fspath__()

An abstract base class for objects representing a file system path, e.g. `pathlib.PurePath`. Added in version 3.6. `__fspath__()` Return the file system path representation of the object. The method should only return a `str` or `bytes` object, with the preference being for `str`.

Return the file system path representation of the object. The method should only return a `str` or `bytes` object, with the preference being for `str`.

os.getenv(key, default=None)

Return the value of the environment variable `key` as a string if it exists, or `default` if it doesn't. `key` is a string. Note that since `getenv()` uses `os.environ`, the mapping of `getenv()` is similarly also captured on import, and the function may not reflect future environment changes. On Unix, keys and values are decoded with `sys.getfilesystemencoding()` and `'surrogateescape'` error handler. Use `os.getenvb()` if you would like to use a different encoding. Availability: Unix, Windows.

`os.getenvb(key,default=None)`

Return the value of the environment variable `key` as bytes if it exists, or `default` if it doesn't. `key` must be bytes. Note that since `getenvb()` uses `os.environb`, the mapping of `getenvb()` is similarly also captured on import, and the function may not reflect future environment changes. `getenvb()` is only available if `supports_bytes_environ` is `True`. Availability: Unix. Added in version 3.2.

`os.get_exec_path(env=None)`

Returns the list of directories that will be searched for a named executable, similar to a shell, when launching a process. `env`, when specified, should be an environment variable dictionary to lookup the `PATH` in. By default, when `env` is `None`, `os.environ` is used. Added in version 3.2.

`os.getegid()`

Return the effective group id of the current process. This corresponds to the "set id" bit on the file being executed in the current process. Availability: Unix, not WASI.

`os.geteuid()`

Return the current process's effective user id. Availability: Unix, not WASI.

`os.getgid()`

Return the real group id of the current process. Availability: Unix. The function is a stub on WASI, see [WebAssembly platforms](#) for more information.

`os.getgrouplist(user,group,/)`

Return list of group ids that `user` belongs to. If `group` is not in the list, it is included; typically, `group` is specified as the group ID field from the password record for `user`, because that group ID will otherwise be potentially omitted. Availability: Unix, not WASI. Added in version 3.3.

`os.getgroups()`

Return list of supplemental group ids associated with the current process. Availability: Unix, not WASI. Note: On macOS, `getgroups()` behavior differs somewhat from other Unix platforms. If the Python interpreter was built with a deployment target of 10.5 or earlier, `getgroups()` returns the list of effective group ids associated with the current user process; this list is limited to a system-defined number of entries, typically 16, and may be modified by calls to `setgroups()` if suitably privileged. If built

`os.getlogin()`

Return the name of the user logged in on the controlling terminal of the process. For most purposes, it is more useful to use `getpass.getuser()` since the latter checks the environment variables `LOGNAME` or `USERNAME` to find out who the user is, and falls back to `pwd.getpwuid(os.getuid())[0]` to get the login name of the current real user id. Availability: Unix, Windows, not WASI.

`os.getpgid(pid)`

Return the process group id of the process with process id `pid`. If `pid` is 0, the process group id of the current process is returned. Availability: Unix, not WASI.

`os.getpgrp()`

Return the id of the current process group. Availability: Unix, not WASI.

os.getpid()

Return the current process id. The function is a stub on WASI, see [WebAssembly platforms](#) for more information.

os.getppid()

Return the parent's process id. When the parent process has exited, on Unix the id returned is the one of the init process (1), on Windows it is still the same id, which may be already reused by another process. Availability: Unix, Windows, not WASI. Changed in version 3.2: Added support for Windows.

os.getpriority(which,who)

Get program scheduling priority. The value *which* is one of `PRIO_PROCESS`, `PRIO_PGRP`, or `PRIO_USER`, and *who* is interpreted relative to *which* (a process identifier for `PRIO_PROCESS`, process group identifier for `PRIO_PGRP`, and a user ID for `PRIO_USER`). A zero value for *who* denotes (respectively) the calling process, the process group of the calling process, or the real user ID of the calling process. Availability: Unix, not WASI. Added in version 3.3.

os.PRIO_PROCESS

os.PRIO_PGRP

os.PRIO_USER

Parameters for the `getpriority()` and `setpriority()` functions. Availability: Unix, not WASI. Added in version 3.3.

os.PRIO_DARWIN_THREAD

os.PRIO_DARWIN_PROCESS

os.PRIO_DARWIN_BG

os.PRIO_DARWIN_NONUI

Parameters for the `getpriority()` and `setpriority()` functions. Availability: macOS. Added in version 3.12.

os.getresuid()

Return a tuple (ruid, euid, suid) denoting the current process's real, effective, and saved user ids. Availability: Unix, not WASI. Added in version 3.2.

os.getresgid()

Return a tuple (rgid, egid, sgid) denoting the current process's real, effective, and saved group ids. Availability: Unix, not WASI. Added in version 3.2.

os.getuid()

Return the current process's real user id. Availability: Unix. The function is a stub on WASI, see [WebAssembly platforms](#) for more information.

os.initgroups(username,gid,/)

Call the system `initgroups()` to initialize the group access list with all of the groups of which the specified *username* is a member, plus the specified group id. Availability: Unix, not WASI, not Android. Added in version 3.2.

os.putenv(key,value,/)

Set the environment variable named *key* to the string *value*. Such changes to the environment affect subprocesses started with `os.system()`, `popen()` or `fork()` and `execv()`. Assignments to items in `os.environ` are automatically translated into corresponding calls to `putenv()`; however, calls to `putenv()` don't update `os.environ`, so it is actually preferable to assign to items of `os.environ`. This also applies to `getenv()` and `getenvb()`, which

respectively use `os.environ`, `os.environ.get`, `os.environ.get` in their implementations. See also `os.environ.get`.

`os.setegid(egid, /)`

Set the current process's effective group id. Availability: Unix, not WASI, not Android.

`os.seteuid(euid, /)`

Set the current process's effective user id. Availability: Unix, not WASI, not Android.

`os.setgid(gid, /)`

Set the current process' group id. Availability: Unix, not WASI, not Android.

`os.setgroups(groups, /)`

Set the list of supplemental group ids associated with the current process to `groups`. `groups` must be a sequence, and each element must be an integer identifying a group. This operation is typically available only to the superuser. Availability: Unix, not WASI. Note: On macOS, the length of `groups` may not exceed the system-defined maximum number of effective group ids, typically 16. See the documentation for `getgroups()` for cases where it may not return the same group list set by calling `setgroups()`.

`os.setns(fd, nstype=0)`

Reassociate the current thread with a Linux namespace. See `setns(2)` and `namespaces(7)` man pages for more details. If `fd` refers to `/proc/pid/ns/link`, `setns()` reassociates the calling thread with the namespace associated with that link, and `nstype` may be set to one of the `CLONE_NEW*` constants to impose constraints on the operation (0 means no constraints). Since Linux 5.8, `fd` may refer to a PID file descriptor obtained from `pidfd_open()`. In this case, `setns()` reassociates the calling thread into one or more of `t`

`os.setpgid()`

Call the system call `setpgid()` or `setpgid(0, 0)` depending on which version is implemented (if any). See the Unix manual for the semantics. Availability: Unix, not WASI.

`os.setpgid(pid, pgrp, /)`

Call the system call `setpgid()` to set the process group id of the process with id `pid` to the process group with id `pgrp`. See the Unix manual for the semantics. Availability: Unix, not WASI.

`os.setpriority(which, who, priority)`

Set program scheduling priority. The value `which` is one of `PRIO_PROCESS`, `PRIO_PGRP`, or `PRIO_USER`, and `who` is interpreted relative to `which` (a process identifier for `PRIO_PROCESS`, process group identifier for `PRIO_PGRP`, and a user ID for `PRIO_USER`). A zero value for `who` denotes (respectively) the calling process, the process group of the calling process, or the real user ID of the calling process. `priority` is a value in the range -20 to 19. The default priority is 0; lower priorities cause more favorable scheduling.

`os.setregid(rgid, egid, /)`

Set the current process's real and effective group ids. Availability: Unix, not WASI, not Android.

`os.setresgid(rgid, egid, sgid, /)`

Set the current process's real, effective, and saved group ids. Availability: Unix, not WASI, not Android. Added in version 3.2.

`os.setresuid(ruid, euid, suid, /)`

Set the current process's real, effective, and saved user ids. Availability: Unix, not WASI, not Android. Added in version 3.2.

`os.setreuid(ruid, euid, /)`

Set the current process's real and effective user ids. Availability: Unix, not WASI, not Android.

os.getsid(pid, /)

Call the system call `getsid()`. See the Unix manual for the semantics. Availability: Unix, not WASI.

os.setsid()

Call the system call `setsid()`. See the Unix manual for the semantics. Availability: Unix, not WASI.

os.setuid(uid, /)

Set the current process's user id. Availability: Unix, not WASI, not Android.

os.strerror(code, /)

Return the error message corresponding to the error code `incode`. On platforms where `strerror()` returns `NULL` when given an unknown error number, `ValueError` is raised.

os.supports_bytes_environ

True if the native OS type of the environment is bytes (eg. False on Windows). Added in version 3.2.

os.umask(mask, /)

Set the current numeric umask and return the previous umask. The function is a stub on WASI, see [WebAssembly platforms](#) for more information.

os.uname()

Returns information identifying the current operating system. The return value is an object with five attributes: `sysname`- operating system name, `nodename`- name of machine on network (implementation-defined), `release`- operating system release, `version`- operating system version, `machine`- hardware identifier. For backwards compatibility, this object is also iterable, behaving like a five-tuple containing `sysname`, `nodename`, `release`, `version`, and `machine` in that order. Some systems truncate `nodename` to 8 characters or

os.unsetenv(key, /)

Unset (delete) the environment variable named `key`. Such changes to the environment affect subprocesses started with `os.system()`, `popen()` or `fork()` and `execv()`. Deletion of items in `os.environ` is automatically translated into a corresponding call to `unsetenv()`; however, calls to `unsetenv()` don't update `os.environ`, so it is actually preferable to delete items from `os.environ`. See also the `os.reload_environ()` function. Raises an `auditing.Event` `os.unsetenv` with argument `key`. Changed in version 3.9: The function is now always

os.unshare(flags)

Disassociate parts of the process execution context, and move them into a newly created namespace. See the `unshare(2)` man page for more details. The `flags` argument is a bit mask, combining zero or more of the `CLONE_*` constants, that specifies which parts of the execution context should be unshared from their existing associations and moved to a new namespace. If the `flags` argument is 0, no changes are made to the calling process's execution context. Availability: Linux >= 2.6.16. Added in version 3.12. See

Flags to the `unshare()` function, if the implementation supports them. See `unshare(2)` in the Linux manual for their exact effect and availability.

os.CLONE_FILES

os.CLONE_FS

os.CLONE_NEWCGROUP

os.CLONE_NEWIPC

os.CLONE_NEWNET
os.CLONE_NEWNS
os.CLONE_NEWPID
os.CLONE_NEWTIME
os.CLONE_NEWUSER
os.CLONE_NEWUTS
os.CLONE_SIGHAND
os.CLONE_SYSVSEM
os.CLONE_THREAD
os.CLONE_VM

File Object Creation

These functions create new file objects. (See also `open()` for opening file descriptors.)

os.fdopen(fd,*args,kwargs)**

Return an open file object connected to the file descriptor `fd`. This is an alias of the `open()` built-in function and accepts the same arguments. The only difference is that the first argument of `fdopen()` must always be an integer.

File Descriptor Operations

These functions operate on I/O streams referenced using file descriptors.

File descriptors are small integers corresponding to a file that has been opened by the current process. For example, standard input is usually file descriptor 0, standard output is 1, and standard error is 2. Further files opened by a process will then be assigned 3, 4, 5, and so forth. The name “file descriptor” is slightly deceptive; on Unix platforms, sockets and pipes are also referenced by file descriptors.

The `fileno()` method can be used to obtain the file descriptor associated with a file object when required. Note that using the file descriptor directly will bypass the file object methods, ignoring aspects such as internal buffering of data.

os.close(fd)

Close file descriptor `fd`. Note This function is intended for low-level I/O and must be applied to a file descriptor as returned by `os.open()` or `pipe()`. To close a “file object” returned by the built-in function `open()` or `popen()` or `fdopen()`, use its `close()` method.

os.closerange(fd_low,fd_high,/))

Close all file descriptors from `fd_low` (inclusive) to `fd_high` (exclusive), ignoring errors. Equivalent to (but much faster than): `for fd in range(fd_low,fd_high): try: os.close(fd) except OSError: pass`

os.copy_file_range(src,dst,count,offset_src=None,offset_dst=None)

Copy `count` bytes from file descriptor `src`, starting from `offset_src`, to file descriptor `dst`, starting from `offset_dst`. If `offset_src` is `None`, then `src` is read from the current position; respectively for `offset_dst`. In Linux kernel older than 5.3, the files pointed to by `src` and `dst` must reside in the same filesystem, otherwise an `OSError` is raised with `errno.EXDEV`. This copy is done without the additional cost of transferring data from the kernel to user space and then back into the kernel.

os.device_encoding(fd)

Return a string describing the encoding of the device associated with `fd` if it is connected to a terminal; else return `None`. On Unix, if the Python UTF-8 Mode is enabled, return 'UTF-8' rather than the device encoding. Changed in version 3.10: On Unix, the function now implements the Python UTF-8 Mode.

os.dup(fd, /)

Return a duplicate of file descriptor `fd`. The new file descriptor is non-inheritable. On Windows, when duplicating a standard stream (0: `stdin`, 1: `stdout`, 2: `stderr`), the new file descriptor is inheritable. Availability: not WASI. Changed in version 3.4: The new file descriptor is now non-inheritable.

os.dup2(fd, fd2, inheritable=True)

Duplicate file descriptor `fd` to `fd2`, closing the latter first if necessary. Return `fd2`. The new file descriptor is inheritable by default or non-inheritable if `inheritable` is `False`. Availability: not WASI. Changed in version 3.4: Add the optional `inheritable` parameter. Changed in version 3.7: Return `fd2` on success. Previously, `None` was always returned.

os.fchmod(fd, mode)

Change the mode of the file given by `fd` to the numeric `mode`. See the docs for `chmod()` for possible values of `mode`. As of Python 3.3, this is equivalent to `os.chmod(fd, mode)`. Raises an `auditing.Event` `os.chmod` with arguments `path`, `mode`, `dir_fd`. Availability: Unix, Windows. The function is limited on WASI, see [WebAssembly platforms](#) for more information. Changed in version 3.13: Added support on Windows.

os.fchown(fd, uid, gid)

Change the owner and group id of the file given by `fd` to the numeric `uid` and `gid`. To leave one of the ids unchanged, set it to `-1`. See `chown()`. As of Python 3.3, this is equivalent to `os.chown(fd, uid, gid)`. Raises an `auditing.Event` `os.chown` with arguments `path`, `uid`, `gid`, `dir_fd`. Availability: Unix. The function is limited on WASI, see [WebAssembly platforms](#) for more information.

os.fdatasync(fd)

Force write of file with file descriptor `fd` to disk. Does not force update of metadata. Availability: Unix. Note: This function is not available on MacOS.

os.fpathconf(fd, name, /)

Return system configuration information relevant to an open file. `name` specifies the configuration value to retrieve; it may be a string which is the name of a defined system value; these names are specified in a number of standards (POSIX.1, Unix 95, Unix 98, and others). Some platforms define additional names as well. The names known to the host operating system are given in the `pathconf_names` dictionary. For configuration variables not included in that mapping, passing an integer for `name` is also

os.fstat(fd)

Get the status of the file descriptor `fd`. Return an `os.stat_result` object. As of Python 3.3, this is equivalent to `os.stat(fd)`. See also `Thstat()` function.

os.fstatvfs(fd, /)

Return information about the filesystem containing the file associated with file descriptor `fd`, like `statvfs()`. As of Python 3.3, this is equivalent to `os.statvfs(fd)`. Availability: Unix.

os.fsync(fd)

Force write of file with file descriptor `fd` to disk. On Unix, this calls the native `fsync()` function; on Windows, the `MS_commit()` function. If you're starting with a buffered Python file object `f`, first do `f.flush()`, and then `doos.fsync(f.fileno())`, to ensure that all internal buffers associated with `f` are written to disk. Availability: Unix, Windows.

os.ftruncate(fd,length,l)

Truncate the file corresponding to file descriptor `fd`, so that it is at most `length` bytes in size. As of Python 3.3, this is equivalent to `os.truncate(fd,length)`. Raises an `auditing event` `os.truncate` with arguments `fd,length`. Availability: Unix, Windows. Changed in version 3.5: Added support for Windows

os.get_blocking(fd,l)

Get the blocking mode of the file descriptor: `False` if the `O_NONBLOCK` flag is set, `True` if the flag is cleared. See also `set_blocking()` and `socket.socket.setblocking()`. Availability: Unix, Windows. The function is limited on WASI, see WebAssembly platforms for more information. On Windows, this function is limited to pipes. Added in version 3.5. Changed in version 3.12: Added support for pipes on Windows.

os.grantpt(fd,l)

Grant access to the slave pseudo-terminal device associated with the master pseudo-terminal device to which the file descriptor `fd` refers. The file descriptor `fd` is not closed upon failure. Calls the C standard library function `grantpt()`. Availability: Unix, not WASI. Added in version 3.13.

os.isatty(fd,l)

Return `True` if the file descriptor `fd` is open and connected to a tty(-like) device, else `False`.

os.lockf(fd,cmd,len,l)

Apply, test or remove a POSIX lock on an open file descriptor. `fd` is an open file descriptor. `cmd` specifies the command to use - one of `F_LOCK`, `F_TLOCK`, `F_ULOCK` or `F_TEST`. `len` specifies the section of the file to lock. Raises an `auditing event` `os.lockf` with arguments `fd,cmd,len`. Availability: Unix. Added in version 3.3.

os.F_LOCK

os.F_TLOCK

os.F_ULOCK

os.F_TEST

Flags that specify what action `lockf()` will take. Availability: Unix. Added in version 3.3.

os.login_tty(fd,l)

Prepare the tty of which `fd` is a file descriptor for a new login session. Make the calling process a session leader; make the tty the controlling tty, the stdin, the stdout, and the stderr of the calling process; close `fd`. Availability: Unix, not WASI. Added in version 3.11.

os.lseek(fd,pos,whence,l)

Set the current position of file descriptor `fd` to position `pos`, modified by `whence`, and return the new position in bytes relative to the start of the file. Valid values for `whence` are: `SEEK_SET` or `0` - set `pos` relative to the beginning of the file; `SEEK_CUR` or `1` - set `pos` relative to the current file position; `SEEK_END` or `2` - set `pos` relative to the end of the file; `SEEK_HOLE` - set `pos` to the next data location, relative to `pos`; `SEEK_DATA` - set `pos` to the next data hole, relative to `pos`. Changed in version 3.3: Add support for `SEEK_HOLE`.a

os.SEEK_SET

os.SEEK_CUR

os.SEEK_END

SEEK_SET

SEEK_CUR

SEEK_END

Parameters to the `seek()` function and `seek()` method on file-like objects, for whence to adjust the file position indicator. `SEEK_SET` Adjust the file position relative to the beginning of the file. `SEEK_CUR` Adjust the file position relative to the current file position. `SEEK_END` Adjust the file position relative to the end of the file. Their values are 0, 1, and 2, respectively.

Adjust the file position relative to the beginning of the file.

Adjust the file position relative to the current file position.

Adjust the file position relative to the end of the file.

os.SEEK_HOLE

os.SEEK_DATA

SEEK_DATA

SEEK_HOLE

Parameters to the `seek()` function and `seek()` method on file-like objects, for seeking file data and holes on sparsely allocated files. `SEEK_DATA` Adjust the file offset to the next location containing data, relative to the seek position. `SEEK_HOLE` Adjust the file offset to the next location containing a hole, relative to the seek position. A hole is defined as a sequence of zeros. Note These operations only make sense for filesystems that support them. Availability: Linux >= 3.1, macOS, Unix Added in ver

Adjust the file offset to the next location containing data, relative to the seek position.

Adjust the file offset to the next location containing a hole, relative to the seek position. A hole is defined as a sequence of zeros.

os.open(path, flags, mode=0o777, *, dir_fd=None)

Open the file `path` and set various flags according to `flags` and possibly its mode according to `mode`. When computing `mode`, the current `umask` value is first masked out. Return the file descriptor for the newly opened file. The new file descriptor is non-inheritable. For a description of the flag and mode values, see the C run-time documentation; flag constants (like `O_RDONLY` and `O_WRONLY`) are defined in the `os` module. In particular, on Windows adding `O_BINARY` is needed to open files in binary mode. This functi

The following constants are options for the `flags` parameter to the `open()` function. They can be combined using the bitwise OR operator. Some of them are not available on all platforms. For descriptions of their availability and use, consult the `open(2)` manual page on Unix or the MSDN on Windows.

os.O_RDONLY

os.O_WRONLY

os.O_RDWR

os.O_APPEND

os.O_CREAT

os.O_EXCL

os.O_TRUNC

The above constants are available on Unix and Windows.

os.O_DSYNC

os.O_RSYNC

os.O_SYNC

os.O_NDELAY

os.O_NONBLOCK

os.O_NOCTTY

os.O_CLOEXEC

The above constants are only available on Unix.Changed in version 3.3:AddO_CLOEXECconstant.

os.O_BINARY

os.O_NOINHERIT

os.O_SHORT_LIVED

os.O_TEMPORARY

os.O_RANDOM

os.O_SEQUENTIAL

os.O_TEXT

The above constants are only available on Windows.

os.O_EVTONLY

os.O_FSYNC

os.O_SYMLINK

os.O_NOFOLLOW_ANY

The above constants are only available on macOS.Changed in version 3.10:AddO_EVTONLY,O_FSYNC,O_SYMLINKandO_NOFOLLOW_ANYconstants.

os.O_ASYNC

os.O_DIRECT

os.O_DIRECTORY

os.O_NOFOLLOW

os.O_NOATIME

os.O_PATH

os.O_TMPFILE

os.O_SHLOCK

os.O_EXLOCK

The above constants are extensions and not present if they are not defined by the C library.Changed in version 3.4:AddO_PATHon systems that support it. AddO_TMPFILE, only available on Linux Kernel 3.11 or newer.

os.openpty()

Open a new pseudo-terminal pair. Return a pair of file descriptors(master,slave)for the pty and the tty, respectively. The new file descriptors arenon-inheritable. For a (slightly) more portable approach, use theptymodule.Availability: Unix, not WASI.Changed in version 3.4:The new file descriptors are now non-inheritable.

os.pipe()

Create a pipe. Return a pair of file descriptors(r,w)usable for reading and writing, respectively. The new file descriptor isnon-inheritable.Availability: Unix, Windows.Changed in version 3.4:The new file descriptors are now

non-inheritable.

os.pipe2(flags,l)

Create a pipe with flags set atomically. flags can be constructed by ORing together one or more of these values: O_NONBLOCK, O_CLOEXEC. Return a pair of file descriptors (r,w) usable for reading and writing, respectively. Availability: Unix, not WASI. Added in version 3.3.

os.posix_fallocate(fd,offset,len,l)

Ensures that enough disk space is allocated for the file specified by fd starting from offset and continuing for len bytes. Availability: Unix. Added in version 3.3.

os.posix_fadvise(fd,offset,len,advice,l)

Announces an intention to access data in a specific pattern thus allowing the kernel to make optimizations. The advice applies to the region of the file specified by fd starting at offset and continuing for len bytes. advice is one of POSIX_FADV_NORMAL, POSIX_FADV_SEQUENTIAL, POSIX_FADV_RANDOM, POSIX_FADV_NOREUSE, POSIX_FADV_WILLNEED or POSIX_FADV_DONTNEED. Availability: Unix. Added in version 3.3.

os.POSIX_FADV_NORMAL

os.POSIX_FADV_SEQUENTIAL

os.POSIX_FADV_RANDOM

os.POSIX_FADV_NOREUSE

os.POSIX_FADV_WILLNEED

os.POSIX_FADV_DONTNEED

Flags that can be used in advice in posix_fadvise() that specify the access pattern that is likely to be used. Availability: Unix. Added in version 3.3.

os.pread(fd,n,offset,l)

Read at most n bytes from file descriptor fd at a position of offset, leaving the file offset unchanged. Return a bytestring containing the bytes read. If the end of the file referred to by fd has been reached, an empty bytes object is returned. Availability: Unix. Added in version 3.3.

os.posix_openpt(oflag,l)

Open and return a file descriptor for a master pseudo-terminal device. Calls the C standard library function posix_openpt(). The oflag argument is used to set file status flags and file access modes as specified in the manual page of posix_openpt() of your system. The returned file descriptor is non-inheritable. If the value O_CLOEXEC is available on the system, it is added to oflag. Availability: Unix, not WASI. Added in version 3.13.

os.preadv(fd,buffers,offset,flags=0,l)

Read from a file descriptor fd at a position of offset into mutable bytes-like objects buffers, leaving the file offset unchanged. Transfer data into each buffer until it is full and then move on to the next buffer in the sequence to hold the rest of the data. The flags argument contains a bitwise OR of zero or more of the following flags: RWF_HIPRI, RWF_NOWAIT. Return the total number of bytes actually read which can be less than the total capacity of all the objects. The operating system may set a limit (

os.RWF_NOWAIT

Do not wait for data which is not immediately available. If this flag is specified, the system call will return instantly if it would have to read data from the backing storage or wait for a lock. If some data was successfully read, it will return the number of bytes read. If no bytes were read, it will return -1 and set errno to errno.EAGAIN. Availability: Linux >= 4.14. Added in version 3.7.

os.RWF_HIPRI

High priority read/write. Allows block-based filesystems to use polling of the device, which provides lower latency, but may use additional resources. Currently, on Linux, this feature is usable only on a file descriptor opened using the `O_DIRECT` flag. Availability: Linux >= 4.6. Added in version 3.7.

os.ptsname(fd, /)

Return the name of the slave pseudo-terminal device associated with the master pseudo-terminal device to which the file descriptor `fd` refers. The file descriptor `fd` is not closed upon failure. Calls the reentrant C standard library function `ptsname_r()` if it is available; otherwise, the C standard library function `ptsname()`, which is not guaranteed to be thread-safe, is called. Availability: Unix, not WASI. Added in version 3.13.

os.pwrite(fd, str, offset, /)

Write the byte string `str` to file descriptor `fd` at position `offset`, leaving the file offset unchanged. Return the number of bytes actually written. Availability: Unix. Added in version 3.3.

os.pwritev(fd, buffers, offset, flags=0, /)

Write the `buffers` contents to file descriptor `fd` at an `offset`, leaving the file offset unchanged. `buffers` must be a sequence of bytes-like objects. Buffers are processed in array order. Entire contents of the first buffer is written before proceeding to the second, and so on. The `flags` argument contains a bitwise OR of zero or more of the following flags: `RWF_DSYNC`, `RWF_SYNC`, `RWF_APPEND`. Return the total number of bytes actually written. The operating system may set a limit (`sysconf()` value `'SC_IOV_MAX'`) on

os.RWF_DSYNC

Provide a per-write equivalent of the `O_DSYNC` `os.open()` flag. This flag effect applies only to the data range written by the system call. Availability: Linux >= 4.7. Added in version 3.7.

os.RWF_SYNC

Provide a per-write equivalent of the `O_SYNC` `os.open()` flag. This flag effect applies only to the data range written by the system call. Availability: Linux >= 4.7. Added in version 3.7.

os.RWF_APPEND

Provide a per-write equivalent of the `O_APPEND` `os.open()` flag. This flag is meaningful only for `os.pwritev()`, and its effect applies only to the data range written by the system call. The `offset` argument does not affect the write operation; the data is always appended to the end of the file. However, if the `offset` argument is -1, the current file offset is updated. Availability: Linux >= 4.16. Added in version 3.10.

os.read(fd, n, /)

Read at most `n` bytes from file descriptor `fd`. Return a byte string containing the bytes read. If the end of the file referred to by `fd` has been reached, an empty bytes object is returned. Note This function is intended for low-level I/O and must be applied to a file descriptor as returned by `os.open()` or `pipe()`. To read a "file object" returned by the built-in function `open()` or by `popen()` or `dopen()`, or `sys.stdin`, use its `read()` or `readline()` methods. Changed in version 3.5: If the system call is interrupted and the

os.readinto(fd, buffer, /)

Read from a file descriptor `fd` into a mutable buffer object `buffer`. The buffer should be mutable and bytes-like. On success, returns the number of bytes read. Less bytes may be read than the size of the buffer. The underlying system call will be retried when interrupted by a signal, unless the signal handler raises an exception. Other errors will not be retried and an error will be raised. Returns 0 if `fd` is at end of file or if the provided buffer has length 0 (which can be used to check for errors without

os.sendfile(out_fd, in_fd, offset, count)

os.sendfile(out_fd, in_fd, offset, count, headers=(), trailers=(), flags=0)

Copycountbytes from file descriptorin_fdto file descriptorout_fdstarting atoffset. Return the number of bytes sent. When EOF is reached return0.The first function notation is supported by all platforms that definesendfile().On Linux, ifoffsetis given asNone, the bytes are read from the current position ofin_fdand the position ofin_fdis updated.The second case may be used on macOS and FreeBSD whereheadersandtrailersare arbitrary sequences of buffers that are written before and after the data from

os.SF_NODISKIO

os.SF_MNOWAIT

os.SF_SYNC

Parameters to thesendfile()function, if the implementation supports them.Availability: Unix, not WASI.Added in version 3.3.

os.SF_NOCACHE

Parameter to thesendfile()function, if the implementation supports it. The data won't be cached in the virtual memory and will be freed afterwards.Availability: Unix, not WASI.Added in version 3.11.

os.set_blocking(fd,blocking,/)

Set the blocking mode of the specified file descriptor. Set theO_NONBLOCKflag if blocking isFalse, clear the flag otherwise.See alsoget_blocking()andsocket.socket.setblocking().Availability: Unix, Windows.The function is limited on WASI, seeWebAssembly platformsfor more information.On Windows, this function is limited to pipes.Added in version 3.5.Changed in version 3.12:Added support for pipes on Windows.

os.splice(src,dst,count,offset_src=None,offset_dst=None,flags=0)

Transfercountbytes from file descriptorsrc, starting from offsetoffset_src, to file descriptordst, starting from offsetoffset_dst.The splicing behaviour can be modified by specifying aflagsvalue. Any of the following variables may used, combined using bitwise OR (the|operator):IfSPLICE_F_MOVEis specified, the kernel is asked to move pages instead of copying, but pages may still be copied if the kernel cannot move the pages from the pipe.IfSPLICE_F_NONBLOCKis specified, the kernel is asked to not

os.SPLICE_F_MOVE

os.SPLICE_F_NONBLOCK

os.SPLICE_F_MORE

Added in version 3.10.

os.readv(fd,buffers,/)

Read from a file descriptorfdinto a number of mutablebytes-like objectsbuffers. Transfer data into each buffer until it is full and then move on to the next buffer in the sequence to hold the rest of the data.Return the total number of bytes actually read which can be less than the total capacity of all the objects.The operating system may set a limit (sysconf(value'SC_IOV_MAX') on the number of buffers that can be used.Availability: Unix.Added in version 3.3.

os.tcgetpgrp(fd,/)

Return the process group associated with the terminal given byfd(an open file descriptor as returned byos.open()).Availability: Unix, not WASI.

os.tcsetpgrp(fd,pg,/)

Set the process group associated with the terminal given byfd(an open file descriptor as returned byos.open()) topg.Availability: Unix, not WASI.

os.ttyname(fd,/)

Return a string which specifies the terminal device associated with file descriptor `fd`. If `fd` is not associated with a terminal device, an exception is raised. Availability: Unix.

os.unlockpt(`fd`,/)

Unlock the slave pseudo-terminal device associated with the master pseudo-terminal device to which the file descriptor `fd` refers. The file descriptor `fd` is not closed upon failure. Calls the C standard library function `unlockpt()`. Availability: Unix, not WASI. Added in version 3.13.

os.write(`fd`,`str`,/)

Write the bytestring `instr` to file descriptor `fd`. Return the number of bytes actually written. Note This function is intended for low-level I/O and must be applied to a file descriptor as returned by `os.open()` or `pipe()`. To write a “file object” returned by the built-in function `open()` or `bypopen()` or `fdopen()`, use `sys.stdout` or `sys.stderr`, use `itswrite()` method. Changed in version 3.5: If the system call is interrupted and the signal handler does not raise an exception, the function now retries the system call `i`

os.writev(`fd`,`buffers`,/)

Write the contents of `buffer`s to file descriptor `fd`. `buffer`s must be a sequence of bytes-like objects. Buffers are processed in array order. Entire contents of the first buffer is written before proceeding to the second, and so on. Returns the total number of bytes actually written. The operating system may set a limit (`sysconf()` value `SC_IOV_MAX`) on the number of buffers that can be used. Availability: Unix. Added in version 3.3.

Querying the size of a terminal

Added in version 3.3.

os.get_terminal_size(`fd=STDOUT_FILENO`,/)

Return the size of the terminal window as (`columns`, `lines`), tuple of type `terminal_size`. The optional argument `fd` (default `STDOUT_FILENO`, or standard output) specifies which file descriptor should be queried. If the file descriptor is not connected to a terminal, an `OSError` is raised. `shutil.get_terminal_size()` is the high-level function which should normally be used, `os.get_terminal_size` is the low-level implementation. Availability: Unix, Windows.

class os.terminal_size

columns

lines

A subclass of tuple, holding (`columns`, `lines`) of the terminal window size. `columns` Width of the terminal window in characters. `lines` Height of the terminal window in characters.

Width of the terminal window in characters.

Height of the terminal window in characters.

Inheritance of File Descriptors

Added in version 3.4.

A file descriptor has an “inheritable” flag which indicates if the file descriptor can be inherited by child processes. Since Python 3.4, file descriptors created by Python are non-inheritable by default.

On UNIX, non-inheritable file descriptors are closed in child processes at the execution of a new program, other file descriptors are inherited. Note that non-inheritable file descriptors are still inherited by child processes on `os.fork()`.

On Windows, non-inheritable handles and file descriptors are closed in child processes, except for standard streams (file descriptors 0, 1 and 2: stdin, stdout and stderr), which are always inherited. Using `spawn* functions`, all inheritable handles and all inheritable file descriptors are inherited. Using `subprocess module`, all file descriptors except standard streams are closed, and inheritable handles are only inherited if the `close_fds` parameter is `False`.

On WebAssembly platforms, the file descriptor cannot be modified.

`os.get_inheritable(fd, /)`

Get the “inheritable” flag of the specified file descriptor (a boolean).

`os.set_inheritable(fd, inheritable, /)`

Set the “inheritable” flag of the specified file descriptor.

`os.get_handle_inheritable(handle, /)`

Get the “inheritable” flag of the specified handle (a boolean). Availability: Windows.

`os.set_handle_inheritable(handle, inheritable, /)`

Set the “inheritable” flag of the specified handle. Availability: Windows.

Files and Directories

On some Unix platforms, many of these functions support one or more of these features:

- specifying a file descriptor: Normally the `path` argument provided to functions in the `os module` must be a string specifying a file path. However, some functions now alternatively accept an open file descriptor for their `path` argument. The function will then operate on the file referred to by the descriptor. For POSIX systems, Python will call the variant of the function prefixed with `f` (e.g. `callfchdir`).
- paths relative to directory descriptors: If `dir_fd` is not `None`, it should be a file descriptor referring to a directory, and the path to operate on should be relative; path will then be relative to that directory. If the path is absolute, `dir_fd` is ignored. For POSIX systems, Python will call the variant of the function with an `at` suffix and possibly prefixed with `f` (e.g. `callfaccessat` instead of `access`). You
- not following symlinks: If `follow_symlinks` is `False`, and the last element of the path to operate on is a symbolic link, the function will operate on the symbolic link itself rather than the file pointed to by the link. For POSIX systems, Python will call the `l...` variant of the function. You can check whether or not `follow_symlinks` is supported for a particular function on your platform using `os.supports_follow_symlinks`.

`os.access(path, mode, *, dir_fd=None, effective_ids=False, follow_symlinks=True)`

Use the real uid/gid to test for access to `path`. Note that most operations will use the effective uid/gid, therefore this routine can be used in a `suid/sgid` environment to test if the invoking user has the specified access to `path`. `mode` should be `F_OK` to test the existence of `path`, or it can be the inclusive OR of one or more of `R_OK`, `W_OK`, and `X_OK` to test permissions. Return `True` if access is allowed, `False` if not. See the Unix man page `access(2)` for more information. This function can support specifying paths

`os.F_OK`

`os.R_OK`

`os.W_OK`

`os.X_OK`

Values to pass as the `mode` parameter of `access()` to test the existence, readability, writability and executability of `path`, respectively.

os.chdir(path)

Change the current working directory to `path`. This function can support specifying a file descriptor. The descriptor must refer to an opened directory, not an open file. This function can raise `OSError` and subclasses such as `FileNotFoundError`, `PermissionError`, and `NotADirectoryError`. Raises an auditing event `os.chdir` with argument `path`. Changed in version 3.3: Added support for specifying `path` as a file descriptor on some platforms. Changed in version 3.6: Accepts `apath`-like object.

os.chflags(path, flags, *, follow_symlinks=True)

Set the flags of `path` to the numeric `flags`. `flags` may take a combination (bitwise OR) of the following values (as defined in the `stat` module): `stat.UF_NODUMP`, `stat.UF_IMMUTABLE`, `stat.UF_APPEND`, `stat.UF_OPAQUE`, `stat.UF_NOUNLINK`, `stat.UF_COMPRESSED`, `stat.UF_HIDDEN`, `stat.SF_ARCHIVED`, `stat.SF_IMMUTABLE`, `stat.SF_APPEND`, `stat.SF_NOUNLINK`, `stat.SF_SNAPSHOT`. This function can support not following symlinks. Raises an auditing event `os.chflags` with arguments `path`, `flags`. Availability: Unix, not WASI. Changed in version 3.3: Added the `follow_symlinks`

os.chmod(path, mode, *, dir_fd=None, follow_symlinks=True)

Change the mode of `path` to the numeric `mode`. `mode` may take one of the following values (as defined in the `stat` module) or bitwise ORed combinations of them: `stat.S_ISUID`, `stat.S_ISGID`, `stat.S_ENFMT`, `stat.S_ISVTX`, `stat.S_IREAD`, `stat.S_IWRITE`, `stat.S_IXEXEC`, `stat.S_IRWXU`, `stat.S_IRUSR`, `stat.S_IWUSR`, `stat.S_IXUSR`, `stat.S_IRWXG`, `stat.S_IRGRP`, `stat.S_IWGRP`, `stat.S_IXGRP`, `stat.S_IRWXO`, `stat.S_IROTH`, `stat.S_IWOTH`, `stat.S_IXOTH`. This function can support specifying a file descriptor, `paths` relative to directory descriptors and not following symlinks. Not

os.chown(path, uid, gid, *, dir_fd=None, follow_symlinks=True)

Change the owner and group id of `path` to the numeric `uid` and `gid`. To leave one of the ids unchanged, set it to `-1`. This function can support specifying a file descriptor, `paths` relative to directory descriptors and not following symlinks. See `shutil.chown()` for a higher-level function that accepts names in addition to numeric ids. Raises an auditing event `os.chown` with arguments `path`, `uid`, `gid`, `dir_fd`. Availability: Unix. The function is limited on WASI, see `WebAssembly` platforms for more information. Changed in version

os.chroot(path)

Change the root directory of the current process to `path`. Availability: Unix, not WASI, not Android. Changed in version 3.6: Accepts `apath`-like object.

os.fchdir(fd)

Change the current working directory to the directory represented by the file descriptor `fd`. The descriptor must refer to an opened directory, not an open file. As of Python 3.3, this is equivalent to `os.chdir(fd)`. Raises an auditing event `os.fchdir` with argument `path`. Availability: Unix.

os.getcwd()

Return a string representing the current working directory.

os.getcwdb()

Return a `bytes` string representing the current working directory. Changed in version 3.8: The function now uses the UTF-8 encoding on Windows, rather than the ANSI code page: see [PEP 529](#) for the rationale. The function is no longer deprecated on Windows.

os.lchflags(path, flags)

Set the flags of `path` to the numeric `flags`, like `chflags()`, but do not follow symbolic links. As of Python 3.3, this is equivalent to `os.chflags(path, flags, follow_symlinks=False)`. Raises an auditing event `os.lchflags` with arguments `path`, `flags`. Availability: Unix, not WASI. Changed in version 3.6: Accepts `apath`-like object.

os.lchmod(path,mode)

Change the mode of `path` to the numeric `mode`. If `path` is a symlink, this affects the symlink rather than the target. See the docs for `chmod()` for possible values of `mode`. As of Python 3.3, this is equivalent to `os.chmod(path,mode,follow_symlinks=False)`. `lchmod()` is not part of POSIX, but Unix implementations may have it if changing the mode of symbolic links is supported. Raises `NotImplementedError` if `os.chmod` with arguments `path,mode,dir_fd`. Availability: Unix, Windows, not Linux, FreeBSD >= 1.3, NetBSD >= 1.3, n

os.lchown(path,uid,gid)

Change the owner and group id of `path` to the numeric `uid` and `gid`. This function will not follow symbolic links. As of Python 3.3, this is equivalent to `os.chown(path,uid,gid,follow_symlinks=False)`. Raises `NotImplementedError` if `os.chown` with arguments `path,uid,gid,dir_fd`. Availability: Unix. Changed in version 3.6: Accepts a path-like object.

os.link(src,dst,*,src_dir_fd=None,dst_dir_fd=None,follow_symlinks=True)

Create a hard link pointing to `src` named `dst`. This function can support specifying `src_dir_fd` and/or `dst_dir_fd` to supply paths relative to directory descriptors, and not following symlinks. The default value of `follow_symlinks` is `False` on Windows. Raises `NotImplementedError` if `os.link` with arguments `src,dst,src_dir_fd,dst_dir_fd`. Availability: Unix, Windows. Changed in version 3.2: Added Windows support. Changed in version 3.3: Added the `src_dir_fd,dst_dir_fd`, and `follow_symlinks` parameters. Changed in version 3.6: Accepts a path-like object.

os.listdir(path='.')

Return a list containing the names of the entries in the directory given by `path`. The list is in arbitrary order, and does not include the special entries `'.'` and `'..'` even if they are present in the directory. If a file is removed from or added to the directory during the call of this function, whether a name for that file be included is unspecified. `path` may be a path-like object. If `path` is of type `bytes` (directly or indirectly through the `PathLike` interface), the filenames returned will also be of type `bytes`.

os.listdir drives()

Return a list containing the names of drives on a Windows system. A drive name typically looks like `'C:\\'`. Not every drive name will be associated with a volume, and some may be inaccessible for a variety of reasons, including permissions, network connectivity or missing media. This function does not test for access. May raise `OSError` if an error occurs collecting the drive names. Raises `NotImplementedError` if `os.listdir drives` with no arguments. Availability: Windows. Added in version 3.12.

os.listmounts(volume)

Return a list containing the mount points for a volume on a Windows system. `volume` must be represented as a GUID path, like those returned by `os.listdir volumes()`. Volumes may be mounted in multiple locations or not at all. In the latter case, the list will be empty. Mount points that are not associated with a volume will not be returned by this function. The mount points returned by this function will be absolute paths, and may be longer than the drive name. Raises `OSError` if the volume is not recognized or

os.listdir volumes()

Return a list containing the volumes in the system. Volumes are typically represented as a GUID path that looks like `\\?\\Volume{xxxxxxx-xxxx-xxxx-xxxx-xxxxxxxxxxx}\\`. Files can usually be accessed through a GUID path, permissions allowing. However, users are generally not familiar with them, and so the recommended use of this function is to retrieve mount points using `os.listdir mounts()`. May raise `OSError` if an error occurs collecting the volumes. Raises `NotImplementedError` if `os.listdir volumes` with no arguments. Av

os.lstat(path,*,dir_fd=None)

Perform the equivalent of `anlstat()` system call on the given path. Similar to `stat()`, but does not follow symbolic links. Return `stat_result` object. On platforms that do not support symbolic links, this is an alias for `stat()`. As of Python 3.3, this is equivalent to `os.stat(path, dir_fd=dir_fd, follow_symlinks=False)`. This function can also support paths relative to directory descriptors. See also `os.stat()` function. Changed in version 3.2: Added support for Windows 6.0 (Vista) symbolic links. Changed in version 3.3: Added support for directory descriptors.

os.mkdir(path, mode=0o777, *, dir_fd=None)

Create a directory named `path` with numeric mode `mode`. If the directory already exists, `FileExistsError` is raised. If a parent directory in the path does not exist, `FileNotFoundError` is raised. On some systems, `mode` is ignored. Where it is used, the current umask value is first masked out. If bits other than the last 9 (i.e. the last 3 digits of the octal representation of the mode) are set, their meaning is platform-dependent. On some platforms, they are ignored and you should call `chmod()` explicitly to set them.

os.makedirs(name, mode=0o777, exist_ok=False)

Recursive directory creation function. Like `mkdir()`, but makes all intermediate-level directories needed to contain the leaf directory. The `mode` parameter is passed to `mkdir()` for creating the leaf directory; see the `mkdir()` description for how it is interpreted. To set the file permission bits of any newly created parent directories you can set the umask before invoking `makedirs()`. The file permission bits of existing parent directories are not changed. If `exist_ok` is `False` (the default), a `FileExistsError` is raised if the directory already exists.

os.mkfifo(path, mode=0o666, *, dir_fd=None)

Create a FIFO (a named pipe) named `path` with numeric mode `mode`. The current umask value is first masked out from the mode. This function can also support paths relative to directory descriptors. FIFOs are pipes that can be accessed like regular files. FIFOs exist until they are deleted (for example with `os.unlink()`). Generally, FIFOs are used as rendezvous between “client” and “server” type processes: the server opens the FIFO for reading, and the client opens it for writing. Note that `mkfifo()` doesn't create the pipe if it already exists.

os.mknod(path, mode=0o600, device=0, *, dir_fd=None)

Create a filesystem node (file, device special file or named pipe) named `path`. `mode` specifies both the permissions to use and the type of node to be created, being combined (bitwise OR) with one of `stat.S_IFREG`, `stat.S_IFCHR`, `stat.S_IFBLK`, and `stat.S_IFIFO` (those constants are available in `stat`). For `stat.S_IFCHR` and `stat.S_IFBLK`, `device` defines the newly created device special file (probably using `os.makedev()`), otherwise it is ignored. This function can also support paths relative to directory descriptors. A `FileExistsError` is raised if the node already exists.

os.major(device, /)

Extract the device major number from a raw device number (usually the `st_dev` or `st_rdev` field from `stat`).

os.minor(device, /)

Extract the device minor number from a raw device number (usually the `st_dev` or `st_rdev` field from `stat`).

os.makedev(major, minor, /)

Compose a raw device number from the major and minor device numbers.

os.pathconf(path, name)

Return system configuration information relevant to a named file. `name` specifies the configuration value to retrieve; it may be a string which is the name of a defined system value; these names are specified in a number of standards (POSIX.1, Unix 95, Unix 98, and others). Some platforms define additional names as well. The names known to the host operating system are given in the `pathconf_names` dictionary. For configuration variables not included in that mapping, passing an integer for `name` is also allowed.

os.pathconf_names

Dictionary mapping names accepted by `pathconf()` and `fpathconf()` to the integer values defined for those names by the host operating system. This can be used to determine the set of names known to the system. Availability: Unix.

`os.readlink(path, *, dir_fd=None)`

Return a string representing the path to which the symbolic link points. The result may be either an absolute or relative pathname; if it is relative, it may be converted to an absolute pathname using `os.path.join(os.path.dirname(path), result)`. If `path` is a string object (directly or indirectly through a `PathLike` interface), the result will also be a string object, and the call may raise a `UnicodeDecodeError`. If `path` is a bytes object (direct or indirectly), the result will be a bytes object. The

`os.remove(path, *, dir_fd=None)`

Remove (delete) the file `path`. If `path` is a directory, an `OSError` is raised. Use `rmdir()` to remove directories. If the file does not exist, a `FileNotFoundError` is raised. This function can support paths relative to directory descriptors. On Windows, attempting to remove a file that is in use causes an exception to be raised; on Unix, the directory entry is removed but the storage allocated to the file is not made available until the original file is no longer in use. This function is semantically identical

`os.removedirs(name)`

Remove directories recursively. Works like `rmdir()` except that, if the leaf directory is successfully removed, `removedirs()` tries to successively remove every parent directory mentioned in `path` until an error is raised (which is ignored, because it generally means that a parent directory is not empty). For example, `os.removedirs('foo/bar/baz')` will first remove the directory `'foo/bar/baz'`, and then remove `'foo/bar'` and `'foo'` if they are empty. Raises `OSError` if the leaf directory could not be successfully removed.

`os.rename(src, dst, *, src_dir_fd=None, dst_dir_fd=None)`

Rename the file or directory `src` to `dst`. If `dst` exists, the operation will fail with an `OSError` subclass in a number of cases: On Windows, if `dst` exists a `FileExistsError` is always raised. The operation may fail if `src` and `dst` are on different filesystems. Use `shutil.move()` to support moves to a different filesystem. On Unix, if `src` is a file and `dst` is a directory or vice-versa, an `IsADirectoryError` or a `NotADirectoryError` will be raised respectively. If both are directories and `dst` is empty, `dst` will be silently replaced.

`os.rename(old, new)`

Recursive directory or file renaming function. Works like `rename()`, except creation of any intermediate directories needed to make the new pathname good is attempted first. After the rename, directories corresponding to rightmost path segments of the old name will be pruned away using `removedirs()`. Note This function can fail with the new directory structure made if you lack permissions needed to remove the leaf directory or file. Raises an auditing event `os.rename` with arguments `src, dst, src_dir_fd, dst_dir_fd`

`os.replace(src, dst, *, src_dir_fd=None, dst_dir_fd=None)`

Rename the file or directory `src` to `dst`. If `dst` is a non-empty directory, `OSError` will be raised. If `dst` exists and is a file, it will be replaced silently if the user has permission. The operation may fail if `src` and `dst` are on different filesystems. If successful, the renaming will be an atomic operation (this is a POSIX requirement). This function can support specifying `src_dir_fd` and/or `dst_dir_fd` to supply paths relative to directory descriptors. Raises an auditing event `os.rename` with arguments `src, dst, src_dir_fd, dst_dir_fd`

`os.rmdir(path, *, dir_fd=None)`

Remove (delete) the directory `path`. If the directory does not exist or is not empty, a `FileNotFoundError` or an `OSError` is raised respectively. In order to remove whole directory trees, `shutil.rmtree()` can be used. This function can support paths relative to directory descriptors. Raises an auditing event `os.rmdir` with arguments `path, dir_fd`. Changed in version 3.3: Added the `dir_fd` parameter. Changed in version 3.6: Accepts a path-like object.

os.scandir(path='.')

scandir.close()

Return an iterator of `os.DirEntry` objects corresponding to the entries in the directory given by `path`. The entries are yielded in arbitrary order, and the special entries `'.'` and `..'` are not included. If a file is removed from or added to the directory after creating the iterator, whether an entry for that file be included is unspecified. Using `scandir()` instead of `listdir()` can significantly increase the performance of code that also needs file type or file attribute information, because `os.DirEntry` object

Close the iterator and free acquired resources. This is called automatically when the iterator is exhausted or garbage collected, or when an error happens during iterating. However it is advisable to call it explicitly or use the `with` statement. Added in version 3.6.

class os.DirEntry

name

path

inode()

is_dir(*, follow_symlinks=True)

is_file(*, follow_symlinks=True)

is_symlink()

is_junction()

stat(*, follow_symlinks=True)

Object yielded by `scandir()` to expose the file path and other file attributes of a directory entry. `scandir()` will provide as much of this information as possible without making additional system calls. When `astat()` or `lstat()` system call is made, the `os.DirEntry` object will cache the result. `os.DirEntry` instances are not intended to be stored in long-lived data structures; if you know the file metadata has changed or if a long time has elapsed since calling `scandir()`, call `os.stat(entry.path)` to fetch up-to-

The entry's base filename, relative to the `scandir()` `path` argument. The `name` attribute will be bytes if the `scandir()` `path` argument is of type bytes and str otherwise. Use `fsdecode()` to decode byte filenames.

The entry's full path name: equivalent to `os.path.join(scandir_path, entry.name)` where `scandir_path` is the `scandir()` `path` argument. The path is only absolute if the `scandir()` `path` argument was absolute. If the `scandir()` `path` argument was a file descriptor, the `path` attribute is the same as the `name` attribute. The `path` attribute will be bytes if the `scandir()` `path` argument is of type bytes and str otherwise. Use `fsdecode()` to decode byte filenames.

Return the inode number of the entry. The result is cached on the `os.DirEntry` object.

Use `os.stat(entry.path, follow_symlinks=False).st_ino` to fetch up-to-date information. On the first, uncached call, a system call is required on Windows but not on Unix.

Return `True` if this entry is a directory or a symbolic link pointing to a directory; return `False` if the entry is or points to any other kind of file, or if it doesn't exist anymore. If `follow_symlinks` is `False`, return `True` only if this entry is a directory (without following symlinks); return `False` if the entry is any other kind of file or if it doesn't exist anymore. The result is cached on the `os.DirEntry` object, with a separate cache for `follow_symlinks` `True` and `False`. Call `os.stat()` along with `stat.S_ISDIR()` to f

Return `True` if this entry is a file or a symbolic link pointing to a file; return `False` if the entry is or points to a directory or other non-file entry, or if it doesn't exist anymore. If `follow_symlinks` is `False`, return `True` only if this entry is a file (without following symlinks); return `False` if the entry is a directory or other non-file entry, or if it doesn't exist anymore. The result is cached on the `os.DirEntry` object. Caching, system calls made, and

exceptions raised are as per `is_dir()`.

Return `True` if this entry is a symbolic link (even if broken); return `False` if the entry points to a directory or any kind of file, or if it doesn't exist anymore. The result is cached on the `os.DirEntry` object. `os.path.islink()` to fetch up-to-date information. On the first, uncached call, no system call is required in most cases. Specifically, neither Windows or Unix require a system call, except on certain Unix file systems, such as network file systems, that return `dirent.d_type==DT_UNKNOWN`. This method

Return `True` if this entry is a junction (even if broken); return `False` if the entry points to a regular directory, any kind of file, a symlink, or if it doesn't exist anymore. The result is cached on the `os.DirEntry` object. `os.path.isjunction()` to fetch up-to-date information. Added in version 3.12.

Return an `os.stat_result` object for this entry. This method follows symbolic links by default; to stat a symbolic link add the `follow_symlinks=False` argument. On Unix, this method always requires a system call. On Windows, it only requires a system call if `follow_symlinks=True` and the entry is a reparse point (for example, a symbolic link or directory junction). On Windows, the `st_ino`, `st_dev` and `st_nlink` attributes of the `stat_result` are always set to zero. `os.stat()` to get these attributes. The result is cached

`os.stat(path, *, dir_fd=None, follow_symlinks=True)`

Get the status of a file or a file descriptor. Perform the equivalent of `stat()` system call on the given `path`. `path` may be specified as either a string or bytes – directly or indirectly through the `PathLike` interface – or as an open file descriptor. Return an `os.stat_result` object. This function normally follows symlinks; to stat a symlink add the argument `follow_symlinks=False`, or `usestat()`. This function can support specifying a file descriptor and not following symlinks. On Windows, passing `follow_symlinks=False`

`class os.stat_result`

`st_mode`

`st_ino`

`st_dev`

`st_nlink`

`st_uid`

`st_gid`

`st_size`

`st_atime`

`st_mtime`

`st_ctime`

`st_atime_ns`

`st_mtime_ns`

`st_ctime_ns`

`st_birthtime`

`st_birthtime_ns`

`st_blocks`

`st_blksize`

`st_rdev`

`st_flags`

st_gen

st_fstype

st_rsize

st_creator

st_type

st_file_attributes

st_reparse_tag

Object whose attributes correspond roughly to the members of the `stat` structure. It is used for the result of `os.stat()`, `os.fstat()` and `os.lstat()`. Attributes: `st_mode` File mode: file type and file mode bits (permissions). `st_ino` Platform dependent, but if non-zero, uniquely identifies the file for a given value of `st_dev`. Typically: the inode number on Unix, the file index on Windows. `st_dev` Identifier of the device on which this file resides. `st_nlink` Number of hard links. `st_uid` User identifier of the file owner. `st`

File mode: file type and file mode bits (permissions).

Platform dependent, but if non-zero, uniquely identifies the file for a given value of `st_dev`. Typically: the inode number on Unix, the file index on Windows

Identifier of the device on which this file resides.

Number of hard links.

User identifier of the file owner.

Group identifier of the file owner.

Size of the file in bytes, if it is a regular file or a symbolic link. The size of a symbolic link is the length of the pathname it contains, without a terminating null byte.

Time of most recent access expressed in seconds.

Time of most recent content modification expressed in seconds.

Time of most recent metadata change expressed in seconds. Changed in version 3.12: `st_ctime` is deprecated on Windows. Use `st_birthtime` for the file creation time. In the future, `st_ctime` will contain the time of the most recent metadata change, as for other platforms.

Time of most recent access expressed in nanoseconds as an integer. Added in version 3.3.

Time of most recent content modification expressed in nanoseconds as an integer. Added in version 3.3.

Time of most recent metadata change expressed in nanoseconds as an integer. Added in version 3.3. Changed in version 3.12: `st_ctime_ns` is deprecated on Windows. Use `st_birthtime_ns` for the file creation time. In the future, `st_ctime` will contain the time of the most recent metadata change, as for other platforms.

Time of file creation expressed in seconds. This attribute is not always available, and may raise `AttributeError`. Changed in version 3.12: `st_birthtime` is now available on Windows.

Time of file creation expressed in nanoseconds as an integer. This attribute is not always available, and may raise `AttributeError`. Added in version 3.12.

Number of 512-byte blocks allocated for file. This may be smaller than `st_size/512` when the file has holes.

“Preferred” blocksize for efficient file system I/O. Writing to a file in smaller chunks may cause an inefficient read-modify-rewrite.

Type of device if an inode device.

User defined flags for file.

File generation number.

String that uniquely identifies the type of the filesystem that contains the file.

Real size of the file.

Creator of the file.

File type.

Windows file attributes: `dwFileAttributes` member of the `BY_HANDLE_FILE_INFORMATION` structure returned by `GetFileInformationByHandle()`. See the `FILE_ATTRIBUTE_*` <`stat.FILE_ATTRIBUTE_ARCHIVE`> constants in the `stat` module. Added in version 3.5.

When `st_file_attributes` has the `FILE_ATTRIBUTE_REPARSE_POINT` set, this field contains the tag identifying the type of reparse point. See the `IO_REPARSE_TAG_*` constants in the `stat` module.

os.statvfs(path)

Perform a `statvfs()` system call on the given path. The return value is an object whose attributes describe the filesystem on the given path, and correspond to the members of the `statvfs` structure, namely: `f_bsize`, `f_frsize`, `f_blocks`, `f_bfree`, `f_bavail`, `f_files`, `f_ffree`, `f_favail`, `f_flag`, `f_namemax`, `f_fsid`. Two module-level constants are defined for the `f_flag` attribute's bit-flags: if `ST_RDONLY` is set, the filesystem is mounted read-only, and if `ST_NOSUID` is set, the semantics of `setuid`/`setgid` bits are disabled or n

os.supports_dir_fd

A set object indicating which functions in the `os` module accept an open file descriptor for their `dir_fd` parameter. Different platforms provide different features, and the underlying functionality Python uses to implement the `dir_fd` parameter is not available on all platforms Python supports. For consistency's sake, functions that may support `dir_fd` always allow specifying the parameter, but will throw an exception if the functionality is used when it's not locally available. (Specifying `None` for `dir_fd` is a

os.supports_effective_ids

A set object indicating whether `os.access()` permits specifying `True` for its `effective_ids` parameter on the local platform. (Specifying `False` for `effective_ids` is always supported on all platforms.) If the local platform supports it, the collection will contain `os.access()`; otherwise it will be empty. This expression evaluates to `True` if `os.access()` supports `effective_ids=True` on the local platform: `os.access` in `os.supports_effective_ids`. Currently `effective_ids` is only supported on Unix platforms; it does not work on Win

os.supports_fd

A set object indicating which functions in the `os` module permit specifying their `path` parameter as an open file descriptor on the local platform. Different platforms provide different features, and the underlying functionality Python uses to accept open file descriptors as `path` arguments is not available on all platforms Python supports. To determine whether a particular function permits specifying an open file descriptor for its `path` parameter, use the `in` operator on `supports_fd`. As an example, this express

os.supports_follow_symlinks

A set object indicating which functions in the `os` module accept `False` for their `follow_symlinks` parameter on the local platform. Different platforms provide different features, and the underlying functionality Python uses to implement `follow_symlinks` is not available on all platforms Python supports. For consistency's sake, functions that may support `follow_symlinks` always allow specifying the parameter, but will throw an exception if the functionality is used when it's not locally available. (Specifying `T`

os.symlink(src,dst,target_is_directory=False,*,dir_fd=None)

Create a symbolic link pointing to `srcname`. The `src` parameter refers to the target of the link (the file or directory being linked to), and `dst` is the name of the link being created. On Windows, a symlink represents either a file or a directory, and does not morph to the target dynamically. If the target is present, the type of the symlink will be created to match. Otherwise, the symlink will be created as a directory if `target_is_directory` is `True` or a file symlink (the default) otherwise. On non-Wi

os.sync()

Force write of everything to disk. Availability: Unix. Added in version 3.3.

os.truncate(path,length)

Truncate the file corresponding to `path`, so that it is at most `length` bytes in size. This function can support specifying a file descriptor. Raises an `auditing` event `os.truncate` with arguments `path,length`. Availability: Unix, Windows. Added in version 3.3. Changed in version 3.5: Added support for Windows. Changed in version 3.6: Accepts `path`-like object.

os.unlink(path,*,dir_fd=None)

Remove (delete) the file `path`. This function is semantically identical to `remove()`; the `unlink` name is its traditional Unix name. Please see the documentation for `remove()` for further information. Raises an `auditing` event `os.remove` with arguments `path,dir_fd`. Changed in version 3.3: Added the `dir_fd` parameter. Changed in version 3.6: Accepts `path`-like object.

os.utime(path,times=None,*,[ns,]dir_fd=None,follow_symlinks=True)

Set the access and modified times of the file specified by `path`. `utime()` takes two optional parameters, `times` and `ns`. These specify the times set on `path` and are used as follows: If `ns` is specified, it must be a 2-tuple of the form `(atime_ns, mtime_ns)` where each member is an int expressing nanoseconds. If `times` is not `None`, it must be a 2-tuple of the form `(atime, mtime)` where each member is an int or float expressing seconds. If `times` is `None` and `ns` is unspecified, this is equivalent to specifying `ns=(atime_ns, mtime_ns)` wh

os.walk(top,topdown=True,onerror=None,followlinks=False)

Generate the file names in a directory tree by walking the tree either top-down or bottom-up. For each directory in the tree rooted at `directory` `top` (including `top` itself), it yields a 3-tuple `(dirpath, dirnames, filenames)`. `dirpath` is a string, the path to the directory. `dirnames` is a list of the names of the subdirectories in `dirpath` (including symlinks to directories, and excluding `'.'` and `'..'`). `filenames` is a list of the names of the non-directory files in `dirpath`. Note that the names in the lists contain no

os.fwalk(top='.',topdown=True,onerror=None,*,follow_symlinks=False,dir_fd=None)

This behaves exactly like `walk()`, except that it yields a 4-tuple `(dirpath, dirnames, filenames, dirfd)`, and it supports `dir_fd`. `dirpath`, `dirnames` and `filenames` are identical to `walk()` output, and `dirfd` is a file descriptor referring to the directory `dirpath`. This function always supports `paths` relative to directory descriptors and not following symlinks. Note however that, unlike other functions, the `fwalk()` default value for `follow_symlinks` is `False`. Note Since `fwalk()` yields file descriptors, those are only valid until

os.memfd_create(name[,flags=os.MFD_CLOEXEC])

Create an anonymous file and return a file descriptor that refers to it. `flags` must be one of the `os.MFD_*` constants available on the system (or a bitwise ORed combination of them). By default, the new file descriptor is non-inheritable. The name supplied in `name` is used as a filename and will be displayed as the target of the corresponding symbolic link in the directory `/proc/self/fd/`. The displayed name is always prefixed with `memfd:` and serves only for debugging purposes. Names do not affect the behavi

os.MFD_CLOEXEC

os.MFD_ALLOW_SEALING

os.MFD_HUGETLB
os.MFD_HUGE_SHIFT
os.MFD_HUGE_MASK
os.MFD_HUGE_64KB
os.MFD_HUGE_512KB
os.MFD_HUGE_1MB
os.MFD_HUGE_2MB
os.MFD_HUGE_8MB
os.MFD_HUGE_16MB
os.MFD_HUGE_32MB
os.MFD_HUGE_256MB
os.MFD_HUGE_512MB
os.MFD_HUGE_1GB
os.MFD_HUGE_2GB
os.MFD_HUGE_16GB

These flags can be passed to `mfd_create()`. Availability: Linux $\geq$ 3.17 with glibc $\geq$ 2.27. The `MFD_HUGE*` flags are only available since Linux 4.14. Added in version 3.8.

os.eventfd(*initval* [, *flags* = `os.EFD_CLOEXEC`])

Create and return an event file descriptor. The file descriptor supports `rawread()` and `write()` with a buffer size of 8, `select()`, `poll()` and similar. See `man page eventfd(2)` for more information. By default, the new file descriptor is non-inheritable. *initval* is the initial value of the event counter. The initial value must be a 32 bit unsigned integer. Please note that the initial value is limited to a 32 bit unsigned int although the event counter is an unsigned 64 bit integer with a maximum value of 2^{64} .

os.eventfd_read(*fd*)

Read value from an `eventfd()` file descriptor and return a 64 bit unsigned int. The function does not verify that *fd* is an `eventfd()`. Availability: Linux $\geq$ 2.6.27. Added in version 3.10.

os.eventfd_write(*fd*, *value*)

Add value to an `eventfd()` file descriptor. *value* must be a 64 bit unsigned int. The function does not verify that *fd* is an `eventfd()`. Availability: Linux $\geq$ 2.6.27. Added in version 3.10.

os.EFD_CLOEXEC

Set close-on-exec flag for new `eventfd()` file descriptor. Availability: Linux $\geq$ 2.6.27. Added in version 3.10.

os.EFD_NONBLOCK

Set `O_NONBLOCK` status flag for new `eventfd()` file descriptor. Availability: Linux $\geq$ 2.6.27. Added in version 3.10.

os.EFD_SEMAPHORE

Provide semaphore-like semantics for reads from an `eventfd()` file descriptor. On read the internal counter is decremented by one. Availability: Linux $\geq$ 2.6.30. Added in version 3.10.

Timer File Descriptors

Added in version 3.13.

These functions provide support for Linux's timer file descriptor API. Naturally, they are all only available on Linux.

os.timerfd_create(clockid, l, *, flags=0)

Create and return a timer file descriptor (timerfd). The file descriptor returned by `timerfd_create()` supports: `read()`, `select()`, `poll()`. The file descriptor's `read()` method can be called with a buffer size of 8. If the timer has already expired one or more times, `read()` returns the number of expirations with the host's endianness, which may be converted to an integer by `int.from_bytes(x, byteorder=sys.byteorder)`. `select()` and `poll()` can be used to wait until timer expires and the file descriptor is readable. `clockid` must

os.timerfd_settime(fd, l, *, flags=flags, initial=0.0, interval=0.0)

Alter a timer file descriptor's internal timer. This function operates the same interval timer as `timerfd_settime_ns()`. `fd` must be a valid timer file descriptor. The timer's behaviour can be modified by specifying a `flags` value. Any of the following variables may be used, combined using bitwise OR (the | operator): `TFD_TIMER_ABSTIME`, `TFD_TIMER_CANCEL_ON_SET`. The timer is disabled by setting `initial` to zero (0). If `initial` is equal to or greater than zero, the timer is enabled. If `initial` is less than zero, it raises

os.timerfd_settime_ns(fd, l, *, flags=0, initial=0, interval=0)

Similar to `timerfd_settime()`, but use time as nanoseconds. This function operates the same interval timer as `timerfd_settime()`. Availability: Linux >= 2.6.27 with glibc >= 2.8. Added in version 3.13.

os.timerfd_gettime(fd, l)

Return a two-item tuple of floats (`next_expiration`, `interval`). `next_expiration` denotes the relative time until the timer next fires, regardless of if the `TFD_TIMER_ABSTIME` flag is set. `interval` denotes the timer's interval. If zero, the timer will only fire once, after `next_expiration` seconds have elapsed. See also `timerfd_gettime(2)`. Availability: Linux >= 2.6.27 with glibc >= 2.8. Added in version 3.13.

os.timerfd_gettime_ns(fd, l)

Similar to `timerfd_gettime()`, but return time as nanoseconds. Availability: Linux >= 2.6.27 with glibc >= 2.8. Added in version 3.13.

os.TFD_NONBLOCK

A flag for the `timerfd_create()` function, which sets the `O_NONBLOCK` status flag for the new timer file descriptor. If `TFD_NONBLOCK` is not set as a flag, `read()` blocks. Availability: Linux >= 2.6.27 with glibc >= 2.8. Added in version 3.13.

os.TFD_CLOEXEC

A flag for the `timerfd_create()` function, If `TFD_CLOEXEC` is set as a flag, set close-on-exec flag for new file descriptor. Availability: Linux >= 2.6.27 with glibc >= 2.8. Added in version 3.13.

os.TFD_TIMER_ABSTIME

A flag for the `timerfd_settime()` and `timerfd_settime_ns()` functions. If this flag is set, `initial` is interpreted as an absolute value on the timer's clock (in UTC seconds or nanoseconds since the Unix Epoch). Availability: Linux >= 2.6.27 with glibc >= 2.8. Added in version 3.13.

os.TFD_TIMER_CANCEL_ON_SET

A flag for the `timerfd_settime()` and `timerfd_settime_ns()` functions along with `TFD_TIMER_ABSTIME`. The timer is cancelled when the time of the underlying clock changes discontinuously. Availability: Linux >= 2.6.27 with glibc >= 2.8. Added in version 3.13.

Linux extended attributes

Added in version 3.3.

These functions are all available on Linux only.

os.getxattr(path,attribute,*,follow_symlinks=True)

Return the value of the extended filesystem attribute `attribute` for `path`. `attribute` can be bytes or str (directly or indirectly through the `PathLike` interface). If it is str, it is encoded with the filesystem encoding. This function can support specifying a file descriptor and not following symlinks. Raises an `audit`ing event `os.getxattr` with arguments `path,attribute`. Changed in version 3.6: Accepts a path-like object for `path` and `attribute`.

os.listxattr(path=None,*,follow_symlinks=True)

Return a list of the extended filesystem attributes on `path`. The attributes in the list are represented as strings decoded with the filesystem encoding. If `path` is `None`, `listxattr()` will examine the current directory. This function can support specifying a file descriptor and not following symlinks. Raises an `audit`ing event `os.listxattr` with argument `path`. Changed in version 3.6: Accepts a path-like object.

os.removexattr(path,attribute,*,follow_symlinks=True)

Removes the extended filesystem attribute `attribute` from `path`. `attribute` should be bytes or str (directly or indirectly through the `PathLike` interface). If it is a string, it is encoded with the filesystem encoding and error handler. This function can support specifying a file descriptor and not following symlinks. Raises an `audit`ing event `os.removexattr` with arguments `path,attribute`. Changed in version 3.6: Accepts a path-like object for `path` and `attribute`.

os.setxattr(path,attribute,value,flags=0,*,follow_symlinks=True)

Set the extended filesystem attribute `attribute` on `path` to `value`. `attribute` must be a bytes or str with no embedded NULs (directly or indirectly through the `PathLike` interface). If it is a str, it is encoded with the filesystem encoding and error handler. `flags` may be `XATTR_REPLACE` or `XATTR_CREATE`. If `XATTR_REPLACE` is given and the attribute does not exist, `ENODATA` will be raised. If `XATTR_CREATE` is given and the attribute already exists, the attribute will not be created and `EEXIST` will be raised. This function can s

os.XATTR_SIZE_MAX

The maximum size the value of an extended attribute can be. Currently, this is 64 KiB on Linux.

os.XATTR_CREATE

This is a possible value for the `flags` argument in `setxattr()`. It indicates the operation must create an attribute.

os.XATTR_REPLACE

This is a possible value for the `flags` argument in `setxattr()`. It indicates the operation must replace an existing attribute.

Process Management

These functions may be used to create and manage processes.

The various `exec*` functions take a list of arguments for the new program loaded into the process. In each case, the first of these arguments is passed to the new program as its own name rather than as an argument a user may have typed on a command line. For the C programmer, this is the `argv[0]` passed to a program's `main()`. For example, `os.execv('/bin/echo',['foo','bar'])` will only print `bar` on standard output; `foo` will seem to be ignored.

os.abort()

Generate a `SIGABRT` signal to the current process. On Unix, the default behavior is to produce a core dump; on Windows, the process immediately returns an exit code of 3. Be aware that calling this function will not call

the Python signal handler registered for SIGABRT with `signal.signal()`.

os.add_dll_directory(path)

Add a path to the DLL search path. This search path is used when resolving dependencies for imported extension modules (the module itself is resolved through `sys.path`), and also by ctypes. Remove the directory by calling `close()` on the returned object or using it in a `with` statement. See the Microsoft documentation for more information about how DLLs are loaded. Raises an `auditing.Event` on `add_dll_directory` with argument `path`. Availability: Windows. Added in version 3.8: Previous versions of CPython would resolve

os.execl(path, arg0, arg1, ...)

os.execlpe(path, arg0, arg1, ..., env)

os.execlp(file, arg0, arg1, ...)

os.execlpe(file, arg0, arg1, ..., env)

os.execv(path, args)

os.execve(path, args, env)

os.execvp(file, args)

os.execvpe(file, args, env)

These functions all execute a new program, replacing the current process; they do not return. On Unix, the new executable is loaded into the current process, and will have the same process id as the caller. Errors will be reported as `OSError` exceptions. The current process is replaced immediately. Open file objects and descriptors are not flushed, so if there may be data buffered on these open files, you should flush them using `flush()` or `os.fsync()` before calling an `exec*` function. The “l” and “v” vari

os._exit(n)

Exit the process with status `n`, without calling cleanup handlers, flushing stdio buffers, etc. Note: The standard way to exit is `sys.exit(n)`. `_exit()` should normally only be used in the child process after a `fork()`.

The following exit codes are defined and can be used with `_exit()`, although they are not required. These are typically used for system programs written in Python, such as a mail server’s external command delivery program.

■ Nota: Note Some of these may not be available on all Unix platforms, since there is some variation. These constants are defined where they are defined by the underlying platform.

os.EX_OK

Exit code that means no error occurred. May be taken from the defined value of `EXIT_SUCCESS` on some platforms. Generally has a value of zero. Availability: Unix, Windows.

os.EX_USAGE

Exit code that means the command was used incorrectly, such as when the wrong number of arguments are given. Availability: Unix, not WASI.

os.EX_DATAERR

Exit code that means the input data was incorrect. Availability: Unix, not WASI.

os.EX_NOINPUT

Exit code that means an input file did not exist or was not readable. Availability: Unix, not WASI.

os.EX_NOUSER

Exit code that means a specified user did not exist. Availability: Unix, not WASI.

os.EX_NOHOST

Exit code that means a specified host did not exist. Availability: Unix, not WASI.

os.EX_UNAVAILABLE

Exit code that means that a required service is unavailable. Availability: Unix, not WASI.

os.EX_SOFTWARE

Exit code that means an internal software error was detected. Availability: Unix, not WASI.

os.EX_OSERR

Exit code that means an operating system error was detected, such as the inability to fork or create a pipe. Availability: Unix, not WASI.

os.EX_OSFILE

Exit code that means some system file did not exist, could not be opened, or had some other kind of error. Availability: Unix, not WASI.

os.EX_CANTCREAT

Exit code that means a user specified output file could not be created. Availability: Unix, not WASI.

os.EX_IOERR

Exit code that means that an error occurred while doing I/O on some file. Availability: Unix, not WASI.

os.EX_TEMPFAIL

Exit code that means a temporary failure occurred. This indicates something that may not really be an error, such as a network connection that couldn't be made during a retryable operation. Availability: Unix, not WASI.

os.EX_PROTOCOL

Exit code that means that a protocol exchange was illegal, invalid, or not understood. Availability: Unix, not WASI.

os.EX_NOPERM

Exit code that means that there were insufficient permissions to perform the operation (but not intended for file system problems). Availability: Unix, not WASI.

os.EX_CONFIG

Exit code that means that some kind of configuration error occurred. Availability: Unix, not WASI.

os.EX_NOTFOUND

Exit code that means something like "an entry was not found". Availability: Unix, not WASI.

os.fork()

Fork a child process. Return 0 in the child and the child's process id in the parent. If an error occurs `OSError` is raised. Note that some platforms including FreeBSD <= 6.3 and Cygwin have known issues when using `fork()` from a thread. Raises an auditing event `os.fork` with no arguments. Warning If you use TLS sockets in an application calling `fork()`, see the warning in the `ssl` documentation. Warning On macOS the use of this function is unsafe when mixed with using higher-level system APIs, and that includes using

os.forkpty()

Fork a child process, using a new pseudo-terminal as the child's controlling terminal. Return a pair of `(pid, fd)`, where `pid` is 0 in the child, the new child's process id in the parent, and `fd` is the file descriptor of the master end of the pseudo-terminal. For a more portable approach, use the `pty` module. If an error occurs `OSError` is

raised. Raises an auditing event `os.forkpty` with no arguments. Warning On macOS the use of this function is unsafe when mixed with using higher-level system APIs, and that includ

os.kill(pid, sig, /)

Send signal `sig` to the process `pid`. Constants for the specific signals available on the host platform are defined in the `signal` module. Windows: The `signal.CTRL_C_EVENT` and `signal.CTRL_BREAK_EVENT` signals are special signals which can only be sent to console processes which share a common console window, e.g., some subprocesses. Any other value for `sig` will cause the process to be unconditionally killed by the `TerminateProcess` API, and the exit code will be set to `sig`. See also `signal.thread_kill()`. Raises an

os.killpg(pgid, sig, /)

Send the signal `sig` to the process group `pgid`. Raises an auditing event `os.killpg` with arguments `pgid, sig`. Availability: Unix, not WASI, not iOS.

os.nice(increment, /)

Add `increment` to the process's "niceness". Return the new niceness. Availability: Unix, not WASI.

os.pidfd_open(pid, flags=0)

os.PIDFD_NONBLOCK

Return a file descriptor referring to the process `pid` with `flags` set. This descriptor can be used to perform process management without races and signals. See the `pidfd_open(2)` man page for more details. Availability: Linux >= 5.3, Android >= build-time API level 31 Added in version 3.9. `os.PIDFD_NONBLOCK` This flag indicates that the file descriptor will be non-blocking. If the process referred to by the file descriptor has not yet terminated, then an attempt to wait on the file descriptor using `waitid(2)` will

This flag indicates that the file descriptor will be non-blocking. If the process referred to by the file descriptor has not yet terminated, then an attempt to wait on the file descriptor using `waitid(2)` will immediately return the error `EAGAIN` rather than blocking.

os.plock(op, /)

Lock program segments into memory. The value of `op` (defined in `<sys/lock.h>`) determines which segments are locked. Availability: Unix, not WASI, not iOS.

os.popen(cmd, mode='r', buffering=-1)

Open a pipe to or from command `cmd`. The return value is an open file object connected to the pipe, which can be read or written depending on whether `mode` is 'r' (default) or 'w'. The `buffering` argument have the same meaning as the corresponding argument to the built-in `open()` function. The returned file object reads or writes text strings rather than bytes. The `close` method returns `None` if the subprocess exited successfully, or the subprocess's return code if there was an error. On POSIX systems, if the return

os.posix_spawn(path, argv, env, *, file_actions=None, setpgroup=None, resetids=False, setsid=False, setsigmask=(), setsigdef=(), scheduler=None)

os.POSIX_SPAWN_OPEN

os.POSIX_SPAWN_CLOSE

os.POSIX_SPAWN_DUP2

os.POSIX_SPAWN_CLOSEFROM

Wraps the `posix_spawn()` C library API for use from Python. Most users should use `subprocess.run()` instead of `posix_spawn()`. The positional-only arguments `path`, `args`, and `env` are similar to `execve()`. `env` is allowed to be `None`, in which case current process' environment is used. The `path` parameter is the path to the executable file. The `path` should contain a directory. Use `posix_spawn()` to pass an executable file without

directory. The `file_actions` argument may be a sequence of tuples describing actions to take on spec

`(os.POSIX_SPAWN_OPEN, fd, path, flags, mode)` Performs `os.dup2(os.open(path, flags, mode), fd)`.

`(os.POSIX_SPAWN_CLOSE, fd)` Performs `os.close(fd)`.

`(os.POSIX_SPAWN_DUP2, fd, new_fd)` Performs `os.dup2(fd, new_fd)`.

`(os.POSIX_SPAWN_CLOSEFROM, fd)` Performs `os.closerange(fd, INF)`.

`os.posix_spawn(path, argv, env, *, file_actions=None, setpgroup=None, resetids=False, setsid=False, setsgmask=(), setsigdef=(), scheduler=None)`

Wraps the `posix_spawn()` C library API for use from Python. Similar to `posix_spawn()` except that the system searches for the executable file in the list of directories specified by the `PATH` environment variable (in the same way as `execvp(3)`). Raises an `auditing` event `os.posix_spawn` with arguments `path, argv, env`. Added in version 3.8. Availability: POSIX, not WASI, not Android, not iOS. See `posix_spawn()` documentation.

`os.register_at_fork(*, before=None, after_in_parent=None, after_in_child=None)`

Register callables to be executed when a new child process is forked using `os.fork()` or similar process cloning APIs. The parameters are optional and keyword-only. Each specifies a different call point. `before` is a function called before forking a child process. `after_in_parent` is a function called from the parent process after forking a child process. `after_in_child` is a function called from the child process. These calls are only made if control is expected to return to the Python interpreter. A typic

`os.spawnl(mode, path, ...)`

`os.spawnle(mode, path, ..., env)`

`os.spawnlp(mode, file, ...)`

`os.spawnlpe(mode, file, ..., env)`

`os.spawnv(mode, path, args)`

`os.spawnve(mode, path, args, env)`

`os.spawnvp(mode, file, args)`

`os.spawnvpe(mode, file, args, env)`

Execute the program `path` in a new process. (Note that the `subprocess` module provides more powerful facilities for spawning new processes and retrieving their results; using that module is preferable to using these functions. Check especially the `Replacing Older Functions with the subprocess Module` section.) If `mode` is `P_NOWAIT`, this function returns the process id of the new process; if `mode` is `P_WAIT`, returns the process's exit code if it exits normally, or `signal`, where `signal` is the signal that killed the p

`os.P_NOWAIT`

`os.P_NOWAITO`

Possible values for the `mode` parameter to the `spawn*` family of functions. If either of these values is given, the `spawn*` functions will return as soon as the new process has been created, with the process id as the return value. Availability: Unix, Windows.

`os.P_WAIT`

Possible value for the `mode` parameter to the `spawn*` family of functions. If this is given as `mode`, the `spawn*` functions will not return until the new process has run to completion and will return the exit code of the process the run is successful, or `signal` if a signal kills the process. Availability: Unix, Windows.

`os.P_DETACH`

os.P_OVERLAY

Possible values for the `mode` parameter to the `spawn*family` of functions. These are less portable than those listed above. `P_DETACH` is similar to `P_NOWAIT`, but the new process is detached from the console of the calling process. If `P_OVERLAY` is used, the current process will be replaced; the `spawn*function` will not return. Availability: Windows.

os.startfile(path[,operation][,arguments][,cwd][,show_cmd])

Start a file with its associated application. When `operation` is not specified, this acts like double-clicking the file in Windows Explorer, or giving the file name as an argument to the `start` command from the interactive command shell: the file is opened with whatever application (if any) its extension is associated. When another `operation` is given, it must be a "command verb" that specifies what should be done with the file. Common verbs documented by Microsoft are 'open', 'print' and 'edit' (to be used on

os.system(command)

Execute the command (a string) in a subshell. This is implemented by calling the Standard C function `system()`, and has the same limitations. Changes to `sys.stdin`, etc. are not reflected in the environment of the executed command. If `command` generates any output, it will be sent to the interpreter standard output stream. The C standard does not specify the meaning of the return value of the C function, so the return value of the Python function is system-dependent. On Unix, the return value is the `exit`

os.times()

Returns the current global process times. The return value is an object with five attributes: `user`- user time, `system`- system time, `children_user`- user time of all child processes, `children_system`- system time of all child processes, `elapsed`- elapsed real time since a fixed point in the past. For backwards compatibility, this object also behaves like a five-tuple containing `user`, `system`, `children_user`, `children_system`, and `elapsed` in that order. See the Unix manual page `times(2)` and `times(3)` manual page on Unix or the

os.wait()

Wait for completion of a child process, and return a tuple containing its pid and exit status indication: a 16-bit number, whose low byte is the signal number that killed the process, and whose high byte is the exit status (if the signal number is zero); the high bit of the low byte is set if a core file was produced. If there are no children that could be waited for, `ChildProcessError` is raised. `waitstatus_to_exitcode()` can be used to convert the exit status into an exit code. Availability: Unix, not

os.waitid(idtype,id,options,/)

Wait for the completion of a child process. `idtype` can be `P_PID`, `P_PGID`, `P_ALL`, or (on Linux) `P_PIDFD`. The interpretation of `id` depends on it; see their individual descriptions. `options` is an OR combination of flags. At least one of `WEXITED`, `WSTOPPED` or `WCONTINUED` is required; `WNOHANG` and `WNOWAIT` are additional optional flags. The return value is an object representing the data contained in the `siginfo_t` structure with the following attributes: `si_pid` (process ID), `si_uid` (real user ID of the child), `si_signo` (always `SIGCHLD`)

os.waitpid(pid,options,/)

The details of this function differ on Unix and Windows. On Unix: Wait for completion of a child process given by process id `pid`, and return a tuple containing its process id and exit status indication (encoded as `wait()`). The semantics of the call are affected by the value of the integer `options`, which should be 0 for normal operation. If `pid` is greater than 0, `waitpid()` requests status information for that specific process. If `pid` is 0, the request is for the status of any child in the process group of

os.wait3(options)

Similar to `waitpid()`, except no process id argument is given and a 3-element tuple containing the child's process id, exit status indication, and resource usage information is returned. Refer to `resource.getrusage()` for

details on resource usage information. The `options` argument is the same as that provided to `waitpid()` and `wait4()`. `waitstatus_to_exitcode()` can be used to convert the exit status into an exit code. Availability: Unix, not WASI, not Android, not iOS.

os.wait4(pid,options)

Similar to `waitpid()`, except a 3-element tuple, containing the child's process id, exit status indication, and resource usage information is returned. Refer to `resource.getrusage()` for details on resource usage information. The arguments to `wait4()` are the same as those provided to `waitpid()`. `waitstatus_to_exitcode()` can be used to convert the exit status into an exit code. Availability: Unix, not WASI, not Android, not iOS.

os.P_PID

os.P_PGID

os.P_ALL

os.P_PIDFD

These are the possible values for `idtype` in `waitid()`. They affect how `id` is interpreted: `P_PID`- wait for the child whose PID is `id`. `P_PGID`- wait for any child whose process group ID is `id`. `P_ALL`- wait for any child; `id` is ignored. `P_PIDFD`- wait for the child identified by the file descriptor `id` (a process file descriptor created with `pidfd_open()`). Availability: Unix, not WASI, not Android, not iOS. Note `P_PIDFD` is only available on Linux >= 5.4. Added in version 3.3. Added in version 3.9: The `P_PIDFD` constant.

os.WCONTINUED

This options flag for `waitpid()`, `wait3()`, `wait4()`, and `waitid()` causes child processes to be reported if they have been continued from a job control stop since they were last reported. Availability: Unix, not WASI, not Android, not iOS.

os.WEXITED

This options flag for `waitid()` causes child processes that have terminated to be reported. The other `wait*` functions always report children that have terminated, so this option is not available for them. Availability: Unix, not WASI, not Android, not iOS. Added in version 3.3.

os.WSTOPPED

This options flag for `waitid()` causes child processes that have been stopped by the delivery of a signal to be reported. This option is not available for the other `wait*` functions. Availability: Unix, not WASI, not Android, not iOS. Added in version 3.3.

os.WUNTRACED

This options flag for `waitpid()`, `wait3()`, and `wait4()` causes child processes to also be reported if they have been stopped but their current state has not been reported since they were stopped. This option is not available for `waitid()`. Availability: Unix, not WASI, not Android, not iOS.

os.WNOHANG

This options flag causes `waitpid()`, `wait3()`, `wait4()`, and `waitid()` to return right away if no child process status is available immediately. Availability: Unix, not WASI, not Android, not iOS.

os.WNOWAIT

This options flag causes `waitid()` to leave the child in a waitable state, so that a later `wait*` call can be used to retrieve the child status information again. This option is not available for the other `wait*` functions. Availability: Unix, not WASI, not Android, not iOS.

os.CLD_EXITED

os.CLD_KILLED

os.CLD_DUMPED

os.CLD_TRAPPED

os.CLD_STOPPED

os.CLD_CONTINUED

These are the possible values for `si_code` in the result returned by `waitid()`. Availability: Unix, not WASI, not Android, not iOS. Added in version 3.3. Changed in version 3.9: Added `CLD_KILLED` and `CLD_STOPPED` values.

os.waitstatus_to_exitcode(status)

Convert a wait status to an exit code. On Unix: If the process exited normally (if `WIFEXITED(status)` is true), return the process exit status (return `WEXITSTATUS(status)`): result greater than or equal to 0. If the process was terminated by a signal (if `WIFSIGNALED(status)` is true), return `-signal` where `signal` is the number of the signal that caused the process to terminate (return `WTERMSIG(status)`): result less than 0. Otherwise, raise a `ValueError`. On Windows, return `status` shifted right by 8 bits. On Unix, if `t`

The following functions take a process status code as returned by `system()`, `wait()`, or `waitpid()` as a parameter. They may be used to determine the disposition of a process.

os.WCOREDUMP(status, /)

Return `True` if a core dump was generated for the process, otherwise return `False`. This function should be employed only if `WIFSIGNALED()` is true. Availability: Unix, not WASI, not Android, not iOS.

os.WIFCONTINUED(status)

Return `True` if a stopped child has been resumed by delivery of `SIGCONT` (if the process has been continued from a job control stop), otherwise return `False`. See `WCONTINUED` option. Availability: Unix, not WASI, not Android, not iOS.

os.WIFSTOPPED(status)

Return `True` if the process was stopped by delivery of a signal, otherwise return `False`. `WIFSTOPPED()` only returns `True` if the `waitpid()` call was done using `WUNTRACED` option or when the process is being traced (see `ptrace(2)`). Availability: Unix, not WASI, not Android, not iOS.

os.WIFSIGNALED(status)

Return `True` if the process was terminated by a signal, otherwise return `False`. Availability: Unix, not WASI, not Android, not iOS.

os.WIFEXITED(status)

Return `True` if the process exited terminated normally, that is, by calling `exit()` or `_exit()`, or by returning from `main()`; otherwise return `False`. Availability: Unix, not WASI, not Android, not iOS.

os.WEXITSTATUS(status)

Return the process exit status. This function should be employed only if `WIFEXITED()` is true. Availability: Unix, not WASI, not Android, not iOS.

os.WSTOPSIG(status)

Return the signal which caused the process to stop. This function should be employed only if `WIFSTOPPED()` is true. Availability: Unix, not WASI, not Android, not iOS.

os.WTERMSIG(status)

Return the number of the signal that caused the process to terminate. This function should be employed only if `WIFSIGNALED()` is true. Availability: Unix, not WASI, not Android, not iOS.

Interface to the scheduler

These functions control how a process is allocated CPU time by the operating system. They are only available on some Unix platforms. For more detailed information, consult your Unix manpages.

Added in version 3.3.

The following scheduling policies are exposed if they are supported by the operating system.

os.SCHED_OTHER

The default scheduling policy.

os.SCHED_BATCH

Scheduling policy for CPU-intensive processes that tries to preserve interactivity on the rest of the computer.

os.SCHED_DEADLINE

Scheduling policy for tasks with deadline constraints. Added in version 3.14.

os.SCHED_IDLE

Scheduling policy for extremely low priority background tasks.

os.SCHED_NORMAL

Alias for SCHED_OTHER. Added in version 3.14.

os.SCHED_SPORADIC

Scheduling policy for sporadic server programs.

os.SCHED_FIFO

A First In First Out scheduling policy.

os.SCHED_RR

A round-robin scheduling policy.

os.SCHED_RESET_ON_FORK

This flag can be OR'ed with any other scheduling policy. When a process with this flag set forks, its child's scheduling policy and priority are reset to the default.

class os.sched_param(sched_priority)

sched_priority

This class represents tunable scheduling parameters used in `sched_setparam()`, `sched_setscheduler()`, and `sched_getparam()`. It is immutable. At the moment, there is only one possible parameter: `sched_priority`. The scheduling priority for a scheduling policy.

The scheduling priority for a scheduling policy.

os.sched_get_priority_min(policy)

Get the minimum priority value for `policy`. `policy` is one of the scheduling policy constants above.

os.sched_get_priority_max(policy)

Get the maximum priority value for `policy`. `policy` is one of the scheduling policy constants above.

os.sched_setscheduler(pid, policy, param, /)

Set the scheduling policy for the process with PID `pid`. A `pid` of 0 means the calling process. `policy` is one of the scheduling policy constants above. `param` is a `sched_param` instance.

os.sched_getscheduler(pid, /)

Return the scheduling policy for the process with PIDpid. Apidof 0 means the calling process. The result is one of the scheduling policy constants above.

os.sched_setparam(pid, param, /)

Set the scheduling parameters for the process with PIDpid. Apidof 0 means the calling process.paramis asched_paraminstance.

os.sched_getparam(pid, /)

Return the scheduling parameters as asched_paraminstance for the process with PIDpid. Apidof 0 means the calling process.

os.sched_rr_get_interval(pid, /)

Return the round-robin quantum in seconds for the process with PIDpid. Apidof 0 means the calling process.

os.sched_yield()

Voluntarily relinquish the CPU. Seesched_yield(2)for details.

os.sched_setaffinity(pid, mask, /)

Restrict the process with PIDpid(or the current process if zero) to a set of CPUs.maskis an iterable of integers representing the set of CPUs to which the process should be restricted.

os.sched_getaffinity(pid, /)

Return the set of CPUs the process with PIDpidis restricted to.Ifpidis zero, return the set of CPUs the calling thread of the current process is restricted to.See also theprocess_cpu_count()function.

Miscellaneous System Information

os.confstr(name, /)

Return string-valued system configuration values.namespecifies the configuration value to retrieve; it may be a string which is the name of a defined system value; these names are specified in a number of standards (POSIX, Unix 95, Unix 98, and others). Some platforms define additional names as well. The names known to the host operating system are given as the keys of theconfstr_namesdictionary. For configuration variables not included in that mapping, passing an integer fornameis also accept

os.confstr_names

Dictionary mapping names accepted byconfstr()to the integer values defined for those names by the host operating system. This can be used to determine the set of names known to the system.Availability: Unix.

os.cpu_count()

Return the number of logical CPUs in thesystem. ReturnsNoneif undetermined.Theprocess_cpu_count()function can be used to get the number of logical CPUs usable by the calling thread of thecurrent process.Added in version 3.4.Changed in version 3.13:If-Xcpu_countis given orPYTHON_CPU_COUNTis set,cpu_count()returns the override valuen.

os.getloadavg()

Return the number of processes in the system run queue averaged over the last 1, 5, and 15 minutes or raisesOSErrorif the load average was unobtainable.Availability: Unix.

os.process_cpu_count()

Get the number of logical CPUs usable by the calling thread of the current process. Returns `None` if undetermined. It can be less than `cpu_count()` depending on the CPU affinity. The `cpu_count()` function can be used to get the number of logical CPUs in the system. If `Xcpu_count` is given or `PYTHON_CPU_COUNT` is set, `process_cpu_count()` returns the override value `n`. See also the `sched_getaffinity()` function. Added in version 3.13.

os.sysconf(name, /)

Return integer-valued system configuration values. If the configuration value specified by `name` isn't defined, `-1` is returned. The comments regarding the `name` parameter for `confstr()` apply here as well; the dictionary that provides information on the known names is given by `sysconf_names`. Availability: Unix.

os.sysconf_names

Dictionary mapping names accepted by `sysconf()` to the integer values defined for those names by the host operating system. This can be used to determine the set of names known to the system. Availability: Unix. Changed in version 3.11: Add `SC_MINSIGSTKSZ` name.

The following data values are used to support path manipulation operations. These are defined for all platforms. Higher-level operations on pathnames are defined in the `os.path` module.

os.curdir

The constant string used by the operating system to refer to the current directory. This is `'.'` for Windows and POSIX. Also available via `os.path`.

os.pardir

The constant string used by the operating system to refer to the parent directory. This is `'..'` for Windows and POSIX. Also available via `os.path`.

os.sep

The character used by the operating system to separate pathname components. This is `'/'` for POSIX and `'\\'` for Windows. Note that knowing this is not sufficient to be able to parse or concatenate pathnames — use `os.path.split()` and `os.path.join()` — but it is occasionally useful. Also available via `os.path`.

os.altsep

An alternative character used by the operating system to separate pathname components, or `None` if only one separator character exists. This is set to `'/'` on Windows systems where `sep` is a backslash. Also available via `os.path`.

os.extsep

The character which separates the base filename from the extension; for example, the `'.'` in `os.py`. Also available via `os.path`.

os.pathsep

The character conventionally used by the operating system to separate search path components (as in `PATH`), such as `':'` for POSIX or `';'` for Windows. Also available via `os.path`.

os.defpath

The default search path used by `exec*()` and `spawn*()` if the environment doesn't have a `'PATH'` key. Also available via `os.path`.

os.linesep

The string used to separate (or, rather, terminate) lines on the current platform. This may be a single character, such as `'\\n'` for POSIX, or multiple characters, for example, `'\\r\\n'` for Windows. Do not use `os.linesep` as a

line terminator when writing files opened in text mode (the default); use a single `\n` instead, on all platforms.

os.devnull

The file path of the null device. For example: `/dev/null` for POSIX, `'nul'` for Windows. Also available via `os.path`.

os.RTLD_LAZY

os.RTLD_NOW

os.RTLD_GLOBAL

os.RTLD_LOCAL

os.RTLD_NODELETE

os.RTLD_NOLOAD

os.RTLD_DEEPBIND

Flags for use with the `setdlopenflags()` and `getdlopenflags()` functions. See the Unix manual `page dlopen(3)` for what the different flags mean. Added in version 3.3.

Random numbers

os.getrandom(size, flags=0)

Get up to `size` random bytes. The function can return less bytes than requested. These bytes can be used to seed user-space random number generators or for cryptographic purposes. `getrandom()` relies on entropy gathered from device drivers and other sources of environmental noise. Unnecessarily reading large quantities of data will have a negative impact on other users of the `/dev/random` and `/dev/urandom` devices. The `flags` argument is a bit mask that can contain zero or more of the following values ORed together.

os.urandom(size, /)

Return a bytes string of `size` random bytes suitable for cryptographic use. This function returns random bytes from an OS-specific randomness source. The returned data should be unpredictable enough for cryptographic applications, though its exact quality depends on the OS implementation. On Linux, if the `getrandom()` syscall is available, it is used in blocking mode: block until the system `urandom` entropy pool is initialized (128 bits of entropy are collected by the kernel). See the [PEP 524](#) for the rationale.

os.GRND_NONBLOCK

By default, when reading from `/dev/random`, `getrandom()` blocks if no random bytes are available, and when reading from `/dev/urandom`, it blocks if the entropy pool has not yet been initialized. If the `GRND_NONBLOCK` flag is set, then `getrandom()` does not block in these cases, but instead immediately raises `BlockingIOError`. Added in version 3.6.

os.GRND_RANDOM

If this bit is set, then random bytes are drawn from the `/dev/random` pool instead of the `/dev/urandom` pool. Added in version 3.6.

io— Core tools for working with streams

Source code: `Lib/io.py`

Overview

The `io` module provides Python's main facilities for dealing with various types of I/O. There are three main types of I/O: text I/O, binary I/O and raw I/O. These are generic categories, and various backing stores can be used for each of them. A concrete object belonging to any of these categories is called a file object. Other common terms are stream and file-like object.

Independent of its category, each concrete stream object will also have various capabilities: it can be read-only, write-only, or read-write. It can also allow arbitrary random access (seeking forwards or backwards to any location), or only sequential access (for example in the case of a socket or pipe).

All streams are careful about the type of data you give to them. For example giving a string object to the `write()` method of a binary stream will raise a `TypeError`. So will giving a bytes object to the `write()` method of a text stream.

Changed in version 3.3: Operations that used to raise `IOError` now raise `OSError`, since `IOError` is now an alias of `OSError`.

Text I/O

Text I/O expects and produces string objects. This means that whenever the backing store is natively made of bytes (such as in the case of a file), encoding and decoding of data is made transparently as well as optional translation of platform-specific newline characters.

The easiest way to create a text stream is with `open()`, optionally specifying an encoding:

```
f = open("myfile.txt", "r", encoding="utf-8")
```

In-memory text streams are also available as `StringIO` objects:

```
f = io.StringIO("some initial text data")
```

■ **Nota:** Note When working with a non-blocking stream, be aware that read operations on text I/O objects might raise a `BlockingIOError` if the stream cannot perform the operation immediately.

The text stream API is described in detail in the documentation of `TextIOBase`.

Binary I/O

Binary I/O (also called buffered I/O) expects bytes-like objects and produces bytes objects. No encoding, decoding, or newline translation is performed. This category of streams can be used for all kinds of non-text data, and also when manual control over the handling of text data is desired.

The easiest way to create a binary stream is with `open()` with `'b'` in the mode string:

```
f = open("myfile.jpg", "rb")
```

In-memory binary streams are also available as `BytesIO` objects:

```
f = io.BytesIO(b"some initial binary data: \x00\x01")
```

The binary stream API is described in detail in the docs of `BufferedIOBase`.

Other library modules may provide additional ways to create text or binary streams.

See `socket.socket.makefile()` for example.

Raw I/O

Raw I/O (also called `unbuffered I/O`) is generally used as a low-level building-block for binary and text streams; it is rarely useful to directly manipulate a raw stream from user code. Nevertheless, you can create a raw stream by opening a file in binary mode with buffering disabled:

```
f = open("myfile.jpg", "rb", buffering=0)
```

The raw stream API is described in detail in the docs of `RawIOBase`.

Text Encoding

The default encoding of `TextIOWrapper` and `open()` is locale-specific (`locale.getencoding()`).

However, many developers forget to specify the encoding when opening text files encoded in UTF-8 (e.g. JSON, TOML, Markdown, etc...) since most Unix platforms use UTF-8 locale by default. This causes bugs because the locale encoding is not UTF-8 for most Windows users. For example:

```
# May not work on Windows when non-ASCII characters in the file.
with open("README.md") as f:
    long_description = f.read()
```

Accordingly, it is highly recommended that you specify the encoding explicitly when opening text files. If you want to use UTF-8, pass `encoding="utf-8"`. To use the current locale encoding, `encoding="locale"` is supported since Python 3.10.

See also

Python UTF-8 Mode

PEP 686

Python UTF-8 Mode can be used to change the default encoding to UTF-8 from locale-specific encoding.

Python 3.15 will make Python UTF-8 Mode default.

Opt-in Encoding Warning

Added in version 3.10: See PEP 597 for more details.

To find where the default locale encoding is used, you can enable the `-Xwarn_default_encoding` command line option or set the `PYTHONWARNDEFAULTENCODING` environment variable, which will emit an `EncodingWarning` when the default encoding is used.

If you are providing an API that uses `open()` or `TextIOWrapper` and passes `encoding=None` as a parameter, you can use `text_encoding()` so that callers of the API will emit an `EncodingWarning` if they don't pass an encoding. However, please consider using UTF-8 by default (i.e. `encoding="utf-8"`) for new APIs.

High-level Module Interface

`io.DEFAULT_BUFFER_SIZE`

An int containing the default buffer size used by the module's buffered I/O classes. `open()` uses the file's `blksize` (as obtained by `os.stat()`) if possible.

`io.open(file, mode='r', buffering=-1, encoding=None, errors=None, newline=None, closefd=True, opener=None)`

This is an alias for the builtin `open()` function. This function raises an `auditing.EventOpen` with arguments `path`, `mode` and `flags`. The `mode` and `flags` arguments may have been modified or inferred from the

original call.

io.open_code(path)

Opens the provided file with mode 'rb'. This function should be used when the intent is to treat the contents as executable code. `path` should be a string and an absolute path. The behavior of this function may be overridden by an earlier call to `PyFile_SetOpenCodeHook()`. However, assuming that `path` is a string and an absolute path, `open_code(path)` should always behave the same as `open(path, 'rb')`. Overriding the behavior is intended for additional validation or preprocessing of the file. Added in version 3.8.

io.text_encoding(encoding, stacklevel=2, /)

This is a helper function for callables that use `open()` or `TextIOWrapper` and have an `encoding=None` parameter. This function returns `sencoding` if it is not `None`. Otherwise, it returns "locale" or "utf-8" depending on UTF-8 Mode. This function emits an `EncodingWarning` if `sys.flags.warn_default_encoding` is true and `sencoding` is `None`. `stacklevel` specifies where the warning is emitted. For example: `def read_text(path, encoding=None): encoding = io.text_encoding(encoding) # stacklevel=2 with open(path, encoding) as f: return f.read()` In this

exception io.BlockingIOError

This is a compatibility alias for the builtin `BlockingIOError` exception.

exception io.UnsupportedOperation

An exception inheriting `OSError` and `ValueError` that is raised when an unsupported operation is called on a stream.

See also

sys

contains the standard IO streams: `sys.stdin`, `sys.stdout`, and `sys.stderr`.

Class hierarchy

The implementation of I/O streams is organized as a hierarchy of classes. First abstract base classes (ABCs), which are used to specify the various categories of streams, then concrete classes providing the standard stream implementations.

■ **Nota:** Note The abstract base classes also provide default implementations of some methods in order to help implementation of concrete stream classes. For example, `BufferedIOBase` provides unoptimized implementations of `readinto()` and `readline()`.

At the top of the I/O hierarchy is the abstract base class `IOBase`. It defines the basic interface to a stream. Note, however, that there is no separation between reading and writing to streams; implementations are allowed to raise `UnsupportedOperation` if they do not support a given operation.

The `RawIOBaseABC` extends `IOBase`. It deals with the reading and writing of bytes to a stream. `FileIO` subclasses `RawIOBase` to provide an interface to files in the machine's file system.

The `BufferedIOBaseABC` extends `IOBase`. It deals with buffering on a raw binary stream (`RawIOBase`). Its subclasses, `BufferedWriter`, `BufferedReader`, and `BufferedRWPair` buffer raw binary streams that are writable, readable, and both readable and writable, respectively. `BufferedRandom` provides a buffered interface to seekable streams. Another `BufferedIOBase` subclass, `BytesIO`, is a stream of in-memory bytes.

The `TextIOBaseABC` extends `IOBase`. It deals with streams whose bytes represent text, and handles encoding and decoding to and from strings. `TextIOWrapper`, which extends `TextIOBase`, is a buffered text interface to a buffered raw stream (`BufferedIOBase`). Finally, `StringIO` is an in-memory stream for text.

Argument names are not part of the specification, and only the arguments of `open()` are intended to be used as keyword arguments.

The following table summarizes the ABCs provided by the `io` module:

ABC	Inherits	Stub Methods	Mixin Methods and Properties
<code>IOBase</code>		<code>fileno()</code>	<code>seek()</code> , <code>truncate()</code> , <code>close()</code> , <code>closed</code> , <code>__enter__()</code> , <code>__exit__()</code> , <code>flush()</code> , <code>isatty()</code> , <code>__iter__()</code> , <code>__next__()</code> , <code>readable()</code> , <code>readline()</code> , <code>readlines()</code> , <code>seekable()</code> , <code>tell()</code> , <code>writable()</code> , and <code>writelines()</code>
<code>RawIOBase</code>	<code>IOBase</code>	<code>readinto()</code>	and <code>write()</code>
<code>BufferedIOBase</code>	<code>IOBase</code>	<code>read()</code> , <code>read1()</code> , and <code>write()</code>	<code>readall()</code> , <code>detach()</code> , <code>readinto1()</code>
<code>TextIOBase</code>	<code>IOBase</code>	<code>detach()</code> , <code>read()</code> , <code>readline()</code> , and <code>write()</code>	Inherited from <code>IOBase</code>
		<code>encoding</code> , <code>errors</code> , and <code>newlines</code>	

I/O Base Classes

`class io.IOBase`

`close()`

`closed`

`fileno()`

`flush()`

`isatty()`

`readable()`

`readline(size=-1, /)`

`readlines(hint=-1, /)`

`seek(offset, whence=os.SEEK_SET, /)`

`seekable()`

`tell()`

`truncate(size=None, /)`

`writable()`

`writelines(lines, /)`

`__del__()`

The abstract base class for all I/O classes. This class provides empty abstract implementations for many methods that derived classes can override selectively; the default implementations represent a file that cannot be read, written or seeked. Even though `IOBase` does not declare `read()` or `write()` because their signatures will vary, implementations and clients should consider those methods part of the interface. Also, implementations may raise `ValueError` (or `UnsupportedOperation`) when operations they do

Flush and close this stream. This method has no effect if the file is already closed. Once the file is closed, any operation on the file (e.g. reading or writing) will raise `ValueError`. As a convenience, it is allowed to call this method more than once; only the first call, however, will have an effect.

True if the stream is closed.

Return the underlying file descriptor (an integer) of the stream if it exists. An `IOError` is raised if the IO object does not use a file descriptor.

Flush the write buffers of the stream if applicable. This does nothing for read-only and non-blocking streams.

Return True if the stream is interactive (i.e., connected to a terminal/tty device).

Return True if the stream can be read from. If False, read() will raise OSError.

Read and return one line from the stream. If size is specified, at most size bytes will be read. The line terminator is always b'\n' for binary files; for text files, the newline argument to open() can be used to select the line terminator(s) recognized.

Read and return a list of lines from the stream. hint can be specified to control the number of lines read: no more lines will be read if the total size (in bytes/characters) of all lines so far exceeds hint. hint values of 0 or less, as well as None, are treated as no hint. Note that it's already possible to iterate on file objects using for line in file: ... without calling file.readlines().

Change the stream position to the given byte offset, interpreted relative to the position indicated by whence, and return the new absolute position. Values for whence are: os.SEEK_SET or 0 – start of the stream (the default); offset should be zero or positive os.SEEK_CUR or 1 – current stream position; offset may be negative os.SEEK_END or 2 – end of the stream; offset is usually negative. Added in version 3.1: The SEEK_* constants. Added in version 3.3: Some operating systems could support additional values, like os.SEEK_HO

Return True if the stream supports random access. If False, seek(), tell() and truncate() will raise OSError.

Return the current stream position.

Resize the stream to the given size in bytes (or the current position if size is not specified). The current stream position isn't changed. This resizing can extend or reduce the current file size. In case of extension, the contents of the new file area depend on the platform (on most systems, additional bytes are zero-filled). The new file size is returned. Changed in version 3.5: Windows will now zero-fill files when extending.

Return True if the stream supports writing. If False, write() and truncate() will raise OSError.

Write a list of lines to the stream. Line separators are not added, so it is usual for each of the lines provided to have a line separator at the end.

Prepare for object destruction. IOBase provides a default implementation of this method that calls the instance's close() method.

class io.RawIOBase

read(size=-1, /)

readall()

readinto(b, /)

write(b, /)

Base class for raw binary streams. It inherits from IOBase. Raw binary streams typically provide low-level access to an underlying OS device or API, and do not try to encapsulate it in high-level primitives (this functionality is done at a higher-level in buffered binary streams and text streams, described later in this page). RawIOBase provides these methods in addition to those from IOBase: read(size=-1, /) Read up to size bytes from the object and return them. As a convenience, if size is unspecified or

Read up to size bytes from the object and return them. As a convenience, if size is unspecified or -1, all bytes until EOF are returned. Otherwise, only one system call is ever made. Fewer than size bytes may be returned if the operating system call returns fewer than size bytes. If 0 bytes are returned, and size was not 0, this indicates end of file. If the object is in non-blocking mode and no bytes are available, None is returned. The default implementation defers to readall() and readinto().

Read and return all the bytes from the stream until EOF, using multiple calls to the stream if necessary.

Read bytes into a pre-allocated, writable bytes-like object b, and return the number of bytes read. For example, b might be a bytearray. If the object is in non-blocking mode and no bytes are available, None is returned.

Write the given bytes-like object, `b`, to the underlying raw stream, and return the number of bytes written. This can be less than the length of `b` in bytes, depending on specifics of the underlying raw stream, and especially if it is in non-blocking mode. `None` is returned if the raw stream is set not to block and no single byte could be readily written to it. The caller may release or mutate `b` after this method returns, so the implementation should only access `b` during the method call.

class io.BufferedIOBase

raw

detach()

read(size=-1, /)

read1(size=-1, /)

readinto(b, /)

readinto1(b, /)

write(b, /)

Base class for binary streams that support some kind of buffering. It inherits from `IOBase`. The main difference with `RawIOBase` is that methods `read()`, `readinto()` and `write()` will try (respectively) to read as much input as requested or to emit all provided data. In addition, if the underlying raw stream is in non-blocking mode, when the system returns would block `write()` will raise `BlockingIOError` with `BlockingIOError.characters_written` and `read()` will return data read so far or `None` if no data is available. Besides

The underlying raw stream (a `RawIOBase` instance) that `BufferedIOBase` deals with. This is not part of the `BufferedIOBase` API and may not exist on some implementations.

Separate the underlying raw stream from the buffer and return it. After the raw stream has been detached, the buffer is in an unusable state. Some buffers, like `BytesIO`, do not have the concept of a single raw stream to return from this method. They raise `UnsupportedOperation`. Added in version 3.1.

Read and return up to `size` bytes. If the argument is omitted, `None`, or negative read as much as possible. Fewer bytes may be returned than requested. An empty bytes object is returned if the stream is already at EOF. More than one read may be made and calls may be retried if specific errors are encountered, see `os.read()` and PEP 475 for more details. Less than `size` bytes being returned does not imply that EOF is imminent. When reading as much as possible the default implementation will use `raw.readall()` if available.

Read and return up to `size` bytes, calling `readinto()` which may retry if `EINTR` is encountered per PEP 475. If `size` is 1 or not provided, the implementation will choose an arbitrary value for `size`. Note When the underlying raw stream is non-blocking, implementations may either raise `BlockingIOError` or return `None` if no data is available. io implementations return `None`.

Read bytes into a pre-allocated, writable bytes-like object `band` and return the number of bytes read. For example, `b` might be a `bytearray`. Like `read()`, multiple reads may be issued to the underlying raw stream, unless the latter is interactive. A `BlockingIOError` is raised if the underlying raw stream is in non blocking-mode, and has no data available at the moment.

Read bytes into a pre-allocated, writable bytes-like object `b`, using at most one call to the underlying raw stream's `read()` (or `readinto()`) method. Return the number of bytes read. A `BlockingIOError` is raised if the underlying raw stream is in non blocking-mode, and has no data available at the moment. Added in version 3.5.

Write the given bytes-like object, `b`, and return the number of bytes written (always equal to the length of `b` in bytes, since if the write fails an `OSError` will be raised). Depending on the actual implementation, these bytes may be readily written to the underlying stream, or held in a buffer for performance and latency reasons. When in non-blocking mode, a `BlockingIOError` is raised if the data needed to be written to the raw stream but it couldn't accept all the data without blocking. The caller may rel

Raw File I/O

class io.FileIO(name, mode='r', closefd=True, opener=None)

mode

name

A raw binary stream representing an OS-level file containing bytes data. It inherits from `RawIOBase`. The name can be one of two things: a character string or bytes object representing the path to the file which will be opened. In this case `closefd` must be `True` (the default) otherwise an error will be raised. an integer representing the number of an existing OS-level file descriptor to which the resulting `FileIO` object will give access. When the `FileIO` object is closed this `fd` will be closed as well, unless

The mode as given in the constructor.

The file name. This is the file descriptor of the file when no name is given in the constructor.

Buffered Streams

Buffered I/O streams provide a higher-level interface to an I/O device than raw I/O does.

class io.BytesIO(initial_bytes=b'')

getbuffer()

getvalue()

read1(size=-1, /)

readinto1(b, /)

A binary stream using an in-memory bytes buffer. It inherits from `BufferedIOBase`. The buffer is discarded when the `close()` method is called. The optional argument `initial_bytes` is a bytes-like object that contains initial data. `BytesIO` provides or overrides these methods in addition to those from `BufferedIOBase` and `IOBase`: `getbuffer()` Return a readable and writable view over the contents of the buffer without copying them. Also, mutating the view will transparently update the contents of the buffer: `>>> b = io.`

Return a readable and writable view over the contents of the buffer without copying them. Also, mutating the view will transparently update the contents of the buffer: `>>> b = io.BytesIO(b"abcdef")>>> view = b.getbuffer()>>> view[2:4] = b"56">>> b.getvalue() b'ab56ef'` Note: As long as the view exists, the `BytesIO` object cannot be resized or closed. Added in version 3.2.

Return bytes containing the entire contents of the buffer.

In `BytesIO`, this is the same as `read()`. Changed in version 3.7: The `size` argument is now optional.

In `BytesIO`, this is the same as `readinto()`. Added in version 3.5.

class io.BufferedReader(raw, buffer_size=DEFAULT_BUFFER_SIZE)

peek(size=0, /)

read(size=-1, /)

read1(size=-1, /)

A buffered binary stream providing higher-level access to a readable, non seekable `RawIOBase` raw binary stream. It inherits from `BufferedIOBase`. When reading data from this object, a larger amount of data may be requested from the underlying raw stream, and kept in an internal buffer. The buffered data can then be returned directly on subsequent reads. The constructor creates a `BufferedReader` for the given readable raw stream and `buffer_size`. If `buffer_size` is omitted, `DEFAULT_BUFFER_SIZE` is used. `BufferedReader`

Return bytes from the stream without advancing the position. The number of bytes returned may be less or more than requested. If the underlying raw stream is non-blocking and the operation would block, returns empty bytes.

InBufferedReaderthis is the same asio.BufferedIOBase.read()

InBufferedReaderthis is the same asio.BufferedIOBase.read1()Changed in version 3.7:The size argument is now optional.

classio.BufferedWriter(raw,buffer_size=DEFAULT_BUFFER_SIZE)

flush()

write(b,/)

A buffered binary stream providing higher-level access to a writeable, non seekableRawIOBase raw binary stream. It inherits fromBufferedIOBase.When writing to this object, data is normally placed into an internal buffer. The buffer will be written out to the underlyingRawIOBaseobject under various conditions, including:when the buffer gets too small for all pending data;whenflush()is called;when aseek()is requested (forBufferedRandomobjects);when theBufferedWriterobject is closed or destroyed.T

Force bytes held in the buffer into the raw stream. ABlockingIOErrorshould be raised if the raw stream blocks.

Write thebytes-like object,b, and return the number of bytes written. When in non-blocking mode, aBlockingIOErrorwithBlockingIOError.characters_writtenset is raised if the buffer needs to be written out but the raw stream blocks.

classio.BufferedRandom(raw,buffer_size=DEFAULT_BUFFER_SIZE)

A buffered binary stream providing higher-level access to a seekableRawIOBase raw binary stream. It inherits fromBufferedReaderandBufferedWriter.The constructor creates a reader and writer for a seekable raw stream, given in the first argument. If thebuffer_sizeis omitted it defaults toDEFAULT_BUFFER_SIZE.BufferedRandomis capable of anythingBufferedReaderorBufferedWritercan do. In addition,seek()andtell()are guaranteed to be implemented.

classio.BufferedRWPair(reader,writer,buffer_size=DEFAULT_BUFFER_SIZE,/)

A buffered binary stream providing higher-level access to two non seekableRawIOBase raw binary streams—one readable, the other writeable. It inherits fromBufferedIOBase.readerandwriterareRawIOBaseobjects that are readable and writeable respectively. If thebuffer_sizeis omitted it defaults toDEFAULT_BUFFER_SIZE.BufferedRWPairimplements all ofBufferedIOBase's methods except fordetach(), which raisesUnsupportedOperation.WarningBufferedRWPairdoes not attempt to synchronize accesses to its underlying

Text I/O

classio.TextIOBase

encoding

errors

newlines

buffer

detach()

read(size=-1,/)

readline(size=-1,/)

seek(offset,whence=SEEK_SET,/)

tell()

write(s,/)

Base class for text streams. This class provides a character and line based interface to stream I/O. It inherits from `IOBase`. `TextIOBase` provides or overrides these data attributes and methods in addition to those from `IOBase`: `encoding` The name of the encoding used to decode the stream's bytes into strings, and to encode strings into bytes. `errors` The error setting of the decoder or encoder. `newlines` A string, a tuple of strings, or `None`, indicating the newlines translated so far. Depending on the implementation

The name of the encoding used to decode the stream's bytes into strings, and to encode strings into bytes.

The error setting of the decoder or encoder.

A string, a tuple of strings, or `None`, indicating the newlines translated so far. Depending on the implementation and the initial constructor flags, this may not be available.

The underlying binary buffer (a `BufferedIOBase` or `RawIOBase` instance) that `TextIOBase` deals with. This is not part of the `TextIOBase` API and may not exist in some implementations.

Separate the underlying binary buffer from the `TextIOBase` and return it. After the underlying buffer has been detached, the `TextIOBase` is in an unusable state. Some `TextIOBase` implementations, like `StringIO`, may not have the concept of an underlying buffer and calling this method will raise `UnsupportedOperation`. Added in version 3.1.

Read and return at most `size` characters from the stream as a single `str`. If `size` is negative or `None`, reads until EOF.

Read until newline or EOF and return a single `str`. If the stream is already at EOF, an empty string is returned. If `size` is specified, at most `size` characters will be read.

Change the stream position to the given `offset`. Behaviour depends on the `whence` parameter. The default value for `whence` is `SEEK_SET`. `SEEK_SET` or 0: seek from the start of the stream (the default); `offset` must either be a number returned by `TextIOBase.tell()`, or zero. Any other `offset` value produces undefined behaviour. `SEEK_CUR` or 1: "seek" to the current position; `offset` must be zero, which is a no-operation (all other values are unsupported). `SEEK_END` or 2: seek to the end of the stream; `offset` must be zero (all o

Return the current stream position as an opaque number. The number does not usually represent a number of bytes in the underlying binary storage.

Write the `string` to the stream and return the number of characters written.

`class io.TextIOWrapper(buffer,encoding=None,errors=None,newline=None,line_buffering=False,write_through=False)`

`line_buffering`

`write_through`

`reconfigure(*,encoding=None,errors=None,newline=None,line_buffering=None,write_through=None)`

`seek(cookie,whence=os.SEEK_SET,/)`

`tell()`

A buffered text stream providing higher-level access to a `BufferedIOBase` buffered binary stream. It inherits from `TextIOBase`. `encoding` gives the name of the encoding that the stream will be decoded or encoded with. In UTF-8 Mode, this defaults to UTF-8. Otherwise, it defaults to `locale.getencoding()`. `encoding="locale"` can be used to specify the current locale's encoding explicitly. See `Text Encoding` for more information. `errors` is an optional string that specifies how encoding and decoding errors are to be

Whether line buffering is enabled.

Whether writes are passed immediately to the underlying binary buffer. Added in version 3.7.

Reconfigure this text stream using new settings

`forencoding, errors, newline, line_buffering` and `write_through`. Parameters not specified keep current settings, except `errors='strict'` is used when `encoding` is specified but `errors` is not specified. It is not possible to change the encoding or newline if some data has already been read from the stream. On the other hand, changing encoding after write is possible. This method does an implicit stream flush before setting the new parameters. Added in version 3.7. Cha

Set the stream position. Return the new stream position as an int. Four operations are supported, given by the following argument combinations: `seek(0, SEEK_SET)`: Rewind to the start of the stream. `seek(cookie, SEEK_SET)`: Restore a previous position; `cookie` must be a number returned by `tell()`. `seek(0, SEEK_END)`: Fast-forward to the end of the stream. `seek(0, SEEK_CUR)`: Leave the current stream position unchanged. Any other argument combinations are invalid, and may raise exceptions. See also `os.SEEK_SET`, `os.SEEK_`

Return the stream position as an opaque number. The return value of `tell()` can be given as input to `seek()`, to restore a previous stream position.

class io.StringIO(initial_value="", newline='\n')

getvalue()

A text stream using an in-memory text buffer. It inherits from `TextIOBase`. The text buffer is discarded when the `close()` method is called. The initial value of the buffer can be set by providing `initial_value`. If newline translation is enabled, newlines will be encoded as if by `write()`. The stream is positioned at the start of the buffer which emulates opening an existing file in `aw+` mode, making it ready for an immediate write from the beginning or for a write that would overwrite the initial value.

Return a `str` containing the entire contents of the buffer. Newlines are decoded as if by `read()`, although the stream position is not changed.

class io.IncrementalNewlineDecoder

A helper codec that decodes newlines for `universal newlines` mode. It inherits from `codecs.IncrementalDecoder`.

Static Typing

The following protocols can be used for annotating function and method arguments for simple stream reading or writing operations. They are decorated with `@typing.runtime_checkable`.

class io.Reader[T]

read()

read(size, /)

Generic protocol for reading from a file or other input stream. `T` will usually be `str` or `bytes`, but can be any type that is read from the stream. Added in version 3.14. `read()` `read(size, /)` Read data from the input stream and return it. If `size` is specified, it should be an integer, and at most `size` items (bytes/characters) will be read. For example: `def read_it(reader: Reader[str]): data = reader.read(11) assert isinstance(data, str)`

Read data from the input stream and return it. If `size` is specified, it should be an integer, and at most `size` items (bytes/characters) will be read.

class io.Writer[T]

write(data, /)

Generic protocol for writing to a file or other output stream. It will usually best for bytes, but can be any type that can be written to the stream. Added in version 3.14. `write(data, /)` Writes data to the output stream and return the number of items (bytes/characters) written. For example: `def write_binary(writer: Writer[bytes]): writer.write(b"Hello world!\n")`

Writes data to the output stream and return the number of items (bytes/characters) written.

See [ABCs and Protocols for working with I/O](#) for other I/O related protocols and classes that can be used for static type checking.

Performance

This section discusses the performance of the provided concrete I/O implementations.

Binary I/O

By reading and writing only large chunks of data even when the user asks for a single byte, buffered I/O hides any inefficiency in calling and executing the operating system's unbuffered I/O routines. The gain depends on the OS and the kind of I/O which is performed. For example, on some modern OSes such as Linux, unbuffered disk I/O can be as fast as buffered I/O. The bottom line, however, is that buffered I/O offers predictable performance regardless of the platform and the backing device. Therefore, it is almost always preferable to use buffered I/O rather than unbuffered I/O for binary data.

Text I/O

Text I/O over a binary storage (such as a file) is significantly slower than binary I/O over the same storage, because it requires conversions between unicode and binary data using a character codec. This can become noticeable handling huge amounts of text data like large log files. Also, `tell()` and `seek()` are both quite slow due to the reconstruction algorithm used.

`StringIO`, however, is a native in-memory unicode container and will exhibit similar speed to `BytesIO`.

Multi-threading

`FileIO` objects are thread-safe to the extent that the operating system calls (such as `read(2)` under Unix) they wrap are thread-safe too.

Binary buffered objects (instances of `BufferedReader`, `BufferedWriter`, `BufferedReader` and `BufferedRWPair`) protect their internal structures using a lock; it is therefore safe to call them from multiple threads at once.

`TextIOWrapper` objects are not thread-safe.

Reentrancy

Binary buffered objects (instances of `BufferedReader`, `BufferedWriter`, `BufferedReader` and `BufferedRWPair`) are not reentrant. While reentrant calls will not happen in normal situations, they can arise from doing I/O in a signal handler. If a thread tries to re-enter a buffered object which it is already accessing, a `RuntimeError` is raised. Note this doesn't prohibit a different thread from entering the buffered object.

The above implicitly extends to text files, since the `open()` function will wrap a buffered object inside a `TextIOWrapper`. This includes standard streams and therefore affects the built-in `print()` function as well.

time— Time access and conversions

This module provides various time-related functions. For related functionality, see also `thedatetimeandcalendar` modules.

Although this module is always available, not all functions are available on all platforms. Most of the functions defined in this module call platform C library functions with the same name. It may sometimes be helpful to consult the platform documentation, because the semantics of these functions varies among platforms.

An explanation of some terminology and conventions is in order.

- The epoch is the point where the time starts, the return value of `time.gmtime(0)`. It is January 1, 1970, 00:00:00 (UTC) on all platforms.
- The term seconds since the epoch refers to the total number of elapsed seconds since the epoch, typically excluding leap seconds. Leap seconds are excluded from this total on all POSIX-compliant platforms.
- The functions in this module may not handle dates and times before the epoch or far in the future. The cut-off point in the future is determined by the C library; for 32-bit systems, it is typically in 2038.
- Function `strptime()` can parse 2-digit years when given `%y` format code. When 2-digit years are parsed, they are converted according to the POSIX and ISO C standards: values 69–99 are mapped to 1969–1999, and values 0–68 are mapped to 2000–2068.
- UTC is Coordinated Universal Time and superseded Greenwich Mean Time or GMT as the basis of international timekeeping. The acronym UTC is not a mistake but conforms to an earlier, language-agnostic naming scheme for time standards such as UT0, UT1, and UT2.
- DST is Daylight Saving Time, an adjustment of the timezone by (usually) one hour during part of the year. DST rules are magic (determined by local law) and can change from year to year. The C library has a table containing the local rules (often it is read from a system file for flexibility) and is the only source of True Wisdom in this respect.
- The precision of the various real-time functions may be less than suggested by the units in which their value or argument is expressed. E.g. on most Unix systems, the clock “ticks” only 50 or 100 times a second.
- On the other hand, the precision of `time()` and `sleep()` is better than their Unix equivalents: times are expressed as floating-point numbers, `time()` returns the most accurate time available (using `Unix gettimeofday()` where available), and `sleep()` will accept a time with a nonzero fraction (`Unix select()` is used to implement this, where available).
- The time value as returned by `gmtime()`, `localtime()`, and `strptime()`, and accepted by `asctime()`, `mktime()` and `strftime()`, is a sequence of 9 integers. The return values of `gmtime()`, `localtime()`, and `strptime()` also offer attribute names for individual fields. See `struct_time` for a description of these objects. Changed in version 3.3: The `struct_time` type was extended to provide the `tm_gmtoff` and `tm_zone` attributes when
- Use the following functions to convert between time representations: `FromToUse` seconds since the epoch `struct_time` in UTC `gmtime()` seconds since the epoch `struct_time` in local time `localtime()` `struct_time` in UTC seconds since the epoch `calendar.timegm()` `struct_time` in local time seconds since the epoch `mktime()`

Functions

`time.asctime([t])`

Convert a tuple or `struct_time` representing a time as returned by `gmtime()` or `localtime()` to a string of the following form: 'SunJun2023:21:051993'. The day field is two characters long and is space padded if the day is a single digit, e.g.: 'WedJun904:26:401993'. If it is not provided, the current time as returned by `localtime()` is used. Locale information is not used by `asctime()`. Note Unlike the C function of the same name, `asctime()` does not add a trailing newline.

time.thread_getcpuclockid(thread_id)

Return the `clk_id` of the thread-specific CPU-time clock for the specified `thread_id`. Use `threading.get_ident()` or the `ident` attribute of `threading.Thread` objects to get a suitable value for `thread_id`. Warning Passing an invalid or expired `thread_id` may result in undefined behavior, such as segmentation fault. Availability: Unix See the man page for `pthread_getcpuclockid(3)` for further information. Added in version 3.7.

time.clock_getres(clk_id)

Return the resolution (precision) of the specified clock `clk_id`. Refer to `Clock ID Constants` for a list of accepted values for `clk_id`. Availability: Unix. Added in version 3.3.

time.clock_gettime(clk_id) → float

Return the time of the specified clock `clk_id`. Refer to `Clock ID Constants` for a list of accepted values for `clk_id`. Use `clock_gettime_ns()` to avoid the precision loss caused by the `float` type. Availability: Unix. Added in version 3.3.

time.clock_gettime_ns(clk_id) → int

Similar to `clock_gettime()` but return time as nanoseconds. Availability: Unix. Added in version 3.7.

time.clock_settime(clk_id, time: float)

Set the time of the specified clock `clk_id`. Currently, `CLOCK_REALTIME` is the only accepted value for `clk_id`. Use `clock_settime_ns()` to avoid the precision loss caused by the `float` type. Availability: Unix, not Android, not iOS. Added in version 3.3.

time.clock_settime_ns(clk_id, time: int)

Similar to `clock_settime()` but set time with nanoseconds. Availability: Unix, not Android, not iOS. Added in version 3.7.

time.ctime([secs])

Convert a time expressed in seconds since the epoch to a string of a form: 'SunJun2023:21:051993' representing local time. The day field is two characters long and is space padded if the day is a single digit, e.g.: 'WedJun904:26:401993'. If `secs` is not provided or `None`, the current time as returned by `time()` is used. `ctime(secs)` is equivalent to `asctime(localtime(secs))`. Locale information is not used by `time()`.

time.get_clock_info(name)

Get information on the specified clock as a namespace object. Supported clock names and the corresponding functions to read their value are: 'monotonic': `time.monotonic()` 'perf_counter': `time.perf_counter()` 'process_time': `time.process_time()` 'thread_time': `time.thread_time()` 'time': `time.time()` The result has the following attributes: `adjustable`: `True` if the clock can be set to jump forward or backward in time, `False` otherwise. Does not refer to gradual NTP rate adjustments. `implementation`: The name of the underlying

time.gmtime([secs])

Convert a time expressed in seconds since the epoch to a `struct_time` in UTC in which the `dst` flag is always zero. If `secs` is not provided or `None`, the current time as returned by `time()` is used. Fractions of a second are ignored. See above for a description of the `struct_time` object. See `calendar.timegm()` for the inverse of this function.

time.localtime([secs])

Like `gmtime()` but converts to local time. If `secs` is not provided or `None`, the current time as returned by `time()` is used. The `dst` flag is set to 1 when DST applies to the given time. `time.localtime()` may raise `OverflowError`, if the timestamp is outside the range of values supported by the platform `clock_gettime()` or `gmtime()` functions, and `OSError` on `localtime()` or `gmtime()` failure. It's common for this to be restricted to years between 1970 and 2038.

time.mktime(t)

This is the inverse function of `localtime()`. Its argument is the `struct_time` or full 9-tuple (since the `dst` flag is needed; use -1 as the `dst` flag if it is unknown) which expresses the time in local time, not UTC. It returns a floating-point number, for compatibility with `time()`. If the input value cannot be represented as a valid time, either `OverflowError` or `ValueError` will be raised (which depends on whether the invalid value is caught by Python or the underlying C libraries). The earliest date for which

time.monotonic() → float

Return the value (in fractional seconds) of a monotonic clock, i.e. a clock that cannot go backwards. The clock is not affected by system clock updates. The reference point of the returned value is undefined, so that only the difference between the results of two calls is valid. Clock: On Windows, call `QueryPerformanceCounter()` and `QueryPerformanceFrequency()`. On macOS, call `mach_absolute_time()` and `mach_timebase_info()`. On HP-UX, call `gethrtime()`. Call `clock_gettime(CLOCK_HIGHRES)` if available. Otherwise, call

time.monotonic_ns() → int

Similar to `monotonic()`, but return time as nanoseconds. Added in version 3.7.

time.perf_counter() → float

Return the value (in fractional seconds) of a performance counter, i.e. a clock with the highest available resolution to measure a short duration. It does include time elapsed during sleep. The clock is the same for all processes. The reference point of the returned value is undefined, so that only the difference between the results of two calls is valid. CPython implementation detail: On CPython, use the same clock as `time.monotonic()` and is a monotonic clock, i.e. a clock that cannot go backwards

time.perf_counter_ns() → int

Similar to `perf_counter()`, but return time as nanoseconds. Added in version 3.7.

time.process_time() → float

Return the value (in fractional seconds) of the sum of the system and user CPU time of the current process. It does not include time elapsed during sleep. It is process-wide by definition. The reference point of the returned value is undefined, so that only the difference between the results of two calls is valid. Use `process_time_ns()` to avoid the precision loss caused by the `float` type. Added in version 3.3.

time.process_time_ns() → int

Similar to `process_time()` but return time as nanoseconds. Added in version 3.7.

time.sleep(secs)

Suspend execution of the calling thread for the given number of seconds. The argument may be a floating-point number to indicate a more precise sleep time. If the sleep is interrupted by a signal and no exception is raised by the signal handler, the sleep is restarted with a recomputed timeout. The suspension time may be longer than requested by an arbitrary amount, because of the scheduling of other activity in the system. Windows implementation On Windows, if `secs` is zero, the thread relinquishes the

time.strftime(format[, t])

Microseconds as a decimal number

Convert a tuple or `struct_time` representing a time as returned by `gmtime()` or `localtime()` to a string as specified by the `format` argument. If it is not provided, the current time as returned by `localtime()` is used. `format` must be a string. `ValueError` is raised if any field in `t` is outside of the allowed range. 0 is a legal argument for any position in the time tuple; if it is normally illegal the value is forced to a correct one. The following directives can be embedded in the `format` string. They are shown without th

[000000,999999].

`time.strptime(string[,format])`

Parse a string representing a time according to a format. The return value is a `struct_time` as returned by `gmtime()` or `localtime()`. The `format` parameter uses the same directives as those used by `strftime()`; it defaults to `"%a%b%d%H:%M:%S%Y"` which matches the formatting returned by `ctime()`. If `string` cannot be parsed according to `format`, or if it has excess data after parsing, `ValueError` is raised. The default values used to fill in any missing data when more accurate values cannot be inferred are (1900,1,1,0,0,0

`time.struct_time`

`tm_year`

`tm_mon`

`tm_mday`

`tm_hour`

`tm_min`

`tm_sec`

`tm_wday`

`tm_yday`

`tm_isdst`

`tm_zone`

`tm_gmtoff`

The type of the time value sequence returned by `gmtime()`, `localtime()`, and `strptime()`. It is an object with an named tuple interface: values can be accessed by index and by attribute name. The following values are present: Index Attribute Values 0 `tm_year` (for example, 1993) 1 `tm_mon` range [1, 12] 2 `tm_mday` range [1, 31] 3 `tm_hour` range [0, 23] 4 `tm_min` range [0, 59] 5 `tm_sec` range [0, 61]; see Note (2) 6 `tm_wday` range [0, 6]; Monday is 0 7 `tm_yday` range [1, 366] 8 `tm_isdst` 0, 1 or -1; see below `N/A` `tm_zone` abbreviation

`time.time()` → float

Return the time in seconds since the epoch as a floating-point number. The handling of leap seconds is platform dependent. On Windows and most Unix systems, the leap seconds are not counted towards the time in seconds since the epoch. This is commonly referred to as Unix time. Note that even though the time is always returned as a floating-point number, not all systems provide time with a better precision than 1 second. While this function normally returns non-decreasing values, it can return a lower v

Changed in version 3.13: On Windows, calls `GetSystemTimePreciseAsFileTime()` instead of `GetSystemTimeAsFileTime()`.

`time.time_ns()` → int

Similar to `time()` but returns time as an integer number of nanoseconds since the epoch. Added in version 3.7.

`time.thread_time()` → float

Return the value (in fractional seconds) of the sum of the system and user CPU time of the current thread. It does not include time elapsed during sleep. It is thread-specific by definition. The reference point of the returned value is undefined, so that only the difference between the results of two calls in the same thread is valid. Use `thread_time_ns()` to avoid the precision loss caused by the float type. Availability: Linux, Unix, Windows. Unix systems supporting `CLOCK_THREAD_CPUTIME_ID`. Added in

`time.thread_time_ns()`→int

Similar to `thread_time()` but return time as nanoseconds. Added in version 3.7.

`time.tzset()`

`stdanddst`

`offset`

`start[/time],end[/time]`

`Jn`

`n`

`Mm.n.d`

Reset the time conversion rules used by the library routines. The environment variable `TZ` specifies how this is done. It will also set the variable `tzname` (from the `TZ` environment variable), `timezone` (non-DST seconds West of UTC), `altzone` (DST seconds west of UTC) and `daylight` (to 0 if this timezone does not have any daylight saving time rules, or to nonzero if there is a time, past, present or future when daylight saving time applies). Availability: Unix. Note Although in many cases, changing the `TZ` environment

Three or more alphanumerics giving the timezone abbreviations. These will be propagated into `time.tzname`

The offset has the form: `±hh[:mm[:ss]]`. This indicates the value added the local time to arrive at UTC. If preceded by a '-', the timezone is east of the Prime Meridian; otherwise, it is west. If no offset follows `dst`, summer time is assumed to be one hour ahead of standard time.

Indicates when to change to and back from DST. The format of the start and end dates are one of the following: `Jn` The Julian day n ($1 \leq n \leq 365$). Leap days are not counted, so in all years February 28 is day 59 and March 1 is day 60. `n` The zero-based Julian day ($0 \leq n \leq 365$). Leap days are counted, and it is possible to refer to February 29. `Mm.n.d` The d 'th day ($0 \leq d \leq 6$) of week n of month m of the year ($1 \leq n \leq 5$, $1 \leq m \leq 12$, where week 5 means "the last day in month m " which may occur in either the fourth

The Julian day n ($1 \leq n \leq 365$). Leap days are not counted, so in all years February 28 is day 59 and March 1 is day 60.

The zero-based Julian day ($0 \leq n \leq 365$). Leap days are counted, and it is possible to refer to February 29.

The d 'th day ($0 \leq d \leq 6$) of week n of month m of the year ($1 \leq n \leq 5$, $1 \leq m \leq 12$, where week 5 means "the last day in month m " which may occur in either the fourth or the fifth week). Week 1 is the first week in which the d 'th day occurs. Day zero is a Sunday.

Clock ID Constants

These constants are used as parameters for `clock_getres()` and `clock_gettime()`.

`time.CLOCK_BOOTTIME`

Identical to `CLOCK_MONOTONIC`, except it also includes any time that the system is suspended. This allows applications to get a suspend-aware monotonic clock without having to deal with the complications of `CLOCK_REALTIME`, which may have discontinuities if the time is changed using `settimeofday()` or

similar.Availability: Linux >= 2.6.39.Added in version 3.7.

time.CLOCK_HIGHRES

The Solaris OS has aCLOCK_HIGHREStimer that attempts to use an optimal hardware source, and may give close to nanosecond resolution.CLOCK_HIGHRESis the nonadjustable, high-resolution clock.Availability: Solaris.Added in version 3.3.

time.CLOCK_MONOTONIC

Clock that cannot be set and represents monotonic time since some unspecified starting point.Availability: Unix.Added in version 3.3.

time.CLOCK_MONOTONIC_RAW

Similar toCLOCK_MONOTONIC, but provides access to a raw hardware-based time that is not subject to NTP adjustments.Availability: Linux >= 2.6.28, macOS >= 10.12.Added in version 3.3.

time.CLOCK_MONOTONIC_RAW_APPROX

Similar toCLOCK_MONOTONIC_RAW, but reads a value cached by the system at context switch and hence has less accuracy.Availability: macOS >= 10.12.Added in version 3.13.

time.CLOCK_PROCESS_CPUTIME_ID

High-resolution per-process timer from the CPU.Availability: Unix.Added in version 3.3.

time.CLOCK_PROF

High-resolution per-process timer from the CPU.Availability: FreeBSD, NetBSD >= 7, OpenBSD.Added in version 3.7.

time.CLOCK_TAI

International Atomic TimeThe system must have a current leap second table in order for this to give the correct answer. PTP or NTP software can maintain a leap second table.Availability: Linux.Added in version 3.9.

time.CLOCK_THREAD_CPUTIME_ID

Thread-specific CPU-time clock.Availability: Unix.Added in version 3.3.

time.CLOCK_UPTIME

Time whose absolute value is the time the system has been running and not suspended, providing accurate uptime measurement, both absolute and interval.Availability: FreeBSD, OpenBSD >= 5.5.Added in version 3.7.

time.CLOCK_UPTIME_RAW

Clock that increments monotonically, tracking the time since an arbitrary point, unaffected by frequency or time adjustments and not incremented while the system is asleep.Availability: macOS >= 10.12.Added in version 3.8.

time.CLOCK_UPTIME_RAW_APPROX

LikeCLOCK_UPTIME_RAW, but the value is cached by the system at context switches and therefore has less accuracy.Availability: macOS >= 10.12.Added in version 3.13.

The following constant is the only parameter that can be sent toclock_settime().

time.CLOCK_REALTIME

Real-time clock. Setting this clock requires appropriate privileges. The clock is the same for all processes.Availability: Unix.Added in version 3.3.

Timezone Constants

time.altzone

The offset of the local DST timezone, in seconds west of UTC, if one is defined. This is negative if the local DST timezone is east of UTC (as in Western Europe, including the UK). Only use this if `daylight` is nonzero. See note below.

time.daylight

Nonzero if a DST timezone is defined. See note below.

time.timezone

The offset of the local (non-DST) timezone, in seconds west of UTC (negative in most of Western Europe, positive in the US, zero in the UK). See note below.

time.tzname

A tuple of two strings: the first is the name of the local non-DST timezone, the second is the name of the local DST timezone. If no DST timezone is defined, the second string should not be used. See note below.

■ Nota: NoteFor the above Timezone constants (`altzone`, `daylight`, `timezone`, and `tzname`), the value is determined by the timezone rules in effect at module load time or the last `time.tzset()` is called and may be incorrect for times in the past. It is recommended to use the `tm_gmtoff` and `tm_zone` results from `localtime()` to obtain timezone information.

See also

Module datetime

Module locale

Module calendar

More object-oriented interface to dates and times.

Internationalization services. The locale setting affects the interpretation of many format specifiers `strftime()` and `strptime()`.

General calendar-related functions. `timegm()` is the inverse of `gmtime()` from this module.

Footnotes

The use of `%Z` is now deprecated, but the `%z` escape that expands to the preferred hour/minute offset is not supported by all ANSI C libraries. Also, a strict reading of the original 1982 RFC 822 standard calls for a two-digit year (`%y` rather than `%Y`), but practice moved to 4-digit years long before the year 2000. After that, RFC 822 became obsolete and the 4-digit year has been first recommended by RFC 1123 and then mandated by RFC 2822, with RFC 5322 continuing this requirement.

logging— Logging facility for Python

Source code: `Lib/logging/__init__.py`

Important

This page contains the API reference information. For tutorial information and discussion of more advanced topics, see

- [Basic Tutorial](#)
- [Advanced Tutorial](#)
- [Logging Cookbook](#)

This module defines functions and classes which implement a flexible event logging system for applications and libraries.

The key benefit of having the logging API provided by a standard library module is that all Python modules can participate in logging, so your application log can include your own messages integrated with messages from third-party modules.

Here's a simple example of idiomatic usage:

```
# myapp.py
import logging
import mylib
logger = logging.getLogger(__name__)

def main():
    logging.basicConfig(filename='myapp.log', level=logging.INFO)
    logger.info('Started')
    mylib.do_something()
    logger.info('Finished')

if __name__ == '__main__':
    main()

# mylib.py
import logging
logger = logging.getLogger(__name__)

def do_something():
    logger.info('Doing something')
```

If you run `myapp.py`, you should see this in `myapp.log`:

```
INFO:__main__:Started
INFO:mylib:Doing something
INFO:__main__:Finished
```

The key feature of this idiomatic usage is that the majority of code is simply creating a module level logger with `getLogger(__name__)`, and using that logger to do any needed logging. This is concise, while allowing downstream code fine-grained control if needed. Logged messages to the module-level logger get forwarded to handlers of loggers in higher-level modules, all the way up to the highest-level logger known as the root logger; this approach is known as hierarchical logging.

For logging to be useful, it needs to be configured: setting the levels and destinations for each logger, potentially changing how specific modules log, often based on command-line arguments or application configuration. In most cases, like the one above, only the root logger needs to be so configured, since all the lower level loggers at module level eventually forward their messages to its handlers. `basicConfig()` provides a quick way to configure the root logger that handles many use cases.

The module provides a lot of functionality and flexibility. If you are unfamiliar with logging, the best way to get to grips with it is to view the tutorials (see the links above and on the right).

The basic classes defined by the module, together with their attributes and methods, are listed in the sections below.

- Loggers expose the interface that application code directly uses.
- Handlers send the log records (created by loggers) to the appropriate destination.
- Filters provide a finer grained facility for determining which log records to output.
- Formatters specify the layout of log records in the final output.

Logger Objects

Loggers have the following attributes and methods. Note that Loggers should NEVER be instantiated directly, but always through the module-level function `logging.getLogger(name)`. Multiple calls to `getLogger()` with the same name will always return a reference to the same Logger object.

The name is potentially a period-separated hierarchical value, like `foo.bar.baz` (though it could also be just plain `foo`, for example). Loggers that are further down in the hierarchical list are children of loggers higher up in the list. For example, given a logger with a name of `foo`, loggers with names `foo.bar`, `foo.bar.baz`, and `foo.bam` are all descendants of `foo`. In addition, all loggers are descendants of the root logger. The logger name hierarchy is analogous to the Python package hierarchy, and identical to it if you organise your loggers on a per-module basis using the recommended construction `logging.getLogger(__name__)`. That's because in a module, `__name__` is the module's name in the Python package namespace.

class `logging.Logger`

name

level

parent

propagate

handlers

disabled

setLevel(level)

isEnabledFor(level)

getEffectiveLevel()

getChild(suffix)

getChildren()

debug(msg,*args,kwargs)**

info(msg,*args,kwargs)**

warning(msg,*args,kwargs)**

error(msg,*args,kwargs)**

critical(msg,*args,kwargs)**

log(level,msg,*args,kwargs)**

exception(msg,*args,kwargs)**

addFilter(filter)

removeFilter(filter)

filter(record)

addHandler(hdlr)

removeHandler(hdlr)

findCaller(stack_info=False,stacklevel=1)

handle(record)

makeRecord(name,level,fn,lno,msg,args,exc_info,func=None,extra=None,sinfo=None)

hasHandlers()

nameThis is the logger's name, and is the value that was passed to `getLogger()` to obtain the logger. **Note** This attribute should be treated as read-only. **level**The threshold of this logger, as set by the `setLevel()` method. **Note** Do not set this attribute directly - always use `setLevel()`, which has checks for the level passed to it. **parent**The parent logger of this logger. It may change based on later instantiation of loggers which are higher up in the namespace hierarchy. **Note** This value should be treated as read-only.

This is the logger's name, and is the value that was passed to `getLogger()` to obtain the logger. **Note** This attribute should be treated as read-only.

The threshold of this logger, as set by the `setLevel()` method. **Note** Do not set this attribute directly - always use `setLevel()`, which has checks for the level passed to it.

The parent logger of this logger. It may change based on later instantiation of loggers which are higher up in the namespace hierarchy. **Note** This value should be treated as read-only.

If this attribute evaluates to true, events logged to this logger will be passed to the handlers of higher level (ancestor) loggers, in addition to any handlers attached to this logger. Messages are passed directly to the ancestor loggers' handlers - neither the level nor filters of the ancestor loggers in question are considered. If this evaluates to false, logging messages are not passed to the handlers of ancestor loggers. Spelling it out with an example: If the `propagate` attribute of the logger is set to true, and the logger has a handler, messages logged to this logger will be passed to that handler, as well as to the handlers of all ancestor loggers. If the `propagate` attribute is set to false, messages logged to this logger will only be passed to the handlers of this logger.

The list of handlers directly attached to this logger instance. **Note** This attribute should be treated as read-only; it is normally changed via the `addHandler()` and `removeHandler()` methods, which use locks to ensure thread-safe operation.

This attribute disables handling of any events. It is set to `False` in the initializer, and only changed by logging configuration code. **Note** This attribute should be treated as read-only.

Sets the threshold for this logger to `level`. Logging messages which are less severe than `level` will be ignored; logging messages which have severity `level` or higher will be emitted by whichever handler or handlers service this logger, unless a handler's level has been set to a higher severity level than `level`. When a logger is created, the level is set to `NOTSET` (which causes all messages to be processed when the logger is the root logger, or delegation to the parent when the logger is a non-root logger).

Indicates if a message of severity `level` would be processed by this logger. This method checks first the module-level level set by `logging.disable(level)` and then the logger's effective level as determined by `getEffectiveLevel()`.

Indicates the effective level for this logger. If a value other than `NOTSET` has been set using `setLevel()`, it is returned. Otherwise, the hierarchy is traversed towards the root until a value other than `NOTSET` is found, and that value is returned. The value returned is an integer, typically one of `logging.DEBUG`, `logging.INFO` etc.

Returns a logger which is a descendant to this logger, as determined by the suffix. Thus, `logging.getLogger('abc').getChild('def.ghi')` would return the same logger as would be returned by `logging.getLogger('abc.def.ghi')`. This is a convenience method, useful when the parent logger is named using e.g. `__name__` rather than a literal string. Added in version 3.2.

Returns a set of loggers which are immediate children of this logger. So for example `logging.getLogger().getChildren()` might return a set containing loggers named `foo` and `bar`, but a logger named `foo.bar` wouldn't be included in the set. Likewise, `logging.getLogger('foo').getChildren()` might return a set including a logger named `foo.bar`, but it wouldn't include one named `foo.bar.baz`. Added in version 3.12.

Logs a message with level `DEBUG` on this logger. The `msg` is the message format string, and the `args` are the arguments which are merged into `msg` using the string formatting operator. (Note that this means that you can use keywords in the format string, together with a single dictionary argument.) No % formatting operation is performed on `msg` when no `args` are supplied. There are four keyword arguments in `kwargs` which are inspected: `exc_info`, `stack_info`, `stacklevel` and `extra`. If `exc_info` does not evaluate as false, it causes

Logs a message with level `INFO` on this logger. The arguments are interpreted as for `debug()`.

Logs a message with level `WARNING` on this logger. The arguments are interpreted as for `debug()`. Note There is an obsolete method `warn` which is functionally identical to `warning`. As `warn` is deprecated, please do not use it - use `warning` instead.

Logs a message with level `ERROR` on this logger. The arguments are interpreted as for `debug()`.

Logs a message with level `CRITICAL` on this logger. The arguments are interpreted as for `debug()`.

Logs a message with integer level `level` on this logger. The other arguments are interpreted as for `debug()`.

Logs a message with level `ERROR` on this logger. The arguments are interpreted as for `debug()`. Exception info is added to the logging message. This method should only be called from an exception handler.

Adds the specified filter `filter` to this logger.

Removes the specified filter `filter` from this logger.

Apply this logger's filters to the record and return `True` if the record is to be processed. The filters are consulted in turn, until one of them returns a false value. If none of them return a false value, the record will be processed (passed to handlers). If one returns a false value, no further processing of the record occurs.

Adds the specified handler `hdlr` to this logger.

Removes the specified handler `hdlr` from this logger.

Finds the caller's source filename and line number. Returns the filename, line number, function name and stack information as a 4-element tuple. The stack information is returned as `None` unless `stack_info` is `True`. The `stacklevel` parameter is passed from code calling the `debug()` and other APIs. If greater than 1, the excess is used to skip stack frames before determining the values to be returned. This will generally be useful when calling logging APIs from helper/wrapper code, so that the information in t

Handles a record by passing it to all handlers associated with this logger and its ancestors (until a false value of `propagate` is found). This method is used for unpickled records received from a socket, as well as those created locally. Logger-level filtering is applied using `filter()`.

This is a factory method which can be overridden in subclasses to create specialized `LogRecord` instances.

Checks to see if this logger has any handlers configured. This is done by looking for handlers in this logger and its parents in the logger hierarchy. Returns `True` if a handler was found, else `False`. The method stops searching up the hierarchy whenever a logger with the 'propagate' attribute set to false is found - that will be the last logger which is checked for the existence of handlers. Added in version 3.2.

Logging Levels

The numeric values of logging levels are given in the following table. These are primarily of interest if you want to define your own levels, and need them to have specific values relative to the predefined levels. If you define a level with the same numeric value, it overwrites the predefined value; the predefined name is lost.

Level	Numeric value	What it means / When to use it
logging.NOTSET	0	When set on a logger, indicates that ancestor loggers are to be consulted to determine the effective level. If that still resolves to NOTSET, then all events are logged. When set on a handler, all events are handled.
logging.DEBUG	10	Detailed information, typically only of interest to a developer trying to diagnose a problem.
logging.INFO	20	Confirmation that things are working as expected.
logging.WARNING	30	An indication that something unexpected happened, or that a problem might occur in the near future (e.g. 'disk space low'). The software is still working as expected.
logging.ERROR	40	Due to a more serious problem, the software has not been able to perform some fun

Handler Objects

Handlers have the following attributes and methods. Note that `Handler` is never instantiated directly; this class acts as a base for more useful subclasses. However, the `__init__()` method in subclasses needs to call `Handler.__init__()`.

class logging.Handler

`__init__(level=NOTSET)`

`createLock()`

`acquire()`

`release()`

`setLevel(level)`

`setFormatter(fmt)`

`addFilter(filter)`

`removeFilter(filter)`

`filter(record)`

`flush()`

`close()`

`handle(record)`

`handleError(record)`

`format(record)`

`emit(record)`

`__init__(level=NOTSET)` Initializes the `Handler` instance by setting its level, setting the list of filters to the empty list and creating a lock (using `createLock()`) for serializing access to an I/O mechanism.
`createLock()` Initializes a thread lock which can be used to serialize access to underlying I/O functionality which may not be threadsafe.
`acquire()` Acquires the thread lock created with `createLock()`.
`release()` Releases the thread lock acquired with `acquire()`.
`setLevel(level)` Sets the threshold for this h

Initializes the `Handler` instance by setting its level, setting the list of filters to the empty list and creating a lock (using `createLock()`) for serializing access to an I/O mechanism.

Initializes a thread lock which can be used to serialize access to underlying I/O functionality which may not be threadsafe.

Acquires the thread lock created with `createLock()`.

Releases the thread lock acquired with `acquire()`.

Sets the threshold for this handler to `level`. Logging messages which are less severe than `level` will be ignored. When a handler is created, the level is set to `NOTSET` (which causes all messages to be processed). See [Logging Levels](#) for a list of levels. Changed in version 3.2: The `level` parameter now accepts a string representation of the level such as 'INFO' as an alternative to the integer constants such as `INFO`.

Sets the formatter for this handler to `fmt`. The `fmt` argument must be a `Formatter` instance or `None`.

Adds the specified filter `filter` to this handler.

Removes the specified filter `filter` from this handler.

Apply this handler's filters to the record and return `True` if the record is to be processed. The filters are consulted in turn, until one of them returns a false value. If none of them return a false value, the record will be emitted. If one returns a false value, the handler will not emit the record.

Ensure all logging output has been flushed. This version does nothing and is intended to be implemented by subclasses.

Tidy up any resources used by the handler. This version does no output but removes the handler from an internal map of handlers, which is used for handler lookup by name. Subclasses should ensure that this gets called from overridden `close()` methods.

Conditionally emits the specified logging record, depending on filters which may have been added to the handler. Wraps the actual emission of the record with acquisition/release of the I/O thread lock.

This method should be called from handlers when an exception is encountered during an `emit()` call. If the module-level attribute `raiseExceptions` is `False`, exceptions get silently ignored. This is what is mostly wanted for a logging system - most users will not care about errors in the logging system, they are more interested in application errors. You could, however, replace this with a custom handler if you wish. The specified record is the one which was being processed when the exception occurred.

Do formatting for a record - if a formatter is set, use it. Otherwise, use the default formatter for the module.

Do whatever it takes to actually log the specified logging record. This version is intended to be implemented by subclasses and so raises a `NotImplementedError`. Warning: This method is called after a handler-level lock is acquired, which is released after this method returns. When you override this method, note that you should be careful when calling anything that invokes other parts of the logging API which might do locking, because that might result in a deadlock. Specifically: [Logging configuration](#)

For a list of handlers included as standard, see [logging.handlers](#).

Formatter Objects

`logging.Formatter(fmt=None, datefmt=None, style='%', validate=True, *, defaults=None)`

Parameters:

`format(record)`

`formatTime(record, datefmt=None)`

`formatException(exc_info)`

`formatStack(stack_info)`

Responsible for converting a `LogRecord` to an output string to be interpreted by a human or external system. Parameters: `fmt(str)` – A format string in the given style for the logged output as a whole. The possible mapping keys are drawn from the `LogRecord` object's `LogRecord` attributes. If not specified, `'%(message)s'` is used, which is just the logged message. `datefmt(str)` – A format string for the date/time portion of the logged output. If not specified, the default described in `formatTime()` is used. `style(str)`

`fmt(str)` – A format string in the given style for the logged output as a whole. The possible mapping keys are drawn from the `LogRecord` object's `LogRecord` attributes. If not specified, `'%(message)s'` is used, which is just the logged message. `datefmt(str)` – A format string for the date/time portion of the logged output. If not specified, the default described in `formatTime()` is used. `style(str)` – Can be one of `'%'`, `'{'` or `'$'` and determines how the format string will be merged with its data: using one of `printf`-st

The record's attribute dictionary is used as the operand to a string formatting operation. Returns the resulting string. Before formatting the dictionary, a couple of preparatory steps are carried out. The `message` attribute of the record is computed using `msg%args`. If the formatting string contains `'(asctime)'`, `formatTime()` is called to format the event time. If there is exception information, it is formatted using `formatException()` and appended to the message. Note that the formatted exception informat

This method should be called from `format()` by a formatter which wants to make use of a formatted time. This method can be overridden in formatters to provide for any specific requirement, but the basic behavior is as follows: if `datefmt(a string)` is specified, it is used with `time.strftime()` to format the creation time of the record. Otherwise, the format `'%Y-%m-%d %H:%M:%S,uuu'` is used, where the `uuu` part is a millisecond value and the other letters are as per the `time.strftime()` documentation. An ex

Formats the specified exception information (a standard exception tuple as returned by `sys.exc_info()`) as a string. This default implementation just uses `traceback.print_exception()`. The resulting string is returned.

Formats the specified stack information (a string as returned by `traceback.print_stack()`, but with the last newline removed) as a string. This default implementation just returns the input value.

`class logging.BufferingFormatter(linefmt=None)`

`formatHeader(records)`

`formatFooter(records)`

`format(records)`

A base formatter class suitable for subclassing when you want to format a number of records. You can pass a `Formatter` instance which you want to use to format each line (that corresponds to a single record). If not specified, the default formatter (which just outputs the event message) is used as the line formatter. `formatHeader(records)` Return a header for a list of records. The base implementation just returns the empty string. You will need to override this method if you want specific behaviour, e

Return a header for a list of records. The base implementation just returns the empty string. You will need to override this method if you want specific behaviour, e.g. to show the count of records, a title or a separator line.

Return a footer for a list of records. The base implementation just returns the empty string. You will need to override this method if you want specific behaviour, e.g. to show the count of records or a separator line.

Return formatted text for a list of records. The base implementation just returns the empty string if there are no records; otherwise, it returns the concatenation of the header, each record formatted with the line formatter, and the footer.

Filter Objects

Filters can be used by `Handlers` and `Loggers` for more sophisticated filtering than is provided by levels. The base filter class only allows events which are below a certain point in the logger hierarchy. For example, a filter

initialized with 'A.B' will allow events logged by loggers 'A.B', 'A.B.C', 'A.B.C.D', 'A.B.D' etc. but not 'A.BB', 'B.A.B' etc. If initialized with the empty string, all events are passed.

class logging.Filter(name="")

filter(record)

Returns an instance of the `Filter` class. If `name` is specified, it names a logger which, together with its children, will have its events allowed through the filter. If `name` is the empty string, allows every event. `filter(record)` Is the specified record to be logged? Returns false for no, true for yes. Filters can either modify log records in-place or return a completely different record instance which will replace the original log record in any future processing of the event.

Is the specified record to be logged? Returns false for no, true for yes. Filters can either modify log records in-place or return a completely different record instance which will replace the original log record in any future processing of the event.

Note that filters attached to handlers are consulted before an event is emitted by the handler, whereas filters attached to loggers are consulted whenever an event is logged (using `debug()`, `info()`, etc.), before sending an event to handlers. This means that events which have been generated by descendant loggers will not be filtered by a logger's filter setting, unless the filter has also been applied to those descendant loggers.

You don't actually need to subclass `Filter`: you can pass any instance which has a `filter` method with the same semantics.

Changed in version 3.2: You don't need to create specialized `Filter` classes, or use other classes with a `filter` method: you can use a function (or other callable) as a filter. The filtering logic will check to see if the filter object has a `filter` attribute: if it does, it's assumed to be a `Filter` and its `filter()` method is called. Otherwise, it's assumed to be a callable and called with the record as the single parameter. The returned value should conform to that returned by `filter()`.

Changed in version 3.12: You can now return a `LogRecord` instance from filters to replace the log record rather than modifying it in place. This allows filters attached to a `Handler` to modify the log record before it is emitted, without having side effects on other handlers.

Although filters are used primarily to filter records based on more sophisticated criteria than levels, they get to see every record which is processed by the handler or logger they're attached to: this can be useful if you want to do things like counting how many records were processed by a particular logger or handler, or adding, changing or removing attributes in the `LogRecord` being processed. Obviously changing the `LogRecord` needs to be done with some care, but it does allow the injection of contextual information into logs (see [Using Filters to impart contextual information](#)).

LogRecord Objects

`LogRecord` instances are created automatically by the `Logger` every time something is logged, and can be created manually via `makeLogRecord()` (for example, from a pickled event received over the wire).

class logging.LogRecord(name, level, pathname, lineno, msg, args, exc_info, func=None, sinfo=None)

Parameters:

getMessage()

Contains all the information pertinent to the event being logged. The primary information is passed in `msg` and `args`, which are combined using `msg%args` to create the `message` attribute of the record. Parameters: `name(str)` – The name of the logger used to log the event represented by this `LogRecord`. Note that the logger name in the `LogRecord` will always have this value, even though it may be emitted by a handler attached to a different (ancestor) logger. `level(int)` – The numeric level of the logging event (such as

`name(str)` – The name of the logger used to log the event represented by this `LogRecord`. Note that the logger name in the `LogRecord` will always have this value, even though it may be emitted by a handler attached to a different (ancestor) logger.

`level(int)` – The numeric level of the logging event (such as 10 for `DEBUG`, 20 for `INFO`, etc). Note that this is converted to two attributes of the `LogRecord`: `levelno` for the numeric value and `levelname` for the corresponding level name.

`pathname(str)` – The full string path

Returns the message for this `LogRecord` instance after merging any user-supplied arguments with the message. If the user-supplied message argument to the logging call is not a string, `str()` is called on it to convert it to a string. This allows use of user-defined classes as messages, whose `__str__` method can return the actual format string to be used.

LogRecord attributes

The `LogRecord` has a number of attributes, most of which are derived from the parameters to the constructor. (Note that the names do not always correspond exactly between the `LogRecord` constructor parameters and the `LogRecord` attributes.) These attributes can be used to merge data from the record into the format string. The following table lists (in alphabetical order) the attribute names, their meanings and the corresponding placeholder in a %-style format string.

If you are using {}-formatting (`str.format()`), you can use `{attrname}` as the placeholder in the format string. If you are using \$-formatting (`string.Template`), use the form `${attrname}`. In both cases, of course, replace `attrname` with the actual attribute name you want to use.

In the case of {}-formatting, you can specify formatting flags by placing them after the attribute name, separated from it with a colon. For example: a placeholder of `{msecs:03.0f}` would format a millisecond value of 4 as 004. Refer to the `str.format()` documentation for full details on the options available to you.

Attribute name	Format	Description	args
<code>msg</code>	<code>%s</code>	You shouldn't need to format this yourself. The tuple of arguments merged into	<code>args</code>
<code>message</code>	<code>%s</code>	to produce	<code>message</code>
<code>asctime</code>	<code>%(asctime)s</code>	, or a dict whose values are used for the merge (when there is only one argument, and it is a dictionary).	<code>asctime</code>
<code>human_readable_time</code>	<code>%(asctime)s</code>	Human-readable time when the <code>LogRecord</code> was created. By default this is of the form	<code>human_readable_time</code>
<code>created</code>	<code>%(created)f</code>	'2003-07-08 16:49:45,896' (the numbers after the comma are millisecond portion of the time).	<code>created</code>
<code>time</code>	<code>%(time)f</code>	Time when the <code>LogRecord</code> was created (as returned by <code>time.time_ns()</code>	<code>time</code>
<code>exc_info</code>	<code>%(exc_info)s</code>	or <code>time.time()</code>).	<code>exc_info</code>
<code>exc_text</code>	<code>%(exc_text)s</code>	You shouldn't need to format this yourself. Exception tuple (à la <code>sys.exc_info()</code>) or, if no exception has occurred, <code>None</code> .	<code>exc_text</code>

Changed in version 3.1: `processName` was added.

Changed in version 3.12: `taskName` was added.

LoggerAdapter Objects

`LoggerAdapter` instances are used to conveniently pass contextual information into logging calls. For a usage example, see the section on adding contextual information to your logging output.

`class logging.LoggerAdapter(logger, extra=None, merge_extra=False)`

`process(msg, kwargs)`

`manager`

`_log`

Returns an instance of `LoggerAdapter` initialized with an underlying `Logger` instance, an optional dict-like object (`extra`), and an optional boolean (`merge_extra`) indicating whether or not the `extra` argument of individual log calls should be merged with the `LoggerAdapter.extra`. The default behavior is to ignore the `extra` argument of individual log calls and only use the one of the `LoggerAdapter` instance.

`process(msg, kwargs)` Modifies the message and/or

keyword arguments passed to a logging call in order to insert

Modifies the message and/or keyword arguments passed to a logging call in order to insert contextual information. This implementation takes the object passed as `extra` to the constructor and adds it to `kwargs` using key 'extra'. The return value is a `(msg,kwargs)` tuple which has the (possibly modified) versions of the arguments passed in.

Delegates to the underlying `manager.onlogger`.

Delegates to the underlying `_log()` method on `logger`.

Thread Safety

The logging module is intended to be thread-safe without any special work needing to be done by its clients. It achieves this through using threading locks; there is one lock to serialize access to the module's shared data, and each handler also creates a lock to serialize access to its underlying I/O.

If you are implementing asynchronous signal handlers using the `signal` module, you may not be able to use logging from within such handlers. This is because lock implementations in the `threading` module are not always re-entrant, and so cannot be invoked from such signal handlers.

Module-Level Functions

In addition to the classes described above, there are a number of module-level functions.

logging.getLogger(name=None)

Return a logger with the specified name or, if `name` is `None`, return the root logger of the hierarchy. If specified, the name is typically a dot-separated hierarchical name like 'a', 'a.b' or 'a.b.c.d'. Choice of these names is entirely up to the developer who is using logging, though it is recommended that `__name__` be used unless you have a specific reason for not doing that, as mentioned in `Logger Objects`. All calls to this function with a given name return the same logger instance. This means that `logg`

logging.getLoggerClass()

Return either the standard `Logger` class, or the last class passed to `setLoggerClass()`. This function may be called from within a new class definition, to ensure that installing a customized `Logger` class will not undo customizations already applied by other code. For example: `class MyLogger(logging.getLoggerClass()):# ... override behaviour here`

logging.getLogRecordFactory()

Return a callable which is used to create a `LogRecord`. Added in version 3.2: This function has been provided, along with `setLogRecordFactory()`, to allow developers more control over how the `LogRecord` representing a logging event is constructed. See `setLogRecordFactory()` for more information about the how the factory is called.

logging.debug(msg,*args,kwargs)**

This is a convenience function that calls `Logger.debug()`, on the root logger. The handling of the arguments is in every way identical to what is described in that method. The only difference is that if the root logger has no handlers, then `basicConfig()` is called, prior to calling `debug` on the root logger. For very short scripts or quick demonstrations of logging facilities, `debug` and the other module-level functions may be convenient. However, most programs will want to carefully and explicitly control th

logging.info(msg,*args,kwargs)**

Logs a message with level `INFO` on the root logger. The arguments and behavior are otherwise the same as `fordebug()`.

logging.warning(msg,*args,kwargs)**

Logs a message with level `WARNING` on the root logger. The arguments and behavior are otherwise the same as `fordebug()`. Note There is an obsolete function `warn` which is functionally identical to `warning`. As `warn` is deprecated, please do not use it - use `warning` instead.

logging.error(msg,*args,kwargs)**

Logs a message with level `ERROR` on the root logger. The arguments and behavior are otherwise the same as `fordebug()`.

logging.critical(msg,*args,kwargs)**

Logs a message with level `CRITICAL` on the root logger. The arguments and behavior are otherwise the same as `fordebug()`.

logging.exception(msg,*args,kwargs)**

Logs a message with level `ERROR` on the root logger. The arguments and behavior are otherwise the same as `fordebug()`. Exception info is added to the logging message. This function should only be called from an exception handler.

logging.log(level,msg,*args,kwargs)**

Logs a message with level `level` on the root logger. The arguments and behavior are otherwise the same as `fordebug()`.

logging.disable(level=CRITICAL)

Provides an overriding level `level` for all loggers which takes precedence over the logger's own level. When the need arises to temporarily throttle logging output down across the whole application, this function can be useful. Its effect is to disable all logging calls of severity `level` and below, so that if you call it with a value of `INFO`, then all `INFO` and `DEBUG` events would be discarded, whereas those of severity `WARNING` and above would be processed according to the logger's effective level. If

logging.addLevelName(level,levelName)

Associates level `level` with text `levelName` in an internal dictionary, which is used to map numeric levels to a textual representation, for example when a `Formatter` formats a message. This function can also be used to define your own levels. The only constraints are that all levels used must be registered using this function, levels should be positive integers and they should increase in increasing order of severity. Note If you are thinking of defining your own levels, please see the section on `Custom Le`

logging.getLevelNamesMapping()

Returns a mapping from level names to their corresponding logging levels. For example, the string `"CRITICAL"` maps to `CRITICAL`. The returned mapping is copied from an internal mapping on each call to this function. Added in version 3.11.

logging.getLevelName(level)

Returns the textual or numeric representation of logging level `level`. If `level` is one of the predefined levels `CRITICAL`, `ERROR`, `WARNING`, `INFO` or `DEBUG` then you get the corresponding string. If you have associated levels with names using `addLevelName()` then the name you have associated with `level` is returned. If a numeric value corresponding to one of the defined levels is passed in, the corresponding string representation is returned. The `level` parameter also accepts a string representation of the level such as '

logging.getHandlerByName(name)

Returns a handler with the specified name, or None if there is no handler with that name. Added in version 3.12.

logging.getHandlerNames()

Returns an immutable set of all known handler names. Added in version 3.12.

logging.makeLogRecord(attrdict)

Creates and returns a new LogRecord instance whose attributes are defined by attrdict. This function is useful for taking a pickled LogRecord attribute dictionary, sent over a socket, and reconstituting it as a LogRecord instance at the receiving end.

logging.basicConfig(kwargs)**

Does basic configuration for the logging system by creating a StreamHandler with a default Formatter and adding it to the root logger. The functions debug(), info(), warning(), error() and critical() will call basicConfig() automatically if no handlers are defined for the root logger. This function does nothing if the root logger already has handlers configured, unless the keyword argument force is set to True. Note This function should be called from the main thread before other threads are started. In versions 0

logging.shutdown()

Informs the logging system to perform an orderly shutdown by flushing and closing all handlers. This should be called at application exit and no further use of the logging system should be made after this call. When the logging module is imported, it registers this function as an exit handler (see atexit), so normally there's no need to do that manually.

logging.setLoggerClass(klass)

Tells the logging system to use the class klass when instantiating a logger. The class should define __init__() such that only a name argument is required, and the __init__() should call Logger.__init__(). This function is typically called before any loggers are instantiated by applications which need to use custom logger behavior. After this call, as at any other time, do not instantiate loggers directly using the subclass: continue to use the logging.getLogger() API to get your loggers.

logging.setLogRecordFactory(factory)

Parameters:

name:

level:

fn:

ln:

msg:

args:

exc_info:

func:

sinfo:

kwargs:

Set a callable which is used to create a LogRecord. Parameters: factory – The factory callable to be used to instantiate a log record. Added in version 3.2: This function has been provided, along with getLogRecordFactory(), to allow developers more control over how the LogRecord representing a logging event is constructed. The factory has the following signature: factory(name, level, fn, ln, msg, args, exc_info, func=None, sinfo=None, **kwargs) name: The logger

name.level:The logging level (numeric).fn:The full path

factory– The factory callable to be used to instantiate a log record.

The logger name.

The logging level (numeric).

The full pathname of the file where the logging call was made.

The line number in the file where the logging call was made.

The logging message.

The arguments for the logging message.

An exception tuple, orNone.

The name of the function or method which invoked the logging call.

A stack traceback such as is provided bytraceback.print_stack(), showing the call hierarchy.

Additional keyword arguments.

Module-Level Attributes

logging.lastResort

A “handler of last resort” is available through this attribute. This is aStreamHandlerwriting tosys.stderrwith a level ofWARNING, and is used to handle logging events in the absence of any logging configuration. The end result is to just print the message tosys.stderr. This replaces the earlier error message saying that “no handlers could be found for logger XYZ”. If you need the earlier behaviour for some reason,lastResortcan be set toNone.Added in version 3.2.

logging.raiseExceptions

Used to see if exceptions during handling should be propagated.Default:True.IfraiseExceptionsisFalse, exceptions get silently ignored. This is what is mostly wanted for a logging system - most users will not care about errors in the logging system, they are more interested in application errors.

Integration with the warnings module

ThecaptureWarnings()function can be used to integrateloggingwith thewarningsmodule.

logging.captureWarnings(capture)

This function is used to turn the capture of warnings by logging on and off.IfcaptureisTrue, warnings issued by thewarningsmodule will be redirected to the logging system. Specifically, a warning will be formatted usingwarnings.formatwarning()and the resulting string logged to a logger named'py.warnings'with a severity ofWARNING.IfcaptureisFalse, the redirection of warnings to the logging system will stop, and warnings will be redirected to their original destinations (i.e. those in effect befor

See also

Modulelogging.config

Modulelogging.handlers

PEP 282- A Logging System

Original Python logging package

Configuration API for the logging module.

Useful handlers included with the logging module.

The proposal which described this feature for inclusion in the Python standard library.

This is the original source for the logging package. The version of the package available from this site is suitable for use with Python 1.5.2, 2.1.x and 2.2.x, which do not include the logging package in the standard library.

logging.config— Logging configuration

Source code: `Lib/logging/config.py`

Important

This page contains only reference information. For tutorials, please see

- Basic Tutorial
- Advanced Tutorial
- Logging Cookbook

This section describes the API for configuring the logging module.

Configuration functions

The following functions configure the logging module. They are located in the `logging.config` module. Their use is optional — you can configure the logging module using these functions or by making calls to the main API (defined in `logging` itself) and defining handlers which are declared either in `logging` or `logging.handlers`.

logging.config.dictConfig(config)

Takes the logging configuration from a dictionary. The contents of this dictionary are described in `Configuration dictionary schema` below. If an error is encountered during configuration, this function will raise a `ValueError`, `TypeError`, `AttributeError` or `ImportError` with a suitably descriptive message. The following is a (possibly incomplete) list of conditions which will raise an error: A level which is not a string or which is a string not corresponding to an actual logging level. A propagate value which

logging.config.fileConfig(fname, defaults=None, disable_existing_loggers=True, encoding=None)

Parameters:

Reads the logging configuration from a `configparser`-format file. The format of the file should be as described in `Configuration file format`. This function can be called several times from an application, allowing an end user to select from various pre-canned configurations (if the developer provides a mechanism to present the choices and load the chosen configuration). It will raise `FileNotFoundError` if the file doesn't exist and `RuntimeError` if the file is invalid or empty. Parameters: `fname`— A filename

`fname`— A filename, or a file-like object, or an instance derived from `RawConfigParser`. If a `RawConfigParser`-derived instance is passed, it is used as is. Otherwise, a `ConfigParser` is instantiated, and the configuration read by it from the object passed in `fname`. If that has a `readline()` method, it is assumed to be a file-like object and read using `read_file()`; otherwise, it is assumed to be a filename and passed to `read()`. `defaults`— Defaults to be passed to the `ConfigParser` can be specified in this argument

logging.config.listen(port=DEFAULT_LOGGING_CONFIG_PORT, verify=None)

Starts up a socket server on the specified port, and listens for new configurations. If no port is specified, the module's default `DEFAULT_LOGGING_CONFIG_PORT` is used. Logging configurations will be sent as a file suitable for processing by `dictConfig()` or `fileConfig()`. Returns a `Thread` instance on which you can call `start()` to start the server, and which you can `join()` when appropriate. To stop the server, call `stopListening()`. The `verify` argument, if specified, should be a callable which should verify whether

logging.config.stopListening()

Stops the listening server which was created with a call to `listen()`. This is typically called before calling `join()` on the return value from `listen()`.

Security considerations

The logging configuration functionality tries to offer convenience, and in part this is done by offering the ability to convert text in configuration files into Python objects used in logging configuration - for example, as described in `User-defined objects`. However, these same mechanisms (importing callables from user-defined modules and calling them with parameters from the configuration) could be used to invoke any code you like, and for this reason you should treat configuration files from untrusted sources with extreme caution and satisfy yourself that nothing bad can happen if you load them, before actually loading them.

Configuration dictionary schema

Describing a logging configuration requires listing the various objects to create and the connections between them; for example, you may create a handler named 'console' and then say that the logger named 'startup' will send its messages to the 'console' handler. These objects aren't limited to those provided by the `logging` module because you might write your own formatter or handler class. The parameters to these classes may also need to include external objects such as `sys.stderr`. The syntax for describing these objects and connections is defined in `Object connections` below.

Dictionary Schema Details

The dictionary passed to `dictConfig()` must contain the following keys:

- **version**- to be set to an integer value representing the schema version. The only valid value at present is 1, but having this key allows the schema to evolve while still preserving backwards compatibility.

All other keys are optional, but if present they will be interpreted as described below. In all cases below where a 'configuring dict' is mentioned, it will be checked for the special '()' key to see if a custom instantiation is required. If so, the mechanism described in `User-defined objects` below is used to create an instance; otherwise, the context is used to determine what to instantiate.

- **formatters**- the corresponding value will be a dict in which each key is a formatter id and each value is a dict describing how to configure the corresponding `Formatter` instance. The configuring dict is searched for the following optional keys which correspond to the arguments passed to create a `Formatter` object: `format`, `datefmt`, `style`, `validate` (since version ≥ 3.8), `defaults` (since version ≥ 3.12). An optional `class`
- **filters**- the corresponding value will be a dict in which each key is a filter id and each value is a dict describing how to configure the corresponding `Filter` instance. The configuring dict is searched for the key `name` (defaulting to the empty string) and this is used to construct a logging `Filter` instance.
- **handlers**- the corresponding value will be a dict in which each key is a handler id and each value is a dict describing how to configure the corresponding `Handler` instance. The configuring dict is searched for the following keys: `class` (mandatory). This is the fully qualified name of the handler class. `level` (optional). The level of the handler. `formatter` (optional). The id of the formatter for this handler.
- **loggers**- the corresponding value will be a dict in which each key is a logger name and each value is a dict describing how to configure the corresponding `Logger` instance. The configuring dict is searched for the following keys: `level` (optional). The level of the logger. `propagate` (optional). The propagation setting of the logger. `filters` (optional). A list of ids of the filters for this logger. `changed`
- **root**- this will be the configuration for the root logger. Processing of the configuration will be as for any logger, except that the `propagate` setting will not be applicable.
- **incremental**- whether the configuration is to be interpreted as incremental to the existing configuration. This value defaults to `False`, which means that the specified configuration replaces the existing configuration with the

same semantics as used by the `existingfileConfig()` API. If the specified value is `True`, the configuration is processed as described in the section on `Incremental Configuration`.

- `disable_existing_loggers`- whether any existing non-root loggers are to be disabled. This setting mirrors the parameter of the same name in `infileConfig()`. If absent, this parameter defaults to `True`. This value is ignored if `incremental` is `True`.

Incremental Configuration

It is difficult to provide complete flexibility for incremental configuration. For example, because objects such as filters and formatters are anonymous, once a configuration is set up, it is not possible to refer to such anonymous objects when augmenting a configuration.

Furthermore, there is not a compelling case for arbitrarily altering the object graph of loggers, handlers, filters, formatters at run-time, once a configuration is set up; the verbosity of loggers and handlers can be controlled just by setting levels (and, in the case of loggers, propagation flags). Changing the object graph arbitrarily in a safe way is problematic in a multi-threaded environment; while not impossible, the benefits are not worth the complexity it adds to the implementation.

Thus, when the `incremental` key of a configuration dict is present and is `True`, the system will completely ignore any `formatters` and `filters` entries, and process only the `level` settings in the `handlers` entries, and the `level` and `propagate` settings in the `loggers` and `root` entries.

Using a value in the configuration dict lets configurations to be sent over the wire as pickled dicts to a socket listener. Thus, the logging verbosity of a long-running application can be altered over time with no need to stop and restart the application.

Object connections

The schema describes a set of logging objects - loggers, handlers, formatters, filters - which are connected to each other in an object graph. Thus, the schema needs to represent connections between the objects. For example, say that, once configured, a particular logger has attached to it a particular handler. For the purposes of this discussion, we can say that the logger represents the source, and the handler the destination, of a connection between the two. Of course in the configured objects this is represented by the logger holding a reference to the handler. In the configuration dict, this is done by giving each destination object an id which identifies it unambiguously, and then using the id in the source object's configuration to indicate that a connection exists between the source and the destination object with that id.

So, for example, consider the following YAML snippet:

```
formatters:
  brief:
    # configuration for formatter with id 'brief' goes here
  precise:
    # configuration for formatter with id 'precise' goes here
handlers:
  h1: #This is an id
    # configuration of handler with id 'h1' goes here
    formatter: brief
  h2: #This is another id
    # configuration of handler with id 'h2' goes here
    formatter: precise
loggers:
  foo.bar.baz:
    # other configuration for logger 'foo.bar.baz'
    handlers: [h1, h2]
```

(Note: YAML used here because it's a little more readable than the equivalent Python source form for the dictionary.)

The ids for loggers are the logger names which would be used programmatically to obtain a reference to those loggers, e.g.foo.bar.baz. The ids for Formatters and Filters can be any string value (such asbrief,preciseabove) and they are transient, in that they are only meaningful for processing the configuration dictionary and used to determine connections between objects, and are not persisted anywhere when the configuration call is complete.

The above snippet indicates that logger namedfoo.bar.bazshould have two handlers attached to it, which are described by the handler idsh1andh2. The formatter forh1is that described by idbrief, and the formatter forh2is that described by idprecise.

User-defined objects

The schema supports user-defined objects for handlers, filters and formatters. (Loggers do not need to have different types for different instances, so there is no support in this configuration schema for user-defined logger classes.)

Objects to be configured are described by dictionaries which detail their configuration. In some places, the logging system will be able to infer from the context how an object is to be instantiated, but when a user-defined object is to be instantiated, the system will not know how to do this. In order to provide complete flexibility for user-defined object instantiation, the user needs to provide a 'factory' - a callable which is called with a configuration dictionary and which returns the instantiated object. This is signalled by an absolute import path to the factory being made available under the special key'(). Here's a concrete example:

```
formatters:
  brief:
    format: '%(message)s'
  default:
    format: '%(asctime)s %(levelname)-8s %(name)-15s %(message)s'
    datefmt: '%Y-%m-%d %H:%M:%S'
  custom:
    (): my.package.customFormatterFactory
    bar: baz
    spam: 99.9
    answer: 42
```

The above YAML snippet defines three formatters. The first, with idbrief, is a standardlogging.Formatterinstance with the specified format string. The second, with iddefault, has a longer format and also defines the time format explicitly, and will result in logging.Formatterinitialized with those two format strings. Shown in Python source form, thebriefanddefaultformatters have configuration sub-dictionaries:

```
{
  'format' : '%(message)s'
}
```

and:

```
{
  'format' : '%(asctime)s %(levelname)-8s %(name)-15s %(message)s',
  'datefmt' : '%Y-%m-%d %H:%M:%S'
}
```

respectively, and as these dictionaries do not contain the special key'()', the instantiation is inferred from the context: as a result, standardlogging.Formatterinstances are created. The configuration sub-dictionary for the third formatter, with idcustom, is:

```
{
  '()' : 'my.package.customFormatterFactory',
  'bar' : 'baz',
  'spam' : 99.9,
  'answer' : 42
}
```

and this contains the special key '()', which means that user-defined instantiation is wanted. In this case, the specified factory callable will be used. If it is an actual callable it will be used directly - otherwise, if you specify a string (as in the example) the actual callable will be located using normal import mechanisms. The callable will be called with the remaining items in the configuration sub-dictionary as keyword arguments. In the above example, the formatter with `idcustom` will be assumed to be returned by the call:

```
my.package.customFormatterFactory(bar='baz', spam=99.9, answer=42)
```

■■ Advertencia: Warning The values for keys such as `bar`, `spam` and `answer` in the above example should not be configuration dictionaries or references such as `ascfg://fooorext://bar`, because they will not be processed by the configuration machinery, but passed to the callable as-is.

The key '()' has been used as the special key because it is not a valid keyword parameter name, and so will not clash with the names of the keyword arguments used in the call. The '()' also serves as a mnemonic that the corresponding value is a callable.

Changed in version 3.11: The `filters` member of `handlers` and `loggers` can take filter instances in addition to `ids`.

You can also specify a special key `'.'` whose value is a mapping of attribute names to values. If found, the specified attributes will be set on the user-defined object before it is returned. Thus, with the following configuration:

```
{
  '()' : 'my.package.customFormatterFactory',
  'bar' : 'baz',
  'spam' : 99.9,
  'answer' : 42,
  '.' : {
    'foo' : 'bar',
    'baz' : 'bozz'
  }
}
```

the returned formatter will have attribute `foo` set to `'bar'` and attribute `baz` set to `'bozz'`.

■■ Advertencia: Warning The values for attributes such as `foo` and `baz` in the above example should not be configuration dictionaries or references such as `ascfg://fooorext://bar`, because they will not be processed by the configuration machinery, but set as attribute values as-is.

Handler configuration order

Handlers are configured in alphabetical order of their keys, and a configured handler replaces the configuration dictionary in (a working copy of) the `handlers` dictionary in the schema. If you use a construct such as `ascfg://handlers.foo`, then initially `handlers['foo']` points to the configuration dictionary for the handler named `foo`, and later (once that handler has been configured) it points to the configured handler instance.

Thus, `cfg://handlers.foo` could resolve to either a dictionary or a handler instance. In general, it is wise to name handlers in a way such that dependent handlers are configured after any handlers they depend on; that allows something like `cfg://handlers.foo` to be used in configuring a handler that depends on handler `foo`. If that dependent handler were named `bar`, problems would result, because the configuration of `bar` would be attempted before that of `foo`, and `foo` would not yet have been configured. However, if the dependent handler were named `foobar`, it would be configured after `foo`, with the result that `cfg://handlers.foo` would resolve to configured handler `foo`, and not its configuration dictionary.

Access to external objects

There are times where a configuration needs to refer to objects external to the configuration, for example `sys.stderr`. If the configuration dict is constructed using Python code, this is straightforward, but a problem arises when the configuration is provided via a text file (e.g. JSON, YAML). In a text file, there is no standard way to distinguish `sys.stderr` from the literal string `'sys.stderr'`. To facilitate this distinction, the configuration system looks for certain special prefixes in string values and treat them specially. For example, if

the literal string `'ext://sys.stderr'` is provided as a value in the configuration, then the `ext://` will be stripped off and the remainder of the value processed using normal import mechanisms.

The handling of such prefixes is done in a way analogous to protocol handling: there is a generic mechanism to look for prefixes which match the regular expression `^(?P<prefix>[a-z]+)://(?P<suffix>.*)$` whereby, if the prefix is recognised, the suffix is processed in a prefix-dependent manner and the result of the processing replaces the string value. If the prefix is not recognised, then the string value will be left as-is.

Access to internal objects

As well as external objects, there is sometimes also a need to refer to objects in the configuration. This will be done implicitly by the configuration system for things that it knows about. For example, the string value `'DEBUG'` for a level in a logger or handler will automatically be converted to the value `logging.DEBUG`, and the `handlers`, `filters` and `formatter` entries will take an object id and resolve to the appropriate destination object.

However, a more generic mechanism is needed for user-defined objects which are not known to the logging module. For example, consider `logging.handlers.MemoryHandler`, which takes a `target` argument which is another handler to delegate to. Since the system already knows about this class, then in the configuration, the given `target` just needs to be the object id of the relevant target handler, and the system will resolve to the handler from the id. If, however, a user defines `my.package.MyHandler` which has an `alternate_handler`, the configuration system would not know that the `alternate` referred to a handler. To cater for this, a generic resolution system allows the user to specify:

```
handlers:
  file:
    # configuration of file handler goes here

  custom:
    (): my.package.MyHandler
    alternate: cfg://handlers.file
```

The literal string `'cfg://handlers.file'` will be resolved in an analogous way to strings with the `ext://` prefix, but looking in the configuration itself rather than the import namespace. The mechanism allows access by dot or by index, in a similar way to that provided by `str.format`. Thus, given the following snippet:

```
handlers:
  email:
    class: logging.handlers.SMTPHandler
    mailhost: localhost
    fromaddr: my_app@domain.tld
    toaddrs:
      - support_team@domain.tld
      - dev_team@domain.tld
    subject: Houston, we have a problem.
```

in the configuration, the string `'cfg://handlers'` would resolve to the dict with key `handlers`, the string `'cfg://handlers.email'` would resolve to the dict with key `email` in the `handlers` dict, and so on. The string `'cfg://handlers.email.toaddrs[1]'` would resolve to `'dev_team@domain.tld'` and the string `'cfg://handlers.email.toaddrs[0]'` would resolve to the value `'support_team@domain.tld'`. The `subject` value could be accessed using either `'cfg://handlers.email.subject'` or, equivalently, `'cfg://handlers.email[subject]'`. The latter form only needs to be used if the key contains spaces or non-alphanumeric characters. Please note that the characters `[` and `]` are not allowed in the keys. If an index value consists only of decimal digits, access will be attempted using the corresponding integer value, falling back to the string value if needed.

Given a string `'cfg://handlers.myhandler.mykey.123'`, this will resolve to `config_dict['handlers']['myhandler']['mykey']['123']`. If the string is specified as `'cfg://handlers.myhandler.mykey[123]'`, the system will attempt to retrieve the value from `config_dict['handlers']['myhandler']['mykey'][123]`, and fall back to `config_dict['handlers']['myhandler']['mykey']['123']` if that fails.

Import resolution and custom importers

Import resolution, by default, uses the builtin `__import__()` function to do its importing. You may want to replace this with your own importing mechanism: if so, you can replace the `importer` attribute of the `DictConfigurator` or its superclass, the `BaseConfigurator` class. However, you need to be careful because of the way functions are accessed from classes via descriptors. If you are using a Python callable to do your imports, and you want to define it at class level rather than instance level, you need to wrap it with `staticmethod()`. For example:

```
from importlib import import_module
from logging.config import BaseConfigurator

BaseConfigurator.importer = staticmethod(import_module)
```

You don't need to wrap with `staticmethod()` if you're setting the import callable on a configurator instance.

Configuring QueueHandler and QueueListener

If you want to configure a `QueueHandler`, noting that this is normally used in conjunction with a `QueueListener`, you can configure both together. After the configuration, the `QueueListener` instance will be available as the `listener` attribute of the created handler, and that in turn will be available to you using `getHandlerByName()` and passing the name you have used for the `QueueHandler` in your configuration. The dictionary schema for configuring the pair is shown in the example YAML snippet below.

```
handlers:
  qhand:
    class: logging.handlers.QueueHandler
    queue: my.module.queue_factory
    listener: my.package.CustomListener
    handlers:
      - hand_name_1
      - hand_name_2
      ...
```

The `queue` and `listener` keys are optional.

If the `queue` key is present, the corresponding value can be one of the following:

- An object implementing the `Queue.put_nowait` and `Queue.get` public API. For instance, this may be an actual instance of `queue.Queue` or a subclass thereof, or a proxy obtained by `multiprocessing.managers.SyncManager.Queue()`. This is of course only possible if you are constructing or modifying the configuration dictionary in code.
- A string that resolves to a callable which, when called with no arguments, returns the queue instance to use. That callable could be a `queue.Queue` subclass or a function which returns a suitable queue instance, such as `my.module.queue_factory()`.
- A dict with a `'()`' key which is constructed in the usual way as discussed in User-defined objects. The result of this construction should be a `queue.Queue` instance.

If the `queue` key is absent, a standard unbounded `queue.Queue` instance is created and used.

If the `listener` key is present, the corresponding value can be one of the following:

- A subclass of `logging.handlers.QueueListener`. This is of course only possible if you are constructing or modifying the configuration dictionary in code.
- A string which resolves to a class which is a subclass of `QueueListener`, such as `'my.package.CustomListener'`.
- A dict with a `'()`' key which is constructed in the usual way as discussed in User-defined objects. The result of this construction should be a callable with the same signature as the `QueueListener` initializer.

If the `listener` key is absent, `logging.handlers.QueueListener` is used.

The values under the `handlers` key are the names of other handlers in the configuration (not shown in the above snippet) which will be passed to the queue listener.

Any custom queue handler and listener classes will need to be defined with the same initialization signatures as `QueueHandler` and `QueueListener`.

Added in version 3.12.

Configuration file format

The configuration file format understood by `fileConfig()` is based on `configparser` functionality. The file must contain sections called `[loggers]`, `[handlers]` and `[formatters]` which identify by name the entities of each type which are defined in the file. For each such entity, there is a separate section which identifies how that entity is configured. Thus, for a logger named `log01` in the `[loggers]` section, the relevant configuration details are held in a section `[logger_log01]`. Similarly, a handler called `hand01` in the `[handlers]` section will have its configuration held in a section called `[handler_hand01]`, while a formatter called `form01` in the `[formatters]` section will have its configuration specified in a section called `[formatter_form01]`. The root logger configuration must be specified in a section called `[logger_root]`.

■ **Nota:** `NoteThefileConfig()` API is older than `dictConfig()` API and does not provide functionality to cover certain aspects of logging. For example, you cannot configure `Filter` objects, which provide for filtering of messages beyond simple integer levels, using `fileConfig()`. If you need to have instances of `Filter` in your logging configuration, you will need to use `dictConfig()`. Note that future enhancements to configuration functionality will be added to `dictConfig()`, so it's worth considering transitioning to this newer API when it's convenient to do so.

Examples of these sections in the file are given below.

```
[loggers]
keys=root, log02, log03, log04, log05, log06, log07

[handlers]
keys=hand01, hand02, hand03, hand04, hand05, hand06, hand07, hand08, hand09

[formatters]
keys=form01, form02, form03, form04, form05, form06, form07, form08, form09
```

The root logger must specify a level and a list of handlers. An example of a root logger section is given below.

```
[logger_root]
level=NOTSET
handlers=hand01
```

The `level` entry can be one of `DEBUG`, `INFO`, `WARNING`, `ERROR`, `CRITICAL` or `NOTSET`. For the root logger only, `NOTSET` means that all messages will be logged. Level values are evaluated in the context of the logging package's namespace.

The `handlers` entry is a comma-separated list of handler names, which must appear in the `[handlers]` section. These names must appear in the `[handlers]` section and have corresponding sections in the configuration file.

For loggers other than the root logger, some additional information is required. This is illustrated by the following example.

```
[logger_parser]
level=DEBUG
handlers=hand01
propagate=1
qualname=compiler.parser
```

The `level` and `handlers` entries are interpreted as for the root logger, except that if a non-root logger's level is specified as `NOTSET`, the system consults loggers higher up the hierarchy to determine the effective level of the logger. The `propagate` entry is set to 1 to indicate that messages must propagate to handlers higher up the logger.

hierarchy from this logger, or 0 to indicate that messages are not propagated to handlers up the hierarchy. The `qualname` entry is the hierarchical channel name of the logger, that is to say the name used by the application to get the logger.

Sections which specify handler configuration are exemplified by the following.

```
[handler_hand01]
class=StreamHandler
level=NOTSET
formatter=form01
args=(sys.stdout,)
```

The `class` entry indicates the handler's class (as determined by `eval()` in the `logging` package's namespace). The `level` is interpreted as for loggers, and `NOTSET` is taken to mean 'log everything'.

The `formatter` entry indicates the key name of the formatter for this handler. If blank, a default formatter (`logging._defaultFormatter`) is used. If a name is specified, it must appear in the `[formatters]` section and have a corresponding section in the configuration file.

The `args` entry, when evaluated in the context of the `logging` package's namespace, is the list of arguments to the constructor for the handler class. Refer to the constructors for the relevant handlers, or to the examples below, to see how typical entries are constructed. If not provided, it defaults to `()`.

The optional `kwargs` entry, when evaluated in the context of the `logging` package's namespace, is the keyword argument dict to the constructor for the handler class. If not provided, it defaults to `{}`.

```
[handler_hand02]
class=FileHandler
level=DEBUG
formatter=form02
args=('python.log', 'w')
```

```
[handler_hand03]
class=handlers.SocketHandler
level=INFO
formatter=form03
args=('localhost', handlers.DEFAULT_TCP_LOGGING_PORT)
```

```
[handler_hand04]
class=handlers.DatagramHandler
level=WARN
formatter=form04
args=('localhost', handlers.DEFAULT_UDP_LOGGING_PORT)
```

```
[handler_hand05]
class=handlers.SysLogHandler
level=ERROR
formatter=form05
args=('localhost', handlers.SYSLOG_UDP_PORT), handlers.SysLogHandler.LOG_USER)
```

```
[handler_hand06]
class=handlers.NTEventLogHandler
level=CRITICAL
formatter=form06
args=('Python Application', '', 'Application')
```

```
[handler_hand07]
class=handlers.SMTPHandler
level=WARN
formatter=form07
args=('localhost', 'from@abc', ['user1@abc', 'user2@xyz'], 'Logger Subject')
kwargs={'timeout': 10.0}
```

```
[handler_hand08]
```

```

class=handlers.MemoryHandler
level=NOTSET
formatter=form08
target=
args=(10, ERROR)

[handler_hand09]
class=handlers.HTTPHandler
level=NOTSET
formatter=form09
args=('localhost:9022', '/log', 'GET')
kwargs={'secure': True}

```

Sections which specify formatter configuration are typified by the following.

```

[formatter_form01]
format=F1 %(asctime)s %(levelname)s %(message)s %(customfield)s
datefmt=
style=%
validate=True
defaults={'customfield': 'defaultvalue'}
class=logging.Formatter

```

The arguments for the formatter configuration are the same as the keys in the dictionary `schemaformatters` section.

The `defaultsentry`, when evaluated in the context of the `logging` package's namespace, is a dictionary of default values for custom formatting fields. If not provided, it defaults to `None`.

■ **Nota:** Due to the use of `eval()` as described above, there are potential security risks which result from using `listen()` to send and receive configurations via sockets. The risks are limited to where multiple users with no mutual trust run code on the same machine; see `listen()` documentation for more information.

See also

Module `logging`

Module `logging.handlers`

API reference for the logging module.

Useful handlers included with the logging module.

logging.handlers— Logging handlers

Source code: `Lib/logging/handlers.py`

Important

This page contains only reference information. For tutorials, please see

- Basic Tutorial
- Advanced Tutorial
- Logging Cookbook

The following useful handlers are provided in the package. Note that three of the handlers (`StreamHandler`, `FileHandler` and `NullHandler`) are actually defined in the `logging` module itself, but have been documented here along with the other handlers.

StreamHandler

The `StreamHandler` class, located in the `corelogging` package, sends logging output to streams such as `sys.stdout`, `sys.stderr` or any file-like object (or, more precisely, any object which supports `write()` and `flush()` methods).

`class logging.StreamHandler(stream=None)`

`emit(record)`

`flush()`

`setStream(stream)`

Parameters:

Returns:

terminator

Returns a new instance of the `StreamHandler` class. If `stream` is specified, the instance will use it for logging output; otherwise, `sys.stderr` will be used. `emit(record)` If a formatter is specified, it is used to format the record. The record is then written to the stream followed by `terminator`. If exception information is present, it is formatted using `traceback.print_exception()` and appended to the stream. `flush()` Flushes the stream by calling its `flush()` method. Note that the `close()` method is inherited from `Handler`.

If a formatter is specified, it is used to format the record. The record is then written to the stream followed by `terminator`. If exception information is present, it is formatted using `traceback.print_exception()` and appended to the stream.

Flushes the stream by calling its `flush()` method. Note that the `close()` method is inherited from `Handler` and so does no output, so an explicit `flush()` call may be needed at times.

Sets the instance's stream to the specified value, if it is different. The old stream is flushed before the new stream is set. **Parameters:** `stream`— The stream that the handler should use. **Returns:** the old stream, if the stream was changed, or `None` if it wasn't. Added in version 3.7.

`stream`— The stream that the handler should use.

the old stream, if the stream was changed, or `None` if it wasn't.

String used as the terminator when writing a formatted record to a stream. Default value is '\n'. If you don't want a newline termination, you can set the handler instance's `terminator` attribute to the empty string. In earlier versions, the terminator was hardcoded as '\n'. Added in version 3.2.

FileHandler

The `FileHandler` class, located in the `corelogging` package, sends logging output to a disk file. It inherits the output functionality from `StreamHandler`.

`class logging.FileHandler(filename, mode='a', encoding=None, delay=False, errors=None)`

`close()`

`emit(record)`

Returns a new instance of the `FileHandler` class. The specified file is opened and used as the stream for logging. If `mode` is not specified, 'a' is used. If `encoding` is not `None`, it is used to open the file with that encoding. If `delay` is `true`, then file opening is deferred until the first call to `emit()`. By default, the file grows indefinitely. If `errors` is specified, it's used to determine how encoding errors are handled. Changed in version 3.6: As well as string values, `Path` objects are also accepted for the `filename` parameter.

Closes the file.

Outputs the record to the file. Note that if the file was closed due to logging shutdown at exit and the file mode is 'w', the record will not be emitted (see [bpo-42378](#)).

NullHandler

Added in version 3.1.

The `NullHandler` class, located in the `corelogging` package, does not do any formatting or output. It is essentially a 'no-op' handler for use by library developers.

`class logging.NullHandler`

`emit(record)`

`handle(record)`

`createLock()`

Returns a new instance of the `NullHandler` class. `emit(record)` This method does nothing. `handle(record)` This method does nothing. `createLock()` This method returns `None` for the lock, since there is no underlying I/O to which access needs to be serialized.

This method does nothing.

This method does nothing.

This method returns `None` for the lock, since there is no underlying I/O to which access needs to be serialized.

See [Configuring Logging for a Library](#) for more information on how to use `NullHandler`.

WatchedFileHandler

The `WatchedFileHandler` class, located in the `logging.handlers` module, is a `FileHandler` which watches the file it is logging to. If the file changes, it is closed and reopened using the file name.

A file change can happen because of usage of programs such as `newsyslog` and `logrotate` which perform log file rotation. This handler, intended for use under Unix/Linux, watches the file to see if it has changed since the last emit. (A file is deemed to have changed if its device or inode have changed.) If the file has changed, the old file stream is closed, and the file opened to get a new stream.

This handler is not appropriate for use under Windows, because under Windows open log files cannot be moved or renamed - logging opens the files with exclusive locks - and so there is no need for such a handler.

Furthermore, `ST_INO` is not supported under Windows; `stat()` always returns zero for this value.

`class logging.handlers.WatchedFileHandler(filename, mode='a', encoding=None, delay=False, errors=None)`

`reopenIfNeeded()`

`emit(record)`

Returns a new instance of the `WatchedFileHandler` class. The specified file is opened and used as the stream for logging. If `mode` is not specified, 'a' is used. If `encoding` is not `None`, it is used to open the file with that encoding. If `delay` is true, then file opening is deferred until the first call to `emit()`. By default, the file grows indefinitely. If `errors` is provided, it determines how encoding errors are handled. Changed in version 3.6: As well as string values, `Path` objects are also accepted for the file.

Checks to see if the file has changed. If it has, the existing stream is flushed and closed and the file opened again, typically as a precursor to outputting the record to the file. Added in version 3.6.

Outputs the record to the file, but first calls `reopenIfNeeded()` to reopen the file if it has changed.

BaseRotatingHandler

The `BaseRotatingHandler` class, located in the `logging.handlers` module, is the base class for the rotating file handlers, `RotatingFileHandler` and `TimedRotatingFileHandler`. You should not need to instantiate this class, but it has attributes and methods you may need to override.

`class logging.handlers.BaseRotatingHandler(filename, mode, encoding=None, delay=False, errors=None)`

`namer`

`rotator`

`rotation_filename(default_name)`

Parameters:

`rotate(source, dest)`

Parameters:

The parameters are as for `FileHandler`. The attributes are: `namer` If this attribute is set to a callable, the `rotation_filename()` method delegates to this callable. The parameters passed to the callable are those passed to `rotation_filename()`. Note The `namer` function is called quite a few times during rollover, so it should be as simple and as fast as possible. It should also return the same output every time for a given input, otherwise the rollover behaviour may not work as expected. It's also worth not

If this attribute is set to a callable, the `rotation_filename()` method delegates to this callable. The parameters passed to the callable are those passed to `rotation_filename()`. Note The `namer` function is called quite a few times during rollover, so it should be as simple and as fast as possible. It should also return the same output every time for a given input, otherwise the rollover behaviour may not work as expected. It's also worth noting that care should be taken when using a `namer` to preserve c

If this attribute is set to a callable, the `rotate()` method delegates to this callable. The parameters passed to the callable are those passed to `rotate()`. Added in version 3.3.

Modify the filename of a log file when rotating. This is provided so that a custom filename can be provided. The default implementation calls the 'namer' attribute of the handler, if it's callable, passing the default name to it. If the attribute isn't callable (the default is `None`), the name is returned unchanged. Parameters: `default_name`— The default name for the log file. Added in version 3.3.

`default_name`— The default name for the log file.

When rotating, rotate the current log. The default implementation calls the 'rotator' attribute of the handler, if it's callable, passing the source and dest arguments to it. If the attribute isn't callable (the default is `None`), the source is simply renamed to the destination. Parameters: `source`— The source filename. This is normally the base filename, e.g. 'test.log'. `dest`— The destination filename. This is normally what the source is rotated to, e.g. 'test.log.1'. Added in version 3.3.

`source`— The source filename. This is normally the base filename, e.g. 'test.log'. `dest`— The destination filename. This is normally what the source is rotated to, e.g. 'test.log.1'.

The reason the attributes exist is to save you having to subclass - you can use the same callables for instances of `RotatingFileHandler` and `TimedRotatingFileHandler`. If either the namer or rotator callable raises an exception, this will be handled in the same way as any other exception during an `emit()` call, i.e. via the `handleError()` method of the handler.

If you need to make more significant changes to rotation processing, you can override the methods.

For an example, see [Using a rotator and namer to customize log rotation processing](#).

RotatingFileHandler

The `RotatingFileHandler` class, located in the `logging.handlers` module, supports rotation of disk log files.

`class logging.handlers.RotatingFileHandler(filename, mode='a', maxBytes=0, backupCount=0, encoding=None, delay=False, errors=None)`

`doRollover()`

`emit(record)`

`shouldRollover(record)`

Returns a new instance of the `RotatingFileHandler` class. The specified file is opened and used as the stream for logging. If `mode` is not specified, 'a' is used. If `encoding` is not `None`, it is used to open the file with that encoding. If `delay` is true, then file opening is deferred until the first call to `emit()`. By default, the file grows indefinitely. If `errors` is provided, it determines how encoding errors are handled. You can use the `maxBytes` and `backupCount` values to allow the file to rollover at a predetermi

Does a rollover, as described above.

Outputs the record to the file, catering for rollover as described previously.

See if the supplied record would cause the file to exceed the configured size limit.

TimedRotatingFileHandler

The `TimedRotatingFileHandler` class, located in the `logging.handlers` module, supports rotation of disk log files at certain timed intervals.

class logging.handlers.TimedRotatingFileHandler(filename, when='h', interval=1, backupCount=0, encoding=None, delay=False, utc=False, atTime=None, errors=None)

doRollover()

emit(record)

getFilesToDelete()

shouldRollover(record)

Returns a new instance of the `TimedRotatingFileHandler` class. The specified file is opened and used as the stream for logging. On rotating it also sets the filename suffix. Rotating happens based on the product of `when` and `interval`. You can use `when` to specify the type of `interval`. The list of possible values is below. Note that they are not case sensitive. `ValueType` of `interval` if `howatTime` is used: 'S' Seconds Ignored 'M' Minutes Ignored 'H' Hours Ignored 'D' Days Ignored 'W0'-'W6' Weekday (0=Monday) Used to compute

Does a rollover, as described above.

Outputs the record to the file, catering for rollover as described above.

Returns a list of filenames which should be deleted as part of rollover. These are the absolute paths of the oldest backup log files written by the handler.

See if enough time has passed for a rollover to occur and if it has, compute the next rollover time.

SocketHandler

The `SocketHandler` class, located in the `logging.handlers` module, sends logging output to a network socket. The base class uses a TCP socket.

class logging.handlers.SocketHandler(host, port)

close()

emit()

handleError()

makeSocket()

makePickle(record)

send(packet)

createSocket()

Returns a new instance of the `SocketHandler` class intended to communicate with a remote machine whose address is given by `host` and `port`. Changed in version 3.4: If `port` is specified as `None`, a Unix domain socket is created using the value in `host` - otherwise, a TCP socket is created. `close()` Closes the socket. `emit()` Pickles the record's attribute dictionary and writes it to the socket in binary format. If there is an error with the socket, silently drops the packet. If the connection was previously lost, re-es

Closes the socket.

Pickles the record's attribute dictionary and writes it to the socket in binary format. If there is an error with the socket, silently drops the packet. If the connection was previously lost, re-establishes the connection. To unpickle the record at the receiving end into a `LogRecord`, use the `makeLogRecord()` function.

Handles an error which has occurred during `emit()`. The most likely cause is a lost connection. Closes the socket so that we can retry on the next event.

This is a factory method which allows subclasses to define the precise type of socket they want. The default implementation creates a TCP socket (`socket.SOCK_STREAM`).

Pickles the record's attribute dictionary in binary format with a length prefix, and returns it ready for transmission across the socket. The details of this operation are equivalent to:
`data=pickle.dumps(record_attr_dict,1)`
`data_len=struct.pack('>L',len(data))`
`return data_len+data`
Note that pickles aren't completely secure. If you are concerned about security, you may want to override this method to implement a more secure mechanism. For example, you can sign pickles using HMAC and then verify them on

Send a pickled byte-string `packet` to the socket. The format of the sent byte-string is as described in the documentation for `makePickle()`. This function allows for partial sends, which can happen when the network is busy.

Tries to create a socket; on failure, uses an exponential back-off algorithm. On initial failure, the handler will drop the message it was trying to send. When subsequent messages are handled by the same instance, it will not try connecting until some time has passed. The default parameters are such that the initial delay is one second, and if after that delay the connection still can't be made, the handler will double the delay each time up to a maximum of 30 seconds. This behaviour is contro

DatagramHandler

The `DatagramHandler` class, located in the `logging.handlers` module, inherits from `SocketHandler` to support sending logging messages over UDP sockets.

`class logging.handlers.DatagramHandler(host,port)`

`emit()`

`makeSocket()`

`send(s)`

Returns a new instance of the `DatagramHandler` class intended to communicate with a remote machine whose address is given by `host` and `port`. Note As UDP is not a streaming protocol, there is no persistent connection between an instance of this handler and `host`. For this reason, when using a network socket, a DNS lookup might have to be made each time an event is logged, which can introduce some latency into the system. If this affects you, you can do a lookup yourself and initialize this handler using the

Pickles the record's attribute dictionary and writes it to the socket in binary format. If there is an error with the socket, silently drops the packet. To unpickle the record at the receiving end into a `LogRecord`, use the `makeLogRecord()` function.

The factory method of `SocketHandler` is here overridden to create a UDP socket (`socket.SOCK_DGRAM`).

Send a pickled byte-string to a socket. The format of the sent byte-string is as described in the documentation for `SocketHandler.makePickle()`.

SysLogHandler

The `SysLogHandler` class, located in the `logging.handlers` module, supports sending logging messages to a remote or local Unix syslog.

`class logging.handlers.SysLogHandler(address=('localhost',SYSLOG_UDP_PORT),facility=LOG_USER,socktype=socket.SOCK_DGRAM,timeout=None)`

`close()`

createSocket()

emit(record)

encodePriority(facility,priority)

mapPriority(levelname)

Returns a new instance of the `SysLogHandler` class intended to communicate with a remote Unix machine whose address is given by `address` in the form of a `(host,port)` tuple. If `address` is not specified, `('localhost',514)` is used. The address is used to open a socket. An alternative to providing a `(host,port)` tuple is providing an address as a string, for example `'/dev/log'`. In this case, a Unix domain socket is used to send the message to the syslog. If `facility` is not specified, `LOG_USER` is used. The type of `s`

Closes the socket to the remote host.

Tries to create a socket and, if it's not a datagram socket, connect it to the other end. This method is called during handler initialization, but it's not regarded as an error if the other end isn't listening at this point - the method will be called again when emitting an event, if there is no socket at that point. Added in version 3.11.

The record is formatted, and then sent to the syslog server. If exception information is present, it is not sent to the server. Changed in version 3.2.1: (See: bpo-12168.) In earlier versions, the message sent to the syslog daemons was always terminated with a NUL byte, because early versions of these daemons expected a NUL terminated message - even though it's not in the relevant specification (RFC 5424). More recent versions of these daemons don't expect the NUL byte but strip it off if it's there,

Encodes the facility and priority into an integer. You can pass in strings or integers - if strings are passed, internal mapping dictionaries are used to convert them to integers. The symbolic `LOG_*` values are defined in `SysLogHandler` and mirror the values defined in the `sys/syslog.h` header file. `PrioritiesName` (string) Symbolic value `alert` `LOG_ALERT` `crit` or `critical` `LOG_CRIT` `debug` `LOG_DEBUG` `emerg` or `panic` `LOG_EMERG` `error` or `error` `LOG_ERR` `info` `LOG_INFO` `notice` `LOG_NOTICE` `warn` or `warning` `LOG_WARNING` `FacilitiesName` (string) Symbolic value

Maps a logging level name to a syslog priority name. You may need to override this if you are using custom levels, or if the default algorithm is not suitable for your needs. The default algorithm maps `DEBUG`, `INFO`, `WARNING`, `ERROR` and `CRITICAL` to the equivalent syslog names, and all other level names to `'warning'`.

NTEventLogHandler

The `NTEventLogHandler` class, located in the `logging.handlers` module, supports sending logging messages to a local Windows NT, Windows 2000 or Windows XP event log. Before you can use it, you need Mark Hammond's Win32 extensions for Python installed.

class logging.handlers.NTEventLogHandler(appname,dllname=None,logtype='Application')

close()

emit(record)

getEventCategory(record)

getEventType(record)

getMessageID(record)

Returns a new instance of the `NTEventLogHandler` class. The `appname` is used to define the application name as it appears in the event log. An appropriate registry entry is created using this name. The `dllname` should give the fully qualified pathname of a `.dll` or `.exe` which contains message definitions to hold in the log (if not specified, `'win32service.pyd'` is used - this is installed with the Win32 extensions and contains some basic

placeholder message definitions. Note that use of these placeholders wi

At this point, you can remove the application name from the registry as a source of event log entries. However, if you do this, you will not be able to see the events as you intended in the Event Log Viewer - it needs to be able to access the registry to get the .dll name. The current version does not do this.

Determines the message ID, event category and event type, and then logs the message in the NT event log.

Returns the event category for the record. Override this if you want to specify your own categories. This version returns 0.

Returns the event type for the record. Override this if you want to specify your own types. This version does a mapping using the handler's `typemap` attribute, which is set up in `__init__()` to a dictionary which contains mappings for `DEBUG`, `INFO`, `WARNING`, `ERROR` and `CRITICAL`. If you are using your own levels, you will either need to override this method or place a suitable dictionary in the handler's `typemap` attribute.

Returns the message ID for the record. If you are using your own messages, you could do this by having `themsg` passed to the logger being an ID rather than a format string. Then, in here, you could use a dictionary lookup to get the message ID. This version returns 1, which is the base message ID in `win32service.pyd`.

SMTPHandler

The `SMTPHandler` class, located in the `logging.handlers` module, supports sending logging messages to an email address via SMTP.

`class logging.handlers.SMTPHandler(mailhost, fromaddr, toaddrs, subject, credentials=None, secure=None, timeout=1.0)`

`emit(record)`

`getSubject(record)`

Returns a new instance of the `SMTPHandler` class. The instance is initialized with the from and to addresses and subject line of the email. The `toaddrs` should be a list of strings. To specify a non-standard SMTP port, use the (host, port) tuple format for the `mailhost` argument. If you use a string, the standard SMTP port is used. If your SMTP server requires authentication, you can specify a (username, password) tuple for the `credentials` argument. To specify the use of a secure protocol (TLS), pass in a t

Formats the record and sends it to the specified addressees.

If you want to specify a subject line which is record-dependent, override this method.

MemoryHandler

The `MemoryHandler` class, located in the `logging.handlers` module, supports buffering of logging records in memory, periodically flushing them to a target handler. Flushing occurs whenever the buffer is full, or when an event of a certain severity or greater is seen.

`MemoryHandler` is a subclass of the more general `BufferingHandler`, which is an abstract class. This buffers logging records in memory. Whenever each record is added to the buffer, a check is made by calling `shouldFlush()` to see if the buffer should be flushed. If it should, then `flush()` is expected to do the flushing.

`class logging.handlers.BufferingHandler(capacity)`

`emit(record)`

`flush()`

`shouldFlush(record)`

Initializes the handler with a buffer of the specified capacity. Here, capacity means the number of logging records buffered. `emit(record)` Append the record to the buffer. `shouldFlush()` returns true, call `flush()` to process the buffer. `flush()` For a `BufferingHandler` instance, flushing means that it sets the buffer to an empty list. This method can be overwritten to implement more useful flushing behavior. `shouldFlush(record)` Return True if the buffer is up to capacity. This method can be overridden to implement

Append the record to the buffer. `shouldFlush()` returns true, call `flush()` to process the buffer.

For a `BufferingHandler` instance, flushing means that it sets the buffer to an empty list. This method can be overwritten to implement more useful flushing behavior.

Return True if the buffer is up to capacity. This method can be overridden to implement custom flushing strategies.

`logging.handlers.MemoryHandler(capacity, flushLevel=ERROR, target=None, flushOnClose=True)`

`close()`

`flush()`

`setTarget(target)`

`shouldFlush(record)`

Returns a new instance of the `MemoryHandler` class. The instance is initialized with a buffer size of capacity (number of records buffered). If flushLevel is not specified, `ERROR` is used. If no target is specified, the target will need to be set using `setTarget()` before this handler does anything useful. If `flushOnClose` is specified as `False`, then the buffer is not flushed when the handler is closed. If not specified or specified as `True`, the previous behaviour of flushing the buffer will occur when the handler is

Calls `flush()`, sets the target to `None` and clears the buffer.

For a `MemoryHandler` instance, flushing means just sending the buffered records to the target, if there is one. The buffer is also cleared when buffered records are sent to the target. Override if you want different behavior.

Sets the target handler for this handler.

Checks for buffer full or a record at the flushLevel or higher.

HTTPHandler

The `HTTPHandler` class, located in the `logging.handlers` module, supports sending logging messages to a web server, using either `GET` or `POST` semantics.

`logging.handlers.HTTPHandler(host, url, method='GET', secure=False, credentials=None, context=None)`

`mapLogRecord(record)`

`emit(record)`

Returns a new instance of the `HTTPHandler` class. The `host` can be of the form `host:port`, should you need to use a specific port number. If no `method` is specified, `GET` is used. If `secure` is true, a `HTTPS` connection will be used. The `context` parameter may be set to `ssl.SSLContext` instance to configure the SSL settings used for the `HTTPS` connection. If `credentials` is specified, it should be a 2-tuple consisting of `userid` and `password`, which will be placed in a `HTTP` 'Authorization' header using Basic authentication

Provides a dictionary, based on `record`, which is to be URL-encoded and sent to the web server. The default implementation just returns `record.__dict__`. This method can be overridden if e.g. only a subset of `LogRecord` is to be sent to the web server, or if more specific customization of what's sent to the server is required.

Sends the record to the web server as a URL-encoded dictionary. The `mapLogRecord()` method is used to convert the record to the dictionary to be sent.

QueueHandler

Added in version 3.2.

The `QueueHandler` class, located in the `logging.handlers` module, supports sending logging messages to a queue, such as those implemented in the `queue` or `multiprocessing` modules.

Along with the `QueueListener` class, `QueueHandler` can be used to let handlers do their work on a separate thread from the one which does the logging. This is important in web applications and also other service applications where threads servicing clients need to respond as quickly as possible, while any potentially slow operations (such as sending an email via `SMTPHandler`) are done on a separate thread.

`class logging.handlers.QueueHandler(queue)`

`emit(record)`

`prepare(record)`

`enqueue(record)`

`listener`

Returns a new instance of the `QueueHandler` class. The instance is initialized with the queue to send messages to. The queue can be any queue-like object; it's used as-is by the `enqueue()` method, which needs to know how to send messages to it. The queue is not required to have the task tracking API, which means that you can use `SimpleQueue` instances for queue. Note if you are using `multiprocessing`, you should avoid using `SimpleQueue` and instead use `multiprocessing.Queue`. Warning: The `multiprocessing` module uses an int

Enqueues the result of preparing the `LogRecord`. Should an exception occur (e.g. because a bounded queue has filled up), the `handleError()` method is called to handle the error. This can result in the record silently being dropped (if `logging.raiseExceptions` is `False`) or a message printed to `sys.stderr` (if `logging.raiseExceptions` is `True`).

Prepares a record for queuing. The object returned by this method is enqueued. The base implementation formats the record to merge the message, arguments, exception and stack information, if present. It also removes unpickleable items from the record in-place. Specifically, it overwrites the record's `msg` and `message` attributes with the merged message (obtained by calling the handler's `format()` method), and sets the `args`, `exc_info` and `exc_text` attributes to `None`. You might want to override this method if you

Enqueues the record on the queue using `put_nowait()`; you may want to override this if you want to use blocking behaviour, or a timeout, or a customized queue implementation.

When created via configuration using `dictConfig()`, this attribute will contain a `QueueListener` instance for use with this handler. Otherwise, it will be `None`. Added in version 3.12.

QueueListener

Added in version 3.2.

The `QueueListener` class, located in the `logging.handlers` module, supports receiving logging messages from a queue, such as those implemented in the `queue` or `multiprocessing` modules. The messages are received from a queue in an internal thread and passed, on the same thread, to one or more handlers for processing. While `QueueListener` is not itself a handler, it is documented here because it works hand-in-hand with `QueueHandler`.

Along with the `QueueHandler` class, `QueueListener` can be used to let handlers do their work on a separate thread from the one which does the logging. This is important in web applications and also other service applications where threads servicing clients need to respond as quickly as possible, while any potentially slow operations (such as sending an email via `SMTPHandler`) are done on a separate thread.

`class logging.handlers.QueueListener(queue,*handlers,respect_handler_level=False)`

`dequeue(block)`

`prepare(record)`

`handle(record)`

`start()`

`stop()`

`enqueue_sentinel()`

Returns a new instance of the `QueueListener` class. The instance is initialized with the queue to send messages to and a list of handlers which will handle entries placed on the queue. The queue can be any queue-like object; it's passed as-is to the `dequeue()` method, which needs to know how to get messages from it. The queue is not required to have the task tracking API (though it's used if available), which means that you can use `SimpleQueue` instances for queue. Note if you are using multiprocessing, you sh

Dequeues a record and return it, optionally blocking. The base implementation uses `get()`. You may want to override this method if you want to use timeouts or work with custom queue implementations.

Prepare a record for handling. This implementation just returns the passed-in record. You may want to override this method if you need to do any custom marshalling or manipulation of the record before passing it to the handlers.

Handle a record. This just loops through the handlers offering them the record to handle. The actual object passed to the handlers is that which is returned from `prepare()`.

Starts the listener. This starts up a background thread to monitor the queue for `LogRecords` to process. Changed in version 3.14: Raises `RuntimeError` if called and the listener is already running.

Stops the listener. This asks the thread to terminate, and then waits for it to do so. Note that if you don't call this before your application exits, there may be some records still left on the queue, which won't be processed.

Writes a sentinel to the queue to tell the listener to quit. This implementation uses `put_nowait()`. You may want to override this method if you want to use timeouts or work with custom queue implementations. Added in version 3.3.

See also

Module `logging`

Module `logging.config`

API reference for the logging module.

Configuration API for the logging module.